JAVA · SPRING BOOT · POSTGRESQL

Maktab Boshqaruvi

Backend qo'llanmasi

Har bir paket, klass va endpoint: nima qiladi, qanday ishlaydi va nega aynan shunday yozilgan

---

Loyiha: maktab-boshqaruv (main, 62c70e3)

Sana: 2026-10-15

Hajm: 351 ta Java klass · 195 ta endpoint · 39 ta entity · 55 ta test fayli

Mundarija

1. Umumiy ko'rinish 5
Loyiha nima qiladi? 5
Texnologiyalar va versiyalar 5
Raqamlarda 6
Bu qo'llanmani qanday o'qish kerak 6

2. Arxitektura: so'rov qanday yo'l bosadi 8
Qatlamli (layered) arxitektura 8
Bitta so'rovning to'liq hayoti (misol) 9
Ikkinchi kirish eshigi: Telegram 10
Ilova ishga tushganda nima bo'ladi? 11

3. Papka va paketlar tuzilmasi 13
Loyiha ildizi 13
Java paketlari 13
Nomlash qoidalari 14

4. Konfiguratsiya 16
build.gradle — loyiha "retsepti" 16
application.properties — ilova sozlamalari 18
Profillar: local va dev 21
@ConfigurationProperties: sozlamalarni klassga yig'ish 22

5. Ma'lumotlar bazasi 24
Umumiy rasm 24
1) Maktab tuzilmasi va odamlar 25
2) O'quv jarayoni 26
3) Telegram va ota-onalar 26
4) Kutubxona 27
Entity anatomiyasi: Attendance misolida 27
Bog'lanish turlari loyihada 29
ddl-auto=update: qulaylik va chegaralar 29

6. Modul: maktab tuzilmasi 31
Andaza: Bino moduli to'liq zanjir bilan 31
Endpoint'lar 35
Fan: o'chirish o'rniga nofaol qilish 36
Xodim: ta'til va hisobni bloklash 37

7. Modul: o'quvchi, hudud va tuman 39
Tarkibi 39
Hudud va tuman: ma'lumotnoma (spravochnik) 39
O'quvchi entity'si 41

So'rov DTO'si va validatsiya 42
So'rov boshidan oxirigacha: yangi o'quvchi qo'shish 43
Qidiruv: JPQL so'rovi 46

8. Modul: dars jadvali 48

Yangi dars qo'shish: qoidalar zanjiri 48
"Hozir qaysi dars?" va "N+1 yo'q" so'rov 50
Haftalik jadval PDF 51

9. Modul: davomat, baholar, jurnal, xulq, arizalar 53

Davomat: butun sinfni bitta so'rovda belgilash 53
Domain event'lar: "nimadir bo'ldi" 55
Baholar: qoidalar 56
Sinf jurnali 57
Oylik davomat kalendari 57
Sababli ariza: botdan saytga va qaytib 58

10. Modul: boshqaruv paneli, profillar, taqvim, e'lonlar 61

Bosh sahifa: bitta so'rov — sakkizta vidjet 61
Profillar 62
E'lonlar: yana bir event 62
Faoliyat lentasi va bitta nozik tuzoq 63

11. Modul: Telegram bot va xabarnomalar 66

Kiruvchi yo'l: update'lardan ekrangacha 66
Chiquvchi yo'l: outbox pattern 72
Rejalashtirilgan ishlar (@Scheduled) 75
Mock rejim: Telegram'siz ishlab chiqish 76
Maxfiylik: token hech qayerga yozilmaydi 76

12. Modul: ota-onalar — murojaatlar, ommaviy xabarlar, Mini App 78

Mini App API: JWT'siz himoya 78
Murojaatlar 80
Ommaviy xabarlar 82

13. Modul: kutubxona 84

Har bir kitob qonuniy bo'lishi shart 84
Fayllar: ishonmang, tekshiring 86
Bot tomoni: kunlik limit va PDF yuborish 87
Ruxsatnomalar va kundalik tekshiruv 87
Wikisource'dan import 87

14. Modul: foydalanuvchilar va maktab chegarasi 90

Foydalanuvchi va rollar 90
Ikki xil foydalanuvchi: maktab xodimi va platforma admini 91
Maktab chegarasi: id qayerda kelmasin, tekshiriladi 92
⚠️ Topilgan jiddiy zaifliklar 95

15. Xavfsizlik: Spring Security va JWT 96

Asosiy tushunchalar 96
SecurityConfig: qoidalar bir joyda 96
Login 98
Token: yaratish va tekshirish 99
Har bir so'rovda: JwtAuthFilter 100
Metod darajasidagi himoya: @PreAuthorize 101
Himoya qatlamlari bir rasmda 102

16. Xatolarni boshqarish 103
Yagona joyda: GlobalExceptionHandler 103
Exception → HTTP status jadvali 104
Exception turlari loyihada 105
Kamchiliklar 105

17. Loyihani ishga tushirish 107
Kerakli dasturlar 107
Muhit o'zgaruvchilari 107
Qadamma-qadam: lokal ishga tushirish 108
Testlar 109
Ishlab chiqarishga (prod) o'tkazish 109

18. Kuchli tomonlar va yaxshilash kerak bo'lgan joylar 111
Kuchli tomonlar 111
Yaxshilash kerak bo'lgan joylar qayerda 111

19. Lug'at 113
Annotatsiyalar 113
Atamalar 115

A-ilova. Barcha endpoint'lar (195 ta) 117

B-ilova. Barcha klasslar katalogi (351 ta) 126

C-ilova. Barcha entity'lar va maydonlari (39 ta) 139

D-ilova. Testlar (55 ta fayl) 153

1. Umumiy ko'rinish

Loyiha nima qiladi?

Maktab Boshqaruvi — bir yoki bir nechta maktabning kundalik ishini raqamlashtiradigan tizim. Uning **backend** qismi (shu qo'llanma mavzusi) — Java'da yozilgan Spring Boot ilovasi. U ma'lumotlar bazasini boshqaradi va ikki xil "mijoz"ga xizmat qiladi:

1. **Sayt (admin panel)** — maktab xodimlari (direktor, o'qituvchi, kotib) brauzerda ishlaydi. Sayt backend bilan **REST API** orqali gaplashadi: `/api/...` manzillariga JSON yuboradi va JSON oladi.
2. **Telegram bot va Mini App** — ota-onalar farzandining davomati, baholari, dars jadvali va maktab e'lonlarini Telegram'da ko'radi, maktabga xat yozadi, sababli ariza beradi, kutubxonadan kitob oladi.

Backend nimalarni boshqaradi (har biri keyingi boblarda alohida yoritiladi):

Soha	Nimalar bor
Maktab tuzilmasi	maktablar, binolar, xonalar, o'quv yillari, sinflar, fanlar, lavozimlar, xodimlar
O'quvchilar	o'quvchi kartochkasi, vasiy, tug'ilgan joyi va yashash manzili (hudud → tuman)
O'quv jarayoni	dars jadvali, davomat, baholar, sinf jurnali, xulq, sababli arizalar
Boshqaruv paneli	bosh sahifa ko'rsatkichlari, profillar, taqvim, e'lonlar, faoliyat lentasi
Foydalanuvchilar	login, rollar (ADMIN, EDITOR, VIEWER), bir nechta maktab va ularning chegarasi
Telegram	bot ekranlari, avtomatik xabarnomalar (outbox), murojaatlar, ommaviy xabarlar, Mini App API
Kutubxona	kitoblar (PDF), bo'limlar, mutolaa statistikasi, mualliflik huquqi, ruxsatnomalar, Wikisource importi

> 🧩 **Backend — maktab kotibiyati**
>
> Backend'ni maktabning kotibiyatiga o'xshatish mumkin. Xodimlar (sayt) va ota-onalar (bot) unga murojaat qiladi. Kotibiyat murojaatchini tanib oladi (login), uning shu ishga haqqi borligini tekshiradi (rollar), kerakli jurnalni arxivdan topadi (bazadan o'qiydi), yozib qo'yadi (bazaga yozadi) va javob beradi. Kotibiyat hech qachon ota-onaga boshqa maktab o'quvchisining jurnalini ko'rsatmaydi — bu ham uning vazifasi.

Texnologiyalar va versiyalar

Texnologiya	Versiya	Loyihada nima uchun
Java	21 (LTS)	dasturlash tili; `record`, `switch` ifodalari, `var`, matn bloklari ishlatilgan
Spring Boot	4.1.0	ilova karkasi: avtomatik sozlash, ichki web-server, bean'lar
Spring Framework	7.0.8	Spring Boot ostidagi yadro: IoC konteyner, Web MVC, tranzaksiyalar
Spring Security	7.1.0	autentifikatsiya va avtorizatsiya, filtrlar zanjiri
Spring Data JPA	4.1.0	repository interfeyslari — SQL yozmasdan bazaga so'rov
Hibernate ORM	7.4.1	JPA'ning amaldagi bajaruvchisi: Java ob'ektlari ↔ jadvallar
PostgreSQL	18 (server)	ma'lumotlar bazasi

Texnologiya	Versiya	Loyihada nima uchun
HikariCP	7.0.2	bazaga ulanishlar hovuzi (connection pool)
Apache Tomcat	11.0.22 (ichki)	HTTP so'rovlarni qabul qiladigan web-server, ilova ichida
Jackson	3.1.4 (`tools.jackson`)	Java ob'ektlarini JSON'ga va aksincha o'girish
Hibernate Validator	9.1.0	`@NotBlank` , `@Size` kabi validatsiya annotatsiyalari
JJWT	0.12.5	JWT token yaratish va tekshirish
Caffeine	(Spring BOM)	xotiradagi tezkor kesh
OpenPDF	2.0.3	PDF yaratish (dars jadvali, ruxsat xati, Wikisource kitoblari)
Apache PDFBox	3.0.5	PDF o'qish: bet soni, birinchi betdan muqova
Gradle	wrapper	qurish (build) vositasi: `build.gradle`

> ⚠️ **pom.xml emas, build.gradle**
>
> Topshiriqda `pom.xml` deb aytilgan, lekin bu loyiha **Maven** emas, **Gradle** bilan qurilgan. Ikkalasi bir xil vazifani bajaradi: qaysi kutubxonalar kerakligini aytadi va loyihani yig'adi. Shuning uchun 4-bobda `build.gradle` tushuntiriladi.

Raqamlarda

Ko'rsatkich	Qiymat
Java fayllari (`src/main/java`)	351 ta, ≈35 000 qator
Paketlar	17 ta
REST endpoint'lar	195 ta (35 ta controller'da)
JPA entity'lar (jadvallar)	39 ta (+ 20 ta enum)
Repository interfeyslari	39 ta
DTO klasslari	74 ta
Test fayllari (`src/test/java`)	55 ta

Bu qo'llanmani qanday o'qish kerak

Qo'llanma tepadan pastga qarab chuqurlashadi:

1. **Arxitektura** (2-bob) — so'rov qanday yo'l bosishini butun rasm sifatida ko'rasiz.
2. **Paketlar va konfiguratsiya** (3–4-boblar) — fayllar qayerda va ilova qanday sozlanadi.
3. **Ma'lumotlar bazasi** (5-bob) — barcha jadvallar va bog'lanishlar.
4. **Modullar** (6–14-boblar) — har bir soha: Entity → Repository → Service → Controller → DTO.
5. **Xavfsizlik va xatolar** (15–16-boblar) — eng muhim ko'ndalang (cross-cutting) mavzular.
6. **Ishga tushirish, tahlil, lug'at** (17–19-boblar).
7. **Ilovalar** — 195 endpoint, 351 klass, 39 entity va 55 test to'liq ro'yxati. "Hech narsa qolib ketmasin" degan talab shu ilovalar orqali ta'minlanadi: kitobda alohida tushuntirilmagan klass ham ilovada o'z izohi bilan bor.

Har bir kod bloki **haqiqiy fayldan** olingan: tepada fayl yo'li, chapda shu fayldagi haqiqiy qator raqamlari. Sariq rang bilan belgilangan qatorlar matnda alohida tushuntiriladi.

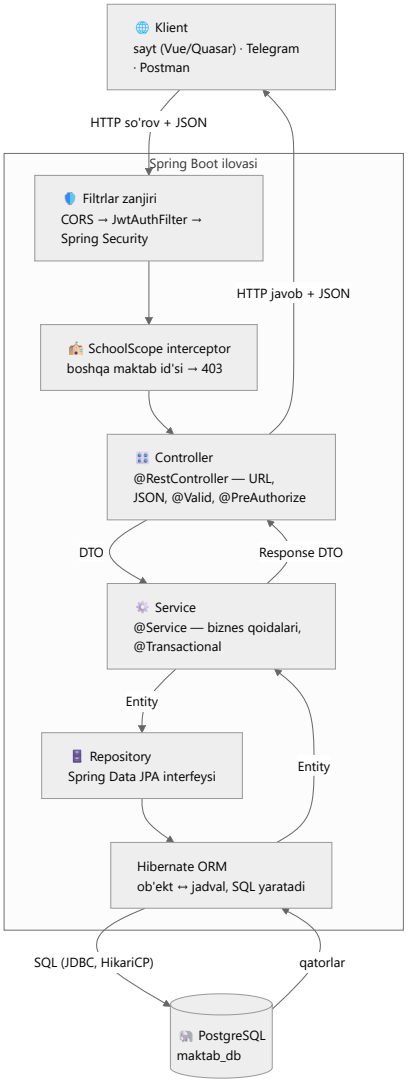
📌 **Esda tut**

- Backend — Spring Boot 4.1 (Java 21) ilovasi, PostgreSQL bazasi bilan ishlaydi.
- Ikki mijozi bor: sayt (JWT bilan) va Telegram bot / Mini App (Telegram imzosi bilan).
- 351 ta klass, 195 ta endpoint, 39 ta jadval — hammasi ilovalarda ro'yxatlangan.
- Loyiha Maven emas, Gradle bilan quriladi.

2. Arxitektura: so'rov qanday yo'l bosadi

Qatlamli (layered) arxitektura

Loyiha klassik **qatlamli arxitektura** asosida qurilgan. Har bir qatlamning bitta vazifasi bor va u faqat o'zidan pastdagi qatlam bilan gaplashadi:



2.1-rasm. Bitta HTTP so'rovning yo'li: tepadan pastga va qaytib.

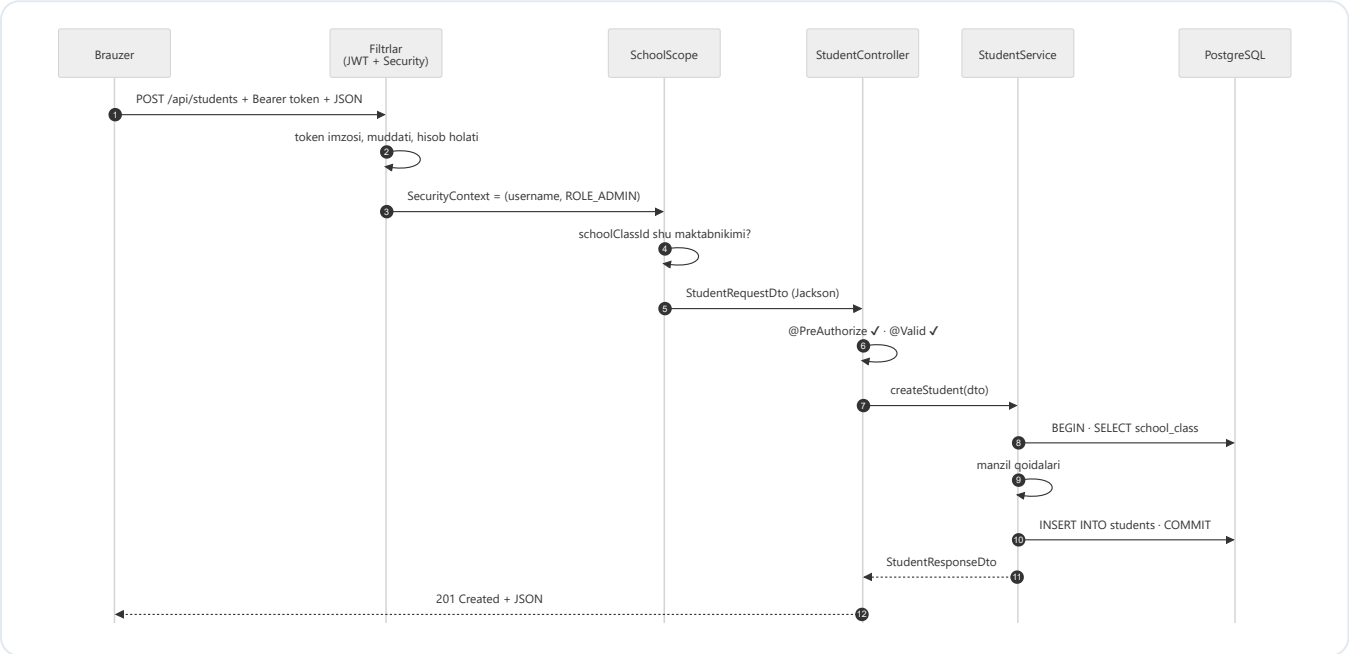
Qatlam	Paket	Vazifasi	Nima QILMAYDI
Controller	`controller`	URL'ni metodga bog'laydi, JSON'ni DTO'ga o'giradi, validatsiya va rolni tekshiradi	biznes qoidasi yozmaydi, bazaga to'g'ridan-to'g'ri murojaat qilmaydi
Service	`service`, `service.*`	biznes qoidalari ("yakshanbaga baho qo'yilmaydi"), tranzaksiya, event chiqarish	HTTP haqida bilmaydi (status kodlar, header'lar)
Repository	`repository`	bazadan o'qish/yozish (Spring Data JPA)	qoidalarni tekshirmaydi
Entity	`entity`	jadvalning Java'dagi aksi	API'ga to'g'ridan-to'g'ri chiqmaydi

Qatlam	Paket	Vazifasi	Nima QILMAYDI
DTO	`dto`	API'ning kirish/chiqish shakli	bazaga saqlanmaydi

> 🤔 **Nega qatlamlarga bo'lingan?**
> Agar hammasi bitta klassda bo'lsa (URL, qoida va SQL aralash), "baho qo'yish" qoidasini o'zgartirish uchun HTTP kodini ham titkilash kerak bo'lardi, va bir qoidani ikki joyda (sayt va bot) takrorlash kerak bo'lardi. Qatlamlar tufayli **bitta qoida bitta joyda** yashaydi: masalan, `GradeService` qoidalarini ham sayt (`GradeController`), ham test ishlatadi. Service'ni HTTP'siz, oddiy Java testi bilan sinash ham oson (loyihadagi 22 ta servis testi aynan shunday — Mockito bilan, bazasiz).

Bitta so'rovning to'liq hayoti (misol)

Xodim saytda yangi o'quvchi qo'shadi: `POST /api/students`. Quyidagi ketma-ketlik diagrammasi har bir qadamni ko'rsatadi:



2.2-rasm. "Yangi o'quvchi qo'shish" so'rovi boshidan oxirigacha (7-bobda kod bilan).

Qadamlarni so'z bilan:

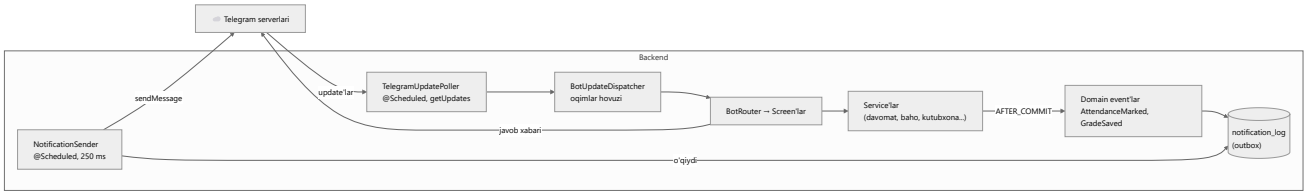
1. **Tomcat** HTTP so'rovni qabul qiladi va Spring'ning `DispatcherServlet` iga uzatishdan oldin **filtrlar zanjiri**dan o'tkazadi.
2. `JwtAuthFilter` `Authorization` header'idagi tokenni tekshiradi. Token to'g'ri bo'lsa, foydalanuvchi nomi va roli `SecurityContextHolder` ga yoziladi. Token yo'q yoki noto'g'ri bo'lsa, hech narsa yozilmaydi.
3. **Spring Security** `SecurityConfig` dagi qoidaga qaraydi: `/api/students` — `authenticated()`. Kontekstda foydalanuvchi bo'lmasa, javob shu yerda **403** bilan tugaydi.
4. `SchoolScopeConfig` URL va query parametrlaridagi id'larni, keyin JSON body ichidagi `schoolClassId` kabi maydonlarni tekshiradi: boshqa maktabniki bo'lsa — **403**.
5. **Jackson** JSON'ni `StudentRequestDto` ob'ektiga aylantiradi; `@Valid` validatsiya annotatsiyalarini tekshiradi (xato bo'lsa — **400**).
6. `@PreAuthorize` rolni tekshiradi (VIEWER bo'lsa — **403**).

7. `StudentService.createStudent` — tranzaksiya ichida biznes qoidalari va saqlash. Xato bo'lsa — tranzaksiya **ROLLBACK** qilinadi, bazaga hech narsa yozilmaydi.
8. Natija **DTO** sifatida qaytadi, Jackson uni JSON'ga aylantiradi, `201 Created`.

> 🔬 **Spring proxy nima?**
> `StudentController` aslida `StudentService` ob'ektining o'zini emas, Spring yaratgan **proxy**ni (o'rinbosar ob'ektni) chaqiradi. Proxy `@Transactional` ko'rgani uchun metoddan oldin `BEGIN` ochadi, metoddan keyin `COMMIT` (yoki xato bo'lsa `ROLLBACK`) qiladi. Shuning uchun bir klass ichida o'z metodini `this.metod()` deb chaqirsangiz, `@Transactional` ishlamaydi — chaqiruv proxy'dan o'tmaydi. Bu tuzoq 7-bobda real misolda ko'rsatiladi.

Ikkinchi kirish eshigi: Telegram

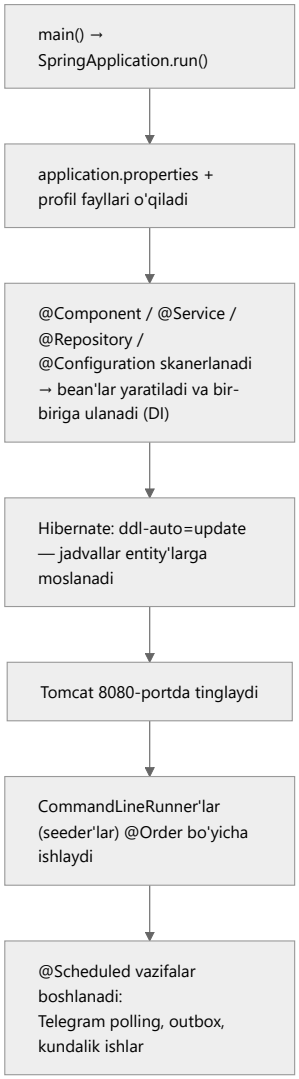
Sayt so'rovlaridan tashqari backend'ning ikkinchi "kirish eshigi" bor — Telegram. Bot uchun webhook ishlatilmaydi, **long polling** ishlatiladi: backend o'zi Telegram'dan "yangi xabar bormi?" deb so'raydi.



2.3-rasm. Telegram oqimi: kiruvchi update'lar va chiquvchi xabarnomalar (11-bobda batafsil).

Bu yerda muhim g'oya — **outbox pattern**: o'qituvchi davomat saqlaganda, ota-onaga xabar **darhol** Telegram'ga yuborilmaydi. Avval bazaga "yuborilishi kerak" degan qator yoziladi, alohida jarayon (`NotificationSender`) uni Telegram limitlariga rioya qilib yuboradi. Natijada Telegram sekinlashsa yoki ishlamay qolsa ham, o'qituvchining "Saqlash" tugmasi kutmaydi va xato bermaydi.

Ilova ishga tushganda nima bo'ladi?



2.4-rasm. Ishga tushish ketma-ketligi.

src/main/java/uz/azizbek/maktabboshqaruv/MaktabBoshqaruvApplication.java 1–14-qatorlar

```
1  package uz.azizbek.maktabboshqaruv;
2
3  import org.springframework.boot.SpringApplication;
4  import org.springframework.boot.autoconfigure.SpringBootApplication;
5  import org.springframework.data.jpa.repository.config.EnableJpaAuditing;
6
7  @SpringBootApplication
8  @EnableJpaAuditing
9  public class MaktabBoshqaruvApplication {
10
11     public static void main(String[] args) {
12         SpringApplication.run(MaktabBoshqaruvApplication.class, args);
13     }
14 }
```

- **7-qator,** `@SpringBootApplication` — uchta annotatsiyaning qisqartmasi: `@Configuration` (bu klass sozlama manbai), `@EnableAutoConfiguration` (classpath'da nima bor bo'lsa, shunga mos sozlash: PostgreSQL drayveri bor → DataSource; spring-web bor → Tomcat), `@ComponentScan` (shu paket va uning ichidagi barcha

paketlardan bean qidirish). Shuning uchun `MaktabBoshqaruvApplication` eng yuqori paketda (`uz.azizbek.maktabboshqaruv`) turadi — pastdagi hamma narsa skanerlanadi.

- **8-qator,** `@EnableJpaAuditing` — `@CreatedDate` va `@LastModifiedDate` bilan belgilangan maydonlarni (masalan, `Position.createdDate`) Spring Data o'zi to'ldiradi.
- **12-qator** — `SpringApplication.run` konteynerni ko'taradi va Tomcat'ni ishga tushiradi.

> 📌 **Esda tut**
>
> - So'rov yo'li: Filtrlar → Interceptor → Controller → Service → Repository → Hibernate → PostgreSQL va qaytish.
> - Har qatlamning bitta vazifasi bor; biznes qoidalari faqat Service'da.
> - `@Transactional` proxy orqali ishlaydi: `this.metod()` chaqiruvi tranzaksiyani chetlab o'tadi.
> - Telegram xabarlari outbox orqali yuboriladi — foydalanuvchi so'rovi Telegram'ni kutmaydi.

3. Papka va paketlar tuzilmasi

Loyiha ildizi

```
maktab-boshqaruv/
├── build.gradle                ← qaysi kutubxonalar kerak, qanday quriladi (4-bob)
├── settings.gradle             ← loyiha nomi
├── gradlew, gradlew.bat        ← Gradle wrapper: Gradle o'rnatmasdan ham quradi
├── .env.example                ← muhit o'zgaruvchilari namunasi (17-bob)
├── src/
│   ├── main/
│   │   ├── java/uz/azizbek/maktabboshqaruv/   ← BACKEND KODI (351 fayl)
│   │   └── resources/
│   │       ├── application.properties          ← asosiy sozlamalar (git'da)
│   │       ├── application-local.properties    ← parollar, secret (git'da EMAS)
│   │       ├── bot/i18n/messages_*.properties  ← bot matnlari: uz, cy, ru
│   │       ├── bot/banners/*.png               ← bot bo'lim bannerlari
│   │       └── fonts/Inter-*.ttf, NotoSerif-*.ttf ← PDF shriftlari (lotin + kirill)
│   └── test/java/...                           ← 55 ta test fayli
├── frontend/                   ← sayt (bu qo'llanma mavzusi emas)
├── docs/                       ← hujjatlar (shu PDF ham shu yerda)
├── scripts/, start.bat, start.sh
└── storage/                    ← yuklangan fayllar (git'da EMAS)
```

Java paketlari

```
uz.azizbek.maktabboshqaruv
├── MaktabBoshqaruvApplication   ← main(), kirish nuqtasi
├── config/        14 fayl   Spring sozlamalari: xavfsizlik, maktab chegarasi, seeder'lar, migratsiyalar
├── controller/    35 fayl   REST endpoint'lar (@RestController) — 195 ta endpoint
├── dto/           74 fayl   API'ning kirish/chiqish shakllari
├── entity/        59 fayl   39 ta JPA entity (jadval) + 20 ta enum
├── repository/    39 fayl   Spring Data JPA interfeyslari
├── service/       45 fayl   biznes-mantiq (asosiy modullar)
│   ├── appeal/     6 fayl   ota-onalar murojaatlari va fayllar
│   ├── library/   15 fayl   kutubxona
│   │   └── wikisource/ 5    Wikisource'dan import va PDF
│   └── parent/     4 fayl   ota-ona ko'radigan ma'lumotlar (bot va Mini App uchun)
├── bot/           17 fayl   Telegram bot karkasi: router, klaviaturalar, matnlar (i18n)
│   └── screens/   15 fayl   botning har bir sahifasi
├── telegram/      14 fayl   Telegram Bot API mijozi, imzo tekshiruvi, sozlamalar
├── event/          4 fayl   domain event'lar va ularni tinglovchi
├── exception/      2 fayl   ForbiddenException, GlobalExceptionHandler
└── util/           2 fayl   JwtUtil, XlsxWriter
```

Paket	Nima uchun alohida	Misol
`config`	ilova qanday **sozlanishi** — biznes emas, "infratuzilma"	`SecurityConfig`, `SchoolScopeConfig`, `DataSeeder`
`controller`	HTTP chegarasi: tashqi dunyo bilan gaplashadigan yagona joy	`StudentController`

Paket	Nima uchun alohida	Misol
`dto`	API shartnomasi: frontend ko'radigan JSON	`StudentRequestDto`, `StudentResponseDto`
`entity`	bazadagi jadvallar va enum'lar	`Student`, `AttendanceStatus`
`repository`	bazaga so'rovlar	`StudentRepository`
`service`	qoidalar va jarayonlar	`GradeService`
`service.appeal`, `.library`, `.parent`	katta modullar o'z kichik paketida — `service` papkasi 100+ faylga to'lib ketmasligi uchun	`AppealService`, `LibraryCatalog`
`bot`, `bot.screens`	Telegram botning ekranlari va navigatsiyasi	`BotRouter`, `GradesScreen`
`telegram`	Telegram API bilan past darajadagi ish (HTTP, imzo)	`HttpTelegramClient`
`event`	"nimadir bo'ldi" degan xabarlar (davomat belgilandi, baho qo'yildi)	`GradeSavedEvent`
`exception`	xatolarni HTTP javobga aylantirish	`GlobalExceptionHandler`
`util`	umumiy yordamchilar	`JwtUtil`

🤔 **Nega "qatlam bo'yicha" (controller/service/...) va "modul bo'yicha" (library/, appeal/) aralash?**
Loyiha boshida hammasi qatlam bo'yicha bo'lingan (`controller`, `service`, ...) — kichik loyihada bu eng sodda yo'l. Kutubxona, murojaatlar va ota-ona ko'rinishlari kattalashgach, ular `service` ichida o'z **modul paketlariga** ajratilgan. Bu — o'sib borayotgan loyihalarda tabiiy holat. Yanada katta loyihada "package by feature" (har modul o'z controller/service/repository'si bilan bitta paketda) qulayroq bo'ladi.

Nomlash qoidalari

Loyihada nomlar izchil, shuning uchun klass nomidan uning vazifasini bilish mumkin:

Qo'shimcha	Ma'nosi	Misol
`...Controller`	REST endpoint'lar	`GradeController`
`...Service`	biznes-mantiq	`GradeService`
`...Repository`	Spring Data interfeysi	`GradeRepository`
`...RequestDto`	klientdan keladigan JSON	`GradeRequestDto`
`...ResponseDto`	klientga qaytadigan JSON	`GradeResponseDto`
`...Seeder`	ishga tushishda ma'lumot qo'shadi	`AdminSeeder`
`...Event`	domain event	`AttendanceMarkedEvent`
`...Screen`	bot sahifasi	`ScheduleScreen`

📌 **Esda tut**

- Barcha kod `uz.azizbek.maktabboshqaruv` ichida — `@SpringBootApplication` shu paketdan boshlab skanerlaydi.
- `config` — sozlash, `controller` — HTTP, `service` — qoidalar, `repository` — baza.
- Katta modullar (`library` , `appeal` , `parent`) `service` ichida alohida paketda.
- Nomlar izchil: qo'shimchadan klassning qatlamini bilasiz.

4. Konfiguratsiya

Ilovaning ikki asosiy sozlama fayli bor: `build.gradle` (qaysi kutubxonalar kerak va loyiha qanday quriladi) va `application.properties` (ilova ishlayotganda qanday sozlamalar bilan ishlaydi).

build.gradle — loyiha "retsepti"

build.gradle plaginlar, Java versiyasi, repozitoriy 1–18-qatorlar

```
1  plugins {
2      id 'java'
3      id 'org.springframework.boot' version '4.1.0'
4      id 'io.spring.dependency-management' version '1.1.7'
5  }
6
7  group = 'uz.azizbek'
8  version = '0.0.1-SNAPSHOT'
9
10 java {
11     toolchain {
12         languageVersion = JavaLanguageVersion.of(21)
13     }
14 }
15
16 repositories {
17     mavenCentral()
18 }
```

- **2-qator,** `java` — Gradle'ga bu Java loyihasi ekanini aytadi: `compileJava`, `test`, `jar` vazifalari paydo bo'ladi.
- **3-qator,** `org.springframework.boot` **4.1.0** — Spring Boot plagini. U `bootRun` (ilovani ishga tushirish) va `bootJar` (barcha kutubxonalar ichida bo'lgan bitta "semiz" jar) vazifalarini qo'shadi. Jonli bot aynan shu jar'dan ishlaydi.
- **4-qator,** `io.spring.dependency-management` — **BOM** (Bill of Materials). Shu tufayli 21–22-qatorlarda versiya yozilmagan: Spring Boot 4.1.0 bilan bir-biriga mos versiyalarni plagin o'zi tanlaydi. Versiyasi qo'lda yozilganlari (`jjwt 0.12.5`, `openpdf 2.0.3`, `pdfbox 3.0.5`) — BOM'da yo'q kutubxonalar.
- **12-qator,** `JavaLanguageVersion.of(21)` — **toolchain**: kompyuterda boshqa Java o'rnatilgan bo'lsa ham, Gradle 21-versiyani topadi (yoki yuklab oladi). Shuning uchun loyiha har bir kompyuterda bir xil Java bilan quriladi.
- **17-qator,** `mavenCentral()` — kutubxonalar shu ochiq ombordan yuklanadi.

build.gradle dependencies 20–41-qatorlar

```
20 dependencies {
21     implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
22     implementation 'org.springframework.boot:spring-boot-starter-webmvc'
23     implementation 'io.jsonwebtoken:jjwt-api:0.12.5'
24     runtimeOnly 'io.jsonwebtoken:jjwt-impl:0.12.5'
25     runtimeOnly 'io.jsonwebtoken:jjwt-jackson:0.12.5'
26     compileOnly 'org.projectlombok:lombok'
27     runtimeOnly 'org.postgresql:postgresql'
28     annotationProcessor 'org.projectlombok:lombok'
29     testImplementation 'org.springframework.boot:spring-boot-starter-test'
30     testCompileOnly 'org.projectlombok:lombok'
```

```
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
    testAnnotationProcessor 'org.projectlombok:lombok'
    implementation 'org.springframework.boot:spring-boot-starter-security'
    implementation 'org.springframework.boot:spring-boot-starter-validation'
    // short-lived in-memory caches (account status, bot lookups)
    implementation 'com.github.ben-manes.caffeine:caffeine'
    // timetable PDF (A4 landscape, embedded Inter font for Latin + Cyrillic)
    implementation 'com.github.librepdf:openpdf:2.0.3'
    // library: page count and the cover drawn from a PDF's first page (once, at upload)
    implementation 'org.apache.pdfbox:pdfbox:3.0.5'
}
```

Har bir dependency nima uchun kerak:

Qator	Dependency	Konfiguratsiya	Nima uchun
21	`spring-boot-starter-data-jpa`	implementation	JPA + Hibernate + Spring Data repository'lar + HikariCP. Barcha `entity` va `repository` paketlari shunga tayanadi
22	`spring-boot-starter-webmvc`	implementation	Spring MVC + ichki Tomcat + Jackson. `@RestController`, `@GetMapping` shu yerdan
23	`jjwt-api`	implementation	JWT bilan ishlash interfeyslari (`Jwts.builder()`) — kod faqat API'ni ko'radi
24–25	`jjwt-impl` , `jjwt-jackson`	runtimeOnly	JJWT'ning ichki bajaruvchisi va JSON qismi — kompilyatsiyada kerak emas, faqat ishlash paytida
26, 28, 30, 32	`lombok`	compileOnly / annotationProcessor	`@Getter` kabi kod generatsiyasi. **Diqqat:** loyihada bir marta ham ishlatilmagan
27	`postgresql`	runtimeOnly	PostgreSQL JDBC drayveri — kod uni to'g'ridan-to'g'ri chaqirmaydi, Hibernate ishlatadi
29	`spring-boot-starter-test`	testImplementation	JUnit 5, Mockito, AssertJ, Spring Test
31	`junit-platform-launcher`	testRuntimeOnly	Gradle testlarni JUnit 5 orqali ishga tushirishi uchun
33	`spring-boot-starter-security`	implementation	Spring Security: filtrlar, `@PreAuthorize`, `BCryptPasswordEncoder`
34	`spring-boot-starter-validation`	implementation	Hibernate Validator: `@NotBlank` , `@Size` , `@Past` va `@Valid`
36	`caffeine`	implementation	xotiradagi tez kesh (hisob holati, maktab egaligi, bot keshlari)
38	`openpdf`	implementation	PDF **yaratish**: dars jadvali, ruxsat xati, Wikisource kitoblari
40	`pdfbox`	implementation	PDF **o'qish**: bet soni, birinchi betdan muqova rasmi

🤔 **implementation, runtimeOnly, compileOnly — farqi nima?**

- **implementation** — kod shu kutubxonani import qiladi; u kompilyatsiyada ham, ishlashda ham kerak.
- **runtimeOnly** — kod uni import qilmaydi, faqat ishlayotganda kerak (masalan, PostgreSQL drayverini Hibernate o'zi topadi). Kod tasodifan unga bog'lanib qolmaydi.
- **compileOnly / annotationProcessor** — faqat kompilyatsiya paytida (Lombok kod yozadi va chiqib ketadi), jar'ga kirmaydi.
- **test...** — faqat testlarda.

build.gradle test vazifalari 43–76-qatorlar

```
43  tasks.named('test') {
44      useJUnitPlatform {
45          excludeTags 'load', 'wikisource'
46      }
47  }
48
49  // the real Wikisource (internet): what is found and imported for five classics — `gradlew wikisourceCh
    eck`
50  tasks.register('wikisourceCheck', Test) {
51      description = 'Wikisource import against the real wikisource.org (needs internet and DB_URL of a te
    st database)'
52      group = 'verification'
53      testClassesDirs = sourceSets.test.output.classesDirs
54      classpath = sourceSets.test.runtimeClasspath
55      useJUnitPlatform {
56          includeTags 'wikisource'
57      }
58      testLogging {
59          showStandardStreams = true
60      }
61  }
62
63  // 10.3: 3000 simulated parents against the bot (Telegram mocked) — `gradlew loadTest`
64  tasks.register('loadTest', Test) {
65      description = 'Bot load test: 3000 simulated parents, mock Telegram'
66      group = 'verification'
67      testClassesDirs = sourceSets.test.output.classesDirs
68      classpath = sourceSets.test.runtimeClasspath
69      useJUnitPlatform {
70          includeTags 'load'
71      }
72      maxHeapSize = '1g'
73      testLogging {
74          showStandardStreams = true
75      }
76  }
```

- **43–47** — oddiy `gradlew test`: `load` va `wikisource` tegli testlar **chiqarib tashlanadi** (`excludeTags`). Biri 3000 ta ota-onani simulyatsiya qiladi (sekin), ikkinchisi internetdagi Wikisource'ga boradi — har build'da kerak emas.
- **50–61,** `wikisourceCheck` va **64–76,** `loadTest` — o'sha tegli testlarni alohida ishga tushiradigan maxsus vazifalar.

Kundalik buyruqlar:

Buyruq	Nima qiladi
`gradlew.bat bootRun`	ilovani ishga tushiradi
`gradlew.bat test`	testlarni ishga tushiradi
`gradlew.bat build`	kompilyatsiya + testlar + jar
`gradlew.bat bootJar`	faqat ishga tushiriladigan jar: `build/libs/maktab-boshqaruv-0.0.1-SNAPSHOT.jar`

application.properties — ilova sozlamalari

Spring Boot ishga tushganda `src/main/resources/application.properties` ni o'qiydi. `${NOM:standart}` yozuvi "muhit o'zgaruvchisi `NOM` bo'lsa uni, bo'lmasa standart qiymatni ol" degani. `${NOM}` (standartsiz) —

o'zgaruvchi **majburiy**: topilmasa ilova ishga tushmaydi.

src/main/resources/application.properties baza, JPA, JWT, admin, CORS 1–14-qatorlar

```
spring.datasource.url=${DB_URL:jdbc:postgresql://localhost:5432/maktab_db}
spring.datasource.username=${DB_USERNAME:postgres}
spring.datasource.password=${DB_PASSWORD}

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.open-in-view=false

jwt.secret=${JWT_SECRET}

admin.seed.username=${ADMIN_USERNAME:admin}
admin.seed.password=${ADMIN_PASSWORD:admin123}

cors.allowed-origins=${CORS_ALLOWED_ORIGINS:http://localhost:9000}
```

Kalit	Qiymat	Ma'nosi
`spring.datasource.url`	`DB_URL`, standart `localhost:5432/maktab_db`	qaysi bazaga ulanish (JDBC URL)
`spring.datasource.username`	`DB_USERNAME`, standart `postgres`	baza foydalanuvchisi
`spring.datasource.password`	`DB_PASSWORD` — **majburiy**	baza paroli; faylga yozilmagan ✓
`spring.jpa.hibernate.ddl-auto`	`update`	Hibernate ishga tushishda jadvallarni entity'larga moslaydi: yangi jadval va ustunlarni **qo'shadi**, hech narsani o'chirmaydi
`spring.jpa.show-sql`	`true`	har bir SQL so'rovni konsolga chiqaradi — o'rganish uchun qulay, prod'da zararli
`spring.jpa.open-in-view`	`false`	Hibernate sessiyasi HTTP so'rov oxirigacha ochiq turmaydi — LAZY ma'lumotlar faqat tranzaksiya ichida o'qiladi
`jwt.secret`	`JWT_SECRET` — **majburiy**	JWT imzo kaliti (Base64); faylga yozilmagan ✓
`admin.seed.username/password`	`ADMIN_USERNAME` / `ADMIN_PASSWORD`, standart `admin` / `admin123`	baza bo'sh bo'lsa yaratiladigan birinchi admin. Standart parol — xavfsizlik zaifligi
`cors.allowed-origins`	`CORS_ALLOWED_ORIGINS`, standart `http://localhost:9000`	qaysi sayt manzillari brauzerdan API'ga murojaat qila oladi

🔬 **open-in-view=false nima beradi?**

Standart holatda (`true`) Spring Hibernate sessiyasini butun HTTP so'rov davomida ochiq ushlab turadi: controller JSON yasayotganda ham LAZY maydon o'qilsa, Hibernate jimgina yangi SQL yuboradi. Bu qulay, lekin yashirin so'rovlar (N+1) va uzoq band turgan ulanishlar keltirib chiqaradi. `false` qilinganda LAZY ma'lumotni faqat `@Transactional` ichida o'qish mumkin, aks holda `LazyInitializationException`. Shuning uchun loyihada o'qish metodlari `@Transactional(readOnly = true)` bilan belgilangan (7-bobda misol bor).

src/main/resources/application.properties Telegram, fayllar, kutubxona 16–49-qatorlar

```
# Telegram bot (ota-onalar uchun xabarnomalar). Token HECH QACHON shu faylga yozilmaydi:
# u faqat application-local.properties (telegram.bot-token=...) yoki TELEGRAM_BOT_TOKEN
# muhit o'zgaruvchisidan olinadi. Batafsil: docs/15-telegram-bot.md
```

```
telegram.enabled=${TELEGRAM_ENABLED:false}
telegram.mock=${TELEGRAM_MOCK:false}
telegram.bot-token=${TELEGRAM_BOT_TOKEN:}
telegram.bot-username=${TELEGRAM_BOT_USERNAME:}
telegram.quiet-hours-start=${TELEGRAM_QUIET_START:22:00}
telegram.quiet-hours-end=${TELEGRAM_QUIET_END:07:00}
# Mini App ("Farzandim kundaligi") uchun HTTPS manzil. Bo'sh bo'lsa — tugma ko'rsatilmaydi.
telegram.webapp-url=${TELEGRAM_WEBAPP_URL:}
# getUpdates long-poll so'rovi oqimni 25 soniyagacha band qiladi — shuning uchun bittadan ortiq scheduler oqimi kerak
spring.task.scheduling.pool.size=3
# Ota-onalar murojaatlaridagi fayllar (rasm, video, ovozli xabar, hujjat) shu papkaga saqlanadi.
# Papka gitga tushmaydi (.gitignore). Prod da doimiy disk yo'li bering: APP_STORAGE_PATH=/var/lib/maktab/storage
app.storage.path=${APP_STORAGE_PATH:./storage}
# Murojaatga javob / ota-onalarga xabar fayllari: bitta fayl 20 MB gacha (Telegram chegarasi), bir so'rovda 10 tagacha.
# Bu chegaralar kodda tekshiriladi; multipart chegarasi kutubxona PDF lari (50 MB) uchun kattaroq.
spring.servlet.multipart.max-file-size=60MB
spring.servlet.multipart.max-request-size=210MB
# Bot update'larini qayta ishlovchi oqimlar (8–16) va navbat chegarasi
telegram.update-threads=${TELEGRAM_UPDATE_THREADS:8}
telegram.update-max-threads=${TELEGRAM_UPDATE_MAX_THREADS:16}
telegram.update-queue=${TELEGRAM_UPDATE_QUEUE:2000}
# Maktab kutubxonasi ("📚 Kitob mutolaasi"). Kitoblarni faqat maktab xodimlari sayt orqali yuklaydi.
app.library.max-pdf-mb=${LIBRARY_MAX_PDF_MB:50}
app.library.max-cover-mb=${LIBRARY_MAX_COVER_MB:2}
# bitta ota-ona bir kunda nechta kitob yuklab olishi mumkin
app.library.daily-download-limit=${LIBRARY_DAILY_LIMIT:20}
# botdagi qidiruv rejimi shuncha daqiqa harakatsizlikdan keyin yopiladi
app.library.search-idle-minutes=10
# haftalik "Dam olish kunlari uchun 3 ta kitob" xabari (Asia/Tashkent vaqti)
app.library.weekly-day=FRIDAY
app.library.weekly-time=19:00
```

Kalit	Standart	Ma'nosi
`telegram.enabled`	`false`	bot yoqilganmi
`telegram.mock`	`false`	mock rejim: hech narsa Telegram'ga ketmaydi (test va demo uchun)
`telegram.bot-token`	bo'sh	BotFather bergan token — **faqat** `application-local.properties` yoki muhit o'zgaruvchisida
`telegram.bot-username`	bo'sh	bot nomi (deep-link havolalar uchun)
`telegram.quiet-hours-start/end`	`22:00` / `07:00`	tinch soatlar: bu vaqtda xabarlar ushlab turiladi
`telegram.webapp-url`	bo'sh	Mini App manzili (HTTPS)
`spring.task.scheduling.pool.size`	`3`	`@Scheduled` vazifalar uchun oqimlar: long-poll bitta oqimni 25 s band qiladi, qolganlari bloklanmasligi uchun
`app.storage.path`	`./storage`	yuklangan fayllar papkasi
`spring.servlet.multipart.max-file-size`	`60MB`	bitta yuklanadigan fayl chegarasi
`spring.servlet.multipart.max-request-size`	`210MB`	bitta so'rovdagi barcha fayllar
`telegram.update-threads / max-threads / queue`	`8 / 16 / 2000`	bot update'larini qayta ishlovchi oqimlar hovuzi

Kalit	Standart	Ma'nosi
`app.library.*`	`50 MB`, `2 MB`, `20`, `10`, `FRIDAY 19:00`	kutubxona chegaralari va haftalik tavsiya vaqti

Fayldan tashqari, kodda `@Value` bilan o'qiladigan va standart qiymati kodda yozilgan yana bir nechta kalit bor:

Kalit	Standart	Qayerda
`telegram.send-interval-ms`	`250`	`NotificationSender` — outbox qancha tez-tez tekshiriladi
`telegram.send-initial-delay-ms`	`5000`	ishga tushgandan keyin birinchi tekshiruv
`telegram.jobs-interval-ms` / `jobs-initial-delay-ms`	`300000` / `60000`	`BotScheduledJobs` (digest'lar)
`app.library.permission-check-cron`	`0 15 6 * * *`	ruxsatnomalar muddatini kundalik tekshirish (06:15)
`app.fake-now`	bo'sh	demo uchun "soxta" vaqt (`TelegramConfig.clock`)
`demo.parent-chat-id`	`990000001`	faqat `dev` profilidagi demo ota-ona

Profillar: local va dev

Profil — bir xil kodni turli sozlamalar bilan ishga tushirish usuli. `SPRING_PROFILES_ACTIVE=local` berilsa, Spring `application.properties` ustiga ** `application-local.properties` **ni ham o'qiydi (ikkinchisi ustun turadi).

Profil	Nima qiladi
`local`	`application-local.properties` yuklanadi: baza paroli, JWT secret, bot tokeni. Bu fayl `.gitignore` da — **git'ga hech qachon tushmaydi**
`dev`	`@Profile("dev")` bilan belgilangan bean'lar yoqiladi: `DemoParentSeeder`, `LibraryDemoSeeder` (demo ota-ona va namunaviy kitoblar). Prod'da ishlamaydi

Repozitoriyda faqat namunasi bor (`.example`), unda haqiqiy qiymat yo'q:

src/main/resources/application-local.properties.example 1–21-qatorlar

```
# Lokal ishlab chiqish sozlamalari shabloni (profil: local).
# 1) Shu faylni shu papkaga "application-local.properties" nomi bilan nusxalang
#    (u .gitignore'da - hech qachon commit qilinmaydi).
# 2) Qiymatlarni o'zingiznikiga almashtiring.
# 3) Backend'ni "local" profil bilan ishga tushiring: start.sh / start.bat yoki
#    IntelliJ'dagi "MaktabBoshqaruvApplication (local)" konfiguratsiyasi.
#
# JWT secret - kamida 32 baytlik tasodifiy qiymatning Base64 ko'rinishi. Generatsiya:
#   openssl rand -base64 64
#   PowerShell: [Convert]::ToBase64String((1..64 | ForEach-Object { Get-Random -Maximum 256 }) -as [byte[]])

spring.datasource.password=o'zgartiring
jwt.secret=o'zgartiring

# Telegram bot (ixtiyoriy). @BotFather'dan olingan tokenni shu yerga yozing - bu fayl
# commit qilinmaydi. Batafsil yo'riqnoma: docs/15-telegram-bot.md
# telegram.enabled=true
# telegram.bot-token=<BotFather bergan token>
# telegram.bot-username=<bot_username, @ belgisisiz>
# Tokensiz sinash uchun: xabarlar faqat logga va bazaga yoziladi
# telegram.mock=true
```

> ⚠️ **Maxfiy qiymatlar qayerda turadi**
>
> Parol, JWT secret va bot tokeni **hech qachon** `application.properties` ga yozilmaydi — u git'da turadi va GitHub'ga ketadi. Ular faqat `application-local.properties` da (git'da emas) yoki muhit o'zgaruvchilarida bo'ladi. Shu qo'llanmada ham ular `<SECRET>` deb belgilangan.

@ConfigurationProperties: sozlamalarni klassga yig'ish

`@Value("${...}")` bitta qiymatni o'qiydi. Ko'p bog'liq sozlamalar uchun Spring'ning ancha qulay usuli — `@ConfigurationProperties`: `telegram.*` bilan boshlanadigan barcha kalitlar bitta klass maydonlariga avtomatik joylanadi.

src/main/java/uz/azizbek/maktabboshqaruv/telegram/TelegramProperties.java — 12–32-qatorlar

```
12 @ConfigurationProperties(prefix = "telegram")
13 public class TelegramProperties {
14
15     public enum Mode { DISABLED, NO_TOKEN, MOCK, LIVE }
16
17     private boolean enabled = false;
18     private boolean mock = false;
19     private String botToken = "";
20     private String botUsername = "";
21     private String apiBaseUrl = "https://api.telegram.org";
22     private int pollTimeoutSeconds = 25;
23     private int maxPerSecond = 25;
24     private int maxAttempts = 3;
25     private LocalTime quietHoursStart = LocalTime.of(22, 0);
26     private LocalTime quietHoursEnd = LocalTime.of(7, 0);
27     /** HTTPS address of the Mini App ("Farzandim kundaligi"); empty = no Mini App button. */
28     private String webappUrl = "";
29     /** How old a Mini App initData may be before it is rejected. */
30     private long initDataMaxAgeSeconds = 86400;
31     /** Per-chat request budget (updates per second) before the bot asks the parent to slow down. */
32     private int chatRatePerSecond = 3;
```

- **12-qator** — `prefix = "telegram"`: `telegram.bot-token` → `botToken` maydoni, `telegram.max-per-second` → `maxPerSecond` (Spring "relaxed binding" bilan `kebab-case` ni `camelCase` ga moslaydi).
- Maydonlardagi `= 25`, `= 86400` — **standart qiymatlar**: properties faylida kalit bo'lmasa shular ishlaydi.
- Bu klass `TelegramConfig` dagi `@EnableConfigurationProperties(TelegramProperties.class)` orqali bean bo'ladi va istalgan servisga `@Autowired` qilinadi.

> 🤔 **Nega @Value emas?**
>
> `@Value` bilan 12 ta Telegram sozlamasi 12 ta joyga tarqalib ketardi va har birida kalit nomi matn sifatida takrorlanardi (xato yozish oson). `@ConfigurationProperties` bilan hammasi **bitta tipli klassda**: IDE avtomatik to'ldiradi, standartlar bir joyda, testda oddiy `new TelegramProperties()` bilan sozlash mumkin.

📌 **Esda tut**

- `build.gradle`: Spring Boot 4.1 plagini + BOM, Java 21 toolchain, 13 ta dependency.
- `application.properties` dagi maxfiy qiymatlar faqat `${DB_PASSWORD}`, `${JWT_SECRET}` kabi o'zgaruvchilar orqali keladi.
- `ddl-auto=update` jadvallarni o'zi yaratadi/to'ldiradi; `open-in-view=false` LAZY o'qishni tranzaksiya ichiga majbur qiladi.
- `local` profil — maxfiy sozlamalar fayli, `dev` profil — demo ma'lumotlar.
- Ko'p bog'liq sozlamalar uchun `@ConfigurationProperties` (`TelegramProperties`).

5. Ma'lumotlar bazasi

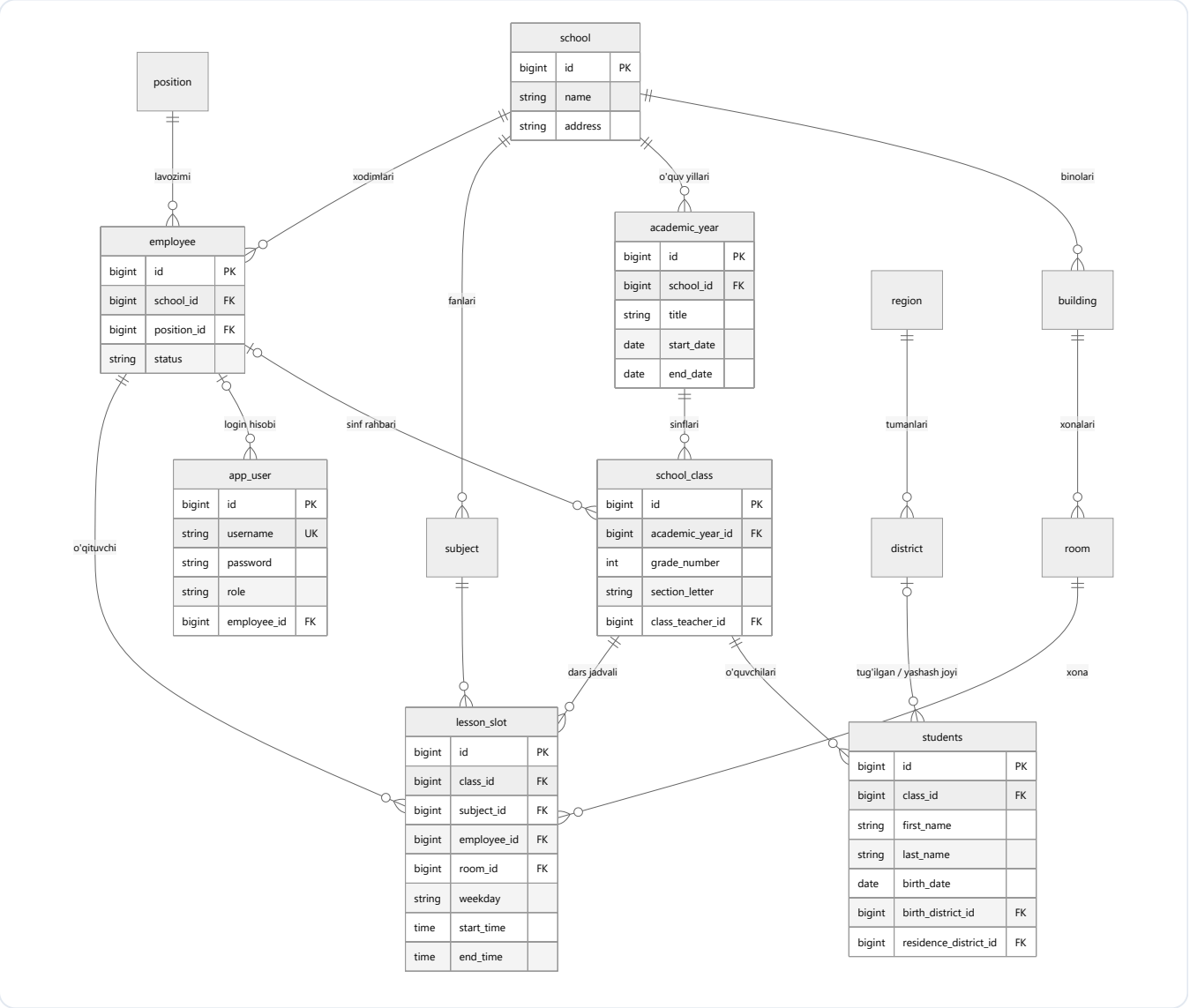
Umumiy rasm

Baza — **PostgreSQL**, nomi `maktab_db`. Unda **39 ta jadval** bor va har biri bitta **JPA entity** klassiga mos keladi. Jadvallar SQL skript bilan emas, **Hibernate** tomonidan yaratiladi: `spring.jpa.hibernate.ddl-auto=update` sozlamasi ilova ishga tushganda entity'larni bazaga solishtiradi va yetishmayotgan jadval va ustunlarni qo'shadi.

> 🧩 **Entity — jadvalning "pasporti"**
> Entity klassi — jadvalning Java'dagi tavsifi. Klass nomi → jadval nomi, maydon → ustun, maydon turi → ustun turi, `@ManyToOne` → boshqa jadvalga tashqi kalit (foreign key). Hibernate bu "pasport"ga qarab jadval quradi, qatorlarni ob'ektga va ob'ektlarni qatorga aylantiradi.

Jadvallarni 4 ta soha bo'yicha ko'ramiz. Diagrammalarda faqat asosiy ustunlar ko'rsatilgan; har bir jadvalning **barcha** maydonlari C-ilovada.

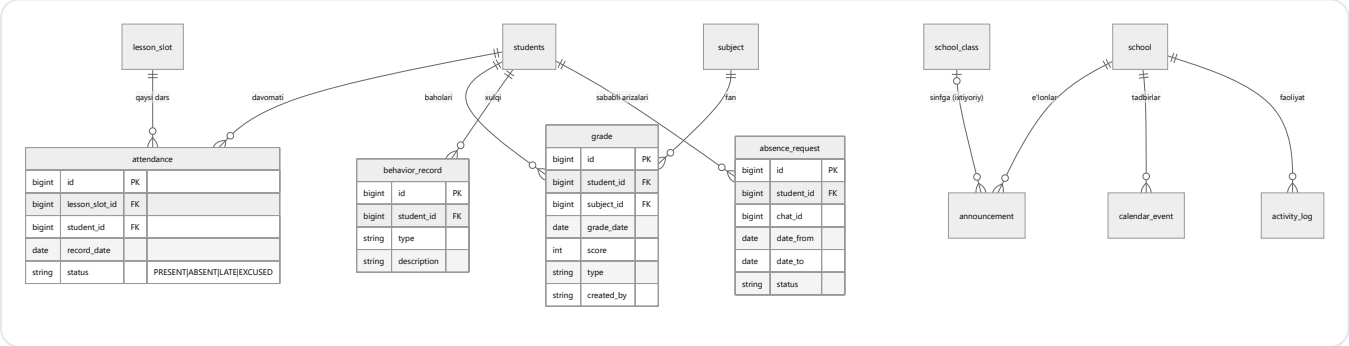
1) Maktab tuzilmasi va odamlar



5.1-rasm. Maktab tuzilmasi: maktab → o'quv yili → sinf → o'quvchi; xodimlar, foydalanuvchilar, dars jadvali, hududlar.

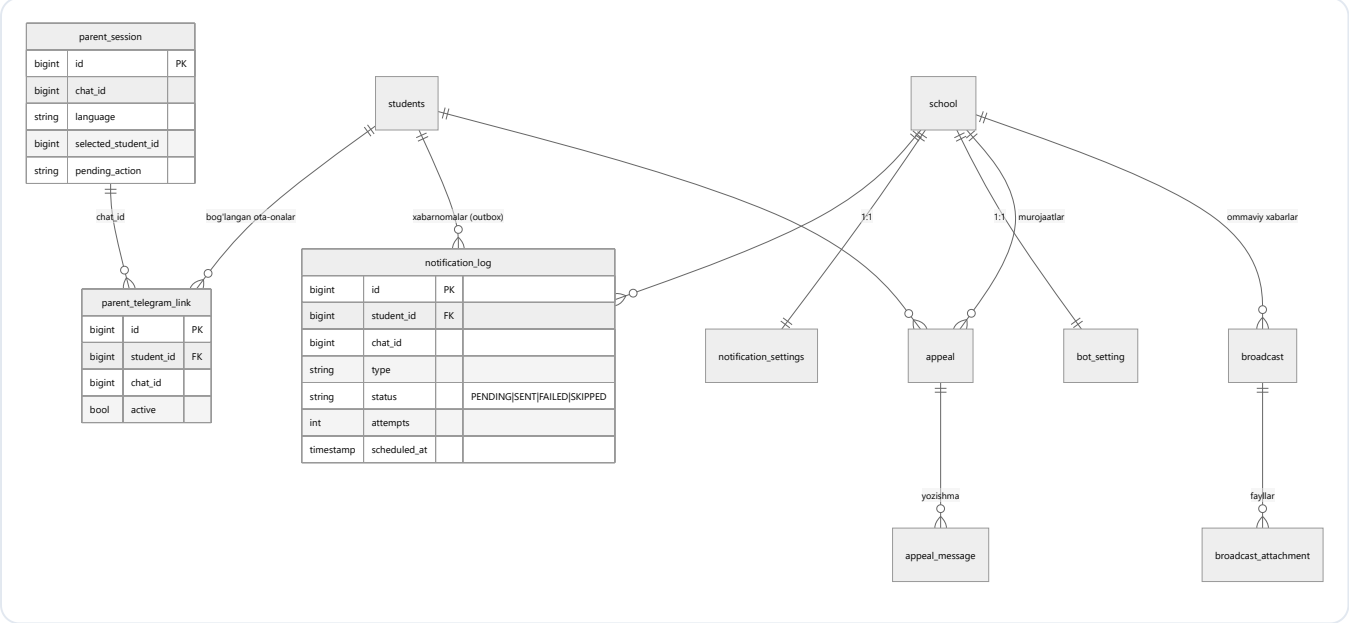
Diqqat qiling: **sinf to'g'ridan-to'g'ri maktabga emas, o'quv yiliga bog'langan** (`school_class.academic_year_id`). Har yangi o'quv yilida yangi sinflar ochiladi, eski yilning sinflari, baholari va davomati tarix sifatida saqlanib qoladi. Shu sababli "o'quvchi qaysi maktabda?" degan savol 3 qadamda topiladi: `students → school_class → academic_year → school`. `SchoolOwnership` klassidagi SQL'lar aynan shunday yozilgan (14-bob).

2) O'quv jarayoni



5.2-rasm. Davomat, baholar, xulq, arizalar, e'lonlar va tadbirlar.

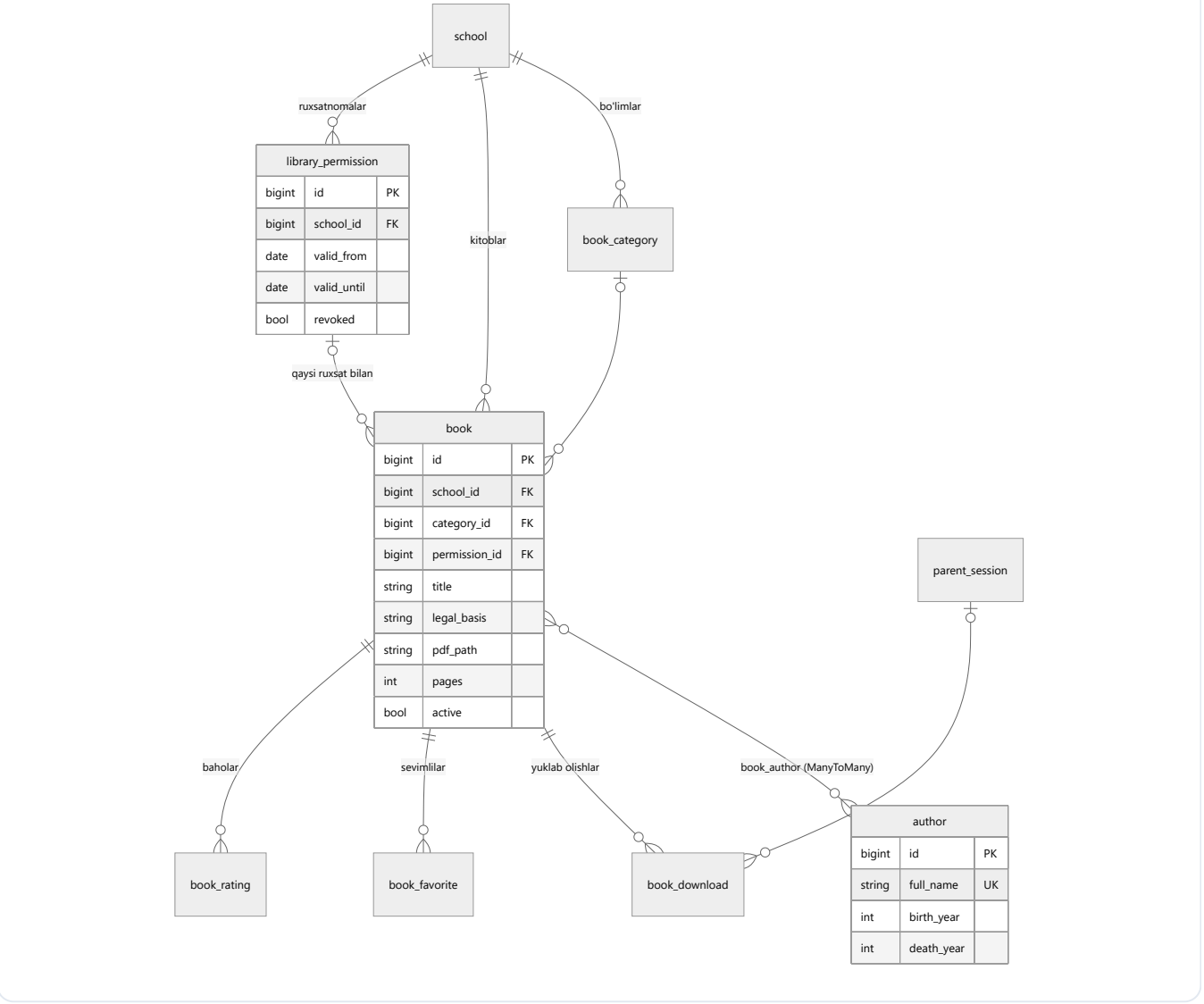
3) Telegram va ota-onalar



5.3-rasm. Telegram qismi: sessiyalar, bog'lanishlar, outbox, murojaatlar va ommaviy xabarlar.

`parent_session` va `parent_telegram_link` orasida JPA bog'lanishi (`@ManyToOne`) yo'q — ular `chat_id` raqami orqali "mantiqan" bog'langan. Telegram chat'i ota-onaning o'zi; u bir nechta farzandga bog'lanishi mumkin, farzandga esa bir nechta ota-ona (otasi, onasi) bog'lanishi mumkin.

4) Kutubxona



5.4-rasm. Kutubxona: kitoblar, bo'limlar, mualliflar, ruxsatnomalar va mutolaa statistikasi.

Entity anatomiyasi: Attendance misolida

src/main/java/uz/azizbek/maktabboshqaruv/entity/Attendance.java 10–38-qatorlar

```
10  @Entity
11  @Table(name = "attendance", uniqueConstraints = @UniqueConstraint(columnNames = {"lesson_slot_id", "stu
    dent_id", "record_date"}))
12  @EntityListeners(AuditingEntityListener.class)
13  public class Attendance {
14
15      @Id
16      @GeneratedValue(strategy = GenerationType.IDENTITY)
17      private Long id;
18
19      @ManyToOne
20      @JoinColumn(name = "lesson_slot_id", nullable = false)
21      private LessonSlot lessonSlot;
22
23      @ManyToOne
24      @JoinColumn(name = "student_id", nullable = false)
```

```
25      private Student student;
26
27      @Column(name = "record_date", nullable = false)
28      private LocalDate recordDate;
29
30      @Enumerated(EnumType.STRING)
31      @Column(nullable = false)
32      private AttendanceStatus status;
33
34      private String comment;
35
36      @CreatedDate
37      @Column(updatable = false)
38      private LocalDateTime createdDate;
```

Har bir annotatsiya nima qiladi:

Qator	Annotatsiya	Ma'nosi
10	`@Entity`	"bu klass jadval" — Hibernate uni boshqaradi
11	`@Table(name = "attendance", uniqueConstraints = ...)`	jadval nomi va **UNIQUE cheklov**: bitta o'quvchi bitta darsda bitta kunda faqat **bitta** davomat yozuviga ega bo'la oladi. Ikki o'qituvchi bir vaqtda "Saqlash"ni bossa ham, baza ikkinchi dublikatni qabul qilmaydi
12	`@EntityListeners(AuditingEntityListener.class)`	saqlashdan oldin Spring Data `@CreatedDate` maydonini to'ldiradi
15	`@Id`	birlamchi kalit (primary key)
16	`@GeneratedValue(strategy = IDENTITY)`	id'ni **baza** beradi (PostgreSQL `IDENTITY` / `serial` ustuni); Java `null` yuboradi, baza raqam qaytaradi
19	`@ManyToOne`	**ko'pdan-birga** bog'lanish: ko'p davomat yozuvi → bitta darsga. Bazada bu `lesson_slot_id` tashqi kaliti
20	`@JoinColumn(name = "lesson_slot_id", nullable = false)`	tashqi kalit ustunining nomi va u bo'sh bo'la olmasligi
30	`@Enumerated(EnumType.STRING)`	enum bazada **matn** sifatida saqlanadi: `'ABSENT'`, `0` emas
36	`@CreatedDate` + `@Column(updatable = false)`	yaratilgan vaqt; keyingi `UPDATE` larda o'zgarmaydi

🤔 **Nega EnumType.STRING, ORDINAL emas?**

`ORDINAL` enum'ni tartib raqami (0, 1, 2...) bilan saqlaydi. Agar kimdir enum o'rtasiga yangi qiymat qo'shsa (masalan, `PRESENT, SICK, ABSENT`), bazadagi barcha eski `1` lar endi boshqa ma'noni bildiradi — ma'lumot jimgina buziladi. `STRING` bilan `'ABSENT'` doim `'ABSENT'`. Loyihadagi barcha enum ustunlari `STRING`.

🔬 **EAGER va LAZY**

`@ManyToOne` standart holatda **EAGER**: `Attendance` o'qilganda Hibernate darhol `LessonSlot` va `Student` ni ham yuklaydi (JOIN yoki qo'shimcha SELECT bilan). Bu kichik ob'ektlar uchun qulay. Yangi yozilgan entity'larda (`Book`, `Appeal`, `District`, `Student.birthDistrict`) `fetch = FetchType.LAZY` ishlatilgan: bog'langan ob'ekt faqat haqiqatan murojaat qilinganda (masalan, `book.getCategory().getName()`) yuklanadi. LAZY maydonni tranzaksiyadan tashqarida o'qish `LazyInitializationException` beradi — 4-bobdagi `open-in-view=false` sababli.

Bog'lanish turlari loyihada

Tur	Loyihada qayerda	Izoh
`@ManyToOne`	50 ta maydon, 30 ta entity'da	asosiy bog'lanish turi
`@OneToOne`	`BotSetting.school`, `NotificationSettings.school`	har maktabda bitta sozlama qatori (`unique = true`)
`@ManyToMany`	`Book.authors` ↔ `Author`	oraliq jadval `book_author` , tartib `@OrderColumn` bilan
`@OneToMany`	**yo'q**	quyida — nega

src/main/java/uz/azizbek/maktabboshqaruv/entity/Book.java 45–56-qatorlar

```
45      /** the authors in the reference list, in order — the latest death year decides "Jamoat mulki" */
46      @ManyToMany(fetch = FetchType.LAZY)
47      @JoinTable(name = "book_author", joinColumns = @JoinColumn(name = "book_id"),
48              inverseJoinColumns = @JoinColumn(name = "author_id"))
49      @OrderColumn(name = "position")
50      @org.hibernate.annotations.BatchSize(size = 100) // a page of books loads its authors in one query
51      private java.util.List<Author> authors = new java.util.ArrayList<>();
52
53      /** MUALLIF_RUXSATI / LITSENZIYA: the permission in force the book is handed out under */
54      @ManyToOne(fetch = FetchType.LAZY)
55      @JoinColumn(name = "permission_id")
56      private LibraryPermission permission;
```

- **46** — `@ManyToMany(fetch = LAZY)` : bitta kitobning bir nechta muallifi, bitta muallifning bir nechta kitobi bo'lishi mumkin.
- **47–48** — `@JoinTable` : bazada oraliq jadval `book_author(book_id, author_id)` .
- **49** — `@OrderColumn(name = "position")` : mualliflar tartibi saqlanadi (birinchi muallif — birinchi).
- **50** — `@BatchSize(size = 100)` : sahifadagi 20 ta kitobning mualliflari 20 ta alohida so'rov bilan emas, **bitta** `WHERE book_id IN (...)` so'rovi bilan yuklanadi — N+1 muammosining yechimi.
- **54** — oddiy `@ManyToOne(fetch = LAZY)` ruxsatnomaga.

> 🤔 **Nega @OneToMany yo'q?**
> Masalan, `School` klassida `List<Student> students` maydoni yo'q. Teskari yo'nalish (`@OneToMany`) qulay ko'rinadi, lekin:
>
> 1. `school.getStudents()` tasodifan 1000 ta o'quvchini xotiraga yuklab yuborishi mumkin;
> 2. JSON'ga aylantirishda cheksiz halqa (`school → students → school → ...`) xavfi bor;
> 3. `cascade` sozlamalari noto'g'ri bo'lsa, ota ob'ektni o'chirish bolalarini ham o'chirib yuboradi.
>
> Loyiha buning o'rniga **bir tomonlama** `@ManyToOne` va kerak bo'lganda **repository so'rovi** ishlatadi: "sinf o'quvchilari" → `studentRepository.findBySchoolClassId(...)` . Bu aniqroq va boshqariladigan yondashuv.

ddl-auto=update: qulaylik va chegaralar

Hibernate `update` rejimida:

- ✅ yangi entity uchun **jadval yaratadi**;

- ✅ yangi maydon uchun **ustun qo'shadi** (`ALTER TABLE ... ADD COLUMN`);
- ✅ yangi `@UniqueConstraint` va `@Index` yaratadi;
- ❌ ustunni **o'chirmaydi** va **nomini o'zgartirmaydi**;
- ❌ mavjud ustun turini ishonchli **o'zgartira olmaydi**;
- ❌ jadvalda qatorlar bo'lsa, yangi ustunga `NOT NULL` qo'ya olmaydi.

Shuning uchun loyihada bir nechta "qo'lda migratsiya" klasslari bor — `config` paketidagi `CommandLineRunner`lar, ular ishga tushishda bir marta ishlaydi:

Klass	Muammo	Yechim
`TelegramLinkCodeBackfill`	`telegram_link_code` ustuni eski qatorlarga `NOT NULL` bo'lib qo'shila olmaydi	ustun nullable qo'shiladi, eski o'quvchilarga kod ishga tushishda beriladi
`TelegramSchemaMigration`	Hibernate enum ustunlari uchun `CHECK (type in (...))` yaratadi, lekin yangi enum qiymati qo'shilganda uni kengaytirmaydi	eskirgan `CHECK` cheklovini SQL bilan yangilaydi
`AppealMigration`	eski `parent_message` jadvalidan yangi `appeal` ga ko'chirish	bir martalik ko'chirish
`LibrarySearchIndex`	trigram indeksni JPA annotatsiyasi bilan yaratib bo'lmaydi	`CREATE INDEX ... USING gin` (pg_trgm bo'lsa)

> ⚠️ **Prod uchun tavsiya**
>
> `ddl-auto=update` o'rganish va tez rivojlantirish uchun ajoyib, lekin prod bazada nazoratsiz o'zgarish xavfli: entity'da xato qilinsa, Hibernate uni jimgina jadvalga "qo'llaydi". Katta loyihalarda **Flyway** yoki **Liquibase** migratsiyalari ishlatiladi: har bir o'zgarish versiyalangan SQL fayl, git'da, ko'rib chiqiladi.

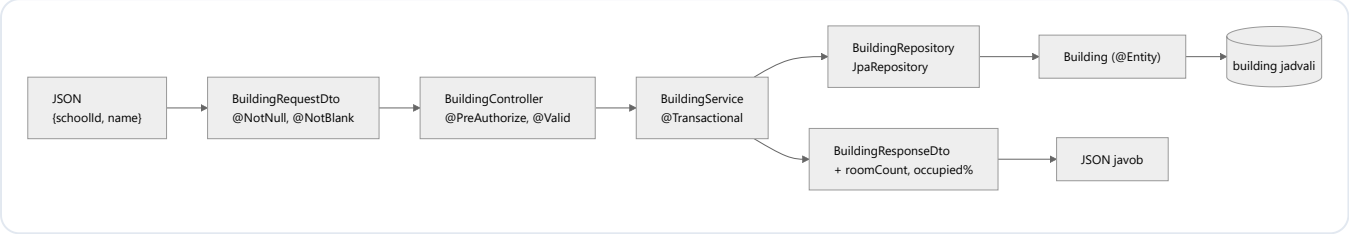
> 📌 **Esda tut**
>
> - 39 ta jadval = 39 ta `@Entity` ; jadvallarni Hibernate `ddl-auto=update` bilan yaratadi.
> - Bog'lanishlar asosan bir tomonlama `@ManyToOne` ; `@OneToMany` ataylab ishlatilmagan.
> - Enum'lar doim `EnumType.STRING` ; muhim dublikatlar `@UniqueConstraint` bilan bazada to'silgan.
> - Yangi entity'larda `LAZY` + `@BatchSize` — N+1 so'rovlarga qarshi.
> - `update` ustun o'chirmaydi va turini o'zgartirmaydi — buning uchun qo'lda migratsiya klasslari bor.

6. Modul: maktab tuzilmasi

Bu bob maktabning "skeletini" tashkil qiladigan sakkizta modulni qamraydi: **Maktab, Bino, Xona, O'quv yili, Sinf, Fan, Lavozim, Xodim**. Ularning hammasi bir xil andaza bilan yozilgan, shuning uchun avval andazani **Bino** misolida to'liq ko'rib chiqamiz, keyin har modulning o'ziga xos qoidalarini ko'ramiz.

Modul	Entity → jadval	Controller	Service	O'ziga xos qoida
Maktab	`School` → `school`	`SchoolController`	`SchoolService`	eng yuqori daraja, boshqa hamma narsa unga bog'lanadi
Bino	`Building` → `building`	`BuildingController`	`BuildingService`	xonalar soni va bandlik foizi hisoblanadi
Xona	`Room` → `room`	`RoomController`	`RoomService`	xonaning haftalik bandligi (`/occupancy`)
O'quv yili	`AcademicYear` → `academic_year`	`AcademicYearController`	`AcademicYearService`	boshlanish sanasi tugashdan oldin bo'lishi shart
Sinf	`SchoolClass` → `school_class`	`SchoolClassController`	`SchoolClassService`	bir o'quv yilida "5-A" ikki marta bo'lmaydi
Fan	`Subject` → `subject`	`SubjectController`	`SubjectService`	**o'chirilmaydi** — nofaol qilinadi
Lavozim	`Position` → `position`	`PositionController`	`PositionService`	nomi takrorlanmaydi (global ro'yxat)
Xodim	`Employee` → `employee`	`EmployeeController`	`EmployeeService`	ta'til / ishdan ketish — login bloklanadi

Andaza: Bino moduli to'liq zanjir bilan



6.1-rasm. Bitta modulning beshta qismi.

1. Entity

src/main/java/uz/azizbek/maktabboshqaruv/entity/Building.java — 5–18-qatorlar

```
5  @Entity
6  @Table(name = "building")
7  public class Building {
8
9      @Id
10     @GeneratedValue(strategy = GenerationType.IDENTITY)
11     private Long id;
12
13     @ManyToOne
14     @JoinColumn(name = "school_id", nullable = false)
15     private School school;
```

```
16
17      @Column(nullable = false)
18      private String name;
```

Bino maktabga tegishli: `@ManyToOne` + `@JoinColumn(name = "school_id", nullable = false)` — bazada `building.school_id` tashqi kaliti, bo'sh bo'la olmaydi. Getter/setter'lar qo'lda yozilgan (Lombok ishlatilmaydi).

2. Repository

src/main/java/uz/azizbek/maktabboshqaruv/repository/BuildingRepository.java 1–10-qatorlar

```
1  package uz.azizbek.maktabboshqaruv.repository;
2
3  import uz.azizbek.maktabboshqaruv.entity.Building;
4  import org.springframework.data.domain.Page;
5  import org.springframework.data.domain.Pageable;
6  import org.springframework.data.jpa.repository.JpaRepository;
7
8  public interface BuildingRepository extends JpaRepository<Building, Long> {
9      Page<Building> findBySchoolId(Long schoolId, Pageable pageable);
10 }
```

Bu — atigi 2 qatorlik **interfeys**, lekin Spring ishga tushganda uning to'liq ishlaydigan amalga oshirilishini (implementation) o'zi yaratadi:

- **8-qator,** `extends JpaRepository<Building, Long>` — bepul tayyor metodlar: `save`, `findById`, `findAll`, `deleteById`, `existsById`, `count`, sahifalash va saralash.
- **9-qator,** `findBySchoolId` — **derived query** (nomdan yasaladigan so'rov). Spring metod nomini tahlil qiladi: `findBy` + `School` + `Id` → `WHERE school.id = ?`. `Pageable` parametri sahifalash va saralashni qo'shadi.

> 🔬 **Hibernate qanday SQL yuboradi?**
> `findBySchoolId(5, PageRequest.of(0, 20))` chaqirilganda taxminan shunday ikki so'rov ketadi (`show-sql=true` bo'lgani uchun ular konsolda ko'rinadi):
>
> ```
> select b.id, b.name, b.school_id from building b where b.school_id = ? offset 0 rows fetch first 20 rows only
> select count(b.id) from building b where b.school_id = ?
> ```
>
> Ikkinchi `count` so'rovi `Page` ob'ektidagi "jami nechta" (`totalElements`) uchun kerak — frontend sahifa raqamlarini shunga qarab chizadi.

3. DTO

src/main/java/uz/azizbek/maktabboshqaruv/dto/BuildingRequestDto.java 6–12-qatorlar

```
6  public class BuildingRequestDto {
7
8      @NotNull(message = "Maktab tanlanishi shart")
9      private Long schoolId;
10
11     @NotBlank(message = "Bino nomi kiritilishi shart")
12     private String name;
```

`@NotNull` — qiymat bo'lishi shart; `@NotBlank` — matn bo'sh yoki faqat bo'shliqdan iborat bo'lmasin. Xabarlar o'zbekcha va ular to'g'ridan-to'g'ri foydalanuvchiga 400 javobida qaytadi (16-bob).

4. Controller

src/main/java/uz/azizbek/maktabboshqaruv/controller/BuildingController.java 14–53-qatorlar

```
14 @RestController
15 @RequestMapping("/api/buildings")
16 public class BuildingController {
17
18     @Autowired
19     private BuildingService buildingService;
20
21     @PreAuthorize("hasAnyRole('ADMIN', 'EDITOR', 'VIEWER')")
22     @GetMapping
23     public Page<BuildingResponseDto> getAllBuildings(@RequestParam Long schoolId, Pageable pageable) {
24         return buildingService.getAllBuildings(schoolId, pageable);
25     }
26
27     @PreAuthorize("hasAnyRole('ADMIN', 'EDITOR', 'VIEWER')")
28     @GetMapping("/{id}")
29     public ResponseEntity<BuildingResponseDto> getBuildingById(@PathVariable Long id) {
30         return ResponseEntity.ok(buildingService.getBuildingById(id));
31     }
32
33     @PreAuthorize("hasAnyRole('ADMIN', 'EDITOR')")
34     @PostMapping
35     public ResponseEntity<BuildingResponseDto> createBuilding(@Valid @RequestBody BuildingRequestDto re
quest) {
36         BuildingResponseDto created = buildingService.createBuilding(request);
37         return ResponseEntity.status(201).body(created);
38     }
39
40     @PreAuthorize("hasAnyRole('ADMIN', 'EDITOR')")
41     @PutMapping("/{id}")
42     public ResponseEntity<BuildingResponseDto> updateBuilding(@PathVariable Long id, @Valid @RequestBod
y BuildingRequestDto request) {
43         BuildingResponseDto updated = buildingService.updateBuilding(id, request);
44         return ResponseEntity.ok(updated);
45     }
46
47     @PreAuthorize("hasRole('ADMIN')")
48     @DeleteMapping("/{id}")
49     public ResponseEntity<String> deleteBuilding(@PathVariable Long id) {
50         buildingService.deleteBuilding(id);
51         return ResponseEntity.ok("ID " + id + " bilan bino muvaffaqiyatli o'chirildi");
52     }
53 }
```

Qator	Nima qiladi
14	`@RestController` = `@Controller` + `@ResponseBody` : metod qaytargan ob'ekt avtomatik JSON'ga aylanadi
15	`@RequestMapping("/api/buildings")` — klassdagi barcha endpoint'lar shu prefiks bilan boshlanadi
21	`@PreAuthorize("hasAnyRole('ADMIN', 'EDITOR', 'VIEWER')")` — o'qishni uchala rol ham qila oladi
23	`@RequestParam Long schoolId` — `?schoolId=5` query parametri (majburiy); `Pageable` — `?page=0&size=20&sort=name` avtomatik
33	yozish — faqat ADMIN va EDITOR
35	`@Valid @RequestBody` — JSON → DTO, keyin validatsiya; xato bo'lsa metod umuman chaqirilmaydi
37	`ResponseEntity.status(201)` — yaratilganda HTTP **201 Created**
47	o'chirish — faqat ADMIN

5. Service

src/main/java/uz/azizbek/maktabboshqaruv/service/BuildingService.java 43–88-qatorlar

```
43      public Page<BuildingResponseDto> getAllBuildings(Long schoolId, Pageable pageable) {
44          return buildingRepository.findBySchoolId(schoolId, pageable)
45                  .map(this::toResponseDto);
46      }
47
48      public BuildingResponseDto getBuildingById(Long id) {
49          Building building = buildingRepository.findById(id)
50                  .orElseThrow(() -> new IllegalStateException("Bunday bino topilmadi: " + id));
51          return toResponseDto(building);
52      }
53
54      @Transactional
55      public BuildingResponseDto createBuilding(BuildingRequestDto request) {
56          School school = schoolRepository.findById(request.getSchoolId())
57                  .orElseThrow(() -> new IllegalStateException("Bunday maktab mavjud emas"));
58
59          Building building = new Building();
60          building.setSchool(school);
61          building.setName(request.getName());
62
63          Building saved = buildingRepository.save(building);
64          return toResponseDto(saved);
65      }
66
67      @Transactional
68      public BuildingResponseDto updateBuilding(Long id, BuildingRequestDto request) {
69          Building building = buildingRepository.findById(id)
70                  .orElseThrow(() -> new IllegalStateException("Bunday bino topilmadi: " + id));
71
72          School school = schoolRepository.findById(request.getSchoolId())
73                  .orElseThrow(() -> new IllegalStateException("Bunday maktab mavjud emas"));
74
75          building.setSchool(school);
76          building.setName(request.getName());
77
78          Building updated = buildingRepository.save(building);
79          return toResponseDto(updated);
80      }
81
82      @Transactional
83      public void deleteBuilding(Long id) {
84          if (!buildingRepository.existsById(id)) {
85              throw new IllegalStateException("Bunday bino topilmadi: " + id);
86          }
87          buildingRepository.deleteById(id);
88      }
```

- **54** — `@Transactional`: metod bitta tranzaksiyada bajariladi.
- **56–57** — avval maktab bormi, tekshiriladi; yo'q bo'lsa `IllegalStateException` → `GlobalExceptionHandler` uni **409** javobga aylantiradi.
- **63** — `save()` yangi ob'ekt uchun `INSERT` yuboradi va bazadan olingan `id` ni ob'ektga yozadi.
- **87** — `deleteById` — haqiqiy o'chirish (`DELETE`). Binoda xonalar bo'lsa, PostgreSQL tashqi kalit cheklovi buni rad etadi.

> ⚠️ **Topilgan kamchilik: N+1 so'rov**
>
> Shu servisning `toResponseDto` metodi (90–118-qatorlar) **har bir bino uchun** xonalarni alohida so'rov bilan o'qiydi, keyin **har bir xona uchun** "hozir dars bormi?" deb yana alohida so'rov yuboradi. 20 ta binoli sahifada 10 tadan xona bo'lsa — 1 + 20 + 200 ≈ **221 ta SQL**. Yechim: bitta guruhlangan so'rov (`select building_id, count(*) ... group by building_id`) yoki band xonalarni bitta `IN (...)` so'rovi bilan olish.

> 🤔 **Nega Service entity'ni emas, DTO qaytaradi?**
>
> Agar controller to'g'ridan-to'g'ri `Building` entity'ni qaytarsa: (1) Jackson `school` ichidagi barcha maydonlarni ham chiqarib yuboradi; (2) LAZY bog'lanishlar tranzaksiyadan tashqarida xato beradi; (3) entity'ga yangi ustun qo'shilsa, API ham o'zgarib, frontend buziladi. DTO — API'ning **barqaror shartnomasi**: `BuildingResponseDto` da `schoolName` , `roomCount` , `occupiedPercentage` kabi frontendga kerakli hisoblangan qiymatlar bor, bazada esa bunday ustunlar yo'q.

Endpoint'lar

Metod	URL	Rol	Java metodi	Nima qiladi
GET	`/api/academic-years`	ADMIN, EDITOR, VIEWER	`getAllAcademicYears()`	O'quv yili ro'yxati (sahifalab)
GET	`/api/academic-years/{id}`	ADMIN, EDITOR, VIEWER	`getAcademicYearById()`	Bitta o'quv yili (id bo'yicha)
POST	`/api/academic-years`	ADMIN, EDITOR	`createAcademicYear()`	Yangi o'quv yili yaratadi
PUT	`/api/academic-years/{id}`	ADMIN, EDITOR	`updateAcademicYear()`	O'quv yilini tahrirlaydi
DELETE	`/api/academic-years/{id}`	ADMIN	`deleteAcademicYear()`	O'quv yilini o'chiradi
GET	`/api/buildings`	ADMIN, EDITOR, VIEWER	`getAllBuildings()`	Bino ro'yxati (sahifalab)
GET	`/api/buildings/{id}`	ADMIN, EDITOR, VIEWER	`getBuildingById()`	Bitta bino (id bo'yicha)
POST	`/api/buildings`	ADMIN, EDITOR	`createBuilding()`	Yangi bino yaratadi
PUT	`/api/buildings/{id}`	ADMIN, EDITOR	`updateBuilding()`	Binoni tahrirlaydi
DELETE	`/api/buildings/{id}`	ADMIN	`deleteBuilding()`	Binoni o'chiradi
GET	`/api/rooms`	ADMIN, EDITOR, VIEWER	`getAllRooms()`	Xona ro'yxati (sahifalab)
GET	`/api/rooms/{id}`	ADMIN, EDITOR, VIEWER	`getRoomById()`	Bitta xona (id bo'yicha)
GET	`/api/rooms/{id}/occupancy`	ADMIN, EDITOR, VIEWER	`getOccupancy()`	Xonaning haftalik bandligi
POST	`/api/rooms`	ADMIN, EDITOR	`createRoom()`	Yangi xona yaratadi
PUT	`/api/rooms/{id}`	ADMIN, EDITOR	`updateRoom()`	Xonani tahrirlaydi
DELETE	`/api/rooms/{id}`	ADMIN	`deleteRoom()`	Xonani o'chiradi
GET	`/api/school-classes`	ADMIN, EDITOR, VIEWER	`getAllSchoolClasses()`	Sinf ro'yxati (sahifalab)
GET	`/api/school-classes/{id}`	ADMIN, EDITOR, VIEWER	`getSchoolClassById()`	Bitta sinf (id bo'yicha)
POST	`/api/school-classes`	ADMIN, EDITOR	`createSchoolClass()`	Yangi sinf yaratadi
PUT	`/api/school-classes/{id}`	ADMIN, EDITOR	`updateSchoolClass()`	Sinfni tahrirlaydi
DELETE	`/api/school-classes/{id}`	ADMIN	`deleteSchoolClass()`	Sinfni o'chiradi
GET	`/api/schools`	ADMIN, EDITOR, VIEWER	`getAllSchools()`	Maktab ro'yxati (sahifalab)

Metod	URL	Rol	Java metodi	Nima qiladi
GET	/api/schools/{id}	ADMIN, EDITOR, VIEWER	getSchoolById()	Bitta maktab (id bo'yicha)
POST	/api/schools	ADMIN, EDITOR	createSchool()	Yangi maktab yaratadi
PUT	/api/schools/{id}	ADMIN, EDITOR	updateSchool()	Maktabni tahrirlaydi
DELETE	/api/schools/{id}	ADMIN	deleteSchool()	Maktabni o'chiradi

Metod	URL	Rol	Java metodi	Nima qiladi
GET	/api/employees	ADMIN, EDITOR, VIEWER	getAllEmployees()	Xodim ro'yxati (sahifalab)
GET	/api/employees/{id}	ADMIN, EDITOR, VIEWER	getEmployeeById()	Bitta xodim (id bo'yicha)
POST	/api/employees	ADMIN, EDITOR	createEmployee()	Yangi xodim yaratadi
PUT	/api/employees/{id}	ADMIN, EDITOR	updateEmployee()	Xodimni tahrirlaydi
DELETE	/api/employees/{id}	ADMIN	deleteEmployee()	Xodimni o'chiradi
GET	/api/positions	ADMIN, EDITOR, VIEWER	getAllPositions()	Lavozim ro'yxati (sahifalab)
GET	/api/positions/{id}	ADMIN, EDITOR, VIEWER	getPositionById()	Bitta lavozim (id bo'yicha)
POST	/api/positions	ADMIN, EDITOR	createPosition()	Yangi lavozim yaratadi
PUT	/api/positions/{id}	ADMIN, EDITOR	updatePosition()	Lavozimni tahrirlaydi
DELETE	/api/positions/{id}	ADMIN	deletePosition()	Lavozimni o'chiradi
GET	/api/subjects	ADMIN, EDITOR, VIEWER	getAllSubjects()	Fan ro'yxati (sahifalab)
PUT	/api/subjects/{id}/activate	ADMIN, EDITOR	activate()	Nofaol fanni qayta faollashtiradi
GET	/api/subjects/{id}	ADMIN, EDITOR, VIEWER	getSubjectById()	Bitta fan (id bo'yicha)
POST	/api/subjects	ADMIN, EDITOR	createSubject()	Yangi fan yaratadi
PUT	/api/subjects/{id}	ADMIN, EDITOR	updateSubject()	Fanni tahrirlaydi
DELETE	/api/subjects/{id}	ADMIN	deleteSubject()	Fanni o'chiradi

Fan: o'chirish o'rniga nofaol qilish

Fanni bazadan o'chirib bo'lmaydi: unga minglab baholar bog'langan. Agar o'chirilsa, eski baholar "yetim" qoladi yoki tashqi kalit xatosi chiqadi. Shuning uchun `Subject` da `active` maydoni bor.

src/main/java/uz/azizbek/maktabboshqaruv/service/SubjectService.java — 82–120-qatorlar

```
82      @Transactional
83      public SubjectResponseDto updateSubject(Long id, SubjectRequestDto request) {
84          Subject subject = find(id);
85          School school = schoolRepository.findById(request.getSchoolId())
86                  .orElseThrow(() -> new IllegalStateException("Bunday maktab mavjud emas"));
87
88          boolean renamed = !request.getName().equals(subject.getName());
89          if (!renamed) return toResponseDto(subject);
90          botCache.evictAllClasses(); // the bot's cached timetables show subject names
91          if (subjectRepository.existsActiveBySchoolIdAndName(school.getId(), request.getName())) {
92              throw new IllegalStateException("Bu fan ushbu maktabda allaqachon mavjud");
93          }
94
95          // nothing graded yet: a plain rename is safe
96          if (!gradeRepository.existsBySubjectId(id)) {
```

```
97              subject.setSchool(school);
98              subject.setName(request.getName());
99              return toResponseDto(subjectRepository.save(subject));
100         }
101
102         // graded: keep the old name for history, continue under the new one
103         Subject renamedSubject = new Subject();
104         renamedSubject.setSchool(school);
105         renamedSubject.setName(request.getName());
106         renamedSubject.setActive(true);
107         renamedSubject = subjectRepository.save(renamedSubject);
108         lessonSlotRepository.moveSubject(id, renamedSubject.getId());
109         subject.setActive(false);
110         subjectRepository.save(subject);
111         return toResponseDto(renamedSubject);
112     }
113
114     /** "O'chirish" = nofaol qilish. */
115     @Transactional
116     public void deleteSubject(Long id) {
117         Subject subject = find(id);
118         subject.setActive(false);
119         subjectRepository.save(subject);
120     }
```

- **88–89** — nom o'zgarmagan bo'lsa, hech narsa qilinmaydi.
- **96–100** — fanga hali baho qo'yilmagan bo'lsa, oddiy nom o'zgartirish xavfsiz.
- **102–111** — baho bor bo'lsa: **yangi faol fan** yaratiladi, dars jadvali unga ko'chiriladi (`moveSubject`), eski fan nofaol bo'ladi. Natijada eski baholar tarixda eski nomi bilan qoladi ("Tabiatshunoslik"), yangilari yangi nom bilan ("Tabiiy fanlar").
- **116–119** — "O'chirish" tugmasi aslida `active = false` qiladi.

`SubjectService.requireActive` metodi baho va dars formalarida nofaol fan tanlanishini to'xtatadi (409, "«...» fani nofaol"). Bu qoidani `SubjectLifecycleIntegrationTest` tekshiradi.

> 🧩 **Arxivga topshirish**
>
> Bu — hujjatni yo'q qilish emas, arxivga topshirish. Arxivdagi hujjat yangi ish uchun berilmaydi, lekin eski jurnalni ochganingizda u joyida turadi.

Xodim: ta'til va hisobni bloklash

Xodimning holati (`EmployeeStatus`): **ACTIVE** (ishda), **ON_LEAVE** (ta'tilda, sanalar bilan), **DISMISSED** (ishdan ketgan).

src/main/java/uz/azizbek/maktabboshqaruv/entity/Employee.java — 71–76-qatorlar

```
71      public boolean isAwayOn(java.time.LocalDate today) {
72          EmployeeStatus s = getStatus();
73          if (s == EmployeeStatus.DISMISSED) return true;
74          if (s != EmployeeStatus.ON_LEAVE) return false;
75          return (leaveFrom == null || !today.isBefore(leaveFrom)) && (leaveTo == null || !today.isAfter
    (leaveTo));
76      }
```

Agar xodim bugun ta'tilda yoki ishdan ketgan bo'lsa (`isAwayOn` = `true`), unga bog'langan foydalanuvchi:

- **tizimga kira olmaydi** — login 403 qaytaradi (`AuthController`);

- **allaqachon kirgan bo'lsa ham** keyingi so'rovda 401 oladi — `JwtAuthFilter` har so'rovda `AccountStatusService.isUsable()` ni tekshiradi (1 daqiqalik kesh bilan).

`AccountStatusService.checkCanSetStatus` ikki xavfli holatni to'xtatadi: administrator **o'zini** bloklay olmaydi va **oxirgi faol administratorni** bloklab bo'lmaydi — aks holda tizimga hech kim kira olmay qolardi. Har kuni 00:05 da (Toshkent) `returnFromLeave()` ta'tili tugagan xodimlarni avtomatik "Ishda" holatiga qaytaradi.

> 📌 **Esda tut**
>
> - Har modul: Entity → Repository (interfeys) → Service (qoidalar, `@Transactional`) → Controller (`@PreAuthorize`, `@Valid`) → DTO.
> - Spring Data metod nomidan SQL yasaydi: `findBySchoolId` → `WHERE school_id = ?`.
> - O'qish — ADMIN/EDITOR/VIEWER, yozish — ADMIN/EDITOR, o'chirish — faqat ADMIN.
> - Fanlar o'chirilmaydi, nofaol qilinadi; baho qo'yilgan fan nomini o'zgartirish yangi fan yaratadi.
> - Ta'tildagi xodimning hisobi avtomatik bloklanadi va ta'til tugagach ochiladi.

7. Modul: o'quvchi, hudud va tuman

Bu bobda loyihaning eng ko'p ishlatiladigan ob'ekti — **o'quvchi** va uning manzilini ta'minlovchi **hudud → tuman** ma'lumotnomasi ko'riladi. Bob oxirida "yangi o'quvchi qo'shish" so'rovi boshidan oxirigacha kuzatiladi.

Tarkibi

Qism	O'quvchi	Hudud	Tuman
Entity	`Student` → `students`	`Region` → `region`	`District` → `district`
Repository	`StudentRepository`	`RegionRepository`	`DistrictRepository`
Service	`StudentService`	`RegionService`	`DistrictService`
Controller	`StudentController`	`RegionController`	`DistrictController`
DTO	`StudentRequestDto`, `StudentResponseDto`, `StudentProfileDto`	`RegionRequestDto`, `RegionResponseDto` (record)	`DistrictRequestDto`, `DistrictResponseDto` (record)
Boshqa	`HousingType` (enum)	—	`TerritorySeeder` (14 hudud, 204 tuman)

Hudud va tuman: ma'lumotnoma (spravochnik)

Hudud (viloyat, Qoraqalpog'iston Respublikasi, Toshkent shahri — 14 ta) va uning tumanlari deyarli o'zgarmaydigan, lekin ko'p joyda ishlatiladigan ro'yxat. Bunday ro'yxatlar uchun loyihada uchta qoida qo'llangan:

1. `code` **(SOATO) — barqaror kalit.** `id` bazadan bazaga o'zgarishi mumkin, davlat kodi esa o'zgarmaydi. `code` UNIQUE.
2. **Soft delete.** O'chirish `state = 'P'` qiladi, qator qoladi — unga bog'langan o'quvchilar "yetim" qolmaydi.
3. **DTO —** `record`**.** Yangi kodda Java 16+ `record` lari ishlatilgan: getter/setter yozish shart emas.

src/main/java/uz/azizbek/maktabboshqaruv/service/RegionService.java — 46–68-qatorlar

```
46      @Transactional
47      public RegionResponseDto update(Long id, RegionRequestDto request) {
48          Region r = findActive(id);
49          String code = request.code().trim();
50          if (!r.getCode().equals(code) && regions.existsByCode(code)) {
51              throw new ResponseStatusException(HttpStatus.CONFLICT, "Bu kod allaqachon mavjud: " + cod
    e);
52          }
53          r.setCode(code);
54          r.setNameUz(request.nameUz().trim());
55          // no save(): the entity is managed, Hibernate writes the change at commit (dirty checking)
56          return RegionResponseDto.from(r);
57      }
58
59      /** Soft delete: districts and students that point at the region keep a valid reference. */
60      @Transactional
61      public void delete(Long id) {
62          findActive(id).setState(Region.DELETED);
63      }
```

```
64
65      private Region findActive(Long id) {
66          return regions.findByIdAndState(id, Region.ACTIVE)
67                  .orElseThrow(() -> new ResponseStatusException(HttpStatus.NOT_FOUND, "Hudud topilmadi:
" + id));
68      }
```

- **55–56** — `save()` chaqirilmagan! Bu xato emas: `findActive` qaytargan ob'ekt Hibernate'ning **persistence context**ida ("boshqariladigan" holatda). Tranzaksiya oxirida Hibernate ob'ektni bazadan o'qilgan nusxa bilan solishtiradi va o'zgarganini ko'rsa, o'zi `UPDATE` yuboradi. Bu **dirty checking** deyiladi.
- **62** — o'chirish ham shunday: faqat `state` maydoni o'zgartiriladi, commit'da `UPDATE region SET state='P'` ketadi.
- **66–67** — topilmasa `ResponseStatusException(NOT_FOUND)` → HTTP **404** (16-bob).

> 🧩 **Dirty checking**
> Kutubxonachi kitobni sizga beradi va daftariga uning holatini yozib qo'yadi. Siz kitob ustida qayd qilasiz va qaytarasiz. Kutubxonachi "qaytarish" paytida daftardagi yozuv bilan solishtiradi va o'zgarishni o'zi qayd qiladi — siz alohida "o'zgarish bor" deb aytishingiz shart emas. Hibernate'ning persistence context'i — o'sha daftar.

Ma'lumotnomani avtomatik to'ldirish: TerritorySeeder

src/main/java/uz/azizbek/maktabboshqaruv/config/TerritorySeeder.java — 99–125-qatorlar

```
99      public void run(String... args) {
100         Map<String, Region> byCode = new LinkedHashMap<>();
101         regions.findAll().forEach(r -> byCode.put(r.getCode(), r));
102         int addedRegions = 0;
103         int addedDistricts = 0;
104         for (RegionSeed seed : REGIONS) {
105             Region region = byCode.get(seed.code());
106             if (region == null) {
107                 region = new Region();
108                 region.setCode(seed.code());
109                 region.setNameUz(seed.name());
110                 region = regions.save(region);
111                 addedRegions++;
112             }
113             if (districts.existsByRegionId(region.getId())) continue;
114             for (String name : seed.districts()) {
115                 District d = new District();
116                 d.setRegion(region);
117                 d.setNameUz(name);
118                 districts.save(d);
119                 addedDistricts++;
120             }
121         }
122         if (addedRegions > 0 || addedDistricts > 0) {
123             log.info("Hududlar ma'lumotnomasi: {} ta hudud, {} ta tuman qo'shildi", addedRegions, adde
dDistricts);
124         }
125     }
```

`CommandLineRunner` — ilova to'liq ishga tushgach `run()` chaqiriladigan bean. Seeder **idempotent** (necha marta ishga tushsa ham natija bir xil):

- **101–106** — hudud faqat **kodi bazada yo'q** bo'lsa qo'shiladi. Jonli bazada 14 ta hudud qo'lda (pgAdmin orqali) kiritilgan — seeder ularga tegmaydi.

- **113** — tumanlar faqat **hali bitta ham tumani yo'q** hududga qo'shiladi. Admin biror tumanni o'chirsa yoki nomini o'zgartirsa, seeder uni qaytarib tiklamaydi.

Tumanni o'quvchi manzili uchun tekshirish

src/main/java/uz/azizbek/maktabboshqaruv/service/DistrictService.java 89–98-qatorlar

```
89      @Transactional(readOnly = true)
90      public District forAddress(Long districtId, Long regionId, String what) {
91          if (districtId == null) return null;
92          District d = districts.findByIdAndState(districtId, Region.ACTIVE)
93                  .orElseThrow(() -> new ResponseStatusException(HttpStatus.BAD_REQUEST, what + ": tuman
    topilmadi"));
94          if (regionId != null && !d.getRegion().getId().equals(regionId)) {
95              throw new ResponseStatusException(HttpStatus.BAD_REQUEST, what + ": tuman tanlangan hududga
    tegishli emas");
96          }
97          return d;
98      }
```

Frontend avval hududni, keyin tumanni tanlaydi va ikkalasini yuboradi. Server **frontendga ishonmaydi**: tanlangan tuman haqiqatan o'sha hududga tegishli ekanini (94-qator) qayta tekshiradi. Aks holda kimdir so'rovni qo'lda o'zgartirib, "Toshkent shahri"ni tanlab, Samarqanddagi tumanni yuborishi mumkin edi.

Metod	URL	Rol	Java metodi	Nima qiladi
GET	/api/districts	ADMIN, EDITOR, VIEWER	getAll()	Tuman ro'yxati
GET	/api/districts/{id}	ADMIN, EDITOR, VIEWER	getById()	Bitta tuman (id bo'yicha)
POST	/api/districts	ADMIN, EDITOR	create()	Yangi tuman yaratadi
PUT	/api/districts/{id}	ADMIN, EDITOR	update()	Tumanni tahrirlaydi
DELETE	/api/districts/{id}	ADMIN	delete()	Tumanni o'chiradi
GET	/api/regions	ADMIN, EDITOR, VIEWER	getAll()	Hudud ro'yxati
GET	/api/regions/{id}	ADMIN, EDITOR, VIEWER	getById()	Bitta hudud (id bo'yicha)
GET	/api/regions/{id}/districts	ADMIN, EDITOR, VIEWER	getDistricts()	Hududning faol tumanlari
POST	/api/regions	ADMIN, EDITOR	create()	Yangi hudud yaratadi
PUT	/api/regions/{id}	ADMIN, EDITOR	update()	Hududni tahrirlaydi
DELETE	/api/regions/{id}	ADMIN	delete()	Hududni o'chiradi

O'quvchi entity'si

`Student` ning asosiy maydonlari: `schoolClass` (`@ManyToOne`, majburiy), `firstName`, `lastName`, `birthDate`, vasiy (`guardianName`, `guardianPhone`, `guardianRelation`), Telegram bog'lash kodi va manzil maydonlari:

Maydon	Ustun	Izoh
`birthDistrict`	`birth_district_id`	`@ManyToOne(fetch = LAZY)` → `District` — tug'ilgan joy
`residenceDistrict`	`residence_district_id`	`@ManyToOne(fetch = LAZY)` → `District` — yashash joyi
`mahalla`	`mahalla`	masalan, "Furqat mahallasi"
`street`	`street`	ko'cha, massiv yoki kvartal

Maydon	Ustun	Izoh
`housingType`	`housing_type`	`KVARTIRA` (ko'p qavatli uy) yoki `HOVLI`
`houseNumber`	`house_number`	uy raqami ("12", "15A")
`apartmentNumber`	`apartment_number`	xonadon raqami (faqat ko'p qavatli uyda)

Barcha manzil maydonlari **nullable**: eski yozuvlar va tezkor kiritish ishlayveradi. Bundan tashqari `ddl-auto=update` qatorlari bor jadvalga `NOT NULL` ustun qo'sha olmaydi (5-bob).

src/main/java/uz/azizbek/maktabboshqaruv/entity/Student.java 122–143-qatorlar

```
122     /** "Furqat mahallasi, Yangizamon ko'chasi, 10-uy, 65-xonadon" / "..., 17-uy"; null when nothing is set */
123     public String addressLine() {
124         List<String> parts = new ArrayList<>();
125         if (mahalla != null && !mahalla.isBlank()) parts.add(mahalla.trim());
126         if (street != null && !street.isBlank()) parts.add(street.trim());
127         if (houseNumber != null && !houseNumber.isBlank()) {
128             if (housingType == HousingType.KVARTIRA) {
129                 parts.add(houseNumber.trim() + "-uy");
130                 if (apartmentNumber != null && !apartmentNumber.isBlank()) parts.add(apartmentNumber.trim() + "-xonadon");
131             } else {
132                 parts.add(houseNumber.trim() + "-uy");
133             }
134         }
135         return parts.isEmpty() ? null : String.join(", ", parts);
136     }
137
138     /** The whole home address to find the house: "Toshkent shahri, Mirzo Ulug'bek tumani, Furqat mahallasi, ..." */
139     public String fullAddress() {
140         String line = addressLine();
141         if (residenceDistrict == null) return line;
142         return line == null ? residenceDistrict.placeName() : residenceDistrict.placeName() + ", " + line;
143     }
```

Entity ichidagi kichik **domain metodlari**: manzilni bitta qatorga yig'ish entity'ning o'zida. Shu sababli ro'yxat (`StudentResponseDto.fullAddress`) va profil (`StudentProfileDto.fullAddress`) bir xil formatni oladi: "*Toshkent shahri, Mirzo Ulug'bek tumani, Furqat mahallasi, Yangizamon ko'chasi, 10-uy, 65-xonadon*".

So'rov DTO'si va validatsiya

src/main/java/uz/azizbek/maktabboshqaruv/dto/StudentRequestDto.java 13–53-qatorlar

```
13      @NotNull(message = "Sinf tanlanishi shart")
14      private Long schoolClassId;
15
16      @NotBlank(message = "Ism kiritilishi shart")
17      @Pattern(regexp = "^[A-Za-zA-Яа-яЎўҚқҒғҲҳ'\\s]+$", message = "Ism faqat harflardan iborat bo'lishi kerak")
18      private String firstName;
19
20      @NotBlank(message = "Familiya kiritilishi shart")
21      @Pattern(regexp = "^[A-Za-zA-Яа-яЎўҚқҒғҲҳ'\\s]+$", message = "Familiya faqat harflardan iborat bo'lishi kerak")
22      private String lastName;
23
24      @NotNull(message = "Tug'ilgan sana kiritilishi shart")
```

```
25      @Past(message = "Tug'ilgan sana o'tmishda bo'lishi kerak")
26      private LocalDate birthDate;
27
28      private String guardianName;
29
30      private String guardianPhone;
31
32      private uz.azizbek.maktabboshqaruv.entity.GuardianRelation guardianRelation;
33
34      // ---- tug'ilgan joyi va yashash manzili (all optional). The region ids drive the pickers and are
35      // checked against the district; only the districts are stored.
36      private Long birthRegionId;
37      private Long birthDistrictId;
38      private Long residenceRegionId;
39      private Long residenceDistrictId;
40
41      @Size(max = 100, message = "Mahalla nomi 100 belgidan oshmasin")
42      private String mahalla;
43
44      @Size(max = 150, message = "Ko'cha nomi 150 belgidan oshmasin")
45      private String street;
46
47      private uz.azizbek.maktabboshqaruv.entity.HousingType housingType;
48
49      @Size(max = 20, message = "Dom / uy raqami 20 belgidan oshmasin")
50      private String houseNumber;
51
52      @Size(max = 20, message = "Kvartira raqami 20 belgidan oshmasin")
53      private String apartmentNumber;
```

Annotatsiya	Qoida	Xato bo'lsa
`@NotNull` (13)	sinf tanlanishi shart	400 `schoolClassId: Sinf tanlanishi shart`
`@NotBlank` + `@Pattern` (16–17)	ism bo'sh emas va faqat harflar (lotin, kirill, `'`)	400
`@Past` (25)	tug'ilgan sana o'tmishda	400
`@Size(max=...)` (41–53)	matn uzunligi ustun uzunligidan oshmasin	400

> 💡 **Validatsiyaning ikki qavati**
>
> **Annotatsiyalar** (`@NotBlank` , `@Size`) — bitta maydonning shaklini tekshiradi va controller'ga kirishdan oldin ishlaydi. **Service'dagi qoidalar** — maydonlar orasidagi bog'liqlik va bazaga qarab tekshiriladigan narsalar: "tuman shu hududdami?", "ko'p qavatli uy bo'lsa, xonadon raqami bormi?". Ikkalasi ham kerak: birinchisi arzon va tez, ikkinchisisiz esa ma'lumot ichidan qarama-qarshi bo'lib qolishi mumkin.

So'rov boshidan oxirigacha: yangi o'quvchi qo'shish

Klient yuboradi:

```
POST /api/students
Authorization: Bearer <JWT>
Content-Type: application/json

{
  "firstName": "Aziza", "lastName": "Karimova", "birthDate": "2014-05-01",
  "schoolClassId": 12,
  "guardianName": "Furqat Karimov", "guardianRelation": "FATHER", "guardianPhone": "+998901234567",
  "birthRegionId": 8, "birthDistrictId": 115,
  "residenceRegionId": 14, "residenceDistrictId": 196,
  "mahalla": "Furqat mahallasi", "street": "Yangizamon ko'chasi",
  "housingType": "KVARTIRA", "houseNumber": "10", "apartmentNumber": "65"
}
```

1-qadam — filtrlar va maktab chegarasi. `JwtAuthFilter` tokenni tekshiradi (15-bob). `SchoolScopeConfig.BodyScope` JSON'dagi `schoolClassId: 12` foydalanuvchining maktabiga tegishli ekanini tekshiradi (14-bob).

2-qadam — controller.

src/main/java/uz/azizbek/maktabboshqaruv/controller/StudentController.java — 36–41-qatorlar

```
36      @PreAuthorize("hasAnyRole('ADMIN', 'EDITOR')")
37      @PostMapping
38      public ResponseEntity<StudentResponseDto> createStudent(@Valid @RequestBody StudentRequestDto reque
        st) {
39          StudentResponseDto created = studentService.createStudent(request);
40          return ResponseEntity.status(201).body(created);
41      }
```

`@PreAuthorize` — ADMIN yoki EDITOR; `@Valid` — DTO validatsiyasi; `201 Created`.

3-qadam — service: tranzaksiya, qoidalar, saqlash.

src/main/java/uz/azizbek/maktabboshqaruv/service/StudentService.java — 56–78-qatorlar

```
56      @Transactional
57      public StudentResponseDto createStudent(StudentRequestDto request) {
58          SchoolClass schoolClass = schoolClassRepository.findById(request.getSchoolClassId())
59                  .orElseThrow(() -> new IllegalStateException("Bunday sinf mavjud emas"));
60
61          Student student = new Student();
62          student.setSchoolClass(schoolClass);
63          student.setFirstName(request.getFirstName());
64          student.setLastName(request.getLastName());
65          student.setBirthDate(request.getBirthDate());
66          student.setGuardianName(request.getGuardianName());
67          student.setGuardianPhone(request.getGuardianPhone());
68          student.setGuardianRelation(request.getGuardianRelation());
69          applyAddress(student, request);
70
71          Student saved = studentRepository.save(student);
72          activityLogService.record(
73                  schoolClass.getAcademicYear().getSchool(),
74                  "person_add",
75                  student.getFirstName() + " " + student.getLastName() + " " + schoolClass.getGradeNumber
        ()
76                          + "-" + schoolClass.getSectionLetter() + " sinfiga qo'shildi");
77          return toResponseDto(saved);
78      }
```

- **56** — `@Transactional`: proxy `BEGIN` ochadi.
- **58** — `SELECT ... FROM school_class WHERE id = 12`. Topilmasa → 409.

- **69** — `applyAddress` manzil qoidalarini tekshiradi (pastda).
- **71** — `save()` → `INSERT INTO students (...)`. Shu payt `@PrePersist` metodi o'quvchiga Telegram bog'lash kodini ham yaratadi.
- **72–76** — "Faoliyat" lentasiga yozuv: "*Aziza Karimova 5-A sinfiga qo'shildi*".
- Metod tugaganda proxy **COMMIT** qiladi. Agar 69-qatordagi tekshiruv xato bersa, **hech narsa** saqlanmaydi.

src/main/java/uz/azizbek/maktabboshqaruv/service/StudentService.java 116–139-qatorlar

```
116     private void applyAddress(Student student, StudentRequestDto request) {
117         District birth = districtService.forAddress(request.getBirthDistrictId(), request.getBirthRegi
        onId(), "Tug'ilgan joyi");
118         District residence = districtService.forAddress(request.getResidenceDistrictId(), request.getR
        esidenceRegionId(), "Yashash joyi");
119         String house = blankToNull(request.getHouseNumber());
120         String apartment = blankToNull(request.getApartmentNumber());
121         HousingType type = request.getHousingType();
122         if (type == null && (house != null || apartment != null)) {
123             throw bad("Uy turini tanlang: ko'p qavatli uy yoki hovli");
124         }
125         if (type == HousingType.KVARTIRA && (house == null || apartment == null)) {
126             throw bad("Ko'p qavatli uy uchun uy raqami va xonadon raqami kiritilishi shart");
127         }
128         if (type == HousingType.HOVLI) {
129             if (house == null) throw bad("Hovli uchun uy raqami kiritilishi shart");
130             if (apartment != null) throw bad("Hovlida xonadon raqami bo'lmaydi");
131         }
132         student.setBirthDistrict(birth);
133         student.setResidenceDistrict(residence);
134         student.setMahalla(blankToNull(request.getMahalla()));
135         student.setStreet(blankToNull(request.getStreet()));
136         student.setHousingType(type);
137         student.setHouseNumber(house);
138         student.setApartmentNumber(type == HousingType.KVARTIRA ? apartment : null);
139     }
```

Qoidalar ketma-ketligi:

1. **117–118** — har ikki tuman faol va tanlangan hududga tegishli (aks holda 400 "*Yashash joyi: tuman tanlangan hududga tegishli emas*").
2. **122** — uy raqami bor-u, uy turi tanlanmagan → 400.
3. **125** — ko'p qavatli uy: uy **va** xonadon raqami shart.
4. **128–130** — hovli: uy raqami shart, xonadon raqami **bo'lmasligi** kerak.
5. **138** — xonadon raqami faqat ko'p qavatli uyda saqlanadi.

4-qadam — javob. `toResponseDto` DTO yasaydi: `birthPlace: "Samarqand viloyati, Urgut tumani"`, `fullAddress: "Toshkent shahri, Yunusobod tumani, Furqat mahallasi, Yangizamon ko'chasi, 10-uy, 65-xonadon"`, shuningdek tahrir formasi uchun barcha id'lar. Javob **201 Created**.

> ⚠️ **Bu yerda o'rganilgan tuzoq: @Transactional va o'z-o'zini chaqirish**
>
> `StudentService.getAllStudents(schoolId, pageable)` (38-qator) ichida boshqa metod — `getAllStudents(schoolId, null, null, pageable)` chaqiriladi. Bu chaqiruv **proxy'dan o'tmaydi** (`this.` orqali). Shuning uchun ikkinchi metoddagi `@Transactional` bu yo'lda ishlamaydi. Manzil maydonlari LAZY bo'lgani uchun tranzaksiyasiz `LazyInitializationException` chiqqan bo'lardi. Yechim — tashqi metodning o'ziga ham `@Transactional(readOnly = true)` qo'yish (37-qator). Bu xato aynan shu modulni yozish paytida topilib tuzatilgan.

src/main/java/uz/azizbek/maktabboshqaruv/service/StudentService.java 37–47-qatorlar

```
37      @Transactional(readOnly = true)
38      public Page<StudentResponseDto> getAllStudents(Long schoolId, Pageable pageable) {
39          return getAllStudents(schoolId, null, null, pageable);
40      }
41
42      @Transactional(readOnly = true)
43      public Page<StudentResponseDto> getAllStudents(Long schoolId, Long schoolClassId, String q, Pageable pageable) {
44          String like = q == null || q.isBlank() ? null : "%" + q.trim().toLowerCase().replaceAll("\\s+", " ") + "%";
45          return studentRepository.search(schoolId, schoolClassId, like, pageable)
46                  .map(this::toResponseDto);
47      }
```

🔬 **readOnly = true nima beradi?**

`@Transactional(readOnly = true)` — Spring va Hibernate'ga "bu tranzaksiyada yozish bo'lmaydi" degan ishora. Hibernate bunday tranzaksiyada dirty checking uchun ob'ektlarning nusxasini saqlamaydi (kamroq xotira) va flush qilmaydi. Ba'zi drayverlar uni bazaga ham "faqat o'qish" sifatida uzatadi.

Qidiruv: JPQL so'rovi

src/main/java/uz/azizbek/maktabboshqaruv/repository/StudentRepository.java 18–29-qatorlar

```
18      /** Students list: optional class filter and a first/last name search (either order). */
19      @org.springframework.data.jpa.repository.Query("""
20              select s from Student s where s.schoolClass.academicYear.school.id = :schoolId
21                and (:classId is null or s.schoolClass.id = :classId)
22                and (:q is null
23                     or lower(concat(s.firstName, ' ', s.lastName)) like :q
24                     or lower(concat(s.lastName, ' ', s.firstName)) like :q)
25              """)
26      Page<Student> search(@org.springframework.data.repository.query.Param("schoolId") Long schoolId,
27                           @org.springframework.data.repository.query.Param("classId") Long classId,
28                           @org.springframework.data.repository.query.Param("q") String q,
29                           Pageable pageable);
```

Bu — **JPQL** (Java Persistence Query Language): SQL'ga o'xshaydi, lekin jadvallar emas, **entity va maydonlar** bilan yoziladi (`Student s`, `s.schoolClass.academicYear.school.id`). Hibernate uni kerakli `JOIN`lar bilan SQL'ga aylantiradi. `(:classId is null or ...)` usuli bilan bitta so'rov ham filtrli, ham filtrsiz holatda ishlaydi. Ism "Aziza Karimova" yoki "Karimova Aziza" ko'rinishida qidirilishi mumkin.

Metod	URL	Rol	Java metodi	Nima qiladi
GET	/api/students	ADMIN, EDITOR, VIEWER	getAllStudents()	O'quvchi ro'yxati (sahifalab)
GET	/api/students/{id}	ADMIN, EDITOR, VIEWER	getStudentById()	Bitta o'quvchi (id bo'yicha)
POST	/api/students	ADMIN, EDITOR	createStudent()	Yangi o'quvchi yaratadi
PUT	/api/students/{id}	ADMIN, EDITOR	updateStudent()	O'quvchini tahrirlaydi
DELETE	/api/students/{id}	ADMIN	deleteStudent()	O'quvchini o'chiradi

O'quvchi profili (`GET /api/profiles/students/{id}`) 10-bobda ko'riladi. U yerda ham to'liq manzil, tug'ilgan joy va uy turi qaytariladi.

📌 **Esda tut**

- Ma'lumotnomalar: barqaror `code`, soft delete (`state = 'A'/'P'`), DTO uchun `record`.
- Boshqariladigan entity o'zgarsa, `save()` shart emas — Hibernate dirty checking bilan o'zi `UPDATE` qiladi.
- Server frontendga ishonmaydi: tuman ↔ hudud mosligi va uy/xonadon qoidalari Service'da qayta tekshiriladi.
- `@Transactional` faqat proxy orqali ishlaydi; LAZY maydonlarni o'qiydigan metod tranzaksiya ichida bo'lishi kerak.
- JPQL entity nomlari bilan yoziladi va Hibernate uni SQL'ga aylantiradi.

8. Modul: dars jadvali

Dars jadvali — maktab hayotining "soati": davomat ham, bot ham, bosh sahifadagi "hozir qaysi dars" ham shunga tayanadi. Jadvalning bitta katagi — `LessonSlot`: qaysi **sinf**, qaysi **fan**, qaysi **o'qituvchi**, qaysi **xonada**, haftaning qaysi **kuni**, soat nechadan nechagacha.

Qism	Klass
Entity	`LessonSlot` → `lesson_slot` (`class_id`, `subject_id`, `employee_id`, `room_id`, `weekday`, `start_time`, `end_time`)
Repository	`LessonSlotRepository` — to'qnashuv so'rovlari, "hozirgi dars", haftalik jadval
Service	`LessonSlotService`, `TimetablePdfService`
Controller	`LessonSlotController` (`/api/lesson-slots`)
DTO	`LessonSlotRequestDto`, `LessonSlotResponseDto`, `TimetableEntryDto`

Yangi dars qo'shish: qoidalar zanjiri

src/main/java/uz/azizbek/maktabboshqaruv/service/LessonSlotService.java — 89–120-qatorlar

```
89      @Transactional
90      public LessonSlotResponseDto createLessonSlot(LessonSlotRequestDto request) {
91          validateTimeRange(request);
92
93          SchoolClass schoolClass = schoolClassRepository.findById(request.getSchoolClassId())
94                  .orElseThrow(() -> new IllegalStateException("Bunday sinf mavjud emas"));
95          Subject subject = subjectRepository.findById(request.getSubjectId())
96                  .orElseThrow(() -> new IllegalStateException("Bunday fan mavjud emas"));
97          Employee employee = employeeRepository.findById(request.getEmployeeId())
98                  .orElseThrow(() -> new IllegalStateException("Bunday o'qituvchi mavjud emas"));
99          Room room = roomRepository.findById(request.getRoomId())
100                 .orElseThrow(() -> new IllegalStateException("Bunday xona mavjud emas"));
101
102         validateSameSchool(schoolClass, subject, employee, room);
103         if (!subject.isActive()) {
104             throw new IllegalStateException("«" + subject.getName() + "» fani nofaol — jadvalga qo'shi
    b bo'lmaydi");
105         }
106         validateNoConflicts(request, null);
107
108         LessonSlot lessonSlot = new LessonSlot();
109         lessonSlot.setSchoolClass(schoolClass);
110         lessonSlot.setSubject(subject);
111         lessonSlot.setEmployee(employee);
112         lessonSlot.setRoom(room);
113         lessonSlot.setWeekday(request.getWeekday());
114         lessonSlot.setStartTime(request.getStartTime());
115         lessonSlot.setEndTime(request.getEndTime());
116
117         LessonSlot saved = lessonSlotRepository.save(lessonSlot);
118         botCache.evictAllClasses();
119         return toResponseDto(saved);
120     }
```

Qoidalar arzondan qimmatga qarab tekshiriladi:

1. **91** — vaqt oralig'i (bazaga bormasdan): boshlanish < tugash.
2. **93–100** — sinf, fan, o'qituvchi, xona mavjudmi (4 ta `SELECT`).
3. **102** — to'rttasi ham **bitta maktabniki**mi.
4. **103** — fan faolmi (6-bob).
5. **106** — **to'qnashuv** yo'qmi: xona, o'qituvchi va sinf o'sha vaqtda bo'sh bo'lishi kerak.
6. **117–118** — saqlash va botning jadval keshini tozalash: ota-onalar yangi jadvalni darhol ko'rsin.

src/main/java/uz/azizbek/maktabboshqaruv/service/LessonSlotService.java — 172–195-qatorlar

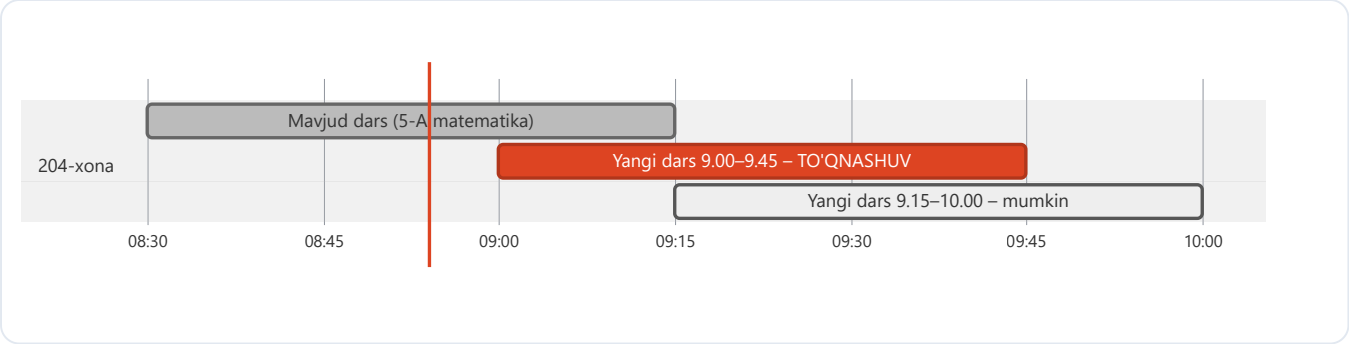
```
172     private void validateSameSchool(SchoolClass schoolClass, Subject subject, Employee employee, Room room) {
173         Long schoolId = schoolClass.getAcademicYear().getSchool().getId();
174         boolean sameSchool = subject.getSchool() != null && schoolId.equals(subject.getSchool().getId())
175                 && employee.getSchool() != null && schoolId.equals(employee.getSchool().getId())
176                 && schoolId.equals(room.getBuilding().getSchool().getId());
177         if (!sameSchool) {
178             throw new IllegalStateException("Sinf, fan, o'qituvchi va xona bitta maktabga tegishli bo'lishi kerak");
179         }
180     }
181
182     private void validateNoConflicts(LessonSlotRequestDto request, Long excludeId) {
183         if (lessonSlotRepository.existsRoomConflict(request.getRoomId(), request.getWeekday(),
184                 request.getStartTime(), request.getEndTime(), excludeId)) {
185             throw new IllegalStateException("Bu xona ushbu vaqtda band");
186         }
187         if (lessonSlotRepository.existsEmployeeConflict(request.getEmployeeId(), request.getWeekday(),
188                 request.getStartTime(), request.getEndTime(), excludeId)) {
189             throw new IllegalStateException("Bu o'qituvchi ushbu vaqtda band");
190         }
191         if (lessonSlotRepository.existsSchoolClassConflict(request.getSchoolClassId(), request.getWeekday(),
192                 request.getStartTime(), request.getEndTime(), excludeId)) {
193             throw new IllegalStateException("Bu sinf ushbu vaqtda band");
194         }
195     }
```

To'qnashuvni bitta so'rov bilan aniqlash

src/main/java/uz/azizbek/maktabboshqaruv/repository/LessonSlotRepository.java — 48–54-qatorlar

```
48     @Query("select count(l) > 0 from LessonSlot l where l.room.id = :roomId and l.weekday = :weekday " +
49             "and l.startTime < :endTime and l.endTime > :startTime and (:excludeId is null or l.id <> :excludeId)")
50     boolean existsRoomConflict(@Param("roomId") Long roomId,
51                                @Param("weekday") String weekday,
52                                @Param("startTime") LocalTime startTime,
53                                @Param("endTime") LocalTime endTime,
54                                @Param("excludeId") Long excludeId);
```

Ikki vaqt oralig'i `[A, B)` va `[C, D)` **kesishadi**, agar `A < D` va `B > C` bo'lsa. So'rovdagi `l.startTime < :endTime and l.endTime > :startTime` aynan shu formula. `(:excludeId is null or l.id <> :excludeId)` — tahrirlashda darsning o'zi o'zi bilan "to'qnashmasligi" uchun.



8.1-rasm. Kesishish qoidasi: 09:15 da tugagan dars 09:15 da boshlanadigan bilan to'qnashmaydi.

> ⚠️ **Bir vaqtdagi so'rovlar (race condition)**
> Tekshiruv ("bo'sh-mi?") va yozish ("saqlash") ikki alohida qadam. Ikki xodim **bir vaqtda** bir xonaga dars qo'shsa, ikkalasi ham "bo'sh" javobini olib, ikkalasi ham saqlashi mumkin. Bazada bunga qarshi cheklov yo'q. Amalda ehtimoli kichik, lekin to'liq kafolat uchun PostgreSQL'ning **exclusion constraint**i (`EXCLUDE USING gist (room_id WITH =, weekday WITH =, tsrange(...) WITH &&)`) yoki tranzaksiya darajasida qulf kerak.

"Hozir qaysi dars?" va "N+1 yo'q" so'rov

src/main/java/uz/azizbek/maktabboshqaruv/repository/LessonSlotRepository.java — 30–46-qatorlar

```
30      /** A class's whole week with subject, teacher and room in one query (the bot's cache loader). */
31      @Query("select distinct l from LessonSlot l join fetch l.subject join fetch l.employee e join fetch e.position " +
32              "left join fetch l.room r left join fetch r.building where l.schoolClass.id = :classId")
33      List<LessonSlot> findWeekOfClass(@Param("classId") Long classId);
34
35      /** A renamed subject: today's timetable moves to the new subject row; grade history stays on the old one. */
36      @org.springframework.data.jpa.repository.Modifying
37      @Query("update LessonSlot l set l.subject.id = :to where l.subject.id = :from")
38      int moveSubject(@Param("from") Long from, @Param("to") Long to);
39
40      @Query("select l from LessonSlot l where l.schoolClass.academicYear.school.id = :schoolId " +
41              "and l.weekday = :weekday and l.startTime <= :time and l.endTime > :time order by l.schoolClass.gradeNumber, l.schoolClass.sectionLetter")
42      List<LessonSlot> findCurrentlyInSession(@Param("schoolId") Long schoolId, @Param("weekday") String weekday, @Param("time") LocalTime time);
43
44      @Query("select l from LessonSlot l where l.schoolClass.academicYear.school.id = :schoolId " +
45              "and l.weekday = :weekday and l.startTime > :time order by l.startTime")
46      List<LessonSlot> findUpcomingToday(@Param("schoolId") Long schoolId, @Param("weekday") String weekday, @Param("time") LocalTime time);
```

- **31–32,** `join fetch` — sinfning butun haftasini fan, o'qituvchi, lavozim, xona va bino bilan **bitta** SQL'da yuklaydi. `join fetch` siz har dars uchun 3–4 ta qo'shimcha so'rov ketgan bo'lardi (N+1). Bot jadval keshini shu bilan to'ldiradi.
- **36–37,** `@Modifying` + `update` — JPQL orqali **ommaviy UPDATE**: fan nomi o'zgarganda barcha darslarni bitta so'rov bilan yangi fanga o'tkazadi (6-bob).
- **40–41** — "hozir o'tayotgan darslar": `startTime <= hozir < endTime`. Bosh sahifadagi "Jonli darslar" shunga tayanadi.

Haftalik jadval PDF

`GET /api/lesson-slots/timetable.pdf?schoolId=1&schoolClassId=12` sinfning butun haftasini bitta A4 albom sahifada PDF qilib beradi. `schoolClassId` berilmasa — maktabning har bir sinfi alohida sahifada.

src/main/java/uz/azizbek/maktabboshqaruv/controller/LessonSlotController.java 42–53-qatorlar

```
42      /** "PDF yuklab olish": the class's week on one A4 landscape page; without schoolClassId — every cl
    ass. */
43      @PreAuthorize("hasAnyRole('ADMIN', 'EDITOR', 'VIEWER')")
44      @GetMapping("/timetable.pdf")
45      public ResponseEntity<byte[]> getTimetablePdf(@RequestParam Long schoolId,
46                                                    @RequestParam(required = false) Long schoolClassId) {
47          byte[] pdf = timetablePdfService.render(schoolId, schoolClassId);
48          String name = schoolClassId == null ? "dars-jadvali-barcha-sinflar.pdf" : "dars-jadvali.pdf";
49          return ResponseEntity.ok()
50                  .contentType(org.springframework.http.MediaType.APPLICATION_PDF)
51                  .header(org.springframework.http.HttpHeaders.CONTENT_DISPOSITION, "attachment; filename
    =\"" + name + "\"")
52                  .body(pdf);
53      }
```

- **50** — `Content-Type: application/pdf`.
- **51** — `Content-Disposition: attachment; filename="..."`: brauzer faylni ochmasdan, yuklab oladi.

src/main/java/uz/azizbek/maktabboshqaruv/service/TimetablePdfService.java 53–66-qatorlar

```
53          this.regular = font("/fonts/Inter-Regular.ttf");
54          this.bold = font("/fonts/Inter-Bold.ttf");
55      }
56
57      private static BaseFont font(String path) {
58          try (InputStream in = TimetablePdfService.class.getResourceAsStream(path)) {
59              if (in == null) throw new IllegalStateException("Shrift topilmadi: " + path);
60              byte[] bytes = in.readAllBytes();
61              return BaseFont.createFont(path.substring(path.lastIndexOf('/') + 1), BaseFont.IDENTITY_H,
62                      BaseFont.EMBEDDED, true, bytes, null);
63          } catch (IOException | DocumentException e) {
64              throw new IllegalStateException("Shriftni yuklab bo'lmadi: " + path, e);
65          }
66      }
```

PDF **OpenPDF** kutubxonasi bilan yasaladi. Shrift (`Inter-Regular.ttf`) `resources/fonts` dan o'qiladi va `IDENTITY_H` + `EMBEDDED` bilan PDF ichiga joylanadi. Shu tufayli kirill va o'zbek harflari foydalanuvchi kompyuterida shu shrift bo'lmasa ham to'g'ri chiqadi. `TimetablePdfServiceTest` bunga kirill matni bilan ham tekshiriladi.

> 🤔 **Nega shriftlar konstruktorda bir marta yuklanadi?**
> Shrift faylini o'qish va tahlil qilish nisbatan qimmat. `TimetablePdfService` — **singleton** bean (Spring uni bitta nusxada yaratadi), shuning uchun shrift ilova ishga tushganda bir marta yuklanadi va keyin har bir PDF so'rovida qayta ishlatiladi.

Metod	URL	Rol	Java metodi	Nima qiladi
GET	/api/lesson-slots	ADMIN, EDITOR, VIEWER	getAllLessonSlots()	Dars ro'yxati (sahifalab)
GET	/api/lesson-slots/timetable	ADMIN, EDITOR, VIEWER	getTimetable()	Haftalik dars jadvali

Metod	URL	Rol	Java metodi	Nima qiladi
GET	`/api/lesson-slots/timetable.pdf`	ADMIN, EDITOR, VIEWER	`getTimetablePdf()`	Haftalik dars jadvali PDF (har sinfga bitta sahifa)
GET	`/api/lesson-slots/{id}`	ADMIN, EDITOR, VIEWER	`getLessonSlotById()`	Bitta dars (id bo'yicha)
POST	`/api/lesson-slots`	ADMIN, EDITOR	`createLessonSlot()`	Yangi dars yaratadi
PUT	`/api/lesson-slots/{id}`	ADMIN, EDITOR	`updateLessonSlot()`	Darsni tahrirlaydi
DELETE	`/api/lesson-slots/{id}`	ADMIN	`deleteLessonSlot()`	Darsni o'chiradi

📌 **Esda tut**

- `LessonSlot` = sinf + fan + o'qituvchi + xona + kun + vaqt.
- Uch xil to'qnashuv (xona, o'qituvchi, sinf) bitta formula bilan tekshiriladi: `start < boshqaEnd && end > boshqaStart`.
- `join fetch` — bog'liq ob'ektlarni bitta so'rovda olish, N+1 ga qarshi asosiy vosita.
- `@Modifying` + JPQL `update` — ko'p qatorni bitta so'rovda o'zgartirish.
- PDF shriftlari ichiga joylanadi, shuning uchun kirill harflari har qanday kompyuterda to'g'ri chiqadi.

9. Modul: davomat, baholar, jurnal, xulq, arizalar

Bu — o'quv jarayonining yuragi. O'qituvchi har kuni shu yerda ishlaydi, ota-onalar esa natijani Telegram'da darhol ko'radi. Shuning uchun bu modullarda ikki muhim mavzu bor: **biznes qoidalari** va **domain event'lar**.

Modul	Entity	Service	Controller
Davomat	`Attendance`	`AttendanceService`, `AttendanceCalendarService`	`AttendanceController`
Baholar	`Grade`	`GradeService`	`GradeController`
Sinf jurnali	(Grade + LessonSlot)	`JournalService`	`GradeController.journal`, `JournalController`
Xulq	`BehaviorRecord`	`BehaviorRecordService`	`BehaviorRecordController`
Sababli ariza	`AbsenceRequest`	`AbsenceRequestService`	`AbsenceRequestController`

Davomat: butun sinfni bitta so'rovda belgilash

O'qituvchi darsni ochadi (`GET /api/attendance/roster?lessonSlotId=..&date=..`), har o'quvchiga holat qo'yadi va "Saqlash"ni bosadi (`POST /api/attendance/bulk`).

src/main/java/uz/azizbek/maktabboshqaruv/service/AttendanceService.java — 119–157-qatorlar

```
119     @Transactional
120     public List<AttendanceRosterEntryDto> bulkMark(AttendanceBulkRequestDto request) {
121         LessonSlot lessonSlot = lessonSlotRepository.findById(request.getLessonSlotId())
122                 .orElseThrow(() -> new IllegalStateException("Bunday dars mavjud emas"));
123
124         Map<Long, Attendance> existing = new HashMap<>();
125         for (Attendance a : attendanceRepository.findByLessonSlotIdAndRecordDate(request.getLessonSlotId(), request.getRecordDate())) {
126             existing.put(a.getStudent().getId(), a);
127         }
128
129         for (AttendanceBulkRequestDto.Entry entry : request.getEntries()) {
130             Student student = studentRepository.findById(entry.getStudentId())
131                     .orElseThrow(() -> new IllegalStateException("Bunday o'quvchi mavjud emas: " + entry.getStudentId()));
132             if (!student.getSchoolClass().getId().equals(lessonSlot.getSchoolClass().getId())) {
133                 throw new IllegalStateException("O'quvchi ushbu darsning sinfiga tegishli emas: " + entry.getStudentId());
134             }
135
136             Attendance attendance = existing.get(entry.getStudentId());
137             if (attendance == null) {
138                 attendance = new Attendance();
139                 attendance.setLessonSlot(lessonSlot);
140                 attendance.setStudent(student);
141                 attendance.setRecordDate(request.getRecordDate());
142             }
143             AttendanceStatus previousStatus = attendance.getStatus();
144             attendance.setStatus(entry.getStatus());
```

```
145             attendance.setComment(entry.getComment());
146             Attendance saved = attendanceRepository.save(attendance);
147             publishIfNotifiable(saved, previousStatus);
148         }
149
150         String className = lessonSlot.getSchoolClass().getGradeNumber() + "-" + lessonSlot.getSchoolClass().getSectionLetter();
151         activityLogService.record(
152                 lessonSlot.getSchoolClass().getAcademicYear().getSchool(),
153                 "fact_check",
154                 className + " sinfida " + lessonSlot.getSubject().getName() + " darsidan davomat olindi");
155
156         return getRoster(request.getLessonSlotId(), request.getRecordDate());
157     }
```

- **124–127** — shu dars va sanadagi **mavjud** yozuvlar bitta so'rov bilan olinib, `Map` ga joylanadi. Qayta saqlash yangi qator yaratmaydi, borini yangilaydi (**upsert** mantiqi).
- **132** — xavfsizlik: yuborilgan o'quvchi haqiqatan shu darsning sinfidami? Begona id → 409.
- **143** — **oldingi** holat eslab qolinadi.
- **146–147** — saqlash va **event**: faqat holat haqiqatan o'zgarganda.

src/main/java/uz/azizbek/maktabboshqaruv/service/AttendanceService.java — 200–206-qatorlar

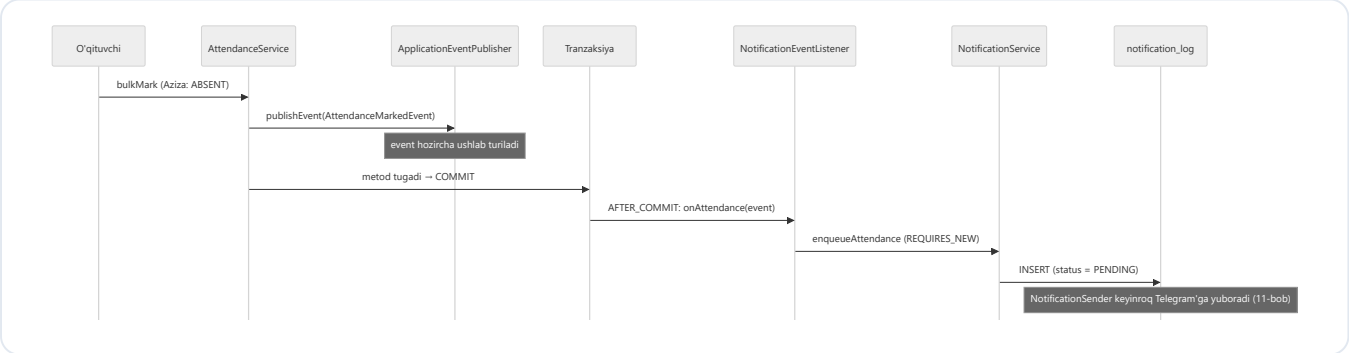
```
200     // Re-saving a roster with unchanged statuses must not re-notify parents;
201     // only a change into ABSENT/LATE (e.g. PRESENT -> ABSENT) does.
202     void publishIfNotifiable(Attendance attendance, AttendanceStatus previousStatus) {
203         if (AttendanceMarkedEvent.isNotifiable(previousStatus, attendance.getStatus())) {
204             eventPublisher.publishEvent(new AttendanceMarkedEvent(attendance.getId(), previousStatus, attendance.getStatus()));
205         }
206     }
```

src/main/java/uz/azizbek/maktabboshqaruv/event/AttendanceMarkedEvent.java — 1–17-qatorlar

```
1  package uz.azizbek.maktabboshqaruv.event;
2
3  import uz.azizbek.maktabboshqaruv.entity.AttendanceStatus;
4
5  /**
6   * Published by AttendanceService only when a record's status actually changed
7   * to ABSENT or LATE. Carries ids, not entities: the listener runs after commit
8   * in its own transaction and reloads what it needs.
9   */
10 public record AttendanceMarkedEvent(Long attendanceId, AttendanceStatus previousStatus, AttendanceStatus newStatus) {
11
12     /** The single rule for "does this save deserve a parent notification?". */
13     public static boolean isNotifiable(AttendanceStatus previous, AttendanceStatus current) {
14         if (current != AttendanceStatus.ABSENT && current != AttendanceStatus.LATE) return false;
15         return previous != current;
16     }
17 }
```

`isNotifiable` — "ota-onaga xabar yuborilsinmi?" degan **yagona** qoida: yangi holat `ABSENT` yoki `LATE` bo'lsa va oldingisidan farq qilsa. O'qituvchi bir sahifani 5 marta saqlasa ham ota-ona 5 ta "farzandingiz kelmadi" xabarini olmaydi.

Domain event'lar: "nimadir bo'ldi"



9.1-rasm. Davomatdan ota-ona xabarigacha: event faqat COMMIT'dan keyin ishlaydi.

src/main/java/uz/azizbek/maktabboshqaruv/event/NotificationEventListener.java — 17–41-qatorlar

```
17  @Component
18  public class NotificationEventListener {
19
20      private static final Logger log = LoggerFactory.getLogger(NotificationEventListener.class);
21
22      @Autowired
23      private NotificationService notificationService;
24
25      @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
26      public void onAttendance(AttendanceMarkedEvent event) {
27          try {
28              notificationService.enqueueAttendance(event);
29          } catch (Exception e) {
30              log.error("Davomat xabarnomasini navbatga qo'shib bo'lmadi (attendance={})", event.attendan
    ceId(), e);
31          }
32      }
33
34      @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
35      public void onGrade(GradeSavedEvent event) {
36          try {
37              notificationService.enqueueGrade(event);
38          } catch (Exception e) {
39              log.error("Baho xabarnomasini navbatga qo'shib bo'lmadi (grade={})", event.gradeId(), e);
40          }
41      }
```

- `@TransactionalEventListener(phase = AFTER_COMMIT)` — tinglovchi **faqat tranzaksiya muvaffaqiyatli commit bo'lgandan keyin** chaqiriladi. Saqlash xato bilan rollback bo'lsa, ota-onaga "kelmadi" degan xato xabar ketmaydi.
- `try/catch` + `log.error` — xabarnoma qo'yishda xato bo'lsa ham, o'qituvchining so'rovi **muvaffaqiyatli** tugaydi: davomat allaqachon saqlangan, xabar muammosi uni buzmasligi kerak.
- `NotificationService.enqueue...` metodlari `@Transactional(propagation = REQUIRES_NEW)` bilan belgilangan: commit'dan keyin eski tranzaksiya yopiq, shuning uchun **yangi** tranzaksiya ochiladi.

🤔 **Nega xabarni to'g'ridan-to'g'ri AttendanceService'dan yubormaslik kerak?**

1. **Bog'liqlik:** `AttendanceService` Telegram haqida hech narsa bilmaydi. Ertaga SMS qo'shilsa, davomat kodiga tegilmaydi — faqat yangi tinglovchi qo'shiladi.
2. **To'g'rilik:** tranzaksiya rollback bo'lishi mumkin. Xabar commit'dan oldin yuborilsa, ota-ona saqlanmagan davomat haqida xabar olardi. `AFTER_COMMIT` buni istisno qiladi.
3. **Tezlik:** Telegram sekin bo'lsa ham o'qituvchi kutmaydi — xabar navbatga (outbox) yoziladi.

🧩 **Hayotiy o'xshatish**

Event — maktab e'lonlar taxtasiga yopishtirilgan qog'oz: "5-A sinfda Aziza kelmadi". Davomatni yozgan o'qituvchi kim o'qishini bilmaydi va qiziqmaydi. Kotib (tinglovchi) taxtani kuzatib, ota-onaga xat tayyorlaydi. Qog'oz esa faqat jurnal rasman imzolangandan (commit) keyin osiladi.

Baholar: qoidalar

src/main/java/uz/azizbek/maktabboshqaruv/service/GradeService.java 52–61-qatorlar

```
52      /** Grades only on school days that have already come: never on a Sunday, never ahead of today (40
        9). */
53      void validateGradeDate(LocalDate date) {
54          if (date == null) return;
55          if (date.getDayOfWeek() == java.time.DayOfWeek.SUNDAY) {
56              throw new IllegalStateException(SUNDAY_MESSAGE);
57          }
58          if (date.isAfter(LocalDate.now(clock))) {
59              throw new IllegalStateException("Kelajak sanaga baho qo'yilmaydi");
60          }
61      }
```

src/main/java/uz/azizbek/maktabboshqaruv/service/GradeService.java 88–116-qatorlar

```
88      @Transactional
89      public GradeResponseDto createGrade(GradeRequestDto request) {
90          Student student = studentRepository.findById(request.getStudentId())
91                  .orElseThrow(() -> new IllegalStateException("Bunday o'quvchi mavjud emas"));
92          Subject subject = subjectRepository.findById(request.getSubjectId())
93                  .orElseThrow(() -> new IllegalStateException("Bunday fan mavjud emas"));
94
95          validateSameSchool(student, subject);
96          requireActive(subject);
97          validateGradeDate(request.getGradeDate());
98
99          Grade grade = new Grade();
100         grade.setStudent(student);
101         grade.setSubject(subject);
102         grade.setGradeDate(request.getGradeDate());
103         grade.setScore(request.getScore());
104         grade.setType(request.getType());
105         grade.setComment(request.getComment());
106         grade.setCreatedBy(currentAuthor());
107
108         Grade saved = gradeRepository.save(grade);
109         activityLogService.record(
110                 student.getSchoolClass().getAcademicYear().getSchool(),
111                 "grade",
112                 student.getFirstName() + " " + student.getLastName() + "ga " + subject.getName()
113                         + " fanidan " + request.getScore() + " baho qo'yildi");
```

```
114         eventPublisher.publishEvent(new GradeSavedEvent(saved.getId(), true));
115         return toResponseDto(saved);
116     }
```

Qoida	Qayerda	Javob
baho 2 dan 5 gacha	`GradeRequestDto` : `@Min(2) @Max(5)`	400
yakshanbaga baho yo'q	`validateGradeDate` , 55-qator	409 "Uzr, bu kun yakshanba..."
kelajak sanaga baho yo'q	58-qator, `LocalDate.now(clock)`	409
o'quvchi va fan bitta maktabda	`validateSameSchool`	409
nofaol fanga baho yo'q	`requireActive`	409
kim qo'ydi	`currentAuthor()` → "Karimova D."	`grade.created_by`

🔬 **Nega LocalDate.now() emas, LocalDate.now(clock)?**
`clock` — `TelegramConfig` da yaratilgan `Clock` bean (Asia/Tashkent). Server boshqa vaqt zonasida (masalan UTC) ishlasa ham, "bugun" Toshkent bo'yicha hisoblanadi: UTC'da yarim tun 05:00 Toshkent. Testlarda esa `MutableClock` yordamida "hozir chorshanba 10:40" deb vaqtni boshqarish mumkin — kelajak sanasi qoidasini sinash shu tufayli oson. Bu — **dependency injection**ning vaqtga qo'llanishi.

`GradeSavedEvent` ham xuddi davomat kabi ishlaydi. Yangi baho yoki bahosi, fani yoki turi o'zgargan baho ota-onaga xabar bo'lib boradi.

Sinf jurnali

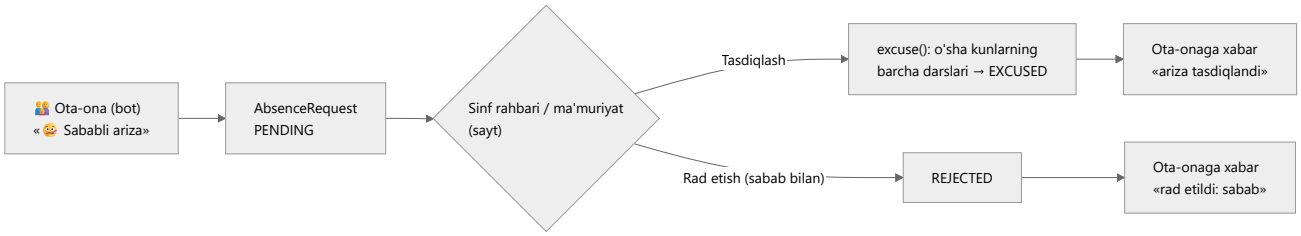
`JournalService` bir oy, bitta fan va bitta sinf uchun qog'oz jurnalga o'xshash jadval yig'adi: qatorlarda o'quvchilar (familiya bo'yicha), ustunlarda oy kunlari (yakshanba va bayramlar belgilangan), kataklarda baholar. Uning javob turlari — servis ichida e'lon qilingan `record` lar (`Journal` , `Day` , `StudentRow` , `GradeCell` , `Overview` , `CurrentLesson`). Bitta servisning DTO'lari uning yonida turibdi.

`JournalQueryCountTest` muhim kafolatni tekshiradi: sinfda 10 ta yoki 40 ta o'quvchi bo'lishidan qat'i nazar, har bir jurnal endpoint'i **1–3 ta SQL** so'rov yuboradi (N+1 yo'q).

Oylik davomat kalendari

`AttendanceCalendarService` sinfning oylik ko'rinishini quradi: har bir o'quvchi uchun har kuni bitta nuqta. Kunning nuqtasi o'sha kundagi darslarning "eng yomon" belgisi bo'ladi (kelmadi > kechikdi > sababli > keldi). Nuqta o'zgartirilsa (`PUT /api/attendance/day`), o'sha kunning **barcha darslari** bir yo'la yangilanadi. Butun oy o'quvchilar soniga bog'liq bo'lmagan holda **4 ta so'rov** bilan o'qiladi.

Sababli ariza: botdan saytga va qaytib



src/main/java/uz/azizbek/maktabboshqaruv/service/AbsenceRequestService.java — 118–150-qatorlar

```
118     /**
119      * Sets every lesson of the requested days to EXCUSED; returns how many lesson
120      * records it touched. Holidays and vacations are skipped — there are no lessons to excuse.
121      */
122     int excuse(AbsenceRequest r) {
123         Student student = r.getStudent();
124         List<LessonSlot> slots = lessonSlotRepository.findBySchoolClassId(student.getSchoolClass().get
    Id());
125         List<CalendarEvent> daysOff = calendarEventRepository.findInRange(r.getSchool().getId(), r.get
    DateFrom(), r.getDateTo())
126                 .stream().filter(e -> e.getType() == CalendarEventType.HOLIDAY || e.getType() == Calen
    darEventType.VACATION).toList();
127         int count = 0;
128         for (LocalDate d = r.getDateFrom(); !d.isAfter(r.getDateTo()); d = d.plusDays(1)) {
129             LocalDate day0 = d;
130             if (daysOff.stream().anyMatch(e -> !day0.isBefore(e.getStartDate()) && !day0.isAfter(e.get
    EndDate()))) continue;
131             String weekday = ParentStats.WEEKDAY_UZ.get(d.getDayOfWeek());
132             for (LessonSlot slot : slots) {
133                 if (!slot.getWeekday().equals(weekday)) continue;
134                 LocalDate day = d;
135                 Attendance a = attendanceRepository.findByLessonSlotIdAndStudentIdAndRecordDate(slot.g
    etId(), student.getId(), d)
136                         .orElseGet(() -> {
137                             Attendance fresh = new Attendance();
138                             fresh.setLessonSlot(slot);
139                             fresh.setStudent(student);
140                             fresh.setRecordDate(day);
141                             return fresh;
142                         });
143                 a.setStatus(AttendanceStatus.EXCUSED);
144                 if (a.getComment() == null) a.setComment("Sababli ariza #" + r.getId());
145                 attendanceRepository.save(a);
146                 count++;
147             }
148         }
149         return count;
150     }
```

- **124** — sinfning butun haftalik jadvali bitta so'rovda.
- **125–126, 130** — bayram va ta'til kunlari o'tkazib yuboriladi: u kunlarda dars yo'q.
- **133** — faqat shu hafta kuniga to'g'ri keladigan darslar.
- **135–142** — davomat yozuvi bo'lsa yangilanadi, bo'lmasa (kelajak kun) **oldindan yaratiladi**.
- **143** — holat `EXCUSED`.

Bitta ariza ko'pi bilan 14 kunlik bo'lishi mumkin (`MAX_DAYS = 14`). Allaqachon ko'rib chiqilgan arizani qayta hal qilib bo'lmaydi (`requirePending` → 409).

Metod	URL	Rol	Java metodi	Nima qiladi
GET	/api/attendance/calendar	ADMIN, EDITOR, VIEWER	calendar()	Sinfning oylik davomat kalendari
PUT	/api/attendance/day	ADMIN, EDITOR	markDay()	O'quvchining butun kunini belgilaydi (barcha darslari)
GET	/api/attendance	ADMIN, EDITOR, VIEWER	getAllAttendance()	Davomat yozuvi ro'yxati (sahifalab)
GET	/api/attendance/today-lessons	ADMIN, EDITOR, VIEWER	getTodayLessons()	Bugungi darslar (o'qituvchi uchun)
GET	/api/attendance/roster	ADMIN, EDITOR, VIEWER	getRoster()	Dars ro'yxati: sinf o'quvchilari va ularning holati
POST	/api/attendance/bulk	ADMIN, EDITOR	bulkMark()	Butun sinf davomatini bitta so'rovda saqlaydi
POST	/api/attendance	ADMIN, EDITOR	createAttendance()	Yangi davomat yozuvi yaratadi
PUT	/api/attendance/{id}	ADMIN, EDITOR	updateAttendance()	Davomat yozuvini tahrirlaydi
DELETE	/api/attendance/{id}	ADMIN	deleteAttendance()	Davomat yozuvini o'chiradi
GET	/api/grades	ADMIN, EDITOR, VIEWER	getAllGrades()	Baho ro'yxati (sahifalab)
GET	/api/grades/gradebook	ADMIN, EDITOR, VIEWER	getGradebook()	Baholar daftari: sinf · fan · sanalar oralig'i
GET	/api/grades/journal	ADMIN, EDITOR, VIEWER	journal()	Sinf jurnali: bir oy · bir fan
POST	/api/grades	ADMIN, EDITOR	createGrade()	Yangi baho yaratadi
PUT	/api/grades/{id}	ADMIN, EDITOR	updateGrade()	Bahoni tahrirlaydi
DELETE	/api/grades/{id}	ADMIN, EDITOR	deleteGrade()	Bahoni o'chiradi
GET	/api/classes/{id}/overview	ADMIN, EDITOR, VIEWER	overview()	Sinf paneli: sinf rahbari, bugungi darslar
GET	/api/me/current-lesson	ADMIN, EDITOR, VIEWER	currentLesson()	Tizimga kirgan o'qituvchining hozirgi darsi

Metod	URL	Rol	Java metodi	Nima qiladi
GET	/api/absence-requests	ADMIN, EDITOR	list()	Ariza ro'yxati (sahifalab)
GET	/api/absence-requests/count-pending	ADMIN, EDITOR	countPending()	Kutilayotgan arizalar soni (menyu nishoni)
GET	/api/absence-requests/{id}/photo	ADMIN, EDITOR	photo()	Shifokor qog'ozi rasmini Telegram'dan oqim qilib beradi
POST	/api/absence-requests/{id}/approve	ADMIN, EDITOR	approve()	Arizani tasdiqlaydi — kunlar EXCUSED bo'ladi
POST	/api/absence-requests/{id}/reject	ADMIN, EDITOR	reject()	Arizani rad etadi (sabab bilan)
GET	/api/behavior-records	ADMIN, EDITOR, VIEWER	getAllRecords()	Xulq yozuvi ro'yxati (sahifalab)
POST	/api/behavior-records	ADMIN, EDITOR	createRecord()	Yangi xulq yozuvi yaratadi
PUT	/api/behavior-records/{id}	ADMIN, EDITOR	updateRecord()	Xulq yozuvini tahrirlaydi
DELETE	/api/behavior-records/{id}	ADMIN	deleteRecord()	Xulq yozuvini o'chiradi

📌 **Esda tut**

- Davomat va arizalar **upsert** qiladi: mavjud yozuv yangilanadi, yo'q bo'lsa yaratiladi; dublikatni `@UniqueConstraint` to'sadi.
- Ota-onaga xabar faqat holat **haqiqatan** ABSENT/LATE ga o'zgarganda — qoida `AttendanceMarkedEvent.isNotifiable` da.
- `@TransactionalEventListener(AFTER_COMMIT)` — xabar faqat commit'dan keyin; xatosi asosiy so'rovni buzmaydi.
- Vaqt `Clock` bean orqali olinadi — Toshkent vaqti va testda boshqariladigan "hozir".
- Jurnal va kalendar so'rovlari o'quvchilar soniga bog'liq emas (1–4 ta SQL) — testlar bilan kafolatlangan.

10. Modul: boshqaruv paneli, profillar, taqvim, e'lonlar

Bu bob saytning "o'qish" tomonini qamraydi: direktor tizimga kirganda ko'radigan **bosh sahifa**, o'quvchi, o'qituvchi va sinf **profillari**, **taqvim**, **e'lonlar** va **faoliyat lentasi**.

Modul	Entity	Service	Controller
Bosh sahifa	(bir nechta jadvaldan hisoblanadi)	`DashboardService`	`DashboardController`
Profillar	(Student, Employee, SchoolClass...)	`ProfileService`	`ProfileController`
Taqvim	`CalendarEvent`	`CalendarEventService`	`CalendarEventController`
E'lonlar	`Announcement`	`AnnouncementService`	`AnnouncementController`
Faoliyat	`ActivityLog`	`ActivityLogService`	`DashboardController.getRecentActivity`
Xabarnomalar sahifasi	`NotificationLog`, `NotificationSettings`	`NotificationQueryService`, `NotificationSettingsService`	`NotificationController`

Bosh sahifa: bitta so'rov — sakkizta vidjet

src/main/java/uz/azizbek/maktabboshqaruv/service/DashboardService.java — 487–502-qatorlar

```
487     /**
488      * Bundles every widget the dashboard needs into one payload so the page
489      * can load with a single request instead of six parallel round trips.
490      */
491     public DashboardOverviewDto getOverview(Long schoolId, int trendDays, int activityLimit) {
492         DashboardOverviewDto dto = new DashboardOverviewDto();
493         dto.setSummary(getSummary(schoolId));
494         dto.setLiveLessons(getLiveLessons(schoolId));
495         dto.setLiveLessonsSummary(getLiveLessonsSummary(schoolId));
496         dto.setAttendanceTrend(getAttendanceTrend(schoolId, trendDays));
497         dto.setClassRankings(getClassRankings(schoolId));
498         dto.setSubjectAverages(getSubjectAverages(schoolId));
499         dto.setAttention(getAttentionItems(schoolId));
500         dto.setActivity(getRecentActivity(schoolId, activityLimit));
501         return dto;
502     }
```

Sahifada 8 ta blok bor: umumiy sonlar, hozirgi darslar, jonli darslar xulosasi, davomat trendi, sinflar reytingi, fanlar bo'yicha o'rtacha, "e'tibor bering" ro'yxati, faoliyat. Ularning har biri uchun alohida endpoint bor (`/summary`, `/live-lessons`, ...). Lekin sahifa birinchi ochilganda frontend **bitta** `/overview` so'rovini yuboradi va hammasini birga oladi.

> 🤔 **Nega bitta "katta" endpoint?**
> Brauzer bir xil serverga bir vaqtda cheklangan miqdordagi so'rov yuboradi, har bir so'rov esa o'z tarmoq kechikishiga ega (ayniqsa mobil internetda). 8 ta kichik so'rov o'rniga bitta so'rov sahifani tezroq ochadi. Kichik endpoint'lar ham saqlab qolingan: masalan, faqat "jonli darslar" bloki har daqiqada alohida yangilanadi.

src/main/java/uz/azizbek/maktabboshqaruv/controller/DashboardController.java 12–24-qatorlar

```
12  @RequestMapping("/api/dashboard")
13  @PreAuthorize("hasAnyRole('ADMIN', 'EDITOR', 'VIEWER')")
14  public class DashboardController {
15
16      @Autowired
17      private DashboardService dashboardService;
18
19      @GetMapping("/overview")
20      public DashboardOverviewDto getOverview(@RequestParam Long schoolId,
21                                              @RequestParam(defaultValue = "30") int trendDays,
22                                              @RequestParam(defaultValue = "8") int activityLimit) {
23          return dashboardService.getOverview(schoolId, trendDays, activityLimit);
24      }
```

- **13** — `@PreAuthorize` **klass darajasida**: klassdagi barcha endpoint'lar uchun bitta qoida. Metod darajasidagi annotatsiya bo'lsa, u klassnikidan ustun turadi.
- **21–22** — `@RequestParam(defaultValue = "30")`: parametr berilmasa, standart qiymat ishlatiladi.

> 💡 **Ish kunlari va "bugun"**
>
> `DashboardService` maktab kalendarini hisobga oladi: faqat yakshanba dam olish kuni hisoblanadi (70-qator), davomat foizi esa **faqat davomat olingan sinflar** bo'yicha hisoblanadi (88-qator). Ertalab soat 8:00 da davomat hali olinmagan paytda foiz "0%" bo'lib qo'rqitmaydi, `null` ("hali ma'lumot yo'q") qaytadi.

Profillar

`ProfileService` uchta katta ko'rinish yig'adi:

Endpoint	Nima qaytaradi
`GET /api/profiles/students/{id}`	yosh, sinf, vasiy, **tug'ilgan joyi**, **to'liq yashash manzili**, 90 kunlik davomat foizi, o'rtacha baho, fanlar bo'yicha o'rtacha, davomat "issiqlik xaritasi", xulq yozuvlari, haftalik jadval
`GET /api/profiles/teachers/{id}`	telefon, lavozim, sinf rahbarligi, haftalik dars yuklamasi, fanlari, sinflari, bugungi darslari, haftalik jadvali
`GET /api/profiles/classes/{id}`	o'quv yili, sinf rahbari, o'quvchilar soni va ro'yxati, davomat foizi, o'rtacha baho, haftalik jadval

O'quvchi profili `@Transactional(readOnly = true)` bilan belgilangan: manzil maydonlari (`birthDistrict`, `residenceDistrict`) LAZY, ularni o'qish tranzaksiya ichida bo'lishi shart (4- va 7-boblar).

E'lonlar: yana bir event

src/main/java/uz/azizbek/maktabboshqaruv/service/AnnouncementService.java 58–71-qatorlar

```
58      @Transactional
59      public AnnouncementResponseDto createAnnouncement(AnnouncementRequestDto request) {
60          School school = schoolRepository.findById(request.getSchoolId())
61                  .orElseThrow(() -> new IllegalStateException("Bunday maktab mavjud emas"));
62
63          Announcement announcement = new Announcement();
64          announcement.setSchool(school);
65          applyFields(announcement, request);
66
```

```
        Announcement saved = announcementRepository.save(announcement);
        activityLogService.record(school, "campaign", "\"" + announcement.getTitle() + "\" e'loni joylandi");
        eventPublisher.publishEvent(new AnnouncementCreatedEvent(saved.getId()));
        return toResponseDto(saved);
    }
```

E'lon yaratilganda `AnnouncementCreatedEvent` chiqariladi va commit'dan keyin e'lon auditoriyasidagi ota-onalarga (butun maktab yoki bitta sinf) Telegram xabari navbatga qo'yiladi. **Tahrirlash** event chiqarmaydi: imlo xatosini to'g'rilash ota-onalarga ikkinchi xabar yubormaydi.

Faoliyat lentasi va bitta nozik tuzoq

src/main/java/uz/azizbek/maktabboshqaruv/service/ActivityLogService.java 12–35-qatorlar

```
/**
 * Thin recorder other services call, one line, right after a write they
 * want to surface on the dashboard's activity feed. Never throws — a
 * logging failure must not break the actual operation it's describing.
 */
@Service
public class ActivityLogService {

    @Autowired
    private ActivityLogRepository activityLogRepository;

    public void record(School school, String icon, String description) {
        try {
            ActivityLog log = new ActivityLog();
            log.setSchool(school);
            log.setIcon(icon);
            log.setDescription(description);
            log.setActorUsername(currentUsername());
            log.setOccurredAt(LocalDateTime.now());
            activityLogRepository.save(log);
        } catch (Exception ignored) {
            // best-effort — the actual write this describes already succeeded
        }
    }
```

Izohda shunday yozilgan: "*hech qachon xato tashlamaydi — log yozishdagi xato asosiy amalni buzmasligi kerak*". Niyat to'g'ri, lekin bu kafolat **to'liq emas**. Sababi:

> ⚠️ **try/catch tranzaksiyani qutqarmaydi**
>
> `record()` chaqiruvchining tranzaksiyasi **ichida** ishlaydi (masalan, `StudentService.createStudent`). Agar 31-qatordagi `save()` baza darajasida xato bersa (masalan, ustun uzunligi oshib ketsa), Spring tranzaksiyani **"faqat rollback"** (rollback-only) deb belgilaydi. `catch` xatoni yutib yuboradi, metod normal tugaydi. Lekin commit vaqtida Spring `UnexpectedRollbackException` tashlaydi va **o'quvchi ham saqlanmaydi**.
>
> To'g'ri yechim: `record()` ga `@Transactional(propagation = Propagation.REQUIRES_NEW)` qo'yish, ya'ni log alohida tranzaksiyada yozilsin. Yoki uni event orqali `AFTER_COMMIT` da yozish kerak.

> 🧩 **Hayotiy o'xshatish**
>
> Bu xuddi pochtaga xat topshirganda konvert ichiga yirtiq kvitansiya solib qo'yishga o'xshaydi: kvitansiya yirtiqligini ko'rmaslikka olsangiz ham, pochta butun konvertni qaytaradi. Kvitansiyani alohida konvertda (alohida tranzaksiya) yuborish kerak.

Metod	URL	Rol	Java metodi	Nima qiladi
GET	`/api/dashboard/overview`	ADMIN, EDITOR, VIEWER	`getOverview()`	Bosh sahifa: umumiy ko'rsatkichlar
GET	`/api/dashboard/summary`	ADMIN, EDITOR, VIEWER	`getSummary()`	Bosh sahifa: qisqa xulosa (sonlar)
GET	`/api/dashboard/live-lessons`	ADMIN, EDITOR, VIEWER	`getLiveLessons()`	Hozir o'tayotgan darslar
GET	`/api/dashboard/live-lessons-summary`	ADMIN, EDITOR, VIEWER	`getLiveLessonsSummary()`	Jonli darslar xulosasi
GET	`/api/dashboard/attendance-trend`	ADMIN, EDITOR, VIEWER	`getAttendanceTrend()`	Davomat trendi (kunlar bo'yicha)
GET	`/api/dashboard/class-rankings`	ADMIN, EDITOR, VIEWER	`getClassRankings()`	Sinflar reytingi
GET	`/api/dashboard/subject-averages`	ADMIN, EDITOR, VIEWER	`getSubjectAverages()`	Fanlar bo'yicha o'rtacha baho
GET	`/api/dashboard/absentees-today`	ADMIN, EDITOR, VIEWER	`getAbsenteesToday()`	Bugun kelmaganlar
GET	`/api/dashboard/attention`	ADMIN, EDITOR, VIEWER	`getAttentionItems()`	E'tibor talab qiladigan holatlar
GET	`/api/dashboard/activity`	ADMIN, EDITOR, VIEWER	`getRecentActivity()`	So'nggi faoliyat lentasi
GET	`/api/profiles/students/{id}`	ADMIN, EDITOR, VIEWER	`getStudentProfile()`	O'quvchi profili (manzil, davomat, baholar, xulq, jadval)
GET	`/api/profiles/teachers/{id}`	ADMIN, EDITOR, VIEWER	`getTeacherProfile()`	O'qituvchi profili
GET	`/api/profiles/classes/{id}`	ADMIN, EDITOR, VIEWER	`getClassProfile()`	Sinf profili

Metod	URL	Rol	Java metodi	Nima qiladi
GET	`/api/announcements`	ADMIN, EDITOR, VIEWER	`getAllAnnouncements()`	E'lon ro'yxati (sahifalab)
GET	`/api/announcements/latest`	ADMIN, EDITOR, VIEWER	`getLatest()`	Eng so'nggi e'lonlar
GET	`/api/announcements/{id}`	ADMIN, EDITOR, VIEWER	`getAnnouncementById()`	Bitta e'lon (id bo'yicha)
POST	`/api/announcements`	ADMIN, EDITOR	`createAnnouncement()`	Yangi e'lon yaratadi
PUT	`/api/announcements/{id}`	ADMIN, EDITOR	`updateAnnouncement()`	E'lonni tahrirlaydi
DELETE	`/api/announcements/{id}`	ADMIN	`deleteAnnouncement()`	E'lonni o'chiradi
GET	`/api/calendar-events`	ADMIN, EDITOR, VIEWER	`getAllEvents()`	Tadbir ro'yxati (sahifalab)
GET	`/api/calendar-events/range`	ADMIN, EDITOR, VIEWER	`getInRange()`	Sanalar oralig'idagi tadbirlar
GET	`/api/calendar-events/upcoming`	ADMIN, EDITOR, VIEWER	`getUpcoming()`	Kelgusi tadbirlar

Metod	URL	Rol	Java metodi	Nima qiladi
POST	`/api/calendar-events`	ADMIN, EDITOR	`createEvent()`	Yangi tadbir yaratadi
PUT	`/api/calendar-events/{id}`	ADMIN, EDITOR	`updateEvent()`	Tadbirni tahrirlaydi
DELETE	`/api/calendar-events/{id}`	ADMIN	`deleteEvent()`	Tadbirni o'chiradi
GET	`/api/health`	ochiq	`health()`	Backend ishlayaptimi (login'siz)
GET	`/api/notifications`	ADMIN, EDITOR	`getNotifications()`	Outbox xabarlari ro'yxati
GET	`/api/notifications/stats`	ADMIN, EDITOR	`getStats()`	Xabarnomalar statistikasi
POST	`/api/notifications/{id}/retry`	ADMIN, EDITOR	`retry()`	Xabarni qayta yuborish
GET	`/api/notifications/settings`	ADMIN, EDITOR	`getSettings()`	Xabarnoma sozlamalari
PUT	`/api/notifications/settings`	ADMIN	`updateSettings()`	Xabarnomani tahrirlaydi

📌 **Esda tut**

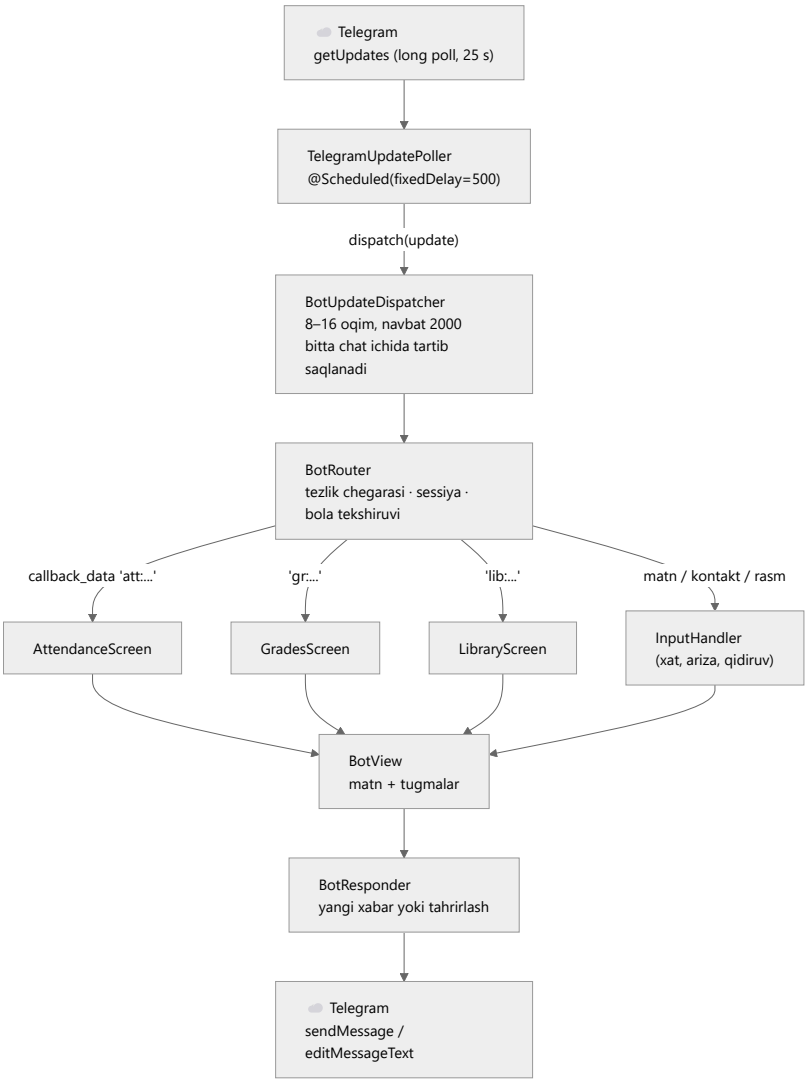
- Bosh sahifa bitta `/overview` so'rovi bilan yuklanadi, bloklar alohida ham yangilanadi.
- `@PreAuthorize` klassga qo'yilsa, klassdagi hamma metodlarga ta'sir qiladi.
- E'lon yaratish event chiqaradi, tahrirlash chiqarmaydi.
- Tranzaksiya ichidagi `try/catch` baza xatosidan keyin tranzaksiyani qutqarmaydi — alohida (`REQUIRES_NEW`) tranzaksiya kerak.

11. Modul: Telegram bot va xabarnomalar

Telegram qismi uchta paketga bo'lingan:

Paket	Vazifa	Asosiy klasslar
`telegram`	Bot API bilan **past darajadagi** aloqa: HTTP, JSON modellari, imzo, sozlamalar	`TelegramClient`, `HttpTelegramClient`, `MockTelegramClient`, `TelegramProperties`, `TelegramInitDataValidator`
`bot`	bot **karkasi**: update'larni qabul qilish, yo'naltirish, ekranni chizish, tillar	`BotUpdateDispatcher`, `BotRouter`, `BotResponder`, `Screen`, `CallbackData`, `BotI18n`
`bot.screens`	botning **15 ta sahifasi**	`HomeScreen`, `GradesScreen`, `AttendanceScreen`, …
`service`	xabarnomalar **outbox**i va rejalashtirilgan ishlar	`NotificationService`, `NotificationSender`, `BotScheduledJobs`, `LinkingService`

Kiruvchi yo'l: update'lardan ekrangacha



11.1-rasm. Ota-ona tugmani bosganda nima bo'ladi.

1. Long polling

src/main/java/uz/azizbek/maktabboshqaruv/service/TelegramUpdatePoller.java 45–90-qatorlar

```
45      @Scheduled(fixedDelay = 500, initialDelay = 3000)
46      public void poll() {
47          if (telegramProperties.mode() != TelegramProperties.Mode.LIVE) return;
48          if (clock.millis() < backoffUntilMillis) return;
49
50          List<TelegramModels.Update> updates;
51          try {
52              updates = telegramClient.getUpdates(offset, telegramProperties.getPollTimeoutSeconds());
53              lastError = null;
54          } catch (TelegramApiException e) {
55              onPollError(e);
56              return;
57          }
58
59          for (TelegramModels.Update update : updates) {
60              // Advance first: a message that crashes the handler must not be re-delivered forever.
61              offset = Math.max(offset, update.updateId() + 1);
62              // handled on the worker pool (per-chat order kept); the poll thread goes straight back to
    Telegram
63              dispatcher.dispatch(update);
64          }
65      }
66
67      private void onPollError(TelegramApiException e) {
68          String msg = TokenMasker.mask(e.getMessage(), telegramProperties.getBotToken());
69          long pauseSeconds;
70          switch (e.getErrorCode()) {
71              case 401, 404 -> {
72                  lastError = "Token noto'g'ri (Telegram " + e.getErrorCode() + ")";
73                  pauseSeconds = 300;
74              }
75              case 409 -> {
76                  lastError = "Boshqa nusxa yoki webhook ishlayapti (409)";
77                  pauseSeconds = 30;
78              }
79              case 429 -> {
80                  lastError = "Telegram limiti (429)";
81                  pauseSeconds = e.getRetryAfter() != null ? e.getRetryAfter() : 5;
82              }
83              default -> {
84                  lastError = "Telegram'ga ulanib bo'lmadi: " + msg;
85                  pauseSeconds = 10;
86              }
87          }
88          backoffUntilMillis = clock.millis() + pauseSeconds * 1000;
89          log.warn("Telegram getUpdates xatosi: {} — {} soniyadan so'ng qayta uriniladi", msg, pauseSecon
    ds);
90      }
```

- **45** — har 500 ms da (oldingi chaqiruv **tugagandan** keyin — `fixedDelay`) ishga tushadi.
- **52** — `getUpdates(offset, 25)`: Telegram yangi xabar bo'lmasa ulanishni 25 soniyagacha ochiq ushlaydi. Xabar kelishi bilan darhol javob beradi. Bu **long polling**: server ochiq port yoki HTTPS sertifikatsiz ham ishlaydi.
- **61** — `offset` **oldin** oshiriladi: biror xabar handler'ni "yiqitsa" ham u qayta-qayta kelavermaydi.
- **63** — update ishchi hovuzga uzatiladi va polling oqimi darhol Telegram'ga qaytadi.
- **67–90** — xatolarga qarab kutish: noto'g'ri token (401/404) — 5 daqiqa, boshqa nusxa ishlayapti (409) — 30 s, limit (429) — Telegram aytgancha.

> 🤔 **Nega spring.task.scheduling.pool.size=3?**
> Spring'ning `@Scheduled` vazifalari standart holatda **bitta** oqimda navbat bilan ishlaydi. `poll()` bu oqimni 25 soniyagacha band qiladi. Agar oqim bitta bo'lsa, shu vaqt davomida outbox (`NotificationSender`) va kundalik ishlar kutib turardi. Shu sababli 4-bobdagi sozlama hovuzni 3 ta oqimga kengaytiradi.

2. Ishchi hovuz va har chat uchun tartib

src/main/java/uz/azizbek/maktabboshqaruv/bot/BotUpdateDispatcher.java 28–61-qatorlar

```
28      private final BotRouter router;
29      private final ThreadPoolExecutor pool;
30      private final ConcurrentHashMap<Long, CompletableFuture<Void>> tails = new ConcurrentHashMap<>();
31
32      public BotUpdateDispatcher(BotRouter router,
33                                 @Value("${telegram.update-threads:8}") int threads,
34                                 @Value("${telegram.update-max-threads:16}") int maxThreads,
35                                 @Value("${telegram.update-queue:2000}") int queue) {
36          this.router = router;
37          AtomicInteger n = new AtomicInteger();
38          this.pool = new ThreadPoolExecutor(threads, Math.max(threads, maxThreads), 60, TimeUnit.SECOND
    S,
39                  new ArrayBlockingQueue<>(queue),
40                  r -> {
41                      Thread t = new Thread(r, "bot-update-" + n.incrementAndGet());
42                      t.setDaemon(true);
43                      return t;
44                  },
45                  new ThreadPoolExecutor.CallerRunsPolicy());
46      }
47
48      /** Queues one update; returns a future that completes when it has been handled. */
49      public CompletableFuture<Void> dispatch(TelegramModels.Update update) {
50          long chatId = chatOf(update);
51          CompletableFuture<Void> next = tails.compute(chatId, (k, tail) -> {
52              CompletableFuture<Void> previous = tail == null ? CompletableFuture.completedFuture(null) :
    tail;
53              // handle*, not then*: a failed update must not block the chat's next one
54              return previous.handleAsync((r, e) -> {
55                  run(update);
56                  return null;
57              }, pool);
58          });
59          next.whenComplete((r, e) -> tails.remove(chatId, next));
60          return next;
61      }
```

- **38–45** — `ThreadPoolExecutor` : 8 ta doimiy, 16 tagacha oqim, 2000 o'rinli **cheklangan** navbat. Navbat to'lsa, `CallerRunsPolicy` ishlaydi: polling oqimi update'ni o'zi bajaradi va shu orqali tabiiy ravishda sekinlashadi (**back-pressure**). Natijada xotira cheksiz o'smaydi.
- **51–59** — har chat uchun `CompletableFuture` zanjiri (`tails`): bitta ota-onaning ketma-ket bosishlari **tartib bilan**, turli ota-onalar esa **parallel** bajariladi.
- **54** — `handleAsync` (`thenAsync` emas): oldingi update xato bilan tugasa ham keyingisi bajariladi.

> 🧩 **Hayotiy o'xshatish**
>
> Pochta bo'limida 8 ta oyna (oqim) bor. Har bir mijozning (chat) xatlari bitta oynaga navbat bilan topshiriladi, chunki ularning tartibi muhim. Turli mijozlar esa turli oynalarda bir vaqtda xizmat oladi. Zal to'lib ketsa (navbat 2000), eshikdagi qorovul yangi kelganlarni o'zi qabul qila boshlaydi va eshik sekinroq ochiladi.

3. Router va ekranlar (Strategy pattern)

src/main/java/uz/azizbek/maktabboshqaruv/bot/Screen.java — 7–18-qatorlar

```
7  public interface Screen {
8
9      /** First segment of callback_data, e.g. "att". */
10     String code();
11
12     BotView render(BotContext ctx, CallbackData data);
13
14     /** Most pages show one child's data; the few that do not (settings, children) override this. */
15     default boolean requiresStudent() {
16         return true;
17     }
18 }
```

src/main/java/uz/azizbek/maktabboshqaruv/bot/BotRouter.java — 71–74-qatorlar

```
71     public BotRouter(List<Screen> screenBeans, List<InputHandler> inputHandlers) {
72         for (Screen s : screenBeans) screens.put(s.code(), s);
73         this.inputHandlers = inputHandlers;
74     }
```

Bu Spring'ning juda kuchli imkoniyati: konstruktor `List<Screen>` so'raydi va Spring `Screen` **interfeysini amalga oshirgan barcha bean'larni** (15 ta `@Component` ekran) ro'yxatga yig'ib beradi. Router ularni `code()` bo'yicha `Map` ga joylaydi. Yangi sahifa qo'shish uchun bitta yangi klass yozish kifoya: routerni o'zgartirish kerak emas. Bu — **Strategy** dizayn patterni va **Open/Closed** tamoyili.

src/main/java/uz/azizbek/maktabboshqaruv/bot/BotRouter.java — 86–135-qatorlar

```
86      void handleCallback(TelegramModels.CallbackQuery cq) {
87          if (cq.message() == null || cq.message().chat() == null || !"private".equals(cq.message().chat
    ().type())) {
88              answer(cq.id(), null, false);
89              return;
90          }
91          long chatId = cq.message().chat().id();
92          ParentSession session = null;
93          try {
94              if (!rateLimiter.allow(chatId)) {
95                  answer(cq.id(), BotI18n.get().t(access.session(chatId).lang(), "common.slow_down"), fa
    lse);
96                  return;
97              }
98              CallbackData data = CallbackData.parse(cq.data());
99              if ("noop".equals(data.screen())) {
100                 answer(cq.id(), null, false);
101                 return;
102             }
103             session = touch(chatId, cq.from());
104             // A new tap elsewhere abandons a half-finished conversation.
105             Screen screen = screens.getOrDefault(data.screen(), screens.get("home"));
106             if (session.getPendingAction() != null && !ownsPending(screen, session.getPendingAction
    ())) {
107                 session.setPendingAction(null);
```

```
                session.setPendingData(null);
                access.save(session);
            }

            List<Student> students = access.linkedStudents(chatId);
            if (students.isEmpty()) {
                answer(cq.id(), null, false);
                BotContext ctx = new BotContext(chatId, null, session, null, students, cq.from());
                responder.deliver(chatId, null, decorate(ctx, onboarding.welcome(ctx), null));
                return;
            }
            Student student = data.get("s") != null
                    ? access.requireLinked(chatId, data.getLong("s"))
                    : access.selected(session);
            if (data.get("s") != null && !student.getId().equals(session.getSelectedStudentId())) {
                session.setSelectedStudentId(student.getId());
                access.save(session);
            }

            if (isSlowText(data)) {
                client.sendChatAction(chatId, "typing");
            }
            // Buttons under an automatic notification open a new page instead of overwriting the noti
fication.
            Long messageId = "1".equals(data.get("n")) ? null : cq.message().messageId();
            BotContext ctx = new BotContext(chatId, messageId, session, student, students, cq.from());
            BotView view = decorate(ctx, render(ctx, screen, data), data);
            answer(cq.id(), view.toast(), false);
            responder.deliver(chatId, messageId, view, messageId != null && cq.message().photo() != nu
ll);
```

- **94** — `ChatRateLimiter`: bitta chatdan soniyasiga bir nechtadan ortiq bosish bo'lsa, "sekinroq" deb javob beriladi. Bu bazani himoya qiladi.
- **98** — `callback_data` tahlili: `att:s:15:k:m:m:2026-10` → ekran `att`, parametrlar `s=15`, `k=m`, `m=2026-10`.
- **119–121** — **xavfsizlik:** tugmadagi o'quvchi id'siga (`s`) ishonilmaydi. `access.requireLinked(chatId, id)` bu bola haqiqatan **shu chatga bog'langan**ligini tekshiradi. Callback ma'lumotini Telegram mijozi orqali soxtalashtirish mumkin, shuning uchun bu tekshiruv shart.

src/main/java/uz/azizbek/maktabboshqaruv/bot/CallbackData.java — 7–39-qatorlar

```
/**
 * Compact, structured callback_data: {@code screen[:key:value]...}, e.g.
 * {@code att:k:m:m:2026-09:v:cal}. Telegram limits callback_data to 64 bytes,
 * which {@link #toString()} enforces. Values never contain ':'.
 *
 * Nothing parsed from here is trusted: a student id ("s") is re-checked
 * against the chat's links by the router before any data is read.
 */
public final class CallbackData {

    public static final int MAX_BYTES = 64;

    private final String screen;
    private final Map<String, String> params;

    private CallbackData(String screen, Map<String, String> params) {
        this.screen = screen;
        this.params = params;
    }

    public static CallbackData of(String screen) {
        return new CallbackData(screen, new LinkedHashMap<>());
```

```
    }

    public static CallbackData parse(String raw) {
        if (raw == null || raw.isBlank()) return of("home");
        String[] parts = raw.split(":");
        Map<String, String> params = new LinkedHashMap<>();
        for (int i = 1; i + 1 < parts.length; i += 2) {
            params.put(parts[i], parts[i + 1]);
        }
        return new CallbackData(parts[0], params);
    }
```

Telegram `callback_data` ni **64 bayt** bilan cheklaydi. `CallbackData` uni ixcham `ekran:kalit:qiymat` ko'rinishida saqlaydi va chegaradan oshib ketmasligini tekshiradi (`CallbackDataTest`).

4. Ota-onani o'quvchiga bog'lash

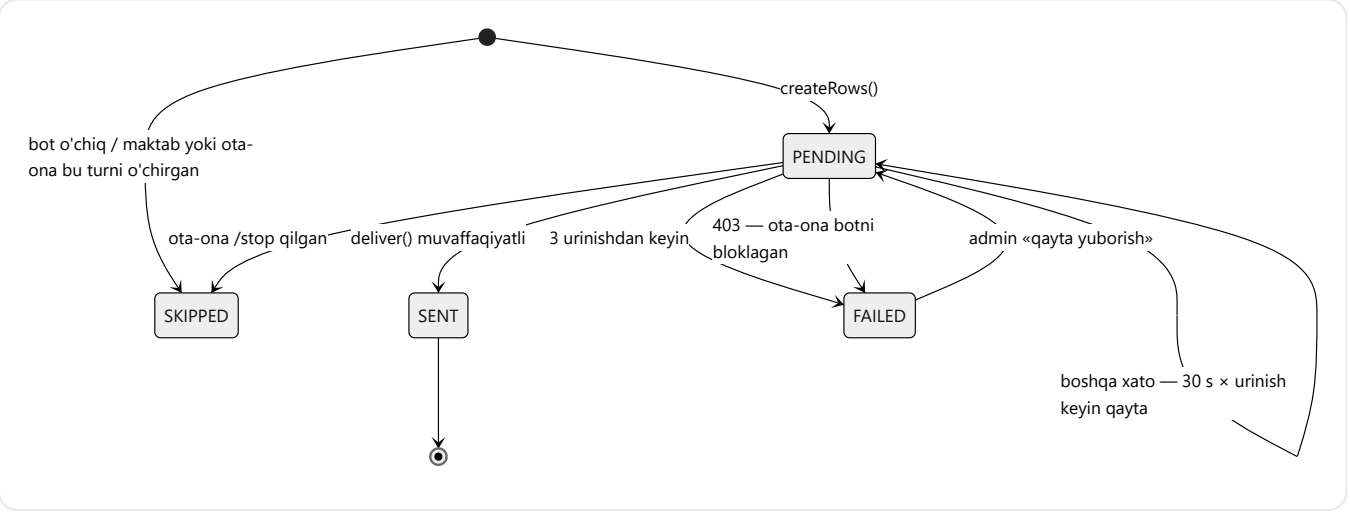
Har bir o'quvchida noyob **bog'lash kodi** bor (`students.telegram_link_code`). Ota-ona `t.me/<bot>?start=<KOD>` havolasini ochadi (yoki QR kodni skanerlaydi) yoki telefon raqamini ulashadi. `LinkingService` kodni yoki raqamni tekshiradi va `parent_telegram_link` qatorini yaratadi.

src/main/java/uz/azizbek/maktabboshqaruv/telegram/LinkCodeGenerator.java — 12–27-qatorlar

```
12  public final class LinkCodeGenerator {
13
14      private static final String ALPHABET = "ABCDEFGHJKLMNPQRSTUVWXYZabcdefghijkmnopqrstuvwxyz23456789";
15      public static final int LENGTH = 10;
16      private static final SecureRandom RANDOM = new SecureRandom();
17
18      private LinkCodeGenerator() {
19      }
20
21      public static String generate() {
22          StringBuilder sb = new StringBuilder(LENGTH);
23          for (int i = 0; i < LENGTH; i++) {
24              sb.append(ALPHABET.charAt(RANDOM.nextInt(ALPHABET.length())));
25          }
26          return sb.toString();
27      }
```

- **14** — 55 ta belgili alifbo: o'xshash belgilar (`0/O` , `1/l/I`) chiqarib tashlangan, chunki ota-ona kodni qog'ozdan terishi mumkin.
- **15–16** — 10 ta belgi, `SecureRandom` (kriptografik tasodif). Kombinatsiyalar soni $55^{10} \approx 2 \cdot 10^{17}$ (≈58 bit), shuning uchun kodni taxmin qilib yoki ketma-ket sanab topib bo'lmaydi. Oddiy `Random` bunday emas: uning keyingi qiymatlarini oldingilaridan hisoblab chiqarish mumkin.

Chiquvchi yo'l: outbox pattern



11.2-rasm. `notification_log` qatorining hayot sikli.

Navbatga yozish

src/main/java/uz/azizbek/maktabboshqaruv/service/NotificationService.java — 82–114-qatorlar

```
82      @Transactional(propagation = Propagation.REQUIRES_NEW)
83      public int enqueueAttendance(AttendanceMarkedEvent event) {
84          if (!AttendanceMarkedEvent.isNotifiable(event.previousStatus(), event.newStatus())) {
85              return 0;
86          }
87          Attendance attendance = attendanceRepository.findById(event.attendanceId()).orElse(null);
88          // Re-marked again (or deleted) before we got here — the newer save decides.
89          if (attendance == null || attendance.getStatus() != event.newStatus()) {
90              return 0;
91          }
92
93          NotificationType type = attendance.getStatus() == AttendanceStatus.ABSENT
94                  ? NotificationType.ATTENDANCE_ABSENT : NotificationType.ATTENDANCE_LATE;
95          Student student = attendance.getStudent();
96          if (notificationLogRepository.existsByStudentIdAndTypeAndReferenceIdAndRecordDate(
97                  student.getId(), type, attendance.getId(), attendance.getRecordDate())) {
98              log.debug("Takroriy xabar o'tkazib yuborildi: {} attendance={}", type, attendance.getId
    ());
99              return 0;
100         }
101
102         LessonSlot slot = attendance.getLessonSlot();
103         School school = schoolOf(student);
104         LocalDate today = LocalDate.now(clock);
105         Integer lessonNumber = lessonNumber(slot);
106         Function<String, String> text = lang -> type == NotificationType.ATTENDANCE_ABSENT
107                 ? MessageFormatter.attendanceAbsent(lang, fullName(student), className(student), lesso
    nNumber,
108                 slot.getSubject().getName(), slot.getStartTime(), attendance.getRecordDate(), today, s
    chool.getName())
109                 : MessageFormatter.attendanceLate(lang, fullName(student), className(student), lessonN
    umber,
110                 slot.getSubject().getName(), slot.getStartTime(), attendance.getRecordDate(), today, s
    chool.getName());
111
112         return createRows(school, type, attendance.getId(), attendance.getRecordDate(), recipientsOf(s
    tudent), text,
113                 actions("att:s:" + student.getId() + ":n:1", "/attendance", student.getId()));
114     }
```

- **82** — `REQUIRES_NEW`: tinglovchi commit'dan keyin chaqiriladi (9-bob), shuning uchun yangi tranzaksiya kerak.
- **89** — event'dan keyin holat yana o'zgargan bo'lsa (o'qituvchi "kelmadi"ni darhol "keldi"ga tuzatgan bo'lsa), xabar yuborilmaydi.
- **96–97** — **dublikatga qarshi kalit**: (o'quvchi, tur, yozuv, sana) bo'yicha xabar allaqachon navbatda bo'lsa, ikkinchisi yaratilmaydi.
- **106–110** — matn **funksiya** sifatida beriladi (`lang -> ...`): har bir ota-ona o'z tilida (uz/cy/ru) oladi.

src/main/java/uz/azizbek/maktabboshqaruv/service/NotificationService.java 280–319-qatorlar

```
280         for (Recipient r : recipients) {
281             ParentSession session = sessions.get(r.chatId());
282             String lang = session == null ? BotI18n.DEFAULT_LANG : session.lang();
283 
284             NotificationStatus status = NotificationStatus.PENDING;
285             String skipReason = null;
286             if (!telegramProperties.isActive()) {
287                 status = NotificationStatus.SKIPPED;
288                 skipReason = telegramProperties.mode() == TelegramProperties.Mode.NO_TOKEN
289                         ? "Bot tokeni berilmagan (TELEGRAM_BOT_TOKEN)"
290                         : "Telegram o'chirilgan (TELEGRAM_ENABLED=false)";
291             } else if (!settings.isTypeEnabled(type)) {
292                 status = NotificationStatus.SKIPPED;
293                 skipReason = "Maktab sozlamalarida bu turdagi xabarlar o'chirilgan";
294             } else if (session != null && !session.wants(type)) {
295                 status = NotificationStatus.SKIPPED;
296                 skipReason = "Ota-ona bot sozlamalarida bu turdagi xabarlarni o'chirgan";
297             }
298 
299             LocalDateTime scheduledAt = type.onDemand() ? now : deliveryTime(now, settings, session);
300 
301             NotificationLog row = new NotificationLog();
302             row.setSchool(school);
303             row.setStudent(r.student());
304             row.setChatId(r.chatId());
305             row.setType(type);
306             row.setReferenceId(referenceId);
307             row.setRecordDate(recordDate);
308             row.setText(text.apply(lang));
309             row.setReplyMarkup(markup == null ? null : TelegramJson.write(markup.apply(lang)));
310             row.setStatus(status);
311             row.setAttempts(0);
312             row.setLastError(skipReason);
313             row.setCreatedAt(now);
314             row.setScheduledAt(scheduledAt);
315             row.setMedia(media);
316             // Held back by quiet hours: goes out in the single morning note with the others
317             // (a message with files keeps its own message).
318             if (scheduledAt.isAfter(now) && status == NotificationStatus.PENDING && media == null) ro
    w.setQuietBundle(true);
319             notificationLogRepository.save(row);
```

Har bir qabul qiluvchi uchun alohida qator: bot o'chiq bo'lsa (286), maktab bu turni o'chirgan bo'lsa (291) yoki ota-onaning o'zi o'chirgan bo'lsa (294) — `SKIPPED`. **Tinch soatlar** (299): kechasi kelgan xabar ertalabga rejalashtiriladi va ertalab bitta " 🌙 Tunda kelgan xabarlar" xabariga jamlanadi (318).

Yuborish

src/main/java/uz/azizbek/maktabboshqaruv/service/NotificationSender.java 79–112-qatorlar

```
79     // short pause between passes; throttleGlobal() keeps the rate at max-per-second (~25/s) anyway
80     @Scheduled(fixedDelayString = "${telegram.send-interval-ms:250}", initialDelayString = "${telegra
    m.send-initial-delay-ms:5000}")
```

```
81      public void tick() {
82          if (!telegramProperties.isActive()) return;
83          try {
84              sendDue();
85          } catch (Exception e) {
86              log.warn("Xabarnomalarni yuborishda xato: {}", TokenMasker.mask(e.getMessage()));
87          }
88      }
89
90      /** One pass over due messages. Returns how many were delivered. */
91      public synchronized int sendDue() {
92          if (clock.millis() < pausedUntilMillis) return 0;
93
94          int batch = Math.max(1, telegramProperties.getMaxPerSecond());
95          List<NotificationLog> due = notificationLogRepository.findDue(LocalDateTime.now(clock), PageRe
    quest.of(0, batch));
96          Set<Long> chatsThisPass = new HashSet<>();
97          int delivered = 0;
98          for (NotificationLog n : due) {
99              if (clock.millis() < pausedUntilMillis) break;
100             // Per-chat limit: a second message to the same chat waits for the next pass.
101             if (!chatsThisPass.add(n.getChatId())) continue;
102             Long last = lastSentPerChat.get(n.getChatId());
103             if (last != null && clock.millis() - last < PER_CHAT_INTERVAL_MS) continue;
104
105             throttleGlobal();
106             if (Boolean.TRUE.equals(n.getQuietBundle())) {
107                 delivered += deliverQuietBundle(n.getChatId());
108             } else if (deliver(n)) {
109                 delivered++;
110             }
111         }
112         return delivered;
```

- **80** — har 250 ms (oldingisi tugagach).
- **94** — bir martada ko'pi bilan 25 ta xabar (`maxPerSecond`).
- **101–103** — **har chatga soniyasiga 1 ta**: Telegram talabi.
- **105** — `throttleGlobal()` : umumiy tezlik soniyasiga ~25 ta (Telegram limiti ~30).

src/main/java/uz/azizbek/maktabboshqaruv/service/NotificationSender.java 218–253-qatorlar

```
218     void handleFailure(NotificationLog n, TelegramApiException e) {
219         String reason = TokenMasker.mask(e.getMessage(), telegramProperties.getBotToken());
220
221         if (e.isFloodLimit()) {
222             int wait = e.getRetryAfter() != null && e.getRetryAfter() > 0 ? e.getRetryAfter() : 5;
223             pausedUntilMillis = clock.millis() + wait * 1000L;
224             n.setLastError("429: Telegram limiti — " + wait + " soniya kutilmoqda");
225             n.setScheduledAt(LocalDateTime.now(clock).plusSeconds(wait));
226             notificationLogRepository.save(n);
227             log.warn("Telegram 429 (flood limit): yuborish {} soniyaga to'xtatildi", wait);
228             return;
229         }
230
231         n.setAttempts(n.getAttempts() + 1);
232
233         if (e.isBlocked()) {
234             n.setStatus(NotificationStatus.FAILED);
235             n.setLastError("403: Ota-ona botni bloklagan — bog'lanish o'chirildi");
236             notificationLogRepository.save(n);
237             int links = linkRepository.deactivateByChatId(n.getChatId());
238             botCache.evictChat(n.getChatId());
239             notificationLogRepository.skipPendingForChat(n.getChatId(), "Ota-ona botni bloklagan");
```

```
            log.info("Telegram 403: chat={} botni bloklagan, {} ta bog'lanish o'chirildi", n.getChatId(), links);
            return;
        }

        n.setLastError((e.getErrorCode() > 0 ? e.getErrorCode() + ": " : "") + reason);
        if (n.getAttempts() >= telegramProperties.getMaxAttempts()) {
            n.setStatus(NotificationStatus.FAILED);
            log.warn("Xabarnoma #{} {} urinishdan keyin FAILED: {}", n.getId(), n.getAttempts(), reason);
        } else {
            n.setScheduledAt(LocalDateTime.now(clock).plusSeconds(RETRY_BACKOFF_SECONDS * n.getAttempts()));
            log.info("Xabarnoma #{} yuborilmadi ({}-urinish), qayta uriniladi: {}", n.getId(), n.getAttempts(), reason);
        }
        notificationLogRepository.save(n);
    }
```

Telegram javobi	Ma'nosi	Loyiha nima qiladi
429 Too Many Requests	limitdan oshdi	butun yuboruvchi `retry_after` soniyaga to'xtaydi; xabar `PENDING` qoladi, urinish **sanalmaydi**
403 Forbidden	ota-ona botni bloklagan	`FAILED`; shu chatning barcha bog'lanishlari o'chiriladi; navbatdagi qolgan xabarlari `SKIPPED`
boshqa xato	tarmoq va h.k.	30 s × urinish soni keyin qayta; 3-urinishdan keyin `FAILED`

> ⚠️ **Bitta nusxa (instance) faraziyasi**
>
> `sendDue()` `synchronized` — bu faqat **bitta JVM** ichida ishlaydi. Agar ilovaning ikki nusxasi bir bazada parallel ishga tushirilsa, ikkalasi bir xil `PENDING` qatorni o'qib, xabarni ikki marta yuborishi mumkin. Shuningdek, ikki nusxa bitta bot tokeni bilan `getUpdates` qilsa, Telegram 409 qaytaradi. Hozirgi o'rnatishda nusxa bitta, shuning uchun muammo yo'q. Gorizontal kengaytirish uchun `SELECT ... FOR UPDATE SKIP LOCKED` kerak bo'ladi.

Rejalashtirilgan ishlar (@Scheduled)

Klass.metod	Jadval	Nima qiladi
`TelegramUpdatePoller.poll`	har 0,5 s	long polling
`NotificationSender.tick`	har 250 ms	outbox'ni yuborish
`BotScheduledJobs`	har 5 daqiqa	ertangi jadval (19:00), haftalik hisobot (shanba 18:00), tadbir eslatmasi (bir kun oldin), ertalabki digest
`BotUsageService`	har 5 s	bot statistikasi hodisalarini bazaga to'plab yozish
`AccountStatusService.returnFromLeave`	har kuni 00:05 (Toshkent)	ta'tili tugaganlarni "Ishda"ga qaytarish
`LibraryPermissions`	har kuni 06:15 (Toshkent)	muddati o'tgan ruxsatnomalar kitoblarini nofaol qilish

`@EnableScheduling` `TelegramConfig` da yoqilgan. `cron` li vazifalarda `zone = "Asia/Tashkent"` berilgan: server qaysi vaqt zonasida bo'lishidan qat'i nazar, 00:05 Toshkent vaqti bo'yicha hisoblanadi.

🔬 **BotScheduledJobs nega har 5 daqiqada, aniq 19:00 da emas?**
Har bir ota-ona "ertangi jadval" vaqtini o'zi tanlaydi (18:30, 19:00, 20:00...). Shuning uchun vazifa har 5 daqiqada uyg'onadi va "vaqti kelgan, lekin hali yuborilmagan" xabarlarni topadi. Xabar o'z vaqtidan boshlab 3 soat ichida yuboriladi, dublikat kaliti esa har xabarni **bir marta** yuboradi. Server o'sha paytda o'chiq bo'lsa ham, qayta yonganda xabarlar baribir boradi.

Mock rejim: Telegram'siz ishlab chiqish

`TelegramConfig.telegramClient` (`@Bean`) rejimga qarab uch xil `TelegramClient` qaytaradi:

Rejim	Shart	Mijoz
`MOCK`	`telegram.mock=true`	`MockTelegramClient` — hech narsa tashqariga chiqmaydi, "yuborilgan" xabarlar xotirada; `/api/telegram/mock/...` orqali soxta ota-ona xabarini berish mumkin
`LIVE`	`enabled=true` + token bor	`HttpTelegramClient` (Spring `RestClient` ustida, uchinchi tomon kutubxonasisiz)
`NO_TOKEN` / `DISABLED`	aks holda	hech narsa yubormaydigan mijoz; outbox qatorlari `SKIPPED`

Bu — **interfeysga dasturlash** (programming to an interface): `NotificationSender` faqat `TelegramClient` interfeysini biladi, qaysi amalga oshirilishi ishlatilayotganini bilmaydi. Testlar (`TelegramMockFlowIntegrationTest`, `BotScreensSeedIntegrationTest`) va yuklama testi (3000 ta ota-ona) shu tufayli internetsiz ishlaydi.

Maxfiylik: token hech qayerga yozilmaydi

`TokenMasker` har qanday log satri yoki bazaga yoziladigan xato matnidan bot tokenini olib tashlaydi (`HttpTelegramClient` xatolari URL'da tokenni o'z ichiga olishi mumkin). Ilova ishga tushganda logda faqat `token: ***` ko'rinadi.

Metod	URL	Rol	Java metodi	Nima qiladi
GET	`/api/bot/stats`	ADMIN, EDITOR	`stats()`	Bot statistikasi
GET	`/api/bot/settings`	ADMIN, EDITOR	`getSettings()`	Bot sozlamalari
PUT	`/api/bot/settings`	ADMIN	`updateSettings()`	Botni tahrirlaydi
POST	`/api/bot/mock/run-jobs`	ADMIN	`runJobs()`	Mock rejimda avtomatik xabarlarni hozir ishga tushiradi
GET	`/api/telegram/status`	ADMIN, EDITOR, VIEWER	`getStatus()`	Telegram bot holati (rejim, ulanish)
GET	`/api/telegram/students/{studentId}`	ADMIN, EDITOR	`getStudentTelegram()`	O'quvchining bog'lash kodi va ulangan ota-onalari
POST	`/api/telegram/students/{studentId}/regenerate-code`	ADMIN, EDITOR	`regenerateCode()`	O'quvchiga yangi bog'lash kodi
DELETE	`/api/telegram/links/{linkId}`	ADMIN, EDITOR	`unlink()`	Telegramni o'chiradi
GET	`/api/telegram/classes/{schoolClassId}/codes`	ADMIN, EDITOR	`getClassCodes()`	Sinf o'quvchilarining kodlari (chop etish uchun)

Metod	URL	Rol	Java metodi	Nima qiladi
GET	`/api/telegram/files/{fileId}`	ADMIN, EDITOR	`file()`	Ota-ona yuborgan rasm
POST	`/api/telegram/mock/updates`	ADMIN	`simulateUpdate()`	Mock: soxta ota-ona xabarini botga beradi
GET	`/api/telegram/mock/messages`	ADMIN	`mockMessages()`	Mock: bot «yuborgan» xabarlar
GET	`/api/telegram/mock/photos/{fileId}`	ADMIN	`mockPhoto()`	Mock: bot «yuborgan» rasm
GET	`/api/telegram/parents`	ADMIN, EDITOR, VIEWER	`byClass()`	Sinf bo'yicha ota-onalar (ulangan / ulanmagan)
GET	`/api/telegram/parents/choices`	ADMIN	`choices()`	Xabar uchun ota-ona tanlash ro'yxati

📌 **Esda tut**

- Kiruvchi: long polling → cheklangan hovuz (har chat ichida tartib) → `BotRouter` → `Screen` → `BotView`.
- Spring `List<Screen>` ni avtomatik yig'adi — yangi ekran routerni o'zgartirmasdan qo'shiladi.
- Tugmadagi id'ga ishonilmaydi: bola har safar chatga bog'langanligi bo'yicha qayta tekshiriladi.
- Chiquvchi: outbox (`notification_log`) — dublikatsiz, tinch soatlar bilan, Telegram limitlariga (25/s, chatga 1/s) rioya qilib.
- 429 — kutish, 403 — bog'lanishni o'chirish, boshqa xatolar — 3 tagacha urinish.
- `TelegramClient` interfeysi tufayli mock rejimda hamma narsa internetsiz sinaladi.

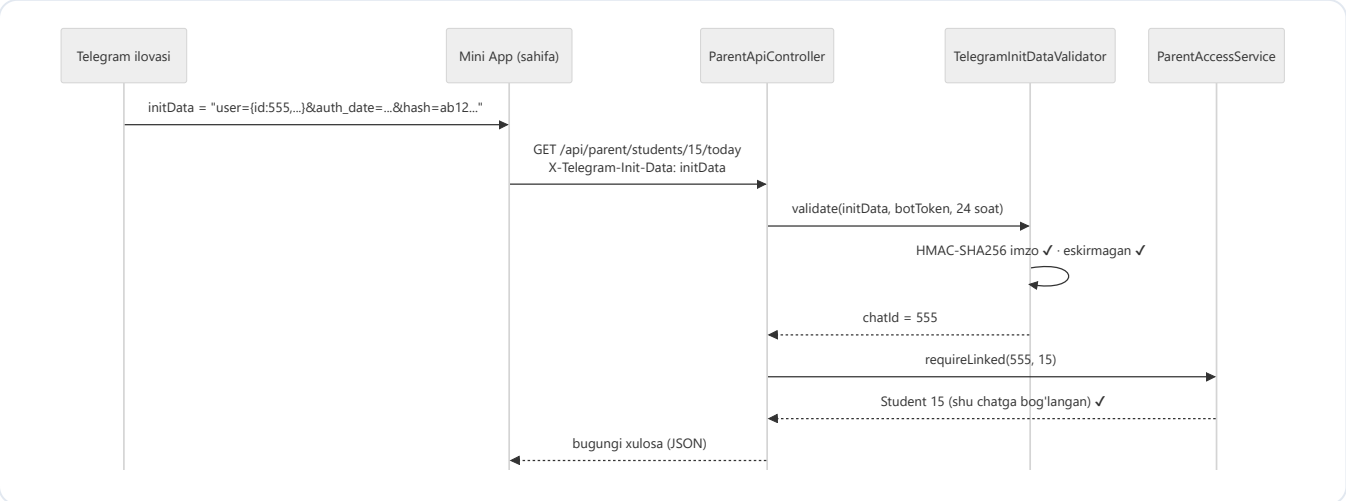
12. Modul: ota-onalar — murojaatlar, ommaviy xabarlar, Mini App

Bu bob ota-ona va maktab orasidagi uch xil aloqani ko'radi:

Yo'nalish	Nima	Klasslar
Ota-ona → maktab	**murojaat** (" Ma'muriyatga xat"): matn, rasm, video, ovoz, hujjat	`Appeal`, `AppealMessage`, `AppealService`, `AppealFiles`, `AttachmentPolicy`, `FileStorage`, `AppealController`
Maktab → ota-onalar	**ommaviy xabar**: butun maktab, sinflar yoki tanlangan ota-onalarga, 10 tagacha fayl bilan	`Broadcast`, `BroadcastAttachment`, `BroadcastService`, `BroadcastMedia`, `BroadcastController`
Ota-ona ← maktab ma'lumoti	**Mini App** "Farzandim kundaligi": bolaning bugungi kuni, jadvali, davomati, baholari	`ParentApiController`, `ParentAccessService`, `ParentDataService`, `ParentViews`, `ParentStats`

Mini App API: JWT'siz himoya

Mini App — Telegram ichida ochiladigan veb-sahifa. Ota-ona saytga login qilmaydi. Uning o'rniga Telegram har bir Mini App ochilishida ota-ona haqida **imzolangan** ma'lumot (`initData`) beradi. Backend shu imzoni tekshiradi.



12.1-rasm. Mini App so'rovi: Telegram imzosi → chat id → faqat o'z farzandi.

src/main/java/uz/azizbek/maktabboshqaruv/telegram/TelegramInitDataValidator.java 46–91-qatorlar

```
46      public static WebAppUser validate(String initData, String botToken, long maxAgeSeconds, Instant no
        w) {
47          if (initData == null || initData.isBlank()) throw new InvalidInitData("initData yo'q");
48          if (botToken == null || botToken.isBlank()) throw new InvalidInitData("Bot tokeni sozlanmaga
        n");
49
50          Map<String, String> fields = new TreeMap<>();
51          for (String pair : initData.split("&")) {
52              int eq = pair.indexOf('=');
53              if (eq <= 0) continue;
54              fields.put(URLDecoder.decode(pair.substring(0, eq), StandardCharsets.UTF_8),
55                      URLDecoder.decode(pair.substring(eq + 1), StandardCharsets.UTF_8));
```

```
56          }
57          String hash = fields.remove("hash");
58          if (hash == null) throw new InvalidInitData("hash yo'q");
59
60          StringBuilder check = new StringBuilder();
61          fields.forEach((k, v) -> {
62              if (check.length() > 0) check.append('\n');
63              check.append(k).append('=').append(v);
64          });
65
66          byte[] expected = hmac(hmac("WebAppData".getBytes(StandardCharsets.UTF_8), botToken), check.toS
    tring());
67          byte[] given;
68          try {
69              given = HexFormat.of().parseHex(hash);
70          } catch (IllegalArgumentException e) {
71              throw new InvalidInitData("hash noto'g'ri formatda");
72          }
73          if (!MessageDigest.isEqual(expected, given)) throw new InvalidInitData("Imzo mos kelmadi");
74
75          long authDate;
76          try {
77              authDate = Long.parseLong(fields.getOrDefault("auth_date", ""));
78          } catch (NumberFormatException e) {
79              throw new InvalidInitData("auth_date yo'q");
80          }
81          long age = now.getEpochSecond() - authDate;
82          if (age > maxAgeSeconds) throw new InvalidInitData("initData eskirgan");
83          if (age < -60) throw new InvalidInitData("auth_date kelajakda");
84
85          String userJson = fields.get("user");
86          if (userJson == null) throw new InvalidInitData("user yo'q");
87          JsonNode user = JSON.readTree(userJson);
88          if (user.get("id") == null) throw new InvalidInitData("user.id yo'q");
89          return new WebAppUser(user.get("id").asLong(), text(user, "first_name"), text(user, "usernam
    e"),
90                  text(user, "language_code"));
91      }
```

Algoritm Telegram hujjatida yozilganidek amalga oshirilgan:

1. **50–56** — `initData` `kalit=qiymat&...` ko'rinishida; har bir juft URL-decode qilinadi va `TreeMap` ga (kalit bo'yicha **saralangan**) joylanadi.
2. **57** — `hash` ajratib olinadi.
3. **60–64** — qolgan maydonlar `\n` bilan birlashtiriladi: bu `data_check_string`.
4. **66** — `secret = HMAC_SHA256("WebAppData", botToken)`, keyin `HMAC_SHA256(secret, data_check_string)`. Bu qiymatni faqat bot tokenini biladigan tomon hisoblay oladi, ya'ni Telegram va bizning server.
5. **73** — `MessageDigest.isEqual` — **doimiy vaqtli** solishtirish. Oddiy `Arrays.equals` birinchi farq qiluvchi baytda to'xtaydi. Javob vaqtini o'lchab, imzoni bayt-bayt "taxmin qilish" (timing attack) mumkin bo'lib qoladi.
6. **81–83** — `auth_date` 24 soatdan eski bo'lsa (yoki kelajakda bo'lsa) rad etiladi: o'g'irlangan `initData` ni abadiy qayta ishlatib bo'lmaydi.

src/main/java/uz/azizbek/maktabboshqaruv/controller/ParentApiController.java 42–49-qatorlar

```
42      private long chatId(String initData) {
43          return TelegramInitDataValidator.validate(initData, properties.initDataSigningToken(),
44                  properties.getInitDataMaxAgeSeconds(), clock.instant()).id();
45      }
46
47      private Student student(String initData, Long studentId) {
```

```
48          return access.requireLinked(chatId(initData), studentId);
49      }
```

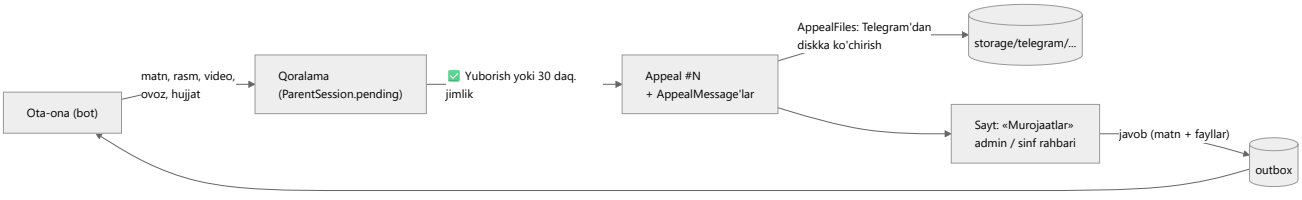
src/main/java/uz/azizbek/maktabboshqaruv/service/parent/ParentAccessService.java 48–64-qatorlar

```
48      /** Cached for a few minutes; linking/unlinking evicts the chat at once. */
49      public List<Student> linkedStudents(long chatId) {
50          return cache.students(chatId, () -> linkRepository.findLinkedStudents(chatId).stream()
51                  .sorted(java.util.Comparator.comparing(Student::getFirstName).thenComparing(Student::ge
    tId))
52                  .toList());
53      }
54
55      /** The student with this id, only if this chat is actively linked to it. */
56      public Student requireLinked(long chatId, Long studentId) {
57          if (studentId != null) {
58              for (Student s : linkedStudents(chatId)) {
59                  if (s.getId().equals(studentId)) return s;
60              }
61          }
62          log.warn("Ruxsatsiz urinish: chat={} o'quvchi #{} ga bog'lanmagan", chatId, studentId);
63          throw new AccessDenied("Bu o'quvchi ushbu chat'ga bog'lanmagan");
64      }
```

`ParentAccessService.requireLinked` — ota-ona ma'lumotiga **yagona darvoza**. Botdagi har bir tugma ham, Mini App'ning har bir so'rovi ham bola id'sini shu metod orqali tekshiradi. Begona id kelsa, xavfsizlik logiga yoziladi va `AccessDenied` → **403** qaytadi. `ParentApiSecurityIntegrationTest` buni real HTTP bilan tekshiradi: soxta yoki eskirgan `initData` → 401, begona bola → 403.

> 🤔 **Nega Mini App uchun JWT emas?**
> Ota-onaning loyihada login va paroli yo'q. Uni majburlab ro'yxatdan o'tkazish noqulay bo'lardi. Telegram esa ota-onani **allaqachon** tanigan va buni imzo bilan tasdiqlab beradi. `SecurityConfig` da `/api/parent/**` `permitAll()`, chunki autentifikatsiya Spring Security filtrida emas, controller ichida (`chatId()` metodida) bajariladi.

Murojaatlar



Qoidalar (`AppealService` izohidan): bir kunda ko'pi bilan 10 ta murojaat, bitta yuborishda 20 tagacha xabar, har bir fayl 20 MB gacha, faqat ruxsat etilgan fayl turlari (`AttachmentPolicy`).

src/main/java/uz/azizbek/maktabboshqaruv/service/appeal/AppealService.java 279–286-qatorlar

```
279     /** Who may see an appeal: admins of its school; for a class-teacher appeal also that class's teac
    her. */
280     void requireVisible(Caller c, Appeal a) {
281         access.requireSchool(c, a.getSchool().getId());
282         if (c.isAdmin()) return;
283         if (a.getTarget() == Target.CLASS_TEACHER
284                 && access.classesLed(c).contains(a.getStudent().getSchoolClass().getId())) return;
```

```
285         throw new ForbiddenException("Bu murojaat sizga ochiq emas");
286     }
```

Kim nimani ko'radi: murojaatni **administratorlar** ko'radi. Sinf rahbariga yozilgan murojaatni esa shu sinfning **rahbari** ham ko'radi. Boshqa o'qituvchi uni ko'ra olmaydi (`ForbiddenException` → 403).

Fayllar uchun imzolangan havola

Sayt murojaatdagi rasm va videoni `<img src="...">` va `<video src="...">` teglari bilan ko'rsatadi. Lekin brauzer bunday teglarda `Authorization` header'ini **yubora olmaydi**. Yechim — har bir fayl va har bir foydalanuvchi uchun alohida, **30 daqiqa** amal qiladigan imzolangan havola:

src/main/java/uz/azizbek/maktabboshqaruv/service/appeal/AttachmentLinks.java 28–55-qatorlar

```
28      public AttachmentLinks(@Value("${jwt.secret}") String secret, Clock clock) {
29          this.key = ("appeal-file:" + secret).getBytes(StandardCharsets.UTF_8);
30          this.clock = clock;
31      }
32
33      /** "?t=<exp>.<user>.<sig>" — the query string for /api/appeals/{id}/attachments/{msgId}. */
34      public String token(Long appealId, Long messageId, String username) {
35          long exp = clock.instant().getEpochSecond() + TTL_SECONDS;
36          String user = Base64.getUrlEncoder().withoutPadding().encodeToString(username.getBytes(Standard
    Charsets.UTF_8));
37          return exp + "." + user + "." + sign(appealId + ":" + messageId + ":" + exp + ":" + username);
38      }
39
40      /** The username the token was issued to, or null when it is forged, for another file, or expired.
    */
41      public String verify(Long appealId, Long messageId, String token) {
42          if (token == null) return null;
43          String[] parts = token.split("\\.");
44          if (parts.length != 3) return null;
45          try {
46              long exp = Long.parseLong(parts[0]);
47              if (clock.instant().getEpochSecond() > exp) return null;
48              String username = new String(Base64.getUrlDecoder().decode(parts[1]), StandardCharsets.UTF_
    8);
49              String expected = sign(appealId + ":" + messageId + ":" + exp + ":" + username);
50              return MessageDigest.isEqual(expected.getBytes(StandardCharsets.UTF_8), parts[2].getBytes(S
    tandardCharsets.UTF_8))
51                      ? username : null;
52          } catch (RuntimeException e) {
53              return null;
54          }
55      }
```

- **29** — kalit `"appeal-file:" + jwtSecret`: JWT secret'dan **alohida maqsadli** kalit yasaladi (domain separation). Shu sababli fayl imzosini JWT o'rnida ishlatib bo'lmaydi va aksincha.
- **37** — imzo `(murojaat, xabar, muddat, foydalanuvchi)` to'rtligiga qo'yiladi: havolani boshqa fayl uchun ishlatib bo'lmaydi.
- **47** — muddat o'tgan bo'lsa → `null` (ruxsat yo'q).
- **50** — yana doimiy vaqtli solishtirish.

`SecurityConfig` da `GET /api/appeals/*/attachments/*` `permitAll()`, chunki tekshiruv shu imzo bilan controller ichida bajariladi.

Fayllar diskda

`FileStorage` fayllarni `{app.storage.path}/{area}/{maktabId}/{yyyy-MM}/{uuid}.ext` ko'rinishida saqlaydi. Fayl nomi **tasodifiy UUID**: ota-ona yuborgan asl nom (`../../etc/passwd` kabi) diskka hech qachon ishlatilmaydi. Bu **path traversal** hujumining oldini oladi. Fayl Telegram'dan alohida kichik oqimlar hovuzida ko'chiriladi (`AppealFiles`): bot yuklab olish tugashini kutib turmaydi.

Ommaviy xabarlar

`BroadcastService` (faqat ADMIN) qabul qiluvchilarni aniqlaydi: butun maktab, tanlangan sinflar yoki tanlangan ota-onalar. Keyin har biriga outbox qatorini yaratadi. Fayllar Telegram'ga **bir marta** yuklanadi, keyingi ota-onalarga esa Telegram bergan `file_id` bilan yuboriladi. Natijada 1000 ta ota-onaga 10 MB'lik video 1000 marta qayta yuklanmaydi. "1250/3000 yuborildi" statistikasi outbox'dan o'qiladi.

`POST /api/broadcasts/preview` yuborishdan **oldin** xabar nechta ota-onaga borishini ko'rsatadi.

Metod	URL	Rol	Java metodi	Nima qiladi
GET	/api/appeals	ADMIN, EDITOR, VIEWER	list()	Murojaat ro'yxati (sahifalab)
GET	/api/appeals/unread	ADMIN, EDITOR, VIEWER	unread()	O'qilmagan murojaatlar soni (menyu nishoni)
GET	/api/appeals/{id}	ADMIN, EDITOR, VIEWER	chat()	Bitta murojaat yozishmasi
POST	/api/appeals/{id}/reply	ADMIN, EDITOR	reply()	Murojaatga javob (matn va fayllar) — ota-onaga Telegram'da boradi
POST	/api/appeals/{id}/close	ADMIN, EDITOR	close()	Murojaatni yopadi
POST	/api/appeals	ADMIN, EDITOR	create()	Yangi murojaat yaratadi
GET	/api/appeals/{id}/attachments/{messageId}	ochiq	attachment()	Murojaat fayli — imzolangan 30 daqiqalik havola bilan
GET	/api/broadcasts	ADMIN	list()	Xabar ro'yxati (sahifalab)
GET	/api/broadcasts/{id}	ADMIN	get()	Bitta xabar
GET	/api/broadcasts/{id}/recipients	ADMIN	recipients()	Xabar kimlarga bordi va holati
POST	/api/broadcasts/preview	ADMIN	preview()	Yuborishdan oldin: nechta ota-onaga boradi
POST	/api/broadcasts	ADMIN	send()	Xabar yuborish (faqat matn, JSON)
POST	/api/broadcasts	ADMIN	sendWithFiles()	Xabar yuborish (fayllar bilan, multipart)
GET	/api/parent/me	Telegram initData	me()	Ota-ona: tili, bolalari, tanlangan bola
GET	/api/parent/students/{id}/today	Telegram initData	today()	Bola: bugungi xulosa
GET	/api/parent/students/{id}/schedule	Telegram initData	schedule()	Bola: dars jadvali (bugun/ertaga/hafta)
GET	/api/parent/students/{id}/attendance	Telegram initData	attendance()	Bola: davomat (oy)
GET	/api/parent/students/{id}/grades	Telegram initData	grades()	Bola: baholar
GET	/api/parent/students/{id}/events	Telegram initData	events()	Bola: tadbirlar

Metod	URL	Rol	Java metodi	Nima qiladi
GET	`/api/parent/students/{id}/grades/{subjectId}`	Telegram initData	`subject()`	Bola: bitta fan bo'yicha baholar
GET	`/api/parent/students/{id}/announcements`	Telegram initData	`announcements()`	Bola: e'lonlar

📌 **Esda tut**

- Mini App JWT emas, Telegram imzosi (`initData` , HMAC-SHA256) bilan himoyalangan; eskirgan imzo rad etiladi.
- Bola id'si har doim `ParentAccessService.requireLinked` orqali tekshiriladi — botda ham, Mini App'da ham.
- Imzolarni solishtirish doim `MessageDigest.isEqual` (doimiy vaqt) bilan.
- `<img>` / `<video>` uchun — faylga va foydalanuvchiga bog'langan, 30 daqiqalik imzolangan havola.
- Disk fayllarining nomi tasodifiy UUID — foydalanuvchi bergan nom ishlatilmaydi.

13. Modul: kutubxona

"📚 Kitob mutolaasi" — maktabning elektron kutubxonasi. Xodimlar kitoblarni saytga yuklaydi, ota-onalar ularni botda qidiradi, tanlaydi va PDF sifatida oladi. Modul `service.library` paketida (15 fayl) va `service.library.wikisource` ichki paketida (5 fayl) joylashgan.

Qism	Klasslar
Entity	`Book`, `BookCategory`, `Author`, `LibraryPermission`, `BookDownload`, `BookFavorite`, `BookRating`, `LibraryEnums`
Sayt servislari	`LibraryAdminService` (kitoblar, javonlar), `LibraryFiles` (fayllar), `LibraryLegal` (huquqiy tekshiruv), `LibraryAuthors`, `LibraryPermissions`, `LibraryStatsService`
Bot servislari	`LibraryCatalog` (qidiruv, javonlar, yuklab olish), `LibraryWeekly` (haftalik tavsiya), `BookMedia` (PDF'ni outbox orqali yuborish)
Yordamchilar	`CopyrightRules`, `LibraryText` (qidiruv normalizatsiyasi), `LibrarySettings`, `LibraryDefaults`, `LibraryPdfFonts`, `PermissionTemplatePdf`
Wikisource	`WikisourceImport`, `WikisourceApi` / `HttpWikisourceApi`, `WikiText`, `WikisourcePdf`
Controller	`LibraryController`, `LibraryLegalController`, `LibraryStatsController`

Har bir kitob qonuniy bo'lishi shart

Kutubxonaning asosiy qoidasi: kitob **faqat** tarqatish huquqi bor bo'lsa saqlanadi. Har bir kitobning `legalBasis` (huquqiy asosi) bor:

Asos	Shart	Tekshiruvchi
`JAMOAT_MULKI`	muallif vafotidan keyin 70 yil o'tgan; asar muallif tirikligida chop etilgan (yoki birinchi nashridan 70 yil o'tgan); faylda zamonaviy so'zboshi yoki sharh yo'q	`LibraryLegal.checkPublicDomain`
`MUALLIF_RUXSATI`, `LITSENZIYA`	amaldagi `LibraryPermission` (imzolangan skan bilan) bog'langan	`LibraryLegal.permissionInForce`

Qonun koddagi formula sifatida

src/main/java/uz/azizbek/maktabboshqaruv/service/library/CopyrightRules.java — 27–50-qatorlar

```
27      public static final int TERM_YEARS = 70;
28
29      private CopyrightRules() {
30      }
31
32      /** Last year of protection for a death (or posthumous publication) in {@code year}. */
33      public static int protectedUntil(int year) {
34          return year + TERM_YEARS;
35      }
36
37      /** First year the work is free ({@code protectedUntil + 1}). */
38      public static int publicFrom(int year) {
39          return year + TERM_YEARS + 1;
40      }
41
```

```
42      public static boolean isFree(int year, int currentYear) {
43          return protectedUntil(year) < currentYear;
44      }
45
46      /** The death year that decides for several authors: the latest one; null when any is unknown. */
47      public static Integer decidingDeathYear(Collection<Integer> deathYears) {
48          if (deathYears.isEmpty() || deathYears.stream().anyMatch(Objects::isNull)) return null;
49          return deathYears.stream().max(Integer::compare).orElse(null);
50      }
```

O'zbekiston "Mualliflik huquqi va turdosh huquqlar to'g'risida"gi qonunining 35-moddasiga ko'ra, huquq muallif vafotidan keyin **70 yil** amal qiladi va muddat keyingi yilning 1-yanvaridan hisoblanadi. Masalan, Abdulla Qodiriy 1938-yilda vafot etgan: 1938 + 70 = 2008-yil oxirigacha himoyada, **2009-yildan** jamoat mulki. Bir nechta muallif bo'lsa, eng **oxirgi** vafot etgani hal qiladi (49-qator).

> 🤔 **Nega bu qoida alohida, statik klassda?**
>
> `CopyrightRules` — sof funksiyalar: na baza, na Spring kerak. Natijada uni bir qatorli unit test bilan sinash oson (`assertEquals(2008, protectedUntil(1938))`). Qoidani ham sayt (`LibraryLegal`), ham mualliflar sahifasidagi "✅ Jamoat mulki (2009-yildan)" belgisi ishlatadi, ya'ni formula bitta joyda yozilgan.

`src/main/java/uz/azizbek/maktabboshqaruv/service/library/LibraryLegal.java` — 83–112-qatorlar

```
83      /** The public-domain rules; throws 400 with the reason. */
84      public void checkPublicDomain(List<Author> list, Boolean publishedInLifetime, Integer firstPublishedYear,
85                                    Boolean noModernCommentary) {
86          int year = LocalDate.now(clock).getYear();
87          if (list.isEmpty()) throw LibraryFiles.bad("Jamoat mulki uchun muallif tanlanishi shart");
88          for (Author a : list) {
89              if (a.getDeathYear() == null) {
90                  throw LibraryFiles.bad("«" + a.getFullName() + "» muallifining vafot yili kiritilmagan – asar jamoat "
91                          + "mulki ekanini aniqlab bo'lmaydi. «Mualliflar» bo'limida vafot yilini kiriting.");
92              }
93          }
94          int deciding = CopyrightRules.decidingDeathYear(list.stream().map(Author::getDeathYear).toList());
95          if (!CopyrightRules.isFree(deciding, year)) {
96              throw LibraryFiles.bad("Bu muallif asarlari " + CopyrightRules.protectedUntil(deciding)
97                      + "-yilgacha mualliflik huquqi bilan himoyalangan. Muallif yoki merosxo'rlar ruxsati kerak.");
98          }
99          if (!Boolean.TRUE.equals(publishedInLifetime)) {
100             if (firstPublishedYear == null) {
101                 throw LibraryFiles.bad("«Asar muallif tirikligida chop etilgan» belgilanmagan – birinchi nashr yilini kiriting");
102             }
103             if (!CopyrightRules.isFree(firstPublishedYear, year)) {
104                 throw LibraryFiles.bad("Asar muallif vafotidan keyin " + firstPublishedYear + "-yilda birinchi marta "
105                         + "chop etilgan – " + CopyrightRules.protectedUntil(firstPublishedYear)
106                         + "-yilgacha mualliflik huquqi bilan himoyalangan. Huquq egasining ruxsati kerak.");
107             }
108         }
109         if (!Boolean.TRUE.equals(noModernCommentary)) {
110             throw LibraryFiles.bad("«Faylda zamonaviy so'zboshi, izoh yoki sharh yo'q» tasdig'i majburiy");
111         }
112     }
```

Har bir tekshiruv xato bo'lsa, foydalanuvchiga **aniq sabab** bilan 400 qaytaradi, masalan: "*Bu muallif asarlari 2038-yilgacha mualliflik huquqi bilan himoyalangan. Muallif yoki merosxo'rlar ruxsati kerak.*" Bu xabarni `LibraryLegalIntegrationTest` Oybek (1968) misolida tekshiradi.

Fayllar: ishonmang, tekshiring

src/main/java/uz/azizbek/maktabboshqaruv/service/library/LibraryFiles.java — 54–92-qatorlar

```
54      /** Refuses anything that is not a PDF within the size limit; returns its hash and page count. */
55      public PdfInfo checkPdf(byte[] bytes) {
56          if (bytes == null || bytes.length == 0) throw bad("PDF fayl tanlanmagan");
57          if (bytes.length > settings.maxPdfBytes()) {
58              throw bad("PDF juda katta: ko'pi bilan " + settings.maxPdfBytes() / 1024 / 1024 + " MB");
59          }
60          if (!isPdf(bytes)) throw bad("Fayl PDF emas (faqat haqiqiy PDF fayllar qabul qilinadi)");
61          Integer pages = null;
62          try (PDDocument doc = Loader.loadPDF(bytes)) {
63              pages = doc.getNumberOfPages();
64          } catch (IOException e) {
65              // a damaged or encrypted PDF can still be sent to parents as it is; only the page count is unknown
66              log.info("PDF sahifalari sanab bo'lmadi: {}", e.getMessage());
67          }
68          return new PdfInfo(sha256(bytes), bytes.length, pages);
69      }
70
71      /** "jpg" or "png" by magic bytes; anything else (or too big) is refused. */
72      public String checkCover(byte[] bytes) {
73          if (bytes.length > settings.maxCoverBytes()) {
74              throw bad("Muqova juda katta: ko'pi bilan " + settings.maxCoverBytes() / 1024 / 1024 + " MB");
75          }
76          if (bytes.length > 3 && (bytes[0] & 0xFF) == 0xFF && (bytes[1] & 0xFF) == 0xD8 && (bytes[2] & 0xFF) == 0xFF) {
77              return "jpg";
78          }
79          if (bytes.length > 8 && (bytes[0] & 0xFF) == 0x89 && bytes[1] == 'P' && bytes[2] == 'N' && bytes[3] == 'G') {
80              return "png";
81          }
82          throw bad("Muqova faqat JPG yoki PNG bo'lishi mumkin");
83      }
84
85      public static boolean isPdf(byte[] bytes) {
86          // "%PDF" — allowed after a few bytes of junk, as PDF readers do
87          int limit = Math.min(bytes.length - 4, 1024);
88          for (int i = 0; i <= limit; i++) {
89              if (bytes[i] == '%' && bytes[i + 1] == 'P' && bytes[i + 2] == 'D' && bytes[i + 3] == 'F') return true;
90          }
91          return false;
92      }
```

- **57** — hajm chegarasi (standart 50 MB, `app.library.max-pdf-mb`).
- **60, 85–92** — **"sehrli baytlar"** (magic bytes): fayl haqiqatan `%PDF` bilan boshlanadimi? Fayl nomidagi `.pdf` va brauzer yuborgan `Content-Type` ga **ishonilmaydi** — `virus.exe` nomini `kitob.pdf` ga o'zgartirish oson.
- **62–67** — PDFBox bilan bet soni hisoblanadi; buzilgan PDF bo'lsa, fayl baribir qabul qilinadi, faqat bet soni noma'lum bo'ladi.
- **68** — **SHA-256** xesh: bitta PDF maktabga ikki marta yuklansa, `GET /books/duplicates` buni aniqlaydi.

- **76, 79** — muqova ham sehrli baytlar bo'yicha tekshiriladi: JPEG `FF D8 FF`, PNG `89 50 4E 47`.

Fayl diskka tasodifiy nom bilan yoziladi (12-bob, `FileStorage`), PDF esa faqat backend orqali, JWT va maktab tekshiruvidan keyin beriladi.

Bot tomoni: kunlik limit va PDF yuborish

`LibraryCatalog.download` ota-ona kitobni so'raganda:

1. kitob shu maktabniki va faolmi — tekshiradi;
2. ota-ona bugun nechta kitob olganini sanaydi: `app.library.daily-download-limit` (standart 20) dan oshsa → `LIMIT`;
3. `BookDownload` yozuvini (statistika uchun) va outbox qatorini (`media = "book:<id>"`) yaratadi.

`BookMedia` PDF'ni Telegram'ga **birinchi marta** yuklaydi va Telegram bergan `file_id` ni `book.telegram_file_id` ga saqlaydi. Keyingi ota-onalarga fayl qayta yuklanmaydi, `file_id` bilan bir zumda yuboriladi.

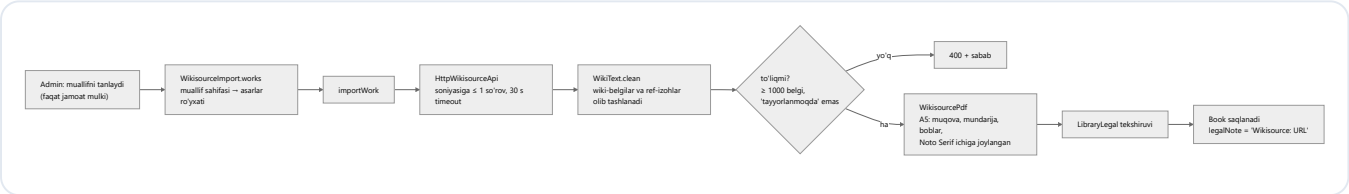
Ruxsatnomalar va kundalik tekshiruv

`LibraryPermission` — muallif, merosxo'r yoki nashriyot bergan ruxsat: kim bergan, telefon, amal qilish muddati (`validUntil` bo'sh bo'lsa — muddatsiz) va imzolangan skan (majburiy).

- Admin ruxsatnomani **bekor qilsa** (`POST /permissions/{id}/revoke`), unga bog'langan kitoblar **darhol** nofaol bo'ladi va botdan yo'qoladi.
- Har kuni **06:15** da (Toshkent) `@Scheduled` vazifa muddati o'tgan ruxsatnomalarning kitoblarini nofaol qiladi. Muddat tugashiga 30 kun qolganda esa bir marta ogohlantiradi.
- `GET /permissions/template` maktab nomi va kitob nomi to'ldirilgan A4 **ruxsat xati shablonini** PDF qilib beradi (`PermissionTemplatePdf`). Uni chop etib imzolatish mumkin.

Wikisource'dan import

Jamoat mulki bo'lgan mumtoz asarlarni qo'lda yuklash o'rniga, ularni Wikisource'dan import qilish mumkin.



13.1-rasm. Wikisource importi: tarmoq → tozalash → tekshirish → PDF → huquqiy tekshiruv.

> 💡 **Tashqi API bilan "odobli" ishlash**
>
> `HttpWikisourceApi` Wikimedia talablariga rioya qiladi: `User-Agent` da loyiha nomi, **butun ilova bo'yicha** soniyasiga ko'pi bilan bitta so'rov va 30 soniyalik timeout. Tashqi API'ga hech qachon cheklovsiz murojaat qilmang: aks holda server sizni bloklaydi yoki sekin javob butun ilovangizni to'xtatib qo'yadi.

`WikisourceApi` — **interfeys**. Testlarda (`LibraryLegalIntegrationTest`) uning o'rniga soxta (mock) amalga oshirilish qo'yiladi, shuning uchun import internetsiz va har doim bir xil natija bilan sinaladi. PDF'dagi o'zbek

harflari (o', g', sh, ch va kirill) PDFBox bilan qayta o'qilib tekshiriladi.

Metod	URL	Rol	Java metodi	Nima qiladi
GET	/api/library/books	ADMIN, EDITOR	list()	Kitob ro'yxati
GET	/api/library/books/{id}	ADMIN, EDITOR	get()	Bitta kitob
GET	/api/library/books/duplicates	ADMIN, EDITOR	duplicates()	Shu PDF maktabda allaqachon bormi (SHA-256)
POST	/api/library/books	ADMIN, EDITOR	create()	Yangi kitob yaratadi
PUT	/api/library/books/{id}	ADMIN, EDITOR	update()	Kitobni tahrirlaydi
POST	/api/library/books/drafts	ADMIN, EDITOR	draft()	Ommaviy yuklash: bitta PDF → bitta qoralama kartochka
DELETE	/api/library/books/{id}	ADMIN, EDITOR	deactivate()	Kitobni o'chiradi
GET	/api/library/books/{id}/pdf	ADMIN, EDITOR	pdf()	Kitob PDF'i
GET	/api/library/books/{id}/cover	ADMIN, EDITOR	cover()	Kitob muqovasi
GET	/api/library/categories	ADMIN, EDITOR	categories()	Kutubxona bo'limlari
POST	/api/library/categories	ADMIN, EDITOR	createCategory()	Yangi kitob yaratadi
PUT	/api/library/categories/{id}	ADMIN, EDITOR	updateCategory()	Kitobni tahrirlaydi
DELETE	/api/library/categories/{id}	ADMIN, EDITOR	deleteCategory()	Kitobni o'chiradi
PUT	/api/library/categories/order	ADMIN, EDITOR	reorder()	Bo'limlar tartibini o'zgartiradi
GET	/api/library/authors	ADMIN, EDITOR	authors()	Mualliflar ro'yxati (holati bilan)
POST	/api/library/authors	ADMIN	createAuthor()	Yangi yozuv yaratadi
PUT	/api/library/authors/{id}	ADMIN	updateAuthor()	Yozuvni tahrirlaydi
DELETE	/api/library/authors/{id}	ADMIN	deleteAuthor()	Yozuvni o'chiradi
GET	/api/library/permissions	ADMIN	permissions()	Ruxsatnomalar ro'yxati
POST	/api/library/permissions	ADMIN	createPermission()	Yangi yozuv yaratadi
PUT	/api/library/permissions/{id}	ADMIN	updatePermission()	Yozuvni tahrirlaydi
POST	/api/library/permissions/{id}/revoke	ADMIN	revoke()	Ruxsatnomani bekor qiladi — kitoblari darhol nofaol
GET	/api/library/permissions/{id}/document	ADMIN	document()	Ruxsatnomaning imzolangan skani
GET	/api/library/permissions/template	ADMIN	template()	Ruxsat xati shabloni (PDF)
GET	/api/library/wikisource/works	ADMIN	wikisourceWorks()	Muallifning Wikisource'dagi asarlari
POST	/api/library/wikisource/import	ADMIN	wikisourceImport()	Asarni Wikisource'dan import qiladi (PDF yaratadi)
GET	/api/library/stats	ADMIN	overview()	Mutolaa statistikasi

Metod	URL	Rol	Java metodi	Nima qiladi
GET	`/api/library/stats/journal`	ADMIN	`journal()`	Yuklab olishlar jurnali
GET	`/api/library/stats/journal/export`	ADMIN	`export()`	Jurnal Excel faylida
GET	`/api/library/stats/parents/{chatId}`	ADMIN	`parent()`	Bitta ota-ona: nimalarni o'qigan

📌 **Esda tut**

- Har bir kitobning huquqiy asosi bor: jamoat mulki (70 yil qoidasi) yoki amaldagi ruxsatnoma.
- Qonun bitta statik klassda (`CopyrightRules`) formula sifatida yozilgan va sof unit test bilan sinaladi.
- Fayl turi nom yoki `Content-Type` bo'yicha emas, **sehrli baytlar** bo'yicha tekshiriladi; dublikatlar SHA-256 bilan aniqlanadi.
- Telegram `file_id` saqlanadi — PDF Telegram'ga bir marta yuklanadi.
- Ruxsatnoma bekor qilinsa yoki muddati o'tsa, kitoblar avtomatik nofaol bo'ladi.
- Tashqi API: interfeys + tezlik chegarasi + timeout + testda mock.

14. Modul: foydalanuvchilar va maktab chegarasi

Tizim bir nechta maktabga xizmat qiladi. Ikkita savol muhim: **kim** tizimga kirdi (foydalanuvchi va uning roli) va u **qaysi maktab** ma'lumotini ko'ra oladi (maktab chegarasi).

Foydalanuvchi va rollar

Klass	Vazifa
`User` → `app_user`	`username` (UNIQUE), `password` (BCrypt xeshi), `role`, ixtiyoriy `employee`
`Role`	`ADMIN`, `EDITOR`, `VIEWER`
`UserService`, `UserController`	foydalanuvchilarni boshqarish — **faqat ADMIN** (`@PreAuthorize` klass darajasida)

Rol	Nima qila oladi
ADMIN	hammasi: foydalanuvchilar, o'chirish, bot sozlamalari, ommaviy xabarlar, kutubxona statistikasi
EDITOR	ma'lumot kiritish va tahrirlash (davomat, baho, o'quvchi...), o'chirishsiz
VIEWER	faqat ko'rish

Endpoint'lar bo'yicha taqsimot (A-ilovadan): 72 tasi ADMIN va EDITOR uchun, 62 tasi uchala rol uchun, 50 tasi faqat ADMIN uchun, 3 tasi ochiq va 8 tasi Telegram imzosi bilan himoyalangan.

Parol hech qachon ochiq saqlanmaydi:

src/main/java/uz/azizbek/maktabboshqaruv/service/UserService.java — 46–60-qatorlar

```
46      @Transactional
47      public UserResponseDto createUser(UserRequestDto request) {
48          if (userRepository.existsByUsername(request.getUsername())) {
49              throw new IllegalStateException("Bu username bilan foydalanuvchi allaqachon mavjud");
50          }
51
52          User user = new User();
53          user.setUsername(request.getUsername());
54          user.setPassword(passwordEncoder.encode(request.getPassword()));
55          user.setRole(request.getRole());
56          user.setEmployee(resolveEmployee(request.getEmployeeId()));
57
58          User saved = userRepository.save(user);
59          return toResponseDto(saved);
60      }
```

`passwordEncoder.encode()` — **BCrypt** (`SecurityConfig`dagi bean). BCrypt har bir parolga tasodifiy "tuz" (salt) qo'shadi va ataylab sekin ishlaydi. Shuning uchun bir xil "123456" parollar bazada har xil ko'rinadi, baza o'g'irlansa ham parollarni tez topib bo'lmaydi. Tekshirishda `passwordEncoder.matches(ochiq, xesh)` ishlatiladi (15-bob).

Ikki xil foydalanuvchi: maktab xodimi va platforma admini

src/main/java/uz/azizbek/maktabboshqaruv/service/SchoolAccessService.java 15–60-qatorlar

```
15  /**
16   * Which school a signed-in user may work with. A user linked to an employee belongs to
17   * that employee's school and sees nothing of other schools (403). An account without an
18   * employee link is a platform account (it manages schools) and may switch between them.
19   */
20  @Service
21  public class SchoolAccessService {
22
23      /** Who is asking: username, role, own school (null = every school), own employee. */
24      public record Caller(String username, Role role, Long schoolId, Long employeeId) {
25          public boolean isAdmin() {
26              return role == Role.ADMIN;
27          }
28      }
29
30      private final UserRepository users;
31      private final SchoolClassRepository classes;
32
33      public SchoolAccessService(UserRepository users, SchoolClassRepository classes) {
34          this.users = users;
35          this.classes = classes;
36      }
37
38      @Transactional(readOnly = true)
39      public Caller caller() {
40          Authentication auth = SecurityContextHolder.getContext().getAuthentication();
41          if (auth == null || auth.getName() == null) throw new ForbiddenException("Avval tizimga kirin
    g");
42          return caller(auth.getName());
43      }
44
45      /** Every request asks who the caller is; a minute is short enough for role or school changes. */
46      private final com.github.benmanes.caffeine.cache.Cache<String, Caller> callers =
47              com.github.benmanes.caffeine.cache.Caffeine.newBuilder()
48                      .maximumSize(5_000).expireAfterWrite(java.time.Duration.ofMinutes(1)).build();
49
50      @Transactional(readOnly = true)
51      public Caller caller(String username) {
52          Caller cached = callers.getIfPresent(username);
53          if (cached != null) return cached;
54          User u = users.findByUsername(username).orElseThrow(() -> new ForbiddenException("Foydalanuvchi
    topilmadi"));
55          Long schoolId = u.getEmployee() != null && u.getEmployee().getSchool() != null
56                  ? u.getEmployee().getSchool().getId() : null;
57          Caller c = new Caller(u.getUsername(), u.getRole(), schoolId, u.getEmployee() != null ? u.getEm
    ployee().getId() : null);
58          callers.put(username, c);
59          return c;
60      }
```

- **Xodimga bog'langan foydalanuvchi** (`user.employee != null`) — shu xodimning maktabiga **qamalgan**: boshqa maktab ma'lumotiga 403.
- **Xodimsiz foydalanuvchi** — **platforma hisobi** (`schoolId = null`): barcha maktablarni boshqaradi va ular orasida almasha oladi.
- **46–48** — `Caller` har so'rovda kerak bo'ladi, shuning uchun u **1 daqiqa** Caffeine keshida saqlanadi. Rol yoki maktab o'zgarsa, `forget(username)` keshni tozalaydi.

Maktab chegarasi: id qayerda kelmasin, tekshiriladi

Boshqa maktab ma'lumotini so'rashning uch yo'li bor. Uchalasi ham to'silgan:

Qayerda	Misol	Kim tekshiradi
URL yo'li	`/api/students/12`	`SchoolScopeConfig` interceptor (`PATH` jadvali)
Query parametr	`?schoolClassId=5`	interceptor (`FIELDS` jadvali)
JSON body	`{"schoolClassId": 5}`	`SchoolScopeConfig.BodyScope` (`RequestBodyAdvice`)

src/main/java/uz/azizbek/maktabboshqaruv/config/SchoolScopeConfig.java — 40–65-qatorlar

```
40      /** Path segment (followed by a numeric id) → what the id names. */
41      static final Map<String, Kind> PATH = Map.ofEntries(
42              Map.entry("schools", Kind.SCHOOL), Map.entry("school-classes", Kind.CLASS), Map.entry("clas
        s", Kind.CLASS), Map.entry("classes", Kind.CLASS),
43              Map.entry("students", Kind.STUDENT), Map.entry("student", Kind.STUDENT),
44              Map.entry("employees", Kind.EMPLOYEE), Map.entry("teacher", Kind.EMPLOYEE),
45              Map.entry("academic-years", Kind.YEAR), Map.entry("buildings", Kind.BUILDING), Map.entry("r
        ooms", Kind.ROOM),
46              Map.entry("subjects", Kind.SUBJECT), Map.entry("lesson-slots", Kind.LESSON_SLOT),
47              Map.entry("announcements", Kind.ANNOUNCEMENT), Map.entry("calendar-events", Kind.CALENDAR_E
        VENT),
48              Map.entry("absence-requests", Kind.ABSENCE), Map.entry("behavior-records", Kind.BEHAVIOR),
49              Map.entry("grades", Kind.GRADE), Map.entry("attendance", Kind.ATTENDANCE), Map.entry("user
        s", Kind.USER),
50              Map.entry("appeals", Kind.APPEAL), Map.entry("broadcasts", Kind.BROADCAST),
51              // library: /api/library/books/{id}, /api/library/categories/{id}
52              Map.entry("books", Kind.BOOK), Map.entry("categories", Kind.BOOK_CATEGORY),
53              Map.entry("permissions", Kind.LIBRARY_PERMISSION)
54      );
55
56      /** Query parameter / body field → what it names. */
57      static final Map<String, Kind> FIELDS = Map.ofEntries(
58              Map.entry("schoolId", Kind.SCHOOL), Map.entry("schoolClassId", Kind.CLASS), Map.entry("clas
        sId", Kind.CLASS),
59              Map.entry("studentId", Kind.STUDENT), Map.entry("employeeId", Kind.EMPLOYEE), Map.entry("te
        acherId", Kind.EMPLOYEE),
60              Map.entry("classTeacherId", Kind.EMPLOYEE), Map.entry("academicYearId", Kind.YEAR),
61              Map.entry("buildingId", Kind.BUILDING), Map.entry("roomId", Kind.ROOM), Map.entry("subjectI
        d", Kind.SUBJECT),
62              Map.entry("lessonSlotId", Kind.LESSON_SLOT),
63              Map.entry("bookId", Kind.BOOK), Map.entry("categoryId", Kind.BOOK_CATEGORY),
64              Map.entry("permissionId", Kind.LIBRARY_PERMISSION)
65      );
```

Ikkita jadval: URL segmenti (`students`) → ob'ekt turi (`Kind.STUDENT`) va maydon nomi (`schoolClassId`) → ob'ekt turi.

src/main/java/uz/azizbek/maktabboshqaruv/config/SchoolScopeConfig.java — 75–124-qatorlar

```
75      @Override
76      public void addInterceptors(@NonNull InterceptorRegistry registry) {
77          registry.addInterceptor(new HandlerInterceptor() {
78              @Override
79              public boolean preHandle(HttpServletRequest request, HttpServletResponse response, Object
         handler) throws Exception {
80                  Long own = ownSchool();
81                  if (own == null) return true;
82                  try {
83                      String[] parts = request.getRequestURI().split("/");
84                      for (int i = 0; i + 1 < parts.length; i++) {
```

```
85                          Kind kind = PATH.get(parts[i]);
86                          if (kind != null && isId(parts[i + 1])) require(own, kind, Long.parseLong(parts[i + 1]));
87                      }
88                      for (Map.Entry<String, Kind> f : FIELDS.entrySet()) {
89                          String value = request.getParameter(f.getKey());
90                          if (value != null && isId(value)) require(own, f.getValue(), Long.parseLong(value));
91                      }
92                      return true;
93                  } catch (ForbiddenException e) {
94                      response.setStatus(403);
95                      response.setContentType("text/plain;charset=UTF-8");
96                      response.getWriter().write(e.getMessage());
97                      return false;
98                  }
99              }
100         }).addPathPatterns("/api/**").excludePathPatterns("/api/auth/**", "/api/parent/**", "/api/health");
101     }
102
103     /** The caller's school, or null when the caller is not limited (anonymous, or a user without a school). */
104     Long ownSchool() {
105         Authentication auth = SecurityContextHolder.getContext().getAuthentication();
106         if (auth == null || !auth.isAuthenticated() || "anonymousUser".equals(auth.getName())) return null;
107         try {
108             return access.caller(auth.getName()).schoolId();
109         } catch (ForbiddenException e) {
110             return null;
111         }
112     }
113
114     void require(Long own, Kind kind, long id) {
115         ownership.schoolOf(kind, id).ifPresent(school -> {
116             if (!school.equals(own)) throw new ForbiddenException("Boshqa maktab ma'lumotiga ruxsat yo'q");
117         });
118     }
119
120     private static boolean isId(String s) {
121         if (s.isEmpty() || s.length() > 18) return false;
122         for (int i = 0; i < s.length(); i++) if (!Character.isDigit(s.charAt(i))) return false;
123         return true;
124     }
```

- **77** — `HandlerInterceptor` controller metodidan **oldin** ishlaydi (`preHandle`).
- **80–81** — foydalanuvchining o'z maktabi; `null` (platforma hisobi) bo'lsa — cheklovsiz.
- **84–86** — URL bo'laklari: `["", "api", "students", "12"]` → `students` jadvalda bor, keyingi bo'lak raqam → `require(own, STUDENT, 12)`.
- **88–90** — query parametrlari.
- **100** — `/api/**` ga qo'llanadi; login, Mini App va health'dan tashqari.
- **114–117** — `SchoolOwnership.schoolOf()` yozuvning maktabini topadi. U boshqa maktab bo'lsa — `ForbiddenException` → 403.

src/main/java/uz/azizbek/maktabboshqaruv/config/SchoolScopeConfig.java 126–152-qatorlar

```
126     /** The same check for ids inside a JSON body (getters or record accessors named like the fields above). */
127     @ControllerAdvice
128     static class BodyScope extends RequestBodyAdviceAdapter {
```

```
129
130         private final SchoolScopeConfig scope;
131
132         BodyScope(SchoolScopeConfig scope) {
133             this.scope = scope;
134         }
135
136         @Override
137         public boolean supports(MethodParameter parameter, Type targetType, Class<? extends HttpMessag
    eConverter<?>> converterType) {
138             return true;
139         }
140
141         @Override
142         public Object afterBodyRead(Object body, HttpInputMessage inputMessage, MethodParameter parame
    ter, Type targetType,
143                                     Class<? extends HttpMessageConverter<?>> converterType) {
144             if (body == null || body instanceof String || body instanceof Map || body instanceof Itera
    ble) return body;
145             Long own = scope.ownSchool();
146             if (own == null) return body;
147             for (Map.Entry<String, Kind> f : FIELDS.entrySet()) {
148                 Object value = read(body, f.getKey());
149                 if (value instanceof Number n) scope.require(own, f.getValue(), n.longValue());
150             }
151             return body;
152         }
```

`@ControllerAdvice` + `RequestBodyAdviceAdapter.afterBodyRead`: JSON DTO'ga aylangandan keyin, controller'dan oldin chaqiriladi. DTO'da `getSchoolClassId()` getter'i yoki `schoolClassId()` record accessor'i bo'lsa, **reflection** bilan o'qiladi va tekshiriladi. Shu tufayli 74 ta DTO'ning birortasiga ham alohida tekshiruv yozish shart bo'lmagan.

Yozuv qaysi maktabniki: SchoolOwnership

src/main/java/uz/azizbek/maktabboshqaruv/service/SchoolOwnership.java 60–73-qatorlar

```
60      private final Cache<String, Optional<Long>> cache = Caffeine.newBuilder()
61              .maximumSize(50_000).expireAfterWrite(Duration.ofMinutes(5)).build();
62
63      public SchoolOwnership(JdbcTemplate jdbc) {
64          this.jdbc = jdbc;
65      }
66
67      /** The school of that record; empty when it does not exist (the controller answers 404). */
68      public Optional<Long> schoolOf(Kind kind, long id) {
69          return cache.get(kind + ":" + id, k -> {
70              List<Long> rows = jdbc.queryForList(SQL.get(kind), Long.class, id);
71              return rows.isEmpty() || rows.get(0) == null ? Optional.empty() : Optional.of(rows.get(0));
72          });
73      }
```

Har bir tur uchun kichik SQL (masalan, o'quvchi uchun `students → school_class → academic_year`) `JdbcTemplate` bilan bajariladi. JPA bu yerda kerak emas: bitta raqam kerak, entity yuklash ortiqcha. Natija **5 daqiqa** keshlanadi, chunki yozuv boshqa maktabga ko'chmaydi. Shu sababli tekshiruv amalda deyarli bepul.

🧩 **Maktab qorovuli**

`SchoolScopeConfig` — bino eshigidagi qorovul. U har bir kiruvchining (so'rov) qo'lidagi qog'ozlardagi (URL, parametrlar, JSON) har bir xona raqamini (id) ro'yxatga (SchoolOwnership) qarab tekshiradi: "bu xona sizning maktabingizdami?". Qorovul biror xonani ro'yxatdan topa olmasa, odatda o'tkazib yuboradi, chunki "bunday xona yo'q, ichkarida 404 bo'ladi" deb hisoblaydi. Aynan shu odat quyidagi zaiflikni keltirib chiqargan.

⚠️ Topilgan jiddiy zaifliklar

⚠️ **Foydalanuvchilar moduli maktab chegarasini to'liq hisobga olmaydi**

`UserService` foydalanuvchi yaratish va tahrirlashda chaqiruvchining maktabini tekshirmaydi. `SchoolOwnership` esa xodimga bog'lanmagan hisoblarning egasini aniqlay olmaydi va bunday holatda tekshiruvni o'tkazib yuboradi. Natijada maktabga qamalgan administrator o'z maktabi doirasidan tashqaridagi hisoblarga ta'sir qila oladi. Bu **imtiyozni oshirish** (privilege escalation) turidagi zaiflik. U shu qo'llanmani yozish paytida test bazasida tasdiqlandi.

Tavsiya:

1. `UserService` da: chaqiruvchi maktabga qamalgan bo'lsa, `employeeId` **majburiy** va o'sha maktab xodimi bo'lishi shart. Platforma hisobini faqat platforma hisobi yarata, tahrirlay va o'chira oladi.
2. `SchoolScopeConfig.require()` da: egasi aniqlanmagan yozuvni maktabga qamalgan foydalanuvchi uchun **rad etish** ("default deny"), keyin aniq istisnolar qo'shish.
3. Bu holatlar uchun integratsiya testi: maktab A admini → platforma adminini tahrirlash = 403.

Metod	URL	Rol	Java metodi	Nima qiladi
POST	`/api/auth/login`	ochiq	`login()`	Login: to'g'ri bo'lsa JWT token qaytaradi
GET	`/api/auth/hash/{raw}`	ADMIN	`hashPassword()`	Parolning BCrypt xeshini qaytaradi (faqat ADMIN)
GET	`/api/users`	ADMIN	`getAllUsers()`	Foydalanuvchi ro'yxati (sahifalab)
GET	`/api/users/{id}`	ADMIN	`getUserById()`	Bitta foydalanuvchi (id bo'yicha)
POST	`/api/users`	ADMIN	`createUser()`	Yangi foydalanuvchi yaratadi
PUT	`/api/users/{id}`	ADMIN	`updateUser()`	Foydalanuvchini tahrirlaydi
DELETE	`/api/users/{id}`	ADMIN	`deleteUser()`	Foydalanuvchini o'chiradi

📌 **Esda tut**

- Uch rol: ADMIN, EDITOR, VIEWER; parollar faqat BCrypt xeshi sifatida saqlanadi.
- Xodimga bog'langan foydalanuvchi o'z maktabiga qamalgan; xodimsiz — platforma hisobi.
- Maktab chegarasi id'ni URL'da, query'da va JSON body'da avtomatik tekshiradi (interceptor + `RequestBodyAdvice` + reflection).
- Egalik `JdbcTemplate` bilan topiladi va keshlanadi — tekshiruv arzon.
- "Egasi topilmadi → o'tkazib yuborish" odati foydalanuvchilar moduli bilan birgalikda ikki jiddiy zaiflik hosil qiladi; "default deny" kerak.

15. Xavfsizlik: Spring Security va JWT

Asosiy tushunchalar

Atama	Ma'nosi	Loyihada
Autentifikatsiya	"Sen kimsan?"	login va parol → JWT token
Avtorizatsiya	"Senga bu mumkinmi?"	rollar (`@PreAuthorize`) va maktab chegarasi (14-bob)
JWT	server imzolagan "pasport": ichida foydalanuvchi nomi va roli	`JwtUtil`, 10 soat amal qiladi
Stateless	server sessiya saqlamaydi, har so'rov tokenini o'zi olib keladi	`SessionCreationPolicy.STATELESS`
Filtr zanjiri	so'rov controller'ga yetguncha o'tadigan tekshiruvlar	CORS → `JwtAuthFilter` → avtorizatsiya

> 🧩 **JWT — muhrlangan ruxsatnoma**
>
> Login — kirish eshigida pasportni ko'rsatish. Qorovul sizga muhrlangan ruxsatnoma beradi: unda ismingiz, lavozimingiz va amal qilish muddati yozilgan. Keyingi har bir xonaga kirishda faqat ruxsatnomani ko'rsatasiz. Qorovul muhrni tekshiradi va ro'yxatga qaramaydi. Muhrni soxtalashtirib bo'lmaydi, chunki muhr (maxfiy kalit) faqat serverda.

SecurityConfig: qoidalar bir joyda

src/main/java/uz/azizbek/maktabboshqaruv/config/SecurityConfig.java — 22–77-qatorlar

```java
@Configuration
@EnableMethodSecurity
public class SecurityConfig {

    @Autowired
    private JwtAuthFilter jwtAuthFilter;

    @Value("${cors.allowed-origins}")
    private String allowedOrigins;

    @Bean
    public FilterRegistrationBean<JwtAuthFilter> jwtFilterRegistration(JwtAuthFilter filter) {
        FilterRegistrationBean<JwtAuthFilter> registration = new FilterRegistrationBean<>(filter);
        registration.setEnabled(false);
        return registration;
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

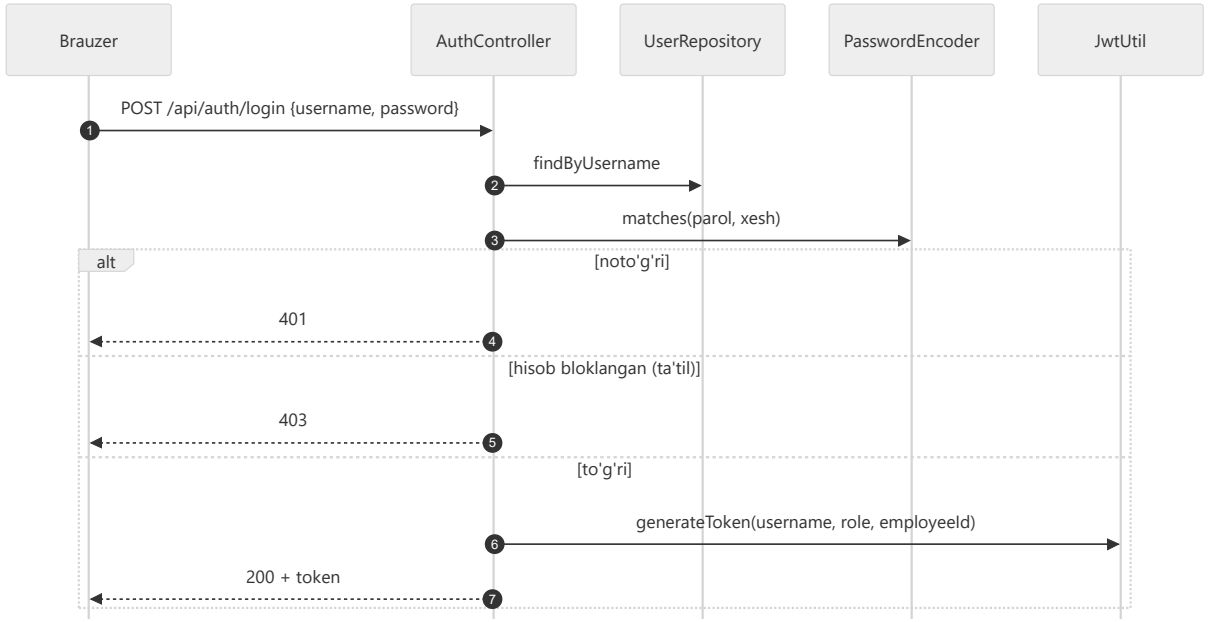
    @Bean
    public CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration configuration = new CorsConfiguration();
        configuration.setAllowedOrigins(Arrays.asList(allowedOrigins.split(",")));
```

```
48          configuration.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));
49          configuration.setAllowedHeaders(List.of("*"));
50          // readable by the frontend: blocked-account marker, download file names, media ranges
51          configuration.setExposedHeaders(List.of("X-Account-Blocked", "Content-Disposition", "Content-Ra
            nge",
52                  "Accept-Ranges", "Content-Length"));
53
54          UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
55          source.registerCorsConfiguration("/**", configuration);
56          return source;
57      }
58
59      @Bean
60      public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
61          http
62                  .cors(cors -> cors.configurationSource(corsConfigurationSource()))
63                  .csrf(csrf -> csrf.disable())
64                  .sessionManagement(sm -> sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
65                  .authorizeHttpRequests(auth -> auth
66                          .requestMatchers("/api/auth/**").permitAll()
67                          .requestMatchers("/api/health").permitAll()
68                          // Mini App: authenticated by Telegram-signed initData inside the controller, n
            ot by JWT.
69                          .requestMatchers("/api/parent/**").permitAll()
70                          // appeal files: a signed 30-minute link for <img>/<video>, checked in the cont
            roller
71                          .requestMatchers(org.springframework.http.HttpMethod.GET, "/api/appeals/*/attac
            hments/*").permitAll()
72                          .anyRequest().authenticated()
73                  )
74                  .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);
75          return http.build();
76      }
77  }
```

Qator	Nima	Nega
23	`@EnableMethodSecurity`	metodlardagi `@PreAuthorize` annotatsiyalarini yoqadi
33–37	`FilterRegistrationBean` + `setEnabled(false)`	`JwtAuthFilter` `@Component` bo'lgani uchun Spring Boot uni oddiy servlet filtri sifatida **ham** ro'yxatga olardi va filtr ikki marta ishlardi. Bu bean uni o'chiradi; filtr faqat Security zanjirida (74-qator) ishlaydi
41	`BCryptPasswordEncoder`	parollarni xeshlash (14-bob)
44–57	CORS	qaysi sayt manzili brauzerdan API'ga murojaat qila olishi (`cors.allowed-origins`)
63	`csrf.disable()`	CSRF hujumi brauzer **cookie**ni avtomatik yuborishiga tayanadi. Token esa cookie'da emas, `Authorization` header'ida qo'lda yuboriladi, shuning uchun CSRF himoyasi bu yerda kerak emas
64	`STATELESS`	`HttpSession` yaratilmaydi: har so'rov mustaqil, server xotirasida sessiya yo'q
66–71	ochiq manzillar	login, health, Mini App (o'z imzosi bilan), murojaat fayllari (imzolangan havola bilan)
72	`anyRequest().authenticated()`	qolgan hamma narsa — faqat tizimga kirganlar uchun
74	`addFilterBefore(jwtAuthFilter, ...)`	JWT filtri Spring'ning standart login filtridan **oldin** joylashadi

> 🤔 **"Default deny" — standart holatda taqiqlash**
>
> 72-qator muhim: ro'yxatda ochiq deb ko'rsatilmagan **har qanday** yangi endpoint avtomatik ravishda login talab qiladi. Dasturchi yangi controller yozib, himoyani unutib qo'ysa ham, u ochiq qolmaydi. Xavfsizlik qoidasi "ruxsat berilganidan boshqa hammasi taqiqlangan" shaklida bo'lishi kerak, aksincha emas.

Login



15.1-rasm. Login ketma-ketligi.

`src/main/java/uz/azizbek/maktabboshqaruv/controller/AuthController.java` — 30–45-qatorlar

```
30      @PostMapping("/login")
31      public ResponseEntity<?> login(@RequestBody LoginRequest request) {
32          User user = userRepository.findByUsername(request.getUsername()).orElse(null);
33
34          if (user == null || !passwordEncoder.matches(request.getPassword(), user.getPassword())) {
35              return ResponseEntity.status(401).body("Login yoki parol noto'g'ri");
36          }
37          // checked only after the password, so the 403 never reveals that a username exists
38          if (accountStatusService.isBlocked(user)) {
39              return ResponseEntity.status(403).body(AccountStatusService.BLOCKED_MESSAGE);
40          }
41
42          Long employeeId = user.getEmployee() != null ? user.getEmployee().getId() : null;
43          String token = jwtUtil.generateToken(user.getUsername(), user.getRole().name(), employeeId);
44          return ResponseEntity.ok(token);
45      }
```

- **34** — foydalanuvchi topilmadi yoki parol noto'g'ri — **bir xil** javob (401). Javobga qarab "bunday login bor-yo'qligi"ni bilib bo'lmaydi.
- **38** — hisob holati faqat parol to'g'ri bo'lgandan **keyin** tekshiriladi (izohda ham shunday yozilgan).
- **43** — token yaratiladi.

Token: yaratish va tekshirish

src/main/java/uz/azizbek/maktabboshqaruv/util/JwtUtil.java 13–69-qatorlar

```
13  @Component
14  public class JwtUtil {
15
16      private final SecretKey key;
17      private final long expirationMs = 1000 * 60 * 60 * 10; // 10 soat
18
19      public JwtUtil(@Value("${jwt.secret}") String secret) {
20          this.key = Keys.hmacShaKeyFor(Decoders.BASE64.decode(secret));
21      }
22
23      public String generateToken(String username, String role) {
24          return generateToken(username, role, null);
25      }
26
27      public String generateToken(String username, String role, Long employeeId) {
28          var builder = Jwts.builder()
29                  .subject(username)
30                  .claim("role", role);
31          if (employeeId != null) {
32              builder.claim("employeeId", employeeId);
33          }
34          return builder
35                  .issuedAt(new Date())
36                  .expiration(new Date(System.currentTimeMillis() + expirationMs))
37                  .signWith(key)
38                  .compact();
39      }
40
41      public String extractUsername(String token) {
42          return extractClaims(token).getSubject();
43      }
44
45      public String extractRole(String token) {
46          return extractClaims(token).get("role", String.class);
47      }
48
49      public Long extractEmployeeId(String token) {
50          Object v = extractClaims(token).get("employeeId");
51          return v == null ? null : Long.valueOf(v.toString());
52      }
53
54      public boolean isTokenValid(String token) {
55          try {
56              extractClaims(token);
57              return true;
58          } catch (Exception e) {
59              return false;
60          }
61      }
62
63      private Claims extractClaims(String token) {
64          return Jwts.parser()
65                  .verifyWith(key)
66                  .build()
67                  .parseSignedClaims(token)
68                  .getPayload();
69      }
```

JWT uch qismdan iborat: `header.payload.signature` (Base64URL bilan kodlangan, nuqta bilan ajratilgan).

Qism	Ichida	Loyihada
header	algoritm	HMAC-SHA (kalit uzunligiga qarab, `Keys.hmacShaKeyFor`)
payload	"claim"lar	`sub` = username (29), `role` (30), `employeeId`, `iat` (35), `exp` (36)
signature	`HMAC(header.payload, maxfiy kalit)`	37-qator

- **20** — maxfiy kalit `jwt.secret` (Base64) dan olinadi. U `${JWT_SECRET}` muhit o'zgaruvchisidan keladi va kodda yozilmagan.
- **17** — amal qilish muddati 10 soat (bitta ish kuni).
- **64–67** — `parseSignedClaims` imzoni **va** muddatni birga tekshiradi. Imzo mos kelmasa yoki muddat o'tgan bo'lsa, istisno tashlanadi.

> ⚠️ **Payload shifrlanmagan**
>
> JWT **imzolangan**, lekin **shifrlanmagan**: payload'ni har kim Base64'dan o'girib o'qiy oladi. Imzo faqat uni **o'zgartirib bo'lmasligini** kafolatlaydi. Shuning uchun tokenga parol yoki maxfiy ma'lumot hech qachon qo'yilmaydi. Bu loyihada ham faqat username, rol va xodim id'si bor.

Har bir so'rovda: JwtAuthFilter

src/main/java/uz/azizbek/maktabboshqaruv/config/JwtAuthFilter.java — 19–64-qatorlar

```
19  @Component
20  public class JwtAuthFilter extends OncePerRequestFilter {
21
22      @Autowired
23      private JwtUtil jwtUtil;
24
25      @Autowired
26      private AccountStatusService accountStatusService;
27
28      @Override
29      protected void doFilterInternal(HttpServletRequest request,
30                                      HttpServletResponse response,
31                                      FilterChain filterChain) throws ServletException, IOException {
32
33          String authHeader = request.getHeader("Authorization");
34
35          if (authHeader != null && authHeader.startsWith("Bearer ")) {
36              String token = authHeader.substring(7);
37
38              if (jwtUtil.isTokenValid(token)) {
39                  String username = jwtUtil.extractUsername(token);
40                  String role = jwtUtil.extractRole(token);
41
42                  // a signed token is not enough: the account may have been put on leave since
43                  // (cached for a minute, so this is not a database query per request)
44                  if (!accountStatusService.isUsable(username)) {
45                      response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
46                      response.setHeader("X-Account-Blocked", "1");
47                      response.setContentType("text/plain;charset=UTF-8");
48                      response.getWriter().write(AccountStatusService.BLOCKED_MESSAGE);
49                      return;
50                  }
51
52                  UsernamePasswordAuthenticationToken authentication =
53                          new UsernamePasswordAuthenticationToken(
```

```
54                                  username,
55                                  null,
56                                  List.of(new SimpleGrantedAuthority("ROLE_" + role))
57                          );
58
59                  SecurityContextHolder.getContext().setAuthentication(authentication);
60              }
61          }
62
63          filterChain.doFilter(request, response);
64      }
```

- **20** — `OncePerRequestFilter`: bitta so'rovda filtr bir martadan ortiq ishlamaydi.
- **33–36** — `Authorization: Bearer <token>` header'idan token ajratiladi.
- **38** — imzo va muddat tekshiruvi.
- **44** — imzo to'g'ri bo'lsa ham, hisob hozir **ishlatilishi mumkinmi** tekshiriladi (ta'til). Buning natijasi 1 daqiqa keshlanadi, shuning uchun bazaga har so'rovda bormaydi.
- **52–59** — `SecurityContextHolder`ga autentifikatsiya qo'yiladi. Rol `ROLE_` prefiksi bilan saqlanadi, chunki `hasRole('ADMIN')` aynan `ROLE_ADMIN`ni qidiradi.
- **63** — zanjir davom etadi. Token bo'lmasa, kontekst bo'sh qoladi va himoyalangan manzil 403 qaytaradi.

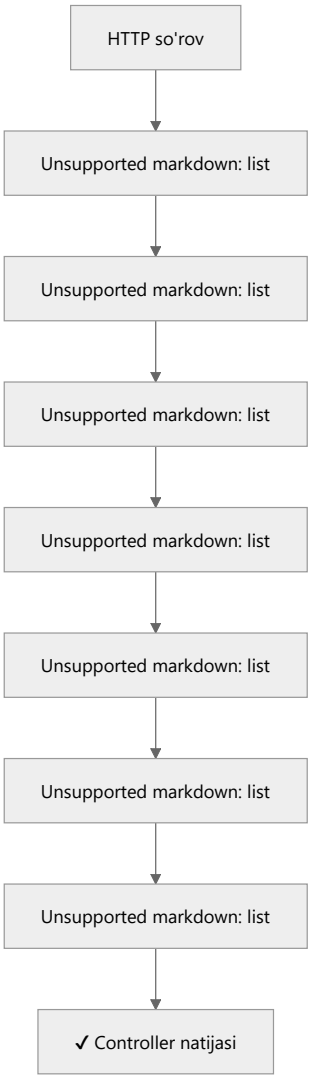
> 🔬 **SecurityContextHolder qayerda saqlanadi?**
> Standart holatda `ThreadLocal` da, ya'ni so'rovni bajarayotgan oqimning shaxsiy "cho'ntagi"da. Shuning uchun istalgan servisda `SecurityContextHolder.getContext().getAuthentication().getName()` bilan joriy foydalanuvchini olish mumkin (`GradeService.currentAuthor`, `ActivityLogService`). So'rov tugagach, Spring kontekstni tozalaydi.

Metod darajasidagi himoya: @PreAuthorize

```
@PreAuthorize("hasAnyRole('ADMIN', 'EDITOR')")
@PostMapping
public ResponseEntity<StudentResponseDto> createStudent(...)
```

Spring controller atrofida **proxy** yaratadi. Metod chaqirilishidan oldin u SpEL ifodasini (`hasAnyRole(...)`) joriy foydalanuvchining rollari bilan solishtiradi. Mos kelmasa, `AccessDeniedException` tashlaydi va `GlobalExceptionHandler` uni 403 javobga aylantiradi (16-bob).

Himoya qatlamlari bir rasmda



15.2-rasm. "Chuqur himoya" (defense in depth): har bir qatlam o'z savoliga javob beradi.

📌 Esda tut

- Login → JWT (10 soat), keyin har so'rovda `Authorization: Bearer` header'i.
- `STATELESS`: server sessiya saqlamaydi; CSRF o'chirilgan, chunki token cookie'da emas.
- `anyRequest().authenticated()` — ochiq deb belgilanmagan hamma narsa yopiq.
- JWT imzolangan, lekin shifrlanmagan — ichiga maxfiy narsa qo'yilmaydi.
- Rollar `@PreAuthorize` bilan metod darajasida, `@EnableMethodSecurity` orqali tekshiriladi.

16. Xatolarni boshqarish

Yagona joyda: GlobalExceptionHandler

Service'lar xato bo'lganda **exception tashlaydi** (`throw new IllegalStateException("...")`), HTTP haqida o'ylamaydi. Exception'ni HTTP javobga aylantirish — bitta klassning vazifasi:

src/main/java/uz/azizbek/maktabboshqaruv/exception/GlobalExceptionHandler.java 11–70-qatorlar

```
11  @RestControllerAdvice
12  public class GlobalExceptionHandler {
13
14      private static final Logger log = LoggerFactory.getLogger(GlobalExceptionHandler.class);
15
16      @ExceptionHandler(IllegalStateException.class)
17      public ResponseEntity<String> handleIllegalState(IllegalStateException e) {
18          return ResponseEntity.status(409).body(e.getMessage());
19      }
20
21      /** Another school's data, someone else's appeal, or a role that may not do this — 403, not 500. */
22      @ExceptionHandler({ForbiddenException.class, org.springframework.security.access.AccessDeniedExcept
        ion.class})
23      public ResponseEntity<String> handleForbidden(RuntimeException e) {
24          String message = e instanceof ForbiddenException ? e.getMessage() : "Bu amal uchun ruxsat y
        o'q";
25          return ResponseEntity.status(403).body(message);
26      }
27
28      @ExceptionHandler(org.springframework.web.multipart.MaxUploadSizeExceededException.class)
29      public ResponseEntity<String> handleTooBig(Exception e) {
30          return ResponseEntity.status(413).body("Fayl juda katta (ilova — 20 MB gacha, kutubxona kitobi
        — 50 MB gacha)");
31      }
32
33      @ExceptionHandler(MissingServletRequestParameterException.class)
34      public ResponseEntity<String> handleMissingParam(MissingServletRequestParameterException e) {
35          return ResponseEntity.status(400).body(e.getParameterName() + " parametri kiritilishi shart");
36      }
37
38      @ExceptionHandler(MethodArgumentNotValidException.class)
39      public ResponseEntity<String> handleValidation(MethodArgumentNotValidException e) {
40          String message = e.getBindingResult().getFieldErrors().stream()
41                  .map(err -> err.getField() + ": " + err.getDefaultMessage())
42                  .findFirst().orElse("Noto'g'ri ma'lumot");
43          return ResponseEntity.status(400).body(message);
44      }
45
46      /** A JSON body sent where files are expected (e.g. an appeal reply). */
47      @ExceptionHandler(org.springframework.web.multipart.MultipartException.class)
48      public ResponseEntity<String> handleNotMultipart(Exception e) {
49          return ResponseEntity.status(415).body("So'rov multipart/form-data bo'lishi kerak");
50      }
51
52      /**
53       * An address nothing answers (e.g. a new page talking to an older backend): a plain 404 for
54       * the user instead of Spring's internal "No static resource …" text.
55       */
56      @ExceptionHandler(org.springframework.web.servlet.resource.NoResourceFoundException.class)
57      public ResponseEntity<java.util.Map<String, String>> handleNoResource(Exception e) {
58          log.warn("Manzil topilmadi: {}", e.getMessage());
```

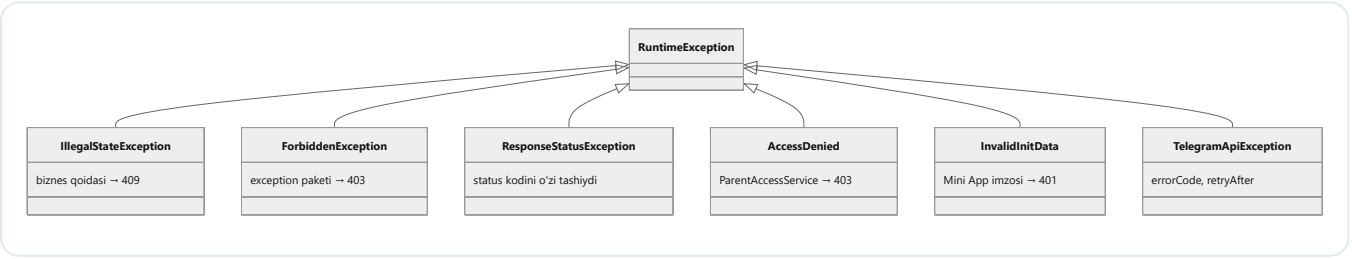
```
59          return ResponseEntity.status(404).body(java.util.Map.of("message", "Manzil topilmadi"));
60      }
61
62      @ExceptionHandler(Exception.class)
63      public ResponseEntity<String> handleGeneral(Exception e) {
64          // Spring's own errors (404, 405, 415, ...) keep their status instead of becoming 500
65          if (e instanceof org.springframework.web.ErrorResponse er) {
66              return ResponseEntity.status(er.getStatusCode()).body(er.getBody().getDetail());
67          }
68          log.error("Kutilmagan xato", e);
69          return ResponseEntity.status(500).body("Kutilmagan xato: " + e.getMessage());
70      }
```

- **11,** `@RestControllerAdvice` — bu klassdagi `@ExceptionHandler` metodlari **barcha** controller'lar uchun ishlaydi.
- Spring tashlangan exception'ga **eng mos** (eng aniq turdagi) handler'ni tanlaydi. Hech biri mos kelmasa, eng umumiy `Exception` handler'i ishlaydi.
- **62–69** — umumiy handler: Spring'ning o'z xatolari (`ResponseStatusException` , 405, 415...) `ErrorResponse` interfeysini amalga oshiradi va **o'z status kodi** bilan qaytariladi. Qolgan kutilmagan xatolar logga to'liq yoziladi va **500** qaytaradi.

Exception → HTTP status jadvali

Exception	Kim tashlaydi	Status	Misol
`MethodArgumentNotValidException`	`@Valid`	**400**	`firstName: Ism kiritilishi shart`
`MissingServletRequestParameterException`	majburiy `?param` yo'q	**400**	`schoolId parametri kiritilishi shart`
`ResponseStatusException(BAD_REQUEST)`	service (manzil, kutubxona)	**400**	`Yashash joyi: tuman tanlangan hududga tegishli emas`
(filtr)	`JwtAuthFilter` , Mini App	**401**	`Hisobingiz vaqtincha o'chirilgan`
`ForbiddenException` , `AccessDeniedException`	maktab chegarasi, `@PreAuthorize`	**403**	`Boshqa maktab ma'lumotiga ruxsat yo'q`
`ResponseStatusException(NOT_FOUND)` , `NoResourceFoundException`	hudud/tuman, noma'lum manzil	**404**	`Hudud topilmadi: 15`
`IllegalStateException`	ko'pchilik service'lar	**409**	`Bu fan ushbu maktabda allaqachon mavjud`
`MaxUploadSizeExceededException`	juda katta fayl	**413**	`Fayl juda katta`
`MultipartException`	fayl o'rniga JSON	**415**	`So'rov multipart/form-data bo'lishi kerak`
boshqa hammasi	kutilmagan xato	**500**	`Kutilmagan xato: ...`

Exception turlari loyihada



16.1-rasm. Loyihadagi asosiy exception'lar. Hammasi `RuntimeException` — `throws` yozish shart emas.

`ParentApiController` o'zining `@ExceptionHandler` metodlariga ega (`InvalidInitData` → 401, `AccessDenied` → 403). **Controller ichidagi** handler shu controller uchun global handler'dan ustun turadi.

> 🤔 **Nega checked emas, RuntimeException?**
> Checked exception (`extends Exception`) har bir chaqiruvchini `throws` yozishga yoki `try/catch` qilishga majburlaydi. Natijada controller → service → repository bo'ylab keraksiz kod ko'payadi. Bundan tashqari, Spring `@Transactional` standart holatda faqat **unchecked** (`RuntimeException`) xatolarda rollback qiladi. Biznes qoidasi buzilganda tranzaksiya aynan bekor qilinishi kerak.

> 💡 **Xabarlar foydalanuvchi uchun**
> Xato matnlari o'zbekcha va to'g'ridan-to'g'ri ekranga chiqadi. Masalan, "*Uzr, bu kun yakshanba — maktab ishlamaydi. Baho qo'yib bo'lmaydi.*" Testlar shu matnlarni aynan tekshiradi (`GradeDateIntegrationTest`). Shu tufayli xabar tasodifan o'zgarib qolsa, test buni ushlaydi.

Kamchiliklar

> ⚠️ **1. "Topilmadi" → 409**
> Eski modullarda "yozuv topilmadi" ham `IllegalStateException` bilan tashlanadi va **409 Conflict** bo'lib qaytadi (masalan, `GET /api/buildings/999999`). HTTP semantikasida bu **404 Not Found** bo'lishi kerak. Yangi modullar (hudud, tuman) buni `ResponseStatusException(NOT_FOUND)` bilan to'g'ri qiladi. Tavsiya: maxsus `NotFoundException` (→ 404) kiritib, eski kodni bosqichma-bosqich o'tkazish.

> ⚠️ **2. Baza cheklovi buzilsa → 500**
> Tashqi kalit yoki UNIQUE cheklovi buzilsa (masalan, xonasi bor binoni o'chirish), Hibernate `DataIntegrityViolationException` tashlaydi. Uning uchun maxsus handler yo'q, shuning uchun foydalanuvchi **500 "Kutilmagan xato"** va texnik SQL matnini ko'radi. Tavsiya: `DataIntegrityViolationException` → 409 va tushunarli xabar ("bu yozuvga bog'liq ma'lumotlar bor"). Ichki SQL matnini foydalanuvchiga ko'rsatmaslik ham kerak.

⚠️ 3. Javob formati bir xil emas

Ba'zi handler'lar oddiy matn (`String`), `NoResourceFoundException` handler'i esa JSON (`{"message": "..."}`) qaytaradi. Frontend ikkala holatni ham qayta ishlashga majbur. Tavsiya: barcha xatolar uchun bitta format, masalan Spring'ning standart **`ProblemDetail`** (RFC 9457).

📌 Esda tut

- Service exception tashlaydi, HTTP javobini esa `@RestControllerAdvice` yasaydi.
- 400 — noto'g'ri kiritish, 401 — kim ekani noma'lum, 403 — ruxsat yo'q, 404 — topilmadi, 409 — qoida buzildi, 500 — kutilmagan xato.
- `RuntimeException` tranzaksiyani avtomatik rollback qiladi.
- Yaxshilash uchun: `NotFoundException` → 404, `DataIntegrityViolationException` → 409, yagona `ProblemDetail` formati.

17. Loyihani ishga tushirish

Kerakli dasturlar

Dastur	Versiya	Tekshirish
JDK	21	`java -version`
PostgreSQL	14+ (loyihada 18)	`psql --version`
Git	istalgan	`git --version`
Gradle	**o'rnatish shart emas** — `gradlew.bat` (wrapper) o'zi yuklab oladi	—
IntelliJ IDEA	ixtiyoriy	—

Muhit o'zgaruvchilari

To'liq ro'yxat loyiha ildizidagi `.env.example` faylida. Eng muhimlari:

O'zgaruvchi	Majburiy	Standart	Ma'nosi
`DB_PASSWORD`	**ha**	—	PostgreSQL paroli
`JWT_SECRET`	**ha**	—	JWT imzo kaliti, **Base64**, kamida 32 bayt (tavsiya: 64)
`DB_URL`	yo'q	`jdbc:postgresql://localhost:5432/maktab_db`	baza manzili
`DB_USERNAME`	yo'q	`postgres`	baza foydalanuvchisi
`ADMIN_USERNAME` / `ADMIN_PASSWORD`	yo'q	`admin` / `admin123`	baza bo'sh bo'lsa yaratiladigan birinchi admin — **o'zgartiring**
`CORS_ALLOWED_ORIGINS`	yo'q	`http://localhost:9000`	sayt manzili
`TELEGRAM_ENABLED`, `TELEGRAM_BOT_TOKEN`, `TELEGRAM_BOT_USERNAME`	yo'q	o'chiq	Telegram bot
`TELEGRAM_MOCK`	yo'q	`false`	botni internetsiz sinash
`APP_STORAGE_PATH`	yo'q	`./storage`	yuklangan fayllar papkasi

JWT kalitini yaratish:

```
openssl rand -base64 64
```

PowerShell'da:

```
[Convert]::ToBase64String((1..64 | ForEach-Object { Get-Random -Maximum 256 }) -as [byte[]])
```

> ⚠️ **Maxfiy qiymatlar faylda emas**
> Haqiqiy parol va kalitlar `application.properties` ga **yozilmaydi**. Ular faqat muhit o'zgaruvchilarida yoki `src/main/resources/application-local.properties` da bo'ladi. Ikkinchisi `.gitignore` da turadi, ya'ni GitHub'ga tushmaydi. Bu qo'llanmada ular `<SECRET>` deb ko'rsatilgan.

Qadamma-qadam: lokal ishga tushirish

1. Baza yaratish (pgAdmin yoki `psql`):

```
CREATE DATABASE maktab_db;
```

Jadvallarni qo'lda yaratish shart emas: Hibernate (`ddl-auto=update`) ularni birinchi ishga tushishda o'zi yaratadi (5-bob).

2. Lokal sozlama fayli:

```
src/main/resources/application-local.properties.example  →  nusxa oling:
src/main/resources/application-local.properties
```

Ichiga yozing:

```
spring.datasource.password=<SECRET>
jwt.secret=<SECRET>
```

3. Ishga tushirish — uchta usuldan biri:

Usul	Buyruq / harakat
Terminal	`set SPRING_PROFILES_ACTIVE=local` keyin `gradlew.bat bootRun`
IntelliJ	Run → `MaktabBoshqaruvApplication (local)` konfiguratsiyasi (`.run/` papkasida tayyor, `local` profil yoqilgan)
Skript	`start.bat` — backend va frontendni alohida oynalarda ochadi

> ⚠️ **Eng ko'p uchraydigan xato**
> IntelliJ'da `main` yonidagi yashil ▶ tugmasi bosilsa, IntelliJ **profilsiz** konfiguratsiya yaratadi. `application-local.properties` yuklanmaydi va ilova quyidagi xato bilan to'xtaydi:
>
> `Could not resolve placeholder 'JWT_SECRET' in value "${JWT_SECRET}"`
>
> Yechim: `(local)` konfiguratsiyasini tanlash yoki `JWT_SECRET` muhit o'zgaruvchisini berish.

4. Tekshirish. Logda quyidagilar chiqadi:

```
Tomcat started on port 8080 (http)
Started MaktabBoshqaruvApplication in 5.4 seconds
```

`http://localhost:8080/api/health` → `200`. Baza bo'sh bo'lsa, `AdminSeeder` birinchi adminni yaratadi, `DataSeeder` demo maktab ma'lumotlarini, `TerritorySeeder` esa hudud va tumanlarni qo'shadi.

5. Login (Postman yoki curl):

```
POST http://localhost:8080/api/auth/login
Content-Type: application/json

{"username": "admin", "password": "<SECRET>"}
```

Javobdagi tokenni keyingi so'rovlarda header'ga qo'ying:

```
GET http://localhost:8080/api/regions
Authorization: Bearer <TOKEN>
```

Testlar

Integratsiya testlari `local` profil bilan va **haqiqiy PostgreSQL** bilan ishlaydi. Ular ishlash bazasini buzmasligi uchun **alohida test bazasini** ko'rsating:

```
DB_URL=jdbc:postgresql://localhost:5432/maktab_tg_test ./gradlew build
```

Buyruq	Nima
`gradlew test`	barcha testlar (`load` va `wikisource` teglarisiz)
`gradlew test --tests '*StudentAddressIntegrationTest'`	bitta test klassi
`gradlew loadTest`	3000 ta ota-ona bilan yuklama testi
`gradlew wikisourceCheck`	haqiqiy Wikisource bilan tekshiruv (internet kerak)

> ⚠️ **DB_URL'ni unutmang**
> `DB_URL` berilmasa, testlar standart `maktab_db` ga ulanadi. Testlar o'zi yaratgan yozuvlarni tozalaydi, lekin ishlab turgan bazada test ishlatish xavfli odat. Har doim alohida test bazasidan foydalaning.

Ishlab chiqarishga (prod) o'tkazish

```
gradlew.bat bootJar
java -jar build/libs/maktab-boshqaruv-0.0.1-SNAPSHOT.jar --spring.profiles.active=local
```

`bootJar` barcha kutubxonalar ichida bo'lgan bitta jar yaratadi. Prod uchun tavsiyalar:

- maxfiy sozlamalarni jar tashqarisida saqlash: `--spring.config.additional-location=file:/path/config/` ;
- `APP_STORAGE_PATH` — doimiy disk yo'li, uni ham zaxiralash;
- yangilashdan oldin bazaning zaxirasi: `pg_dump -Fc maktab_db > zaxira.dump` ;
- `spring.jpa.show-sql=false` .

📌 **Esda tut**

- Majburiy: `DB_PASSWORD` va `JWT_SECRET`; qolganlarining standart qiymati bor.
- Lokal ishlash: `application-local.properties` + `local` profil.
- Jadvallarni Hibernate yaratadi — faqat bo'sh baza kerak.
- Testlar uchun doim alohida baza (`DB_URL=.../maktab_tg_test`).
- Prod: `bootJar`, maxfiy sozlamalar jar tashqarisida, yangilashdan oldin `pg_dump`.

18. Kuchli tomonlar va yaxshilash kerak bo'lgan joylar

Kuchli tomonlar

Soha	Nima yaxshi qilingan	Qayerda
Arxitektura	aniq qatlamlar; biznes qoidalari Service'da, controller'lar yupqa	barcha modullar
Maxfiylik	parol, JWT secret va bot tokeni faylda yozilmagan, muhit o'zgaruvchilaridan keladi; tokenlar logdan tozalanadi	`application.properties`, `TokenMasker`
Parollar	BCrypt; login javobi "foydalanuvchi bormi" degan savolga javob bermaydi	`AuthController`
Imzolar	Telegram `initData` va fayl havolalari HMAC-SHA256 bilan; solishtirish doimiy vaqtli	`TelegramInitDataValidator`, `AttachmentLinks`
Ishonchsiz kiritish	bola id'si chatga bog'lanish bo'yicha qayta tekshiriladi; fayl turi sehrli baytlar bo'yicha; disk nomlari UUID	`ParentAccessService`, `LibraryFiles`, `FileStorage`
Ishonchlilik	outbox pattern, dublikat kaliti, 429/403 ishlovi, tinch soatlar	`NotificationService`, `NotificationSender`
To'g'rilik	event'lar `AFTER_COMMIT` — rollback bo'lgan amal haqida xabar ketmaydi	`NotificationEventListener`
Vaqt	`Clock` bean (Asia/Tashkent), testda boshqariladi	`TelegramConfig`, `GradeService`
Samaradorlik	`join fetch`, `@BatchSize`, guruhlangan so'rovlar, Caffeine keshlar; SQL soni testda o'lchanadi	`JournalQueryCountTest`
Testlar	55 ta fayl: unit (Mockito), haqiqiy HTTP integratsiya, mock Telegram, yuklama testi	`src/test`
Domen qoidalari	o'chirmaslik (fanlar, hududlar — soft delete), huquqiy qoidalar formula sifatida	`SubjectService`, `CopyrightRules`
Matnlar	bot uch tilda, matnlar kodda emas, fayllarda	`BotI18n`

Yaxshilash kerak bo'lgan joylar qayerda

Kamchiliklar alohida ro'yxatga yig'ilmagan. Har biri o'zi tegishli bo'lgan bobda, kod yonida ⚠️ **Diqqat** ramkasida va tavsiyasi bilan berilgan:

Bob	Mavzu
5	`ddl-auto=update` va migratsiya vositalari
6	bino ro'yxatidagi N+1 so'rovlar
8	dars to'qnashuvi tekshiruvi va bir vaqtdagi so'rovlar
10	tranzaksiya ichidagi `try/catch`
11	outbox va bir nechta nusxa

Bob	Mavzu
14	foydalanuvchilar moduli va maktab chegarasi
16	"topilmadi" statusi, baza cheklovlari, javob formati

📌 **Esda tut**

- Loyihaning asosiy kuchi: aniq qatlamlar, outbox, imzolar, testlar va domen qoidalarining kodda aniq ifodalanishi.
- Yaxshilash kerak bo'lgan joylar har bir bobning "Diqqat" ramkalarida, aniq tavsiyalar bilan.

19. Lug'at

Annotatsiyalar

Annotatsiya	Ma'nosi	Qayerda uchraydi
`@SpringBootApplication`	`@Configuration` + `@EnableAutoConfiguration` + `@ComponentScan`	`MaktabBoshqaruvApplication`
`@EnableJpaAuditing`	`@CreatedDate`, `@LastModifiedDate` maydonlarini avtomatik to'ldirish	`MaktabBoshqaruvApplication`
`@Configuration`	bean'lar manbai bo'lgan sozlama klassi	`SecurityConfig`, `TelegramConfig`
`@Bean`	metod qaytargan ob'ekt Spring bean'i bo'ladi	`passwordEncoder()`, `clock()`
`@Component`	umumiy bean	`JwtAuthFilter`, ekranlar
`@Service`	biznes-mantiq bean'i	barcha `*Service`
`@Repository`	ma'lumotlar bean'i (Spring Data interfeyslari uchun shart emas)	—
`@RestController`	`@Controller` + `@ResponseBody`: qaytgan ob'ekt JSON bo'ladi	barcha controller'lar
`@RequestMapping`	klass yoki metod uchun URL prefiksi	`/api/students`
`@GetMapping` va boshqalar	HTTP metodi + URL (`Post`, `Put`, `Delete`, `Patch`)	controller metodlari
`@PathVariable`	URL bo'lagini parametrga olish: `/{id}`	`getById(@PathVariable Long id)`
`@RequestParam`	query parametr: `?schoolId=1`	ro'yxat endpoint'lari
`@RequestBody`	JSON tanasini ob'ektga aylantirish	`create`, `update`
`@RequestHeader`	HTTP header'ini olish	`ParentApiController`
`@ResponseStatus`	muvaffaqiyatli javob statusi	`201`, `204`
`@Valid`	DTO validatsiyasini ishga tushirish	controller'lar
`@NotNull`, `@NotBlank`, `@Size`, `@Min`, `@Max`, `@Past`, `@Pattern`	validatsiya qoidalari	DTO'lar

Annotatsiya	Ma'nosi	Qayerda uchraydi
`@PreAuthorize`	metoddan oldin ruxsat tekshiruvi (SpEL)	controller'lar
`@EnableMethodSecurity`	`@PreAuthorize` ni yoqadi	`SecurityConfig`
`@Transactional`	metod bitta tranzaksiyada; xato bo'lsa rollback	service'lar
`@Transactional(readOnly = true)`	faqat o'qish tranzaksiyasi	o'qish metodlari
`Propagation.REQUIRES_NEW`	har doim yangi, alohida tranzaksiya	`NotificationService.enqueue*`
`@TransactionalEventListener(AFTER_COMMIT)`	event'ni faqat commit'dan keyin qabul qilish	`NotificationEventListener`
`@Scheduled`	vaqt bo'yicha qayta-qayta ishlash (`fixedDelay` , `cron`)	poller, outbox, kundalik ishlar
`@EnableScheduling`	`@Scheduled` ni yoqadi	`TelegramConfig`
`@ConfigurationProperties`	`prefix.*` sozlamalarini klassga bog'lash	`TelegramProperties`
`@Value("${...}")`	bitta sozlamani o'qish	`JwtUtil` , `SecurityConfig`
`@Profile("dev")`	bean faqat shu profilda yaratiladi	demo seeder'lar
`@Order`	bir xil turdagi bean'lar tartibi	seeder'lar
`@Autowired`	bog'liqlikni maydonga kiritish (field injection)	eski klasslar
`@RestControllerAdvice`	barcha controller'lar uchun umumiy xato ushlagich	`GlobalExceptionHandler`
`@ControllerAdvice`	barcha controller'lar uchun umumiy maslahatchi	`SchoolScopeConfig.BodyScope`
`@ExceptionHandler`	exception → HTTP javob	`GlobalExceptionHandler`
`@Entity`	JPA entity (jadval)	`entity` paketi
`@Table`	jadval nomi, UNIQUE cheklovlar, indekslar	entity'lar
`@Id` , `@GeneratedValue(IDENTITY)`	birlamchi kalit, qiymatini baza beradi	entity'lar
`@Column`	ustun: nomi, uzunligi, `nullable` , `unique` , `updatable`	entity'lar
`@ManyToOne` , `@JoinColumn`	ko'pdan-birga bog'lanish va tashqi kalit ustuni	`Student.schoolClass`
`@OneToOne`	birga-bir	`BotSetting.school`
`@ManyToMany` , `@JoinTable` , `@OrderColumn`	ko'pdan-ko'pga, oraliq jadval, tartib	`Book.authors`
`@Enumerated(EnumType.STRING)`	enum bazada matn sifatida	barcha enum maydonlar

Annotatsiya	Ma'nosi	Qayerda uchraydi
`@JdbcTypeCode(SqlTypes.CHAR)`	Hibernate ustun turini aniq belgilash	`Region.state`
`@BatchSize`	LAZY ob'ektlarni guruhlab yuklash (N+1 ga qarshi)	`Book.authors`, `District`
`@PrePersist`, `@PreUpdate`	saqlash yoki yangilashdan oldin chaqiriladigan metod	`Student`, `Region`
`@CreatedDate`, `@LastModifiedDate`	yaratilgan va o'zgargan vaqt	`Grade`, `Position`
`@EntityListeners(AuditingEntityListener.class)`	auditingni entity'ga ulash	`Attendance`, `Grade`
`@Query`	qo'lda yozilgan JPQL yoki SQL so'rovi	repository'lar
`@Modifying`	`@Query` UPDATE/DELETE qiladi	`LessonSlotRepository.moveSubject`
`@Param`	so'rovdagi `:nom` parametri	repository'lar
`@EntityGraph`	so'rovda bog'lanishni birga yuklash	`DistrictRepository`
`@Test`, `@Tag`	JUnit 5 testi va uning tegi	testlar
`@SpringBootTest`, `@ActiveProfiles`	to'liq kontekst bilan test, profil	integratsiya testlari
`@Mock`, `@InjectMocks`	Mockito soxta ob'ektlari	unit testlar

Atamalar

Atama	Izoh
Bean	Spring yaratadigan va boshqaradigan ob'ekt (standart holatda bitta nusxa — singleton)
IoC / DI	Inversion of Control / Dependency Injection: ob'ekt bog'liqliklarini o'zi yaratmaydi, Spring beradi
Proxy	Spring bean atrofida yaratadigan "o'rinbosar" ob'ekt; `@Transactional`, `@PreAuthorize` shu orqali ishlaydi
Entity	jadvalning Java'dagi aksi
DTO	Data Transfer Object — API'ga kiradigan va chiqadigan ma'lumot shakli
Repository	bazaga murojaat qatlami (Spring Data interfeysi)
JPA	Java Persistence API — ORM standarti; **Hibernate** uning amaldagi bajaruvchisi
ORM	ob'ektlar ↔ jadvallar xaritasi
JPQL	entity nomlari bilan yoziladigan so'rov tili
Derived query	metod nomidan avtomatik yasaladigan so'rov (`findBySchoolId`)
Persistence context	Hibernate boshqarayotgan ob'ektlar "daftari"
Dirty checking	o'zgargan boshqariladigan ob'ektni Hibernate'ning o'zi `UPDATE` qilishi
LAZY / EAGER	bog'langan ob'ekt kerak bo'lganda / darhol yuklanadi
N+1	ro'yxat uchun 1 so'rov + har element uchun yana 1 tadan — sekinlik manbai
Tranzaksiya	"hammasi yoki hech narsa": COMMIT yoki ROLLBACK

Atama	Izoh
Soft delete	o'chirish o'rniga holatni belgilash (`active=false` , `state='P'`)
Upsert	bor bo'lsa yangilash, yo'q bo'lsa yaratish
Domain event	"nimadir bo'ldi" degan xabar ob'ekti
Outbox pattern	tashqi tizimga yuboriladigan xabarlarni avval bazaga yozish, keyin alohida yuborish
Long polling	server tashqi API'dan yangilikni o'zi uzoq ochiq so'rov bilan so'raydi
JWT	JSON Web Token — imzolangan, shifrlanmagan token
BCrypt	tuzli va sekin parol xeshlash algoritmi
HMAC-SHA256	maxfiy kalit bilan imzo
CORS	brauzerning boshqa domen API'siga murojaat qilish qoidalari
CSRF	brauzer cookie'larini suiiste'mol qiluvchi hujum turi
Stateless	server so'rovlar orasida sessiya saqlamaydi
Interceptor	controller'dan oldin va keyin ishlaydigan Spring MVC "tutqichi"
Filter	Servlet darajasidagi tutqich (Spring Security filtrlari)
SpEL	Spring Expression Language: `hasAnyRole('ADMIN')`
Back-pressure	navbat to'lganda manbani sekinlashtirish
Idempotent	necha marta bajarilsa ham natija bir xil
Magic bytes	fayl turini aniqlaydigan dastlabki baytlar (`%PDF`)
BOM	Bill of Materials — mos versiyalar ro'yxati
Toolchain	Gradle'ning kerakli JDK versiyasini topishi
SOATO	hududlarning davlat tasniflagich kodi
Mock	test uchun soxta amalga oshirilish

> 📌 **Esda tut**
> Bu lug'at — eslatma. Har bir atamaning to'liq tushuntirishi va loyihadagi misoli o'z bobida keltirilgan.

A-ilova. Barcha endpoint'lar (195 ta)

Bu ilovada backend'dagi **har bir** REST endpoint controller bo'yicha guruhlangan. Jadval kodni avtomatik skanerlash orqali tuzilgan, shuning uchun hech biri tushib qolmagan. **Rol** ustunida `@PreAuthorize` talab qiladigan rollar ko'rsatilgan; `ochiq` — token talab qilinmaydi; `Telegram initData` — Mini App imzosi bilan himoyalangan.

AbsenceRequestController

Metod	URL	Rol	Java metodi	Nima qiladi
GET	`/api/absence-requests`	ADMIN, EDITOR	`list()`	Ariza ro'yxati (sahifalab)
GET	`/api/absence-requests/count-pending`	ADMIN, EDITOR	`countPending()`	Kutilayotgan arizalar soni (menyu nishoni)
GET	`/api/absence-requests/{id}/photo`	ADMIN, EDITOR	`photo()`	Shifokor qog'ozi rasmini Telegram'dan oqim qilib beradi
POST	`/api/absence-requests/{id}/approve`	ADMIN, EDITOR	`approve()`	Arizani tasdiqlaydi — kunlar EXCUSED bo'ladi
POST	`/api/absence-requests/{id}/reject`	ADMIN, EDITOR	`reject()`	Arizani rad etadi (sabab bilan)

AcademicYearController

Metod	URL	Rol	Java metodi	Nima qiladi
GET	`/api/academic-years`	ADMIN, EDITOR, VIEWER	`getAllAcademicYears()`	O'quv yili ro'yxati (sahifalab)
GET	`/api/academic-years/{id}`	ADMIN, EDITOR, VIEWER	`getAcademicYearById()`	Bitta o'quv yili (id bo'yicha)
POST	`/api/academic-years`	ADMIN, EDITOR	`createAcademicYear()`	Yangi o'quv yili yaratadi
PUT	`/api/academic-years/{id}`	ADMIN, EDITOR	`updateAcademicYear()`	O'quv yilini tahrirlaydi
DELETE	`/api/academic-years/{id}`	ADMIN	`deleteAcademicYear()`	O'quv yilini o'chiradi

AnnouncementController

Metod	URL	Rol	Java metodi	Nima qiladi
GET	`/api/announcements`	ADMIN, EDITOR, VIEWER	`getAllAnnouncements()`	E'lon ro'yxati (sahifalab)
GET	`/api/announcements/latest`	ADMIN, EDITOR, VIEWER	`getLatest()`	Eng so'nggi e'lonlar
GET	`/api/announcements/{id}`	ADMIN, EDITOR, VIEWER	`getAnnouncementById()`	Bitta e'lon (id bo'yicha)
POST	`/api/announcements`	ADMIN, EDITOR	`createAnnouncement()`	Yangi e'lon yaratadi
PUT	`/api/announcements/{id}`	ADMIN, EDITOR	`updateAnnouncement()`	E'lonni tahrirlaydi
DELETE	`/api/announcements/{id}`	ADMIN	`deleteAnnouncement()`	E'lonni o'chiradi

AppealController

Metod	URL	Rol	Java metodi	Nima qiladi
GET	`/api/appeals`	ADMIN, EDITOR, VIEWER	`list()`	Murojaat ro'yxati (sahifalab)
GET	`/api/appeals/unread`	ADMIN, EDITOR, VIEWER	`unread()`	O'qilmagan murojaatlar soni (menyu nishoni)
GET	`/api/appeals/{id}`	ADMIN, EDITOR, VIEWER	`chat()`	Bitta murojaat yozishmasi

Metod	URL	Rol	Java metodi	Nima qiladi
POST	/api/appeals/{id}/reply	ADMIN, EDITOR	reply()	Murojaatga javob (matn va fayllar) — ota-onaga Telegram'da boradi
POST	/api/appeals/{id}/close	ADMIN, EDITOR	close()	Murojaatni yopadi
POST	/api/appeals	ADMIN, EDITOR	create()	Yangi murojaat yaratadi
GET	/api/appeals/{id}/attachments/{messageId}	ochiq	attachment()	Murojaat fayli — imzolangan 30 daqiqalik havola bilan

AttendanceController

Metod	URL	Rol	Java metodi	Nima qiladi
GET	/api/attendance/calendar	ADMIN, EDITOR, VIEWER	calendar()	Sinfning oylik davomat kalendari
PUT	/api/attendance/day	ADMIN, EDITOR	markDay()	O'quvchining butun kunini belgilaydi (barcha darslari)
GET	/api/attendance	ADMIN, EDITOR, VIEWER	getAllAttendance()	Davomat yozuvi ro'yxati (sahifalab)
GET	/api/attendance/today-lessons	ADMIN, EDITOR, VIEWER	getTodayLessons()	Bugungi darslar (o'qituvchi uchun)
GET	/api/attendance/roster	ADMIN, EDITOR, VIEWER	getRoster()	Dars ro'yxati: sinf o'quvchilari va ularning holati
POST	/api/attendance/bulk	ADMIN, EDITOR	bulkMark()	Butun sinf davomatini bitta so'rovda saqlaydi
POST	/api/attendance	ADMIN, EDITOR	createAttendance()	Yangi davomat yozuvi yaratadi
PUT	/api/attendance/{id}	ADMIN, EDITOR	updateAttendance()	Davomat yozuvini tahrirlaydi
DELETE	/api/attendance/{id}	ADMIN	deleteAttendance()	Davomat yozuvini o'chiradi

AuthController

Metod	URL	Rol	Java metodi	Nima qiladi
POST	/api/auth/login	ochiq	login()	Login: to'g'ri bo'lsa JWT token qaytaradi
GET	/api/auth/hash/{raw}	ADMIN	hashPassword()	Parolning BCrypt xeshini qaytaradi (faqat ADMIN)

BehaviorRecordController

Metod	URL	Rol	Java metodi	Nima qiladi
GET	/api/behavior-records	ADMIN, EDITOR, VIEWER	getAllRecords()	Xulq yozuvi ro'yxati (sahifalab)
POST	/api/behavior-records	ADMIN, EDITOR	createRecord()	Yangi xulq yozuvi yaratadi
PUT	/api/behavior-records/{id}	ADMIN, EDITOR	updateRecord()	Xulq yozuvini tahrirlaydi
DELETE	/api/behavior-records/{id}	ADMIN	deleteRecord()	Xulq yozuvini o'chiradi

BotAdminController

Metod	URL	Rol	Java metodi	Nima qiladi
GET	/api/bot/stats	ADMIN, EDITOR	stats()	Bot statistikasi
GET	/api/bot/settings	ADMIN, EDITOR	getSettings()	Bot sozlamalari

Metod	URL	Rol	Java metodi	Nima qiladi
PUT	/api/bot/settings	ADMIN	updateSettings()	Botni tahrirlaydi
POST	/api/bot/mock/run-jobs	ADMIN	runJobs()	Mock rejimda avtomatik xabarlarni hozir ishga tushiradi

BroadcastController

Metod	URL	Rol	Java metodi	Nima qiladi
GET	/api/broadcasts	ADMIN	list()	Xabar ro'yxati (sahifalab)
GET	/api/broadcasts/{id}	ADMIN	get()	Bitta xabar
GET	/api/broadcasts/{id}/recipients	ADMIN	recipients()	Xabar kimlarga bordi va holati
POST	/api/broadcasts/preview	ADMIN	preview()	Yuborishdan oldin: nechta ota-onaga boradi
POST	/api/broadcasts	ADMIN	send()	Xabar yuborish (faqat matn, JSON)
POST	/api/broadcasts	ADMIN	sendWithFiles()	Xabar yuborish (fayllar bilan, multipart)

BuildingController

Metod	URL	Rol	Java metodi	Nima qiladi
GET	/api/buildings	ADMIN, EDITOR, VIEWER	getAllBuildings()	Bino ro'yxati (sahifalab)
GET	/api/buildings/{id}	ADMIN, EDITOR, VIEWER	getBuildingById()	Bitta bino (id bo'yicha)
POST	/api/buildings	ADMIN, EDITOR	createBuilding()	Yangi bino yaratadi
PUT	/api/buildings/{id}	ADMIN, EDITOR	updateBuilding()	Binoni tahrirlaydi
DELETE	/api/buildings/{id}	ADMIN	deleteBuilding()	Binoni o'chiradi

CalendarEventController

Metod	URL	Rol	Java metodi	Nima qiladi
GET	/api/calendar-events	ADMIN, EDITOR, VIEWER	getAllEvents()	Tadbir ro'yxati (sahifalab)
GET	/api/calendar-events/range	ADMIN, EDITOR, VIEWER	getInRange()	Sanalar oralig'idagi tadbirlar
GET	/api/calendar-events/upcoming	ADMIN, EDITOR, VIEWER	getUpcoming()	Kelgusi tadbirlar
POST	/api/calendar-events	ADMIN, EDITOR	createEvent()	Yangi tadbir yaratadi
PUT	/api/calendar-events/{id}	ADMIN, EDITOR	updateEvent()	Tadbirni tahrirlaydi
DELETE	/api/calendar-events/{id}	ADMIN	deleteEvent()	Tadbirni o'chiradi

DashboardController

Metod	URL	Rol	Java metodi	Nima qiladi
GET	/api/dashboard/overview	ADMIN, EDITOR, VIEWER	getOverview()	Bosh sahifa: umumiy ko'rsatkichlar
GET	/api/dashboard/summary	ADMIN, EDITOR, VIEWER	getSummary()	Bosh sahifa: qisqa xulosa (sonlar)
GET	/api/dashboard/live-lessons	ADMIN, EDITOR, VIEWER	getLiveLessons()	Hozir o'tayotgan darslar
GET	/api/dashboard/live-lessons-summary	ADMIN, EDITOR, VIEWER	getLiveLessonsSummary()	Jonli darslar xulosasi

Metod	URL	Rol	Java metodi	Nima qiladi
GET	/api/dashboard/attendance-trend	ADMIN, EDITOR, VIEWER	getAttendanceTrend()	Davomat trendi (kunlar bo'yicha)
GET	/api/dashboard/class-rankings	ADMIN, EDITOR, VIEWER	getClassRankings()	Sinflar reytingi
GET	/api/dashboard/subject-averages	ADMIN, EDITOR, VIEWER	getSubjectAverages()	Fanlar bo'yicha o'rtacha baho
GET	/api/dashboard/absentees-today	ADMIN, EDITOR, VIEWER	getAbsenteesToday()	Bugun kelmaganlar
GET	/api/dashboard/attention	ADMIN, EDITOR, VIEWER	getAttentionItems()	E'tibor talab qiladigan holatlar
GET	/api/dashboard/activity	ADMIN, EDITOR, VIEWER	getRecentActivity()	So'nggi faoliyat lentasi

DistrictController

Metod	URL	Rol	Java metodi	Nima qiladi
GET	/api/districts	ADMIN, EDITOR, VIEWER	getAll()	Tuman ro'yxati
GET	/api/districts/{id}	ADMIN, EDITOR, VIEWER	getById()	Bitta tuman (id bo'yicha)
POST	/api/districts	ADMIN, EDITOR	create()	Yangi tuman yaratadi
PUT	/api/districts/{id}	ADMIN, EDITOR	update()	Tumanni tahrirlaydi
DELETE	/api/districts/{id}	ADMIN	delete()	Tumanni o'chiradi

EmployeeController

Metod	URL	Rol	Java metodi	Nima qiladi
GET	/api/employees	ADMIN, EDITOR, VIEWER	getAllEmployees()	Xodim ro'yxati (sahifalab)
GET	/api/employees/{id}	ADMIN, EDITOR, VIEWER	getEmployeeById()	Bitta xodim (id bo'yicha)
POST	/api/employees	ADMIN, EDITOR	createEmployee()	Yangi xodim yaratadi
PUT	/api/employees/{id}	ADMIN, EDITOR	updateEmployee()	Xodimni tahrirlaydi
DELETE	/api/employees/{id}	ADMIN	deleteEmployee()	Xodimni o'chiradi

GradeController

Metod	URL	Rol	Java metodi	Nima qiladi
GET	/api/grades	ADMIN, EDITOR, VIEWER	getAllGrades()	Baho ro'yxati (sahifalab)
GET	/api/grades/gradebook	ADMIN, EDITOR, VIEWER	getGradebook()	Baholar daftari: sinf · fan · sanalar oralig'i
GET	/api/grades/journal	ADMIN, EDITOR, VIEWER	journal()	Sinf jurnali: bir oy · bir fan
POST	/api/grades	ADMIN, EDITOR	createGrade()	Yangi baho yaratadi
PUT	/api/grades/{id}	ADMIN, EDITOR	updateGrade()	Bahoni tahrirlaydi
DELETE	/api/grades/{id}	ADMIN, EDITOR	deleteGrade()	Bahoni o'chiradi

HealthController

Metod	URL	Rol	Java metodi	Nima qiladi
GET	/api/health	ochiq	health()	Backend ishlayaptimi (login'siz)

JournalController

Metod	URL	Rol	Java metodi	Nima qiladi
GET	/api/classes/{id}/overview	ADMIN, EDITOR, VIEWER	overview()	Sinf paneli: sinf rahbari, bugungi darslar
GET	/api/me/current-lesson	ADMIN, EDITOR, VIEWER	currentLesson()	Tizimga kirgan o'qituvchining hozirgi darsi

LessonSlotController

Metod	URL	Rol	Java metodi	Nima qiladi
GET	/api/lesson-slots	ADMIN, EDITOR, VIEWER	getAllLessonSlots()	Dars ro'yxati (sahifalab)
GET	/api/lesson-slots/timetable	ADMIN, EDITOR, VIEWER	getTimetable()	Haftalik dars jadvali
GET	/api/lesson-slots/timetable.pdf	ADMIN, EDITOR, VIEWER	getTimetablePdf()	Haftalik dars jadvali PDF (har sinfga bitta sahifa)
GET	/api/lesson-slots/{id}	ADMIN, EDITOR, VIEWER	getLessonSlotById()	Bitta dars (id bo'yicha)
POST	/api/lesson-slots	ADMIN, EDITOR	createLessonSlot()	Yangi dars yaratadi
PUT	/api/lesson-slots/{id}	ADMIN, EDITOR	updateLessonSlot()	Darsni tahrirlaydi
DELETE	/api/lesson-slots/{id}	ADMIN	deleteLessonSlot()	Darsni o'chiradi

LibraryController

Metod	URL	Rol	Java metodi	Nima qiladi
GET	/api/library/books	ADMIN, EDITOR	list()	Kitob ro'yxati
GET	/api/library/books/{id}	ADMIN, EDITOR	get()	Bitta kitob
GET	/api/library/books/duplicates	ADMIN, EDITOR	duplicates()	Shu PDF maktabda allaqachon bormi (SHA-256)
POST	/api/library/books	ADMIN, EDITOR	create()	Yangi kitob yaratadi
PUT	/api/library/books/{id}	ADMIN, EDITOR	update()	Kitobni tahrirlaydi
POST	/api/library/books/drafts	ADMIN, EDITOR	draft()	Ommaviy yuklash: bitta PDF → bitta qoralama kartochka
DELETE	/api/library/books/{id}	ADMIN, EDITOR	deactivate()	Kitobni o'chiradi
GET	/api/library/books/{id}/pdf	ADMIN, EDITOR	pdf()	Kitob PDF'i
GET	/api/library/books/{id}/cover	ADMIN, EDITOR	cover()	Kitob muqovasi
GET	/api/library/categories	ADMIN, EDITOR	categories()	Kutubxona bo'limlari
POST	/api/library/categories	ADMIN, EDITOR	createCategory()	Yangi kitob yaratadi
PUT	/api/library/categories/{id}	ADMIN, EDITOR	updateCategory()	Kitobni tahrirlaydi

Metod	URL	Rol	Java metodi	Nima qiladi
DELETE	/api/library/categories/{id}	ADMIN, EDITOR	deleteCategory()	Kitobni o'chiradi
PUT	/api/library/categories/order	ADMIN, EDITOR	reorder()	Bo'limlar tartibini o'zgartiradi

LibraryLegalController

Metod	URL	Rol	Java metodi	Nima qiladi
GET	/api/library/authors	ADMIN, EDITOR	authors()	Mualliflar ro'yxati (holati bilan)
POST	/api/library/authors	ADMIN	createAuthor()	Yangi yozuv yaratadi
PUT	/api/library/authors/{id}	ADMIN	updateAuthor()	Yozuvni tahrirlaydi
DELETE	/api/library/authors/{id}	ADMIN	deleteAuthor()	Yozuvni o'chiradi
GET	/api/library/permissions	ADMIN	permissions()	Ruxsatnomalar ro'yxati
POST	/api/library/permissions	ADMIN	createPermission()	Yangi yozuv yaratadi
PUT	/api/library/permissions/{id}	ADMIN	updatePermission()	Yozuvni tahrirlaydi
POST	/api/library/permissions/{id}/revoke	ADMIN	revoke()	Ruxsatnomani bekor qiladi — kitoblari darhol nofaol
GET	/api/library/permissions/{id}/document	ADMIN	document()	Ruxsatnomaning imzolangan skani
GET	/api/library/permissions/template	ADMIN	template()	Ruxsat xati shabloni (PDF)
GET	/api/library/wikisource/works	ADMIN	wikisourceWorks()	Muallifning Wikisource'dagi asarlari
POST	/api/library/wikisource/import	ADMIN	wikisourceImport()	Asarni Wikisource'dan import qiladi (PDF yaratadi)

LibraryStatsController

Metod	URL	Rol	Java metodi	Nima qiladi
GET	/api/library/stats	ADMIN	overview()	Mutolaa statistikasi
GET	/api/library/stats/journal	ADMIN	journal()	Yuklab olishlar jurnali
GET	/api/library/stats/journal/export	ADMIN	export()	Jurnal Excel faylida
GET	/api/library/stats/parents/{chatId}	ADMIN	parent()	Bitta ota-ona: nimalarni o'qigan

NotificationController

Metod	URL	Rol	Java metodi	Nima qiladi
GET	/api/notifications	ADMIN, EDITOR	getNotifications()	Outbox xabarlari ro'yxati
GET	/api/notifications/stats	ADMIN, EDITOR	getStats()	Xabarnomalar statistikasi
POST	/api/notifications/{id}/retry	ADMIN, EDITOR	retry()	Xabarni qayta yuborish
GET	/api/notifications/settings	ADMIN, EDITOR	getSettings()	Xabarnoma sozlamalari
PUT	/api/notifications/settings	ADMIN	updateSettings()	Xabarnomani tahrirlaydi

ParentApiController

Metod	URL	Rol	Java metodi	Nima qiladi
GET	/api/parent/me	Telegram initData	me()	Ota-ona: tili, bolalari, tanlangan bola
GET	/api/parent/students/{id}/today	Telegram initData	today()	Bola: bugungi xulosa
GET	/api/parent/students/{id}/schedule	Telegram initData	schedule()	Bola: dars jadvali (bugun/ertaga/hafta)
GET	/api/parent/students/{id}/attendance	Telegram initData	attendance()	Bola: davomat (oy)
GET	/api/parent/students/{id}/grades	Telegram initData	grades()	Bola: baholar
GET	/api/parent/students/{id}/events	Telegram initData	events()	Bola: tadbirlar
GET	/api/parent/students/{id}/grades/{subjectId}	Telegram initData	subject()	Bola: bitta fan bo'yicha baholar
GET	/api/parent/students/{id}/announcements	Telegram initData	announcements()	Bola: e'lonlar

PositionController

Metod	URL	Rol	Java metodi	Nima qiladi
GET	/api/positions	ADMIN, EDITOR, VIEWER	getAllPositions()	Lavozim ro'yxati (sahifalab)
GET	/api/positions/{id}	ADMIN, EDITOR, VIEWER	getPositionById()	Bitta lavozim (id bo'yicha)
POST	/api/positions	ADMIN, EDITOR	createPosition()	Yangi lavozim yaratadi
PUT	/api/positions/{id}	ADMIN, EDITOR	updatePosition()	Lavozimni tahrirlaydi
DELETE	/api/positions/{id}	ADMIN	deletePosition()	Lavozimni o'chiradi

ProfileController

Metod	URL	Rol	Java metodi	Nima qiladi
GET	/api/profiles/students/{id}	ADMIN, EDITOR, VIEWER	getStudentProfile()	O'quvchi profili (manzil, davomat, baholar, xulq, jadval)
GET	/api/profiles/teachers/{id}	ADMIN, EDITOR, VIEWER	getTeacherProfile()	O'qituvchi profili
GET	/api/profiles/classes/{id}	ADMIN, EDITOR, VIEWER	getClassProfile()	Sinf profili

RegionController

Metod	URL	Rol	Java metodi	Nima qiladi
GET	/api/regions	ADMIN, EDITOR, VIEWER	getAll()	Hudud ro'yxati
GET	/api/regions/{id}	ADMIN, EDITOR, VIEWER	getById()	Bitta hudud (id bo'yicha)
GET	/api/regions/{id}/districts	ADMIN, EDITOR, VIEWER	getDistricts()	Hududning faol tumanlari
POST	/api/regions	ADMIN, EDITOR	create()	Yangi hudud yaratadi
PUT	/api/regions/{id}	ADMIN, EDITOR	update()	Hududni tahrirlaydi
DELETE	/api/regions/{id}	ADMIN	delete()	Hududni o'chiradi

RoomController

Metod	URL	Rol	Java metodi	Nima qiladi
GET	/api/rooms	ADMIN, EDITOR, VIEWER	getAllRooms()	Xona ro'yxati (sahifalab)
GET	/api/rooms/{id}	ADMIN, EDITOR, VIEWER	getRoomById()	Bitta xona (id bo'yicha)
GET	/api/rooms/{id}/occupancy	ADMIN, EDITOR, VIEWER	getOccupancy()	Xonaning haftalik bandligi
POST	/api/rooms	ADMIN, EDITOR	createRoom()	Yangi xona yaratadi
PUT	/api/rooms/{id}	ADMIN, EDITOR	updateRoom()	Xonani tahrirlaydi
DELETE	/api/rooms/{id}	ADMIN	deleteRoom()	Xonani o'chiradi

SchoolClassController

Metod	URL	Rol	Java metodi	Nima qiladi
GET	/api/school-classes	ADMIN, EDITOR, VIEWER	getAllSchoolClasses()	Sinf ro'yxati (sahifalab)
GET	/api/school-classes/{id}	ADMIN, EDITOR, VIEWER	getSchoolClassById()	Bitta sinf (id bo'yicha)
POST	/api/school-classes	ADMIN, EDITOR	createSchoolClass()	Yangi sinf yaratadi
PUT	/api/school-classes/{id}	ADMIN, EDITOR	updateSchoolClass()	Sinfni tahrirlaydi
DELETE	/api/school-classes/{id}	ADMIN	deleteSchoolClass()	Sinfni o'chiradi

SchoolController

Metod	URL	Rol	Java metodi	Nima qiladi
GET	/api/schools	ADMIN, EDITOR, VIEWER	getAllSchools()	Maktab ro'yxati (sahifalab)
GET	/api/schools/{id}	ADMIN, EDITOR, VIEWER	getSchoolById()	Bitta maktab (id bo'yicha)
POST	/api/schools	ADMIN, EDITOR	createSchool()	Yangi maktab yaratadi
PUT	/api/schools/{id}	ADMIN, EDITOR	updateSchool()	Maktabni tahrirlaydi
DELETE	/api/schools/{id}	ADMIN	deleteSchool()	Maktabni o'chiradi

StudentController

Metod	URL	Rol	Java metodi	Nima qiladi
GET	/api/students	ADMIN, EDITOR, VIEWER	getAllStudents()	O'quvchi ro'yxati (sahifalab)
GET	/api/students/{id}	ADMIN, EDITOR, VIEWER	getStudentById()	Bitta o'quvchi (id bo'yicha)
POST	/api/students	ADMIN, EDITOR	createStudent()	Yangi o'quvchi yaratadi
PUT	/api/students/{id}	ADMIN, EDITOR	updateStudent()	O'quvchini tahrirlaydi
DELETE	/api/students/{id}	ADMIN	deleteStudent()	O'quvchini o'chiradi

SubjectController

Metod	URL	Rol	Java metodi	Nima qiladi
GET	/api/subjects	ADMIN, EDITOR, VIEWER	getAllSubjects()	Fan ro'yxati (sahifalab)
PUT	/api/subjects/{id}/activate	ADMIN, EDITOR	activate()	Nofaol fanni qayta faollashtiradi
GET	/api/subjects/{id}	ADMIN, EDITOR, VIEWER	getSubjectById()	Bitta fan (id bo'yicha)
POST	/api/subjects	ADMIN, EDITOR	createSubject()	Yangi fan yaratadi

Metod	URL	Rol	Java metodi	Nima qiladi
PUT	/api/subjects/{id}	ADMIN, EDITOR	updateSubject()	Fanni tahrirlaydi
DELETE	/api/subjects/{id}	ADMIN	deleteSubject()	Fanni o'chiradi

TelegramController

Metod	URL	Rol	Java metodi	Nima qiladi
GET	/api/telegram/status	ADMIN, EDITOR, VIEWER	getStatus()	Telegram bot holati (rejim, ulanish)
GET	/api/telegram/students/{studentId}	ADMIN, EDITOR	getStudentTelegram()	O'quvchining bog'lash kodi va ulangan ota-onalari
POST	/api/telegram/students/{studentId}/regenerate-code	ADMIN, EDITOR	regenerateCode()	O'quvchiga yangi bog'lash kodi
DELETE	/api/telegram/links/{linkId}	ADMIN, EDITOR	unlink()	Telegramni o'chiradi
GET	/api/telegram/classes/{schoolClassId}/codes	ADMIN, EDITOR	getClassCodes()	Sinf o'quvchilarining kodlari (chop etish uchun)
GET	/api/telegram/files/{fileId}	ADMIN, EDITOR	file()	Ota-ona yuborgan rasm
POST	/api/telegram/mock/updates	ADMIN	simulateUpdate()	Mock: soxta ota-ona xabarini botga beradi
GET	/api/telegram/mock/messages	ADMIN	mockMessages()	Mock: bot «yuborgan» xabarlar
GET	/api/telegram/mock/photos/{fileId}	ADMIN	mockPhoto()	Mock: bot «yuborgan» rasm

TelegramParentsController

Metod	URL	Rol	Java metodi	Nima qiladi
GET	/api/telegram/parents	ADMIN, EDITOR, VIEWER	byClass()	Sinf bo'yicha ota-onalar (ulangan / ulanmagan)
GET	/api/telegram/parents/choices	ADMIN	choices()	Xabar uchun ota-ona tanlash ro'yxati

UserController

Metod	URL	Rol	Java metodi	Nima qiladi
GET	/api/users	ADMIN	getAllUsers()	Foydalanuvchi ro'yxati (sahifalab)
GET	/api/users/{id}	ADMIN	getUserById()	Bitta foydalanuvchi (id bo'yicha)
POST	/api/users	ADMIN	createUser()	Yangi foydalanuvchi yaratadi
PUT	/api/users/{id}	ADMIN	updateUser()	Foydalanuvchini tahrirlaydi
DELETE	/api/users/{id}	ADMIN	deleteUser()	Foydalanuvchini o'chiradi

B-ilova. Barcha klasslar katalogi (351 ta)

Har bir Java klassi paketi, turi, qator soni va bir qatorlik vazifasi bilan. Kitobda alohida bob ajratilmagan klasslar ham shu yerda o'z izohi bilan bor.

uz.azizbek.maktabboshqaruv

Klass	Tur	Qator	Vazifasi
`MaktabBoshqaruvApplication`	class	15	Ilovaning kirish nuqtasi: main() Spring Boot'ni ishga tushiradi; @EnableJpaAuditing @CreatedDate/@LastModifiedDate maydonlarini avtomatik to'ldiradi.

bot

Klass	Tur	Qator	Vazifasi
`BotBanners`	class	103	Bo'lim bannerlari — tayyor statik rasmlar; Telegram'ga bir marta yuklanadi va keyin file_id bilan yuboriladi.
`BotContext`	class	140	Bitta ekranni chizish uchun kerak bo'lgan hamma narsa: chat, ota-ona sessiyasi, til va tekshirilgan bola.
`BotI18n`	class	123	Botning barcha matnlari bot/i18n/messages_*.properties fayllaridan (uz lotin, cy kirill, ru) olinadi — kodda matn yozilmaydi.
`BotProfileInitializer`	class	60	Ishga tushishda botning ochiq profilini (buyruqlar menyusi, tavsif, Mini App tugmasi) koddan sozlaydi.
`BotResponder`	class	125	BotView'ni ekranga chiqaradi: «bitta sahifa = bitta xabar» qoidasi bilan yangi xabar yuboradi yoki eskisini tahrirlaydi.
`BotRouter`	class	339	Botga kelgan har bir update'ning markaziy dispetcheri: tugma bosilishi, buyruqlar, kontakt va matnni kerakli ekranga yo'naltiradi.
`BotUpdateDispatcher`	class	97	Update'larni cheklangan oqimlar hovuzida bajaradi: bitta chat ichida tartib saqlanadi, turli chatlar parallel ishlaydi.
`BotView`	class	120	Ekran nimani ko'rsatmoqchi: matn + inline tugmalar, ixtiyoriy banner bilan.
`CallbackData`	class	90	Ixcham callback_data formati «ekran:kalit:qiymat»; 64 bayt chegarasiga sig'adi.
`ChatRateLimiter`	class	40	Bitta chatdan soniyasiga ko'pi bilan N ta update — bazani himoya qiladi.
`InputHandler`	interface	27	Interfeys: suhbat ochiq bo'lganda (ariza, xat) yozilgan matnni qabul qiladigan ekran.
`Keyboards`	class	110	Doimiy pastki klaviatura (asosiy menyu) va bosilgan tugma matnidan ekranni topish.
`MiniAppLinks`	class	27	«📱 Batafsil» tugmasi — mos Mini App sahifasini ochadi.
`Onboarding`	class	90	Birinchi tanishuv: hali bog'lanmagan ota-onaga xush kelibsiz xabari va bog'langandan keyingi qisqa yo'riqnoma.
`PendingData`	class	62	Davom etayotgan suhbat holati («k=v;k=v» ko'rinishida ParentSession'da saqlanadi).
`Screen`	interface	19	Interfeys: bitta bot sahifasi; router callback_data'ning birinchi qismi bo'yicha kerakli Screen'ni chaqiradi.
`SubjectIcons`	class	80	Har bir fan, tadbir turi va baho uchun bir xil emoji.

bot.screens

Klass	Tur	Qator	Vazifasi
AbsenceScreen	class	209	Botdagi «😷 Sababli ariza»: boshlanish sanasi → kunlar soni → sabab → izoh → shifokor qog'ozi rasmi; oxirida ariza yaratiladi.
AnnouncementsScreen	class	94	Botdagi «📢 E'lonlar»: sahifalangan ro'yxat (🆕 o'qilmagan, 🔴 muhim); bosilganda to'liq matn ochiladi va o'qilgan deb belgilanadi.
AttendanceScreen	class	225	Botdagi «✅ Davomat»: oy, chorak yoki o'quv yili bo'yicha kelgan/kelmagan/kechikkan kunlar.
BehaviorScreen	class	74	Botdagi «⭐ Xulq»: avval rag'batlar, keyin ogohlantirishlar, oyma-oy.
ChildrenScreen	class	132	Botdagi «👨‍👧 Farzandlarim»: bog'langan bolalar, faol bolani tanlash va kod bilan yangi bola qo'shish.
EventsScreen	class	61	Botdagi «🗓 Tadbirlar»: kelgusi 60 kundagi bayramlar, imtihonlar, majlislar va ta'tillar.
GradesScreen	class	181	Botdagi «📘 Baholar»: so'nggi baholar, fanlar bo'yicha o'rtacha (chiziqli grafik bilan) va bitta fanning batafsil ro'yxati.
HomeScreen	class	107	Botdagi «🏠 Bosh menyu» — «Bugun» xulosasi va barcha bo'limlarga bitta bosishda o'tish.
LibraryScreen	class	398	Botdagi «📚 Kitob mutolaasi»: bo'limlar, qidiruv, tavsiyalar, sevimlilar, PDF yuborish.
MessageScreen	class	206	Botdagi «✉️ Ma'muriyatga xat»: ota-ona ma'muriyatga yoki sinf rahbariga yozadi (matn, rasm, ovoz, hujjat).
ReportScreen	class	97	Botdagi «📊 Hisobot»: hafta/oy bo'yicha davomat, baho va xulq bitta sahifada.
ScheduleScreen	class	109	Botdagi «🗓 Dars jadvali»: Bugun · Ertaga · Hafta; ▶️ hozirgi darsni ko'rsatadi.
SchoolInfoScreen	class	52	Botdagi «🏫 Maktab haqida»: nom, manzil, telefon, qo'ng'iroq jadvali, direktor qabul kunlari.
SettingsScreen	class	199	Botdagi «⚙️ Sozlamalar»: har bir avtomatik xabarni yoqish/o'chirish, tinch soatlar, bot tili.
TeachersScreen	class	58	Botdagi «👩‍🏫 O'qituvchilar»: sinf rahbari va fan o'qituvchilari (telefon — maktab ruxsat bersa).

config

Klass	Tur	Qator	Vazifasi
AdminSeeder	class	55	Baza bo'sh bo'lsa, ADMIN_USERNAME/ADMIN_PASSWORD dan birinchi administratorni yaratadi.
AppealMigration	class	95	Eski «Maktabga yozish» xabarlarini (parent_message) yangi «Murojaatlar»ga bir marta ko'chiradi.
AuthorSeeder	class	90	19 ta mumtoz o'zbek muallifini (kitobsiz, faqat ma'lumotnoma) barcha muhitlarda qo'shadi; eski kitoblarni mualliflarga bog'laydi.
DataSeeder	class	739	Har bir maktab uchun katta, izchil demo ma'lumot: binolar, xonalar, xodimlar, sinflar, o'quvchilar, dars jadvali.
DemoParentSeeder	class	204	Faqat «dev» profilda: demo ota-ona va uning farzandi uchun so'nggi haftaning ma'lumotlari.
JwtAuthFilter	class	66	Har bir so'rovda Authorization: Bearer tokenini tekshiradi va SecurityContext'ga foydalanuvchini qo'yadi.
LibraryDemoSeeder	class	223	Faqat «dev» profilda: kutubxona uchun namunaviy kitoblar.

Klass	Tur	Qator	Vazifasi
`LibrarySearchIndex`	class	37	PostgreSQL'da pg_trgm bo'lsa, kitob qidiruvi uchun trigram GIN indeks yaratadi.
`OperationalDataSeeder`	class	590	DataSeeder ustiga davomat, baho, e'lon, tadbir va xulq yozuvlarini qo'shadi.
`SchoolScopeConfig`	class	170	Har bir xodimni o'z maktabi doirasida ushlab turadi: URL, query va JSON body'dagi id'lar boshqa maktabniki bo'lsa — 403.
`SecurityConfig`	class	78	Spring Security sozlamasi: qaysi URL ochiq, sessiyasiz (STATELESS) ishlash, CORS, BCrypt va JWT filtri.
`TelegramLinkCodeBackfill`	class	49	Telegram kodi qo'shilishidan oldin bo'lgan o'quvchilarga ishga tushishda bog'lash kodi beradi.
`TelegramSchemaMigration`	class	54	Enum ustunlaridagi eskirgan CHECK cheklovlarini yangilaydi (ddl-auto=update buni o'zi qilmaydi).
`TerritorySeeder`	class	127	14 ta hudud (faqat kodi yo'qlarini) va har bir hududning tumanlarini (faqat tumani yo'q hududga) qo'shadi.

controller

Klass	Tur	Qator	Vazifasi
`AbsenceRequestController`	class	56	«Sababli arizalar»: ota-onalar arizalari; tasdiqlansa kunlar EXCUSED bo'ladi.
`AcademicYearController`	class	54	O'quv yillari CRUD.
`AnnouncementController`	class	61	E'lonlar CRUD va eng so'nggi e'lonlar.
`AppealController`	class	150	«Murojaatlar»: ota-onalar xatlari (botdan) — ro'yxat, yozishma, javob, yopish, fayllar.
`AttendanceController`	class	97	Davomat: dars ro'yxati, ommaviy belgilash, oylik kalendar.
`AuthController`	class	52	Login: login/parolni tekshirib JWT qaytaradi.
`BehaviorRecordController`	class	49	Xulq yozuvlari (rag'bat / ogohlantirish) CRUD.
`BotAdminController`	class	71	«Bot statistikasi» va «Bot sozlamalari» sahifalari.
`BroadcastController`	class	88	«Ota-onalarga xabar»: butun maktab, sinflar yoki tanlangan ota-onalarga matn va fayllar.
`BuildingController`	class	54	Binolar CRUD.
`CalendarEventController`	class	66	Taqvim tadbirlari CRUD, oraliq va kelgusi tadbirlar.
`DashboardController`	class	72	Bosh sahifa ko'rsatkichlari: umumiy holat, jonli darslar, davomat trendi, reytinglar, faoliyat.
`DistrictController`	class	56	Tumanlar ma'lumotnomasi CRUD (soft delete).
`EmployeeController`	class	56	Xodimlar CRUD.
`GradeController`	class	78	Baholar CRUD, baholar daftari va sinf jurnali.
`HealthController`	class	21	Login'siz «tirikmisan?» tekshiruvi — frontend backend ishlayotganini bilishi uchun.
`JournalController`	class	41	Jurnal tepasidagi sinf paneli va o'qituvchining hozirgi darsi.
`LessonSlotController`	class	82	Dars jadvali yozuvlari CRUD, haftalik jadval va PDF.
`LibraryController`	class	154	Kutubxona: kitoblar (PDF, muqova) va javonlar — ADMIN, EDITOR.
`LibraryLegalController`	class	156	Kutubxonaning huquqiy qismi: mualliflar, ruxsatnomalar, Wikisource'dan import.
`LibraryStatsController`	class	78	Mutolaa statistikasi (faqat ADMIN): kim nimani o'qidi, Excel eksport.

Klass	Tur	Qator	Vazifasi
`NotificationController`	class	67	«Xabarnomalar» sahifasi: outbox ro'yxati, statistika, qayta yuborish.
`ParentApiController`	class	138	Mini App «Farzandim kundaligi» API'si: JWT o'rniga Telegram imzolagan initData bilan himoyalangan.
`PhotoResponses`	class	28	Ota-ona yuborgan rasmni Telegram'dan oqim qilib beradigan yordamchi.
`PositionController`	class	53	Lavozimlar CRUD.
`ProfileController`	class	34	O'quvchi, o'qituvchi va sinf profillari.
`RegionController`	class	66	Hududlar ma'lumotnomasi CRUD va hudud tumanlari.
`RoomController`	class	63	Xonalar CRUD va xonaning bandligi.
`SchoolClassController`	class	54	Sinflar CRUD.
`SchoolController`	class	53	Maktablar CRUD.
`StudentController`	class	57	O'quvchilar CRUD (manzil bilan).
`SubjectController`	class	63	Fanlar CRUD; o'chirish o'rniga nofaol qilish va qayta faollashtirish.
`TelegramController`	class	177	Telegram boshqaruvi: holat, o'quvchi kodlari, sinf kodlari, rasm fayllari, mock rejim vositalari.
`TelegramParentsController`	class	40	«Sinf bo'yicha ota-onalar» va xabar uchun ota-ona tanlash.
`UserController`	class	51	Foydalanuvchilar CRUD — faqat ADMIN.

dto

Klass	Tur	Qator	Vazifasi
`AbsenceRequestDto`	class	153	Absence uchun so'rov DTO: klient yuboradigan JSON shakli va validatsiya qoidalari.
`AbsenteeDto`	class	52	Absentee — API javobidagi ma'lumot shakli (DTO).
`AcademicYearRequestDto`	class	54	AcademicYear uchun so'rov DTO: klient yuboradigan JSON shakli va validatsiya qoidalari.
`AcademicYearResponseDto`	class	62	AcademicYear uchun javob DTO: API qaytaradigan JSON shakli.
`ActivityItemDto`	class	60	ActivityItem — API javobidagi ma'lumot shakli (DTO).
`AnnouncementRequestDto`	class	87	Announcement uchun so'rov DTO: klient yuboradigan JSON shakli va validatsiya qoidalari.
`AnnouncementResponseDto`	class	102	Announcement uchun javob DTO: API qaytaradigan JSON shakli.
`AttendanceBulkRequestDto`	class	81	AttendanceBulk uchun so'rov DTO: klient yuboradigan JSON shakli va validatsiya qoidalari.
`AttendanceRequestDto`	class	64	Attendance uchun so'rov DTO: klient yuboradigan JSON shakli va validatsiya qoidalari.
`AttendanceResponseDto`	class	91	Attendance uchun javob DTO: API qaytaradigan JSON shakli.
`AttendanceRosterEntryDto`	class	53	AttendanceRosterEntry — API javobidagi ma'lumot shakli (DTO).
`AttendanceTrendPointDto`	class	34	AttendanceTrendPoint — API javobidagi ma'lumot shakli (DTO).
`AttentionItemDto`	class	52	AttentionItem — API javobidagi ma'lumot shakli (DTO).
`BehaviorRecordRequestDto`	class	55	BehaviorRecord uchun so'rov DTO: klient yuboradigan JSON shakli va validatsiya qoidalari.

Klass	Tur	Qator	Vazifasi
`BehaviorRecordResponseDto`	class	119	BehaviorRecord uchun javob DTO: API qaytaradigan JSON shakli.
`BotSettingDto`	class	149	BotSetting — API javobidagi ma'lumot shakli (DTO).
`BotStatsDto`	class	274	BotStats — API javobidagi ma'lumot shakli (DTO).
`BroadcastDto`	class	141	Broadcast — API javobidagi ma'lumot shakli (DTO).
`BroadcastRecipientDto`	record	15	BroadcastRecipient — API javobidagi ma'lumot shakli (DTO).
`BroadcastRequestDto`	class	68	Broadcast uchun so'rov DTO: klient yuboradigan JSON shakli va validatsiya qoidalari.
`BuildingRequestDto`	class	30	Building uchun so'rov DTO: klient yuboradigan JSON shakli va validatsiya qoidalari.
`BuildingResponseDto`	class	69	Building uchun javob DTO: API qaytaradigan JSON shakli.
`CalendarEventRequestDto`	class	76	CalendarEvent uchun so'rov DTO: klient yuboradigan JSON shakli va validatsiya qoidalari.
`CalendarEventResponseDto`	class	73	CalendarEvent uchun javob DTO: API qaytaradigan JSON shakli.
`ClassProfileDto`	class	99	ClassProfile — API javobidagi ma'lumot shakli (DTO).
`ClassRankingDto`	class	87	ClassRanking — API javobidagi ma'lumot shakli (DTO).
`ClassTelegramCodesDto`	class	54	ClassTelegramCodes — API javobidagi ma'lumot shakli (DTO).
`DashboardOverviewDto`	class	85	DashboardOverview — API javobidagi ma'lumot shakli (DTO).
`DashboardSummaryDto`	class	129	DashboardSummary — API javobidagi ma'lumot shakli (DTO).
`DistrictRequestDto`	record	12	District uchun so'rov DTO: klient yuboradigan JSON shakli va validatsiya qoidalari.
`DistrictResponseDto`	record	12	District uchun javob DTO: API qaytaradigan JSON shakli.
`EmployeeRequestDto`	class	94	Employee uchun so'rov DTO: klient yuboradigan JSON shakli va validatsiya qoidalari.
`EmployeeResponseDto`	class	143	Employee uchun javob DTO: API qaytaradigan JSON shakli.
`GradebookResponseDto`	class	56	Gradebook uchun javob DTO: API qaytaradigan JSON shakli.
`GradeRequestDto`	class	79	Grade uchun so'rov DTO: klient yuboradigan JSON shakli va validatsiya qoidalari.
`GradeResponseDto`	class	119	Grade uchun javob DTO: API qaytaradigan JSON shakli.
`HeatmapDayDto`	class	34	HeatmapDay — API javobidagi ma'lumot shakli (DTO).
`LessonSlotRequestDto`	class	87	LessonSlot uchun so'rov DTO: klient yuboradigan JSON shakli va validatsiya qoidalari.
`LessonSlotResponseDto`	class	116	LessonSlot uchun javob DTO: API qaytaradigan JSON shakli.
`LibraryDtos`	class	132	Kutubxona DTO'lari bitta faylda (record'lar): kitob, javon, muallif, ruxsatnoma, Wikisource.
`LiveLessonDto`	class	89	LiveLesson — API javobidagi ma'lumot shakli (DTO).
`LiveLessonsSummaryDto`	class	42	LiveLessonsSummary — API javobidagi ma'lumot shakli (DTO).
`LoginRequest`	class	16	Login so'rovi: username va password.
`NotificationLogDto`	class	138	NotificationLog — API javobidagi ma'lumot shakli (DTO).
`NotificationSettingsDto`	class	76	NotificationSettings — API javobidagi ma'lumot shakli (DTO).

Klass	Tur	Qator	Vazifasi
`NotificationStatsDto`	class	87	NotificationStats — API javobidagi ma'lumot shakli (DTO).
`ParentLinkDto`	class	44	ParentLink — API javobidagi ma'lumot shakli (DTO).
`PositionRequestDto`	class	18	Position uchun so'rov DTO: klient yuboradigan JSON shakli va validatsiya qoidalari.
`PositionResponseDto`	class	25	Position uchun javob DTO: API qaytaradigan JSON shakli.
`RegionRequestDto`	record	10	Region uchun so'rov DTO: klient yuboradigan JSON shakli va validatsiya qoidalari.
`RegionResponseDto`	record	11	Region uchun javob DTO: API qaytaradigan JSON shakli.
`RoomOccupancySlotDto`	class	60	RoomOccupancySlot — API javobidagi ma'lumot shakli (DTO).
`RoomRequestDto`	class	63	Room uchun so'rov DTO: klient yuboradigan JSON shakli va validatsiya qoidalari.
`RoomResponseDto`	class	78	Room uchun javob DTO: API qaytaradigan JSON shakli.
`SchoolClassRequestDto`	class	68	SchoolClass uchun so'rov DTO: klient yuboradigan JSON shakli va validatsiya qoidalari.
`SchoolClassResponseDto`	class	88	SchoolClass uchun javob DTO: API qaytaradigan JSON shakli.
`SchoolRequestDto`	class	28	School uchun so'rov DTO: klient yuboradigan JSON shakli va validatsiya qoidalari.
`SchoolResponseDto`	class	59	School uchun javob DTO: API qaytaradigan JSON shakli.
`StudentProfileDto`	class	221	StudentProfile — API javobidagi ma'lumot shakli (DTO).
`StudentRequestDto`	class	183	Student uchun so'rov DTO: klient yuboradigan JSON shakli va validatsiya qoidalari.
`StudentResponseDto`	class	218	Student uchun javob DTO: API qaytaradigan JSON shakli.
`StudentTelegramDto`	class	72	StudentTelegram — API javobidagi ma'lumot shakli (DTO).
`SubjectAverageDto`	class	33	SubjectAverage — API javobidagi ma'lumot shakli (DTO).
`SubjectRequestDto`	class	30	Subject uchun so'rov DTO: klient yuboradigan JSON shakli va validatsiya qoidalari.
`SubjectResponseDto`	class	52	Subject uchun javob DTO: API qaytaradigan JSON shakli.
`TeacherProfileDto`	class	117	TeacherProfile — API javobidagi ma'lumot shakli (DTO).
`TelegramSimulateRequestDto`	class	163	TelegramSimulate uchun so'rov DTO: klient yuboradigan JSON shakli va validatsiya qoidalari.
`TelegramStatusDto`	class	71	TelegramStatus — API javobidagi ma'lumot shakli (DTO).
`TextRequestDto`	class	21	Faqat matndan iborat so'rov (masalan, murojaatga javob).
`TimetableEntryDto`	class	125	TimetableEntry — API javobidagi ma'lumot shakli (DTO).
`TodayLessonDto`	class	87	TodayLesson — API javobidagi ma'lumot shakli (DTO).
`UserRequestDto`	class	54	User uchun so'rov DTO: klient yuboradigan JSON shakli va validatsiya qoidalari.
`UserResponseDto`	class	53	User uchun javob DTO: API qaytaradigan JSON shakli.
`UserUpdateRequestDto`	class	53	UserUpdate uchun so'rov DTO: klient yuboradigan JSON shakli va validatsiya qoidalari.

entity

Klass	Tur	Qator	Vazifasi
AbsenceReason	enum	6	Enum: sababli ariza sababi (kasallik, oilaviy va h.k.).
AbsenceRequest	class	196	AbsenceRequest jadvali uchun JPA entity.
AbsenceStatus	enum	6	Enum: ariza holati (kutilmoqda, tasdiqlangan, rad etilgan).
AcademicYear	class	64	AcademicYear jadvali uchun JPA entity.
ActivityLog	class	84	ActivityLog jadvali uchun JPA entity.
Announcement	class	115	Announcement jadvali uchun JPA entity.
AnnouncementAudience	enum	6	Enum: e'lon kimlar uchun.
AnnouncementPriority	enum	6	Enum: e'lon muhimligi.
Appeal	class	175	Appeal jadvali uchun JPA entity.
AppealEnums	class	42	Murojaatlar uchun enum'lar to'plami (holat, yo'nalish, fayl turi va h.k.).
AppealMessage	class	227	AppealMessage jadvali uchun JPA entity.
Attendance	class	92	Attendance jadvali uchun JPA entity.
AttendanceStatus	enum	6	Enum: PRESENT, ABSENT, LATE, EXCUSED.
Author	class	98	Author jadvali uchun JPA entity.
BehaviorRecord	class	93	BehaviorRecord jadvali uchun JPA entity.
BehaviorType	enum	6	Enum: rag'bat yoki ogohlantirish.
Book	class	415	Book jadvali uchun JPA entity.
BookCategory	class	95	BookCategory jadvali uchun JPA entity.
BookDownload	class	103	BookDownload jadvali uchun JPA entity.
BookFavorite	class	72	BookFavorite jadvali uchun JPA entity.
BookRating	class	82	BookRating jadvali uchun JPA entity.
BotAsset	class	49	BotAsset jadvali uchun JPA entity.
BotSetting	class	194	BotSetting jadvali uchun JPA entity.
BotUsageEvent	class	80	BotUsageEvent jadvali uchun JPA entity.
Broadcast	class	126	Broadcast jadvali uchun JPA entity.
BroadcastAttachment	class	115	BroadcastAttachment jadvali uchun JPA entity.
BroadcastAudience	enum	6	Enum: xabar kimlarga (maktab, sinflar, tanlangan ota-onalar).
Building	class	43	Building jadvali uchun JPA entity.
CalendarEvent	class	115	CalendarEvent jadvali uchun JPA entity.
CalendarEventType	enum	6	Enum: tadbir turi (bayram, imtihon, majlis, ta'til).
District	class	104	District jadvali uchun JPA entity.
Employee	class	140	Employee jadvali uchun JPA entity.
EmployeeStatus	enum	9	Enum: Ishda / Ta'tilda / Ishdan ketgan.
Grade	class	115	Grade jadvali uchun JPA entity.
GradeType	enum	6	Enum: baho turi (joriy, nazorat va h.k.).

Klass	Tur	Qator	Vazifasi
`GuardianRelation`	enum	7	Enum: vasiy kim (otasi, onasi, boshqa).
`HousingType`	enum	10	Enum: KVARTIRA (ko'p qavatli uy: uy + xonadon) yoki HOVLI (xususiy uy).
`LessonSlot`	class	102	LessonSlot jadvali uchun JPA entity.
`LibraryEnums`	class	47	Kutubxona enum'lari: huquqiy asos, yosh toifasi, til va h.k.
`LibraryPermission`	class	208	LibraryPermission jadvali uchun JPA entity.
`NotificationLog`	class	239	NotificationLog jadvali uchun JPA entity.
`NotificationSettings`	class	130	NotificationSettings jadvali uchun JPA entity.
`NotificationStatus`	enum	6	Enum: outbox xabari holati (PENDING, SENT, FAILED, SKIPPED).
`NotificationType`	enum	16	Enum: xabarnoma turi (davomat, baho, e'lon, digest va h.k.).
`ParentMessage`	class	169	ParentMessage jadvali uchun JPA entity.
`ParentMessageRecipient`	enum	6	Enum: eski xat kimga yozilgan (ma'muriyat / sinf rahbari).
`ParentMessageStatus`	enum	6	Enum: eski xat holati.
`ParentSession`	class	300	ParentSession jadvali uchun JPA entity.
`ParentTelegramLink`	class	96	ParentTelegramLink jadvali uchun JPA entity.
`Position`	class	50	Position jadvali uchun JPA entity.
`Region`	class	91	Region jadvali uchun JPA entity.
`Role`	enum	7	Enum: ADMIN, EDITOR, VIEWER — foydalanuvchi rollari.
`Room`	class	75	Room jadvali uchun JPA entity.
`RoomType`	enum	10	Enum: xona turi (sinfxona, laboratoriya, sport zali va h.k.).
`School`	class	63	School jadvali uchun JPA entity.
`SchoolClass`	class	77	SchoolClass jadvali uchun JPA entity.
`Student`	class	229	Student jadvali uchun JPA entity.
`Subject`	class	58	Subject jadvali uchun JPA entity.
`User`	class	42	User jadvali uchun JPA entity.

event

Klass	Tur	Qator	Vazifasi
`AnnouncementCreatedEvent`	record	6	Event: yangi e'lon yaratildi (tahrir qayta xabar yubormaydi).
`AttendanceMarkedEvent`	record	18	Event: davomat holati haqiqatan ABSENT yoki LATE'ga o'zgardi.
`GradeSavedEvent`	record	6	Event: baho yaratildi yoki bahosi/fani/turi o'zgardi.
`NotificationEventListener`	class	52	Commit bo'lgan event'larni outbox'ga (NotificationLog) aylantiradi — AFTER_COMMIT.

exception

Klass	Tur	Qator	Vazifasi
`ForbiddenException`	class	10	Foydalanuvchi tizimga kirgan, lekin bu ma'lumot uniki emas → 403.

Klass	Tur	Qator	Vazifasi
`GlobalExceptionHandler`	class	71	Barcha controller'lar uchun yagona xato ushlagich: exception → HTTP status + o'zbekcha xabar.

repository

Klass	Tur	Qator	Vazifasi
`AbsenceRequestRepository`	interface	20	AbsenceRequest entity uchun Spring Data JPA repository (bazaga so'rovlar).
`AcademicYearRepository`	interface	11	AcademicYear entity uchun Spring Data JPA repository (bazaga so'rovlar).
`ActivityLogRepository`	interface	14	ActivityLog entity uchun Spring Data JPA repository (bazaga so'rovlar).
`AnnouncementRepository`	interface	26	Announcement entity uchun Spring Data JPA repository (bazaga so'rovlar).
`AppealMessageRepository`	interface	43	AppealMessage entity uchun Spring Data JPA repository (bazaga so'rovlar).
`AppealRepository`	interface	101	Appeal entity uchun Spring Data JPA repository (bazaga so'rovlar).
`AttendanceRepository`	interface	95	Attendance entity uchun Spring Data JPA repository (bazaga so'rovlar).
`AuthorRepository`	interface	24	Author entity uchun Spring Data JPA repository (bazaga so'rovlar).
`BehaviorRecordRepository`	interface	34	BehaviorRecord entity uchun Spring Data JPA repository (bazaga so'rovlar).
`BookCategoryRepository`	interface	24	BookCategory entity uchun Spring Data JPA repository (bazaga so'rovlar).
`BookDownloadRepository`	interface	41	BookDownload entity uchun Spring Data JPA repository (bazaga so'rovlar).
`BookFavoriteRepository`	interface	12	BookFavorite entity uchun Spring Data JPA repository (bazaga so'rovlar).
`BookRatingRepository`	interface	12	BookRating entity uchun Spring Data JPA repository (bazaga so'rovlar).
`BookRepository`	interface	147	Book entity uchun Spring Data JPA repository (bazaga so'rovlar).
`BotAssetRepository`	interface	8	BotAsset entity uchun Spring Data JPA repository (bazaga so'rovlar).
`BotSettingRepository`	interface	11	BotSetting entity uchun Spring Data JPA repository (bazaga so'rovlar).
`BotUsageEventRepository`	interface	20	BotUsageEvent entity uchun Spring Data JPA repository (bazaga so'rovlar).
`BroadcastAttachmentRepository`	interface	27	BroadcastAttachment entity uchun Spring Data JPA repository (bazaga so'rovlar).
`BroadcastRepository`	interface	35	Broadcast entity uchun Spring Data JPA repository (bazaga so'rovlar).
`BuildingRepository`	interface	11	Building entity uchun Spring Data JPA repository (bazaga so'rovlar).
`CalendarEventRepository`	interface	35	CalendarEvent entity uchun Spring Data JPA repository (bazaga so'rovlar).
`DistrictRepository`	interface	28	District entity uchun Spring Data JPA repository (bazaga so'rovlar).
`EmployeeRepository`	interface	31	Employee entity uchun Spring Data JPA repository (bazaga so'rovlar).
`GradeRepository`	interface	93	Grade entity uchun Spring Data JPA repository (bazaga so'rovlar).
`LessonSlotRepository`	interface	108	LessonSlot entity uchun Spring Data JPA repository (bazaga so'rovlar).

Klass	Tur	Qator	Vazifasi
`LibraryPermissionRepository`	interface	30	LibraryPermission entity uchun Spring Data JPA repository (bazaga so'rovlar).
`NotificationLogRepository`	interface	101	NotificationLog entity uchun Spring Data JPA repository (bazaga so'rovlar).
`NotificationSettingsRepository`	interface	11	NotificationSettings entity uchun Spring Data JPA repository (bazaga so'rovlar).
`ParentMessageRepository`	interface	20	ParentMessage entity uchun Spring Data JPA repository (bazaga so'rovlar).
`ParentSessionRepository`	interface	15	ParentSession entity uchun Spring Data JPA repository (bazaga so'rovlar).
`ParentTelegramLinkRepository`	interface	75	ParentTelegramLink entity uchun Spring Data JPA repository (bazaga so'rovlar).
`PositionRepository`	interface	8	Position entity uchun Spring Data JPA repository (bazaga so'rovlar).
`RegionRepository`	interface	18	Region entity uchun Spring Data JPA repository (bazaga so'rovlar).
`RoomRepository`	interface	21	Room entity uchun Spring Data JPA repository (bazaga so'rovlar).
`SchoolClassRepository`	interface	36	SchoolClass entity uchun Spring Data JPA repository (bazaga so'rovlar).
`SchoolRepository`	interface	6	School entity uchun Spring Data JPA repository (bazaga so'rovlar).
`StudentRepository`	interface	45	Student entity uchun Spring Data JPA repository (bazaga so'rovlar).
`SubjectRepository`	interface	29	Subject entity uchun Spring Data JPA repository (bazaga so'rovlar).
`UserRepository`	interface	15	User entity uchun Spring Data JPA repository (bazaga so'rovlar).

service

Klass	Tur	Qator	Vazifasi
`AbsenceRequestService`	class	189	«😳 Sababli ariza»: ota-ona botda yaratadi, sinf rahbari yoki ma'muriyat saytda hal qiladi.
`AcademicYearService`	class	96	O'quv yillari biznes-mantiqi (joriy yil, sanalar tekshiruvi).
`AccountStatusService`	class	114	Xodim hisobi hozir ishlatilishi mumkinmi (ta'tilda bo'lsa — bloklangan); 1 daqiqalik kesh.
`ActivityLogService`	class	42	Bosh sahifadagi «Faoliyat» lentasiga bir qatorlik yozuv qo'shadi.
`AnnouncementService`	class	132	E'lonlar: yaratish (event chiqaradi), tahrirlash, ro'yxat.
`AttendanceCalendarService`	class	180	Sinfning oylik davomat kalendari: qatorda o'quvchilar, ustunda kunlar.
`AttendanceService`	class	231	Davomat: dars ro'yxati, ommaviy belgilash, holat o'zgarsa event chiqarish.
`BehaviorRecordService`	class	97	Xulq yozuvlari biznes-mantiqi.
`BotCache`	class	73	Bot uchun qisqa keshlar: chatning bolalari (5 daq.) va sinf jadvali (10 daq.).
`BotScheduledJobs`	class	265	Avtomatik xabarlar: ertangi jadval, haftalik hisobot, ertalabki digest — hammasi outbox orqali.
`BotSettingService`	class	87	«Bot sozlamalari»: maktab kontaktlari va avtomatik xabar vaqtlari.
`BotStatsService`	class	132	Bot statistikasi: faol ota-onalar, bo'limlar, xabarlar.
`BotUsageService`	class	77	Ota-ona qaysi bo'limni ochganini statistikaga yozadi.
`BroadcastMedia`	class	53	Outbox media «broadcast:<id>»: xabar fayllarini tartib bilan yuboradi.

Klass	Tur	Qator	Vazifasi
`BroadcastService`	class	281	«Ota-onalarga xabar»: qabul qiluvchilarni aniqlash va outbox'ga yozish.
`BuildingService`	class	120	Binolar biznes-mantiqi.
`CalendarEventService`	class	112	Taqvim tadbirlari biznes-mantiqi.
`DashboardService`	class	504	Bosh sahifadagi barcha ko'rsatkichlarni hisoblaydi.
`DistrictService`	class	114	Tumanlar ma'lumotnomasi: faqat faol yozuvlar, soft delete, o'quvchi manzili uchun tekshiruv.
`EmployeeService`	class	192	Xodimlar: yaratish, tahrirlash, ta'tilga chiqarish (bloklash qoidalari bilan).
`GradeService`	class	206	Baholar: qoidalar (yakshanba, kelajak sana, nofaol fan), event chiqarish, baholar daftari.
`JournalService`	class	215	Sinf jurnali: bir oy · bir fan · bir sinf, sinf paneli, o'qituvchining hozirgi darsi.
`LessonSlotService`	class	214	Dars jadvali: to'qnashuvlarni (sinf, o'qituvchi, xona) tekshirish va haftalik jadval.
`LinkingService`	class	146	Telegram chatni o'quvchiga bog'lash: deep-link kodi yoki telefon raqami orqali.
`NotificationQueryService`	class	115	Outbox'ni «Xabarnomalar» sahifasi uchun o'qish va qo'lda qayta yuborish.
`NotificationSender`	class	271	Outbox'ni bo'shatadi: Telegram limitlari (≈25 xabar/s, chatga 1 xabar/s), 429/403 va qayta urinishlar.
`NotificationService`	class	395	Event'larni, digest'larni va javoblarni outbox qatorlariga (NotificationLog) aylantiradi.
`NotificationSettingsService`	class	76	Maktab bo'yicha xabarnoma sozlamalari.
`OutboxMedia`	class	170	Outbox qatorining fayllarini yuboradi: bitta fayl izoh bilan yoki albom (sendMediaGroup).
`PositionService`	class	72	Lavozimlar biznes-mantiqi.
`ProfileService`	class	240	O'quvchi, o'qituvchi va sinf profillarini yig'adi (davomat, baholar, xulq, jadval).
`RegionService`	class	70	Hududlar ma'lumotnomasi: faqat faol ('A') yozuvlar, soft delete ('P').
`RoomService`	class	144	Xonalar va ularning haftalik bandligi.
`SchoolAccessService`	class	85	Kim so'rayapti va qaysi maktab bilan ishlay oladi (1 daqiqalik kesh).
`SchoolClassService`	class	130	Sinflar biznes-mantiqi.
`SchoolOwnership`	class	75	Yozuv qaysi maktabga tegishli — har tur uchun kichik SQL, 5 daqiqalik kesh.
`SchoolService`	class	97	Maktablar biznes-mantiqi.
`StudentService`	class	179	O'quvchilar: yaratish, tahrirlash (manzil qoidalari bilan), ro'yxat.
`SubjectService`	class	150	Fanlar hech qachon o'chirilmaydi — nofaol qilinadi.
`TelegramLinkService`	class	146	Admin tomonidan ota-ona bog'lanishlari va bog'lash kodlarini boshqarish.
`TelegramParentsService`	class	110	«Sinf bo'yicha ota-onalar»: sinfning har bir o'quvchisi va botga ulangan ota-onalari.
`TelegramUpdatePoller`	class	96	Long polling: server Telegram'dan getUpdates bilan yangiliklarni o'zi so'raydi (webhook shart emas).
`TimetablePdfService`	class	214	Haftalik dars jadvali PDF: har sinf uchun bitta A4 albom sahifa.
`UserService`	class	115	Foydalanuvchilar: yaratish, tahrirlash, o'chirish, parolni BCrypt bilan saqlash.
`WeeklyReportService`	class	84	Shanba kungi haftalik hisobot matni: davomat, o'rtacha baho, fanlar.

service.appeal

Klass	Tur	Qator	Vazifasi
AppealFiles	class	109	Ota-ona Telegram'dan yuborgan faylni o'zimizning diskka alohida kichik oqimda ko'chiradi.
AppealService	class	520	Murojaatlar biznes-mantiqi: yaratish, javob, yopish, kim ko'ra olishi.
AppealViews	class	27	«Murojaatlar» sahifasiga qaytadigan ko'rinishlar (record'lar).
AttachmentLinks	class	67	Murojaat fayllari uchun qisqa muddatli (30 daqiqa) imzolangan havolalar.
AttachmentPolicy	class	71	Qaysi fayllar qabul qilinadi: rasm, video, audio, PDF, Office — 20 MB gacha.
FileStorage	class	77	Fayllarni diskda saqlash: {storage}/{area}/{maktab}/{yyyy-MM}/{uuid}.ext.

service.library

Klass	Tur	Qator	Vazifasi
BookMedia	class	65	Outbox media «book:<id>»: kitob PDF'ini «Muallif - Nomi.pdf» nomi bilan yuboradi.
CopyrightRules	class	70	Mualliflik huquqi qoidasi (qonun, 35/37-moddalar): asar qachon jamoat mulkiga o'tadi.
LibraryAdminService	class	404	Saytdagi kutubxona: kitoblarni (PDF va muqova bilan) va javonlarni boshqarish.
LibraryAuthors	class	143	Mualliflar ma'lumotnomasi: kim yozgan va qachon vafot etgan.
LibraryCatalog	class	321	Botdagi «📚 Kitob mutolaasi» o'qiydigan va bajaradigan hamma narsa.
LibraryDefaults	class	56	Maktab kutubxonasi boshlanadigan standart bo'limlar.
LibraryFiles	class	160	Kitob fayllari: tekshirish (haqiqiy PDF, hajm), saqlash va o'qish.
LibraryLegal	class	128	Har bir saqlanadigan kitob o'tadigan huquqiy tekshiruvlar (jamoat mulki, ruxsatnoma).
LibraryPdfFonts	class	44	PDF'larga joylanadigan Noto Serif shrifti (o'zbek lotin va kirill harflari).
LibraryPermissions	class	298	Ruxsatnomalar: yaratish, bekor qilish, muddati o'tganlarni kundalik tekshirish.
LibrarySettings	class	58	Kutubxona chegaralari (app.library.*), har birining standart qiymati bor.
LibraryStatsService	class	337	«Mutolaa statistikasi» — kim nimani o'qiyapti.
LibraryText	class	79	Kitob nomi va muallifini qidiruv uchun normallashtirish (lotin/kirill, apostroflar).
LibraryWeekly	class	133	Har juma 19:00 da ota-onalarga «Dam olish kunlari uchun 3 ta kitob» tavsiyasi.
PermissionTemplatePdf	class	89	Chop etib imzolash uchun A4 ruxsat xati shabloni.

service.library.wikisource

Klass	Tur	Qator	Vazifasi
HttpWikisourceApi	class	110	Wikisource MediaWiki API mijozi: User-Agent, soniyasiga ≤1 so'rov, 30 s timeout.
WikisourceApi	interface	20	Interfeys: Wikisource importiga kerakli MediaWiki chaqiruvlari.
WikisourceImport	class	235	«📜 Wikisource dan qo'shish»: muallif sahifasi, asarlar ro'yxati va asarni import qilish.
WikisourcePdf	class	189	Wikisource matnidan A5 kitob PDF'i: muqova, mundarija, boblar, Noto Serif shrifti.
WikiText	class	189	Wiki-belgilarni o'qish va toza matnga aylantirish.

service.parent

Klass	Tur	Qator	Vazifasi
ParentAccessService	class	99	Telegram chat va o'quvchi ma'lumoti orasidagi yagona darvoza: faqat bog'langan bolalar.

Klass	Tur	Qator	Vazifasi
`ParentDataService`	class	475	Ota-ona bitta bola haqida ko'ra oladigan hamma narsani maktab ma'lumotlaridan hisoblaydi.
`ParentStats`	class	120	Ota-ona ko'rinishlari ortidagi sof hisob-kitoblar (bazasiz, unit-test uchun qulay).
`ParentViews`	class	100	Bot ekranlari va Mini App API ishlatadigan bola ma'lumotlari ko'rinishlari.

telegram

Klass	Tur	Qator	Vazifasi
`HttpTelegramClient`	class	373	Haqiqiy Telegram Bot API mijozi (Spring RestClient ustida, uchinchi tomon kutubxonasisiz).
`LinkCodeGenerator`	class	39	Ota-ona bog'lash kodlari: SecureRandom'dan 10 belgi (~58 bit) — taxmin qilib bo'lmaydi.
`MessageFormatter`	class	137	Avtomatik xabarnomalar matnini (Telegram HTML) til fayllaridan yig'adi.
`MockTelegramClient`	class	179	telegram.mock=true: hech narsa tashqariga chiqmaydi, xabarlar faqat xotirada.
`PhoneNormalizer`	class	46	Telefon raqamini yagona «+998XXXXXXXXX» ko'rinishga keltiradi.
`QuietHours`	class	33	Tinch soatlar hisobi (Toshkent vaqti).
`TelegramApiException`	class	35	Muvaffaqiyatsiz Bot API chaqiruvi (status kodi va retry_after bilan).
`TelegramClient`	interface	106	Interfeys: ilovaga kerakli Bot API chaqiruvlari (mock va haqiqiy).
`TelegramConfig`	class	69	Telegram bean'lari: Clock (Asia/Tashkent) va rejimga qarab Mock/Http mijoz.
`TelegramInitDataValidator`	class	123	Mini App initData imzosini Telegram hujjatidagidek tekshiradi (HMAC-SHA256) va eskirganini rad etadi.
`TelegramJson`	class	19	Klaviaturalar uchun umumiy JSON mapper.
`TelegramModels`	class	291	Bot API'ning loyiha ishlatadigan qismi — oddiy record'lar.
`TelegramProperties`	class	182	Barcha «telegram.*» sozlamalari (@ConfigurationProperties).
`TokenMasker`	class	31	Log va bazaga yozishdan oldin matndan bot tokenini yashiradi.

util

Klass	Tur	Qator	Vazifasi
`JwtUtil`	class	70	JWT yaratish va tekshirish (HMAC-SHA kalit, 10 soat amal qiladi).
`XlsxWriter`	class	139	Oddiy .xlsx (Excel) fayl yozuvchi — tashqi kutubxonasiz.

C-ilova. Barcha entity'lar va maydonlari (39 ta)

Har bir jadvalning har bir maydoni: Java turi, bazadagi ustun yoki bog'lanish (`@ManyToOne` va h.k.) va cheklovlar. Ustun nomi berilmagan bo'lsa, Hibernate uni maydon nomidan `snake_case` qilib yasaydi (`birthDate` → `birth_date`).

AbsenceRequest → jadval `absence_request`

Maydon	Java turi	Ustun / bog'lanish	Cheklov
`id`	`Long`	`id`	PRIMARY KEY
`school`	`School`	`ManyToOne → school_id`	NOT NULL
`student`	`Student`	`ManyToOne → student_id`	NOT NULL
`chatId`	`Long`	`chat_id`	NOT NULL
`parentName`	`String`	`parent_name`	
`dateFrom`	`LocalDate`	`date_from`	NOT NULL
`dateTo`	`LocalDate`	`date_to`	NOT NULL
`reason`	`AbsenceReason`	`reason`	NOT NULL, enum (STRING)
`comment`	`String`	`comment`	
`photoFileId`	`String`	`photo_file_id`	
`status`	`AbsenceStatus`	`status`	NOT NULL, enum (STRING)
`decisionNote`	`String`	`decision_note`	
`decidedBy`	`String`	`decided_by`	
`createdAt`	`LocalDateTime`	`created_at`	NOT NULL
`decidedAt`	`LocalDateTime`	`decided_at`	
`excusedLessons`	`Integer`	`excused_lessons`	

AcademicYear → jadval `academic_year`

Maydon	Java turi	Ustun / bog'lanish	Cheklov
`id`	`Long`	`id`	PRIMARY KEY
`school`	`School`	`ManyToOne → school_id`	NOT NULL
`title`	`String`	`title`	NOT NULL
`startDate`	`LocalDate`	`start_date`	NOT NULL
`endDate`	`LocalDate`	`end_date`	NOT NULL

ActivityLog → jadval `activity_log`

Maydon	Java turi	Ustun / bog'lanish	Cheklov
`id`	`Long`	`id`	PRIMARY KEY

Maydon	Java turi	Ustun / bog'lanish	Cheklov
school	School	ManyToOne → school_id	NOT NULL
description	String	description	NOT NULL
icon	String	icon	NOT NULL
actorUsername	String	actor_username	
occurredAt	LocalDateTime	occurred_at	NOT NULL

Announcement → jadval `announcement`

Maydon	Java turi	Ustun / bog'lanish	Cheklov
id	Long	id	PRIMARY KEY
school	School	ManyToOne → school_id	NOT NULL
title	String	title	NOT NULL
content	String	content	NOT NULL
audience	AnnouncementAudience	audience	NOT NULL, enum (STRING)
schoolClass	SchoolClass	ManyToOne → class_id	
priority	AnnouncementPriority	priority	NOT NULL, enum (STRING)
deadline	LocalDate	deadline	
createdDate	LocalDateTime	created_date	

Appeal → jadval `appeal`

Maydon	Java turi	Ustun / bog'lanish	Cheklov
id	Long	id	PRIMARY KEY
school	School	ManyToOne (LAZY) → school_id	NOT NULL
student	Student	ManyToOne (LAZY) → student_id	NOT NULL
chatId	Long	chat_id	
parentName	String	parent_name	
parentUsername	String	parent_username	
target	AppealEnums.Target	target	NOT NULL, enum (STRING)
status	AppealEnums.Status	status	NOT NULL, enum (STRING)
source	AppealEnums.Source	source	enum (STRING)
createdAt	LocalDateTime	created_at	NOT NULL
lastMessageAt	LocalDateTime	last_message_at	NOT NULL
unreadCount	int	unread_count	
lastPreview	String	last_preview	
createdBy	String	created_by	

AppealMessage → jadval `appeal_message`

Maydon	Java turi	Ustun / bog'lanish	Cheklov
id	Long	id	PRIMARY KEY

Maydon	Java turi	Ustun / bog'lanish	Cheklov
appeal	Appeal	ManyToOne (LAZY) → appeal_id	NOT NULL
direction	AppealEnums.Direction	direction	NOT NULL, enum (STRING)
kind	AppealEnums.Kind	kind	NOT NULL, enum (STRING)
text	String	text	
mediaGroupId	String	media_group_id	
fileId	String	file_id	
fileUniqueId	String	file_unique_id	
mimeType	String	mime_type	
fileSize	Long	file_size	
duration	Integer	duration	
originalName	String	original_name	
storagePath	String	storage_path	
fileState	AppealEnums.FileState	file_state	enum (STRING)
sentBy	String	sent_by	
pending	Boolean	pending	
telegramMessageId	Long	telegram_message_id	
createdAt	LocalDateTime	created_at	NOT NULL

Attendance → jadval `attendance`

Maydon	Java turi	Ustun / bog'lanish	Cheklov
id	Long	id	PRIMARY KEY
lessonSlot	LessonSlot	ManyToOne → lesson_slot_id	NOT NULL
student	Student	ManyToOne → student_id	NOT NULL
recordDate	LocalDate	record_date	NOT NULL
status	AttendanceStatus	status	NOT NULL, enum (STRING)
comment	String	comment	
createdDate	LocalDateTime	created_date	

Author → jadval `author`

Maydon	Java turi	Ustun / bog'lanish	Cheklov
id	Long	id	PRIMARY KEY
fullName	String	full_name	NOT NULL
aliases	String	aliases	
birthYear	Integer	birth_year	
deathYear	Integer	death_year	
note	String	note	
searchKey	String	search_key	NOT NULL

BehaviorRecord → jadval `behavior_record`

Maydon	Java turi	Ustun / bog'lanish	Cheklov
id	Long	id	PRIMARY KEY
student	Student	ManyToOne → student_id	NOT NULL
recordDate	LocalDate	record_date	NOT NULL
type	BehaviorType	type	NOT NULL, enum (STRING)
description	String	description	NOT NULL
createdBy	String	created_by	
createdDate	LocalDateTime	created_date	

Book → jadval `book`

Maydon	Java turi	Ustun / bog'lanish	Cheklov
id	Long	id	PRIMARY KEY
school	School	ManyToOne (LAZY) → school_id	NOT NULL
title	String	title	NOT NULL
author	String	author	
authors	List<Author>	ManyToMany (LAZY) → book_id	
permission	LibraryPermission	ManyToOne (LAZY) → permission_id	
publishedInLifetime	Boolean	published_in_lifetime	
firstPublishedYear	Integer	first_published_year	
noModernCommentary	Boolean	no_modern_commentary	
sourceUrl	String	source_url	
category	BookCategory	ManyToOne (LAZY) → category_id	
description	String	description	
language	String	language	NOT NULL
ageGroup	AgeGroup	age_group	NOT NULL, enum (STRING)
pages	Integer	pages	
year	Integer	publish_year	
coverPath	String	cover_path	
pdfPath	String	pdf_path	NOT NULL
pdfSize	Long	pdf_size	NOT NULL
pdfSha256	String	pdf_sha256	NOT NULL
telegramFileId	String	telegram_file_id	
legalBasis	LegalBasis	legal_basis	NOT NULL, enum (STRING)
legalNote	String	legal_note	
active	Boolean	active	NOT NULL
featured	Boolean	featured	NOT NULL
draft	Boolean	draft	NOT NULL

Maydon	Java turi	Ustun / bog'lanish	Cheklov
searchKey	String	search_key	NOT NULL
downloadCount	Long	download_count	NOT NULL
likes	Integer	likes	NOT NULL
dislikes	Integer	dislikes	NOT NULL
createdBy	String	created_by	
createdAt	LocalDateTime	created_at	NOT NULL
updatedAt	LocalDateTime	updated_at	

BookCategory → jadval `book_category`

Maydon	Java turi	Ustun / bog'lanish	Cheklov
id	Long	id	PRIMARY KEY
school	School	ManyToOne (LAZY) → school_id	NOT NULL
nameUz	String	name_uz	NOT NULL
nameRu	String	name_ru	
emoji	String	emoji	
position	Integer	position	NOT NULL
active	Boolean	active	NOT NULL

BookDownload → jadval `book_download`

Maydon	Java turi	Ustun / bog'lanish	Cheklov
id	Long	id	PRIMARY KEY
school	School	ManyToOne (LAZY) → school_id	NOT NULL
book	Book	ManyToOne (LAZY) → book_id	NOT NULL
parent	ParentSession	ManyToOne (LAZY) → parent_session_id	
chatId	Long	chat_id	NOT NULL
downloadedAt	LocalDateTime	downloaded_at	NOT NULL
source	DownloadSource	source	NOT NULL, enum (STRING)

BookFavorite → jadval `book_favorite`

Maydon	Java turi	Ustun / bog'lanish	Cheklov
id	Long	id	PRIMARY KEY
book	Book	ManyToOne (LAZY) → book_id	NOT NULL
parent	ParentSession	ManyToOne (LAZY) → parent_session_id	
chatId	Long	chat_id	NOT NULL
createdAt	LocalDateTime	created_at	NOT NULL

BookRating → jadval `book_rating`

Maydon	Java turi	Ustun / bog'lanish	Cheklov
id	Long	id	PRIMARY KEY
book	Book	ManyToOne (LAZY) → book_id	NOT NULL
parent	ParentSession	ManyToOne (LAZY) → parent_session_id	
chatId	Long	chat_id	NOT NULL
value	Integer	value	NOT NULL
updatedAt	LocalDateTime	updated_at	NOT NULL

BotAsset → jadval `bot_asset`

Maydon	Java turi	Ustun / bog'lanish	Cheklov
key	String	asset_key	PRIMARY KEY
fileId	String	file_id	NOT NULL
updatedAt	LocalDateTime	updated_at	

BotSetting → jadval `bot_setting`

Maydon	Java turi	Ustun / bog'lanish	Cheklov
id	Long	id	PRIMARY KEY
school	School	OneToOne → school_id	NOT NULL, UNIQUE
phone	String	phone	
directorName	String	director_name	
receptionHours	String	reception_hours	
bellScheduleNote	String	bell_schedule_note	
showTeacherPhones	Boolean	show_teacher_phones	NOT NULL
tomorrowScheduleTime	LocalTime	tomorrow_schedule_time	NOT NULL
weeklyReportDay	DayOfWeek	weekly_report_day	NOT NULL, enum (STRING)
weeklyReportTime	LocalTime	weekly_report_time	NOT NULL
eventReminderTime	LocalTime	event_reminder_time	NOT NULL
lowGradeThreshold	Integer	low_grade_threshold	NOT NULL
morningDigestTime	LocalTime	morning_digest_time	
libraryWeeklyEnabled	Boolean	library_weekly_enabled	

BotUsageEvent → jadval `bot_usage_event`

Maydon	Java turi	Ustun / bog'lanish	Cheklov
id	Long	id	PRIMARY KEY
chatId	Long	chat_id	NOT NULL
schoolId	Long	school_id	
studentId	Long	student_id	
section	String	section	NOT NULL

Maydon	Java turi	Ustun / bog'lanish	Cheklov
createdAt	LocalDateTime	created_at	NOT NULL

Broadcast → jadval `broadcast`

Maydon	Java turi	Ustun / bog'lanish	Cheklov
id	Long	id	PRIMARY KEY
school	School	ManyToOne → school_id	NOT NULL
text	String	text	NOT NULL
audience	BroadcastAudience	audience	NOT NULL, enum (STRING)
classIds	String	class_ids	
createdBy	String	created_by	
createdAt	LocalDateTime	created_at	NOT NULL
recipientCount	Integer	recipient_count	
chatIds	String	chat_ids	
mediaCount	Integer	media_count	

BroadcastAttachment → jadval `broadcast_attachment`

Maydon	Java turi	Ustun / bog'lanish	Cheklov
id	Long	id	PRIMARY KEY
broadcast	Broadcast	ManyToOne (LAZY) → broadcast_id	NOT NULL
position	Integer	position	NOT NULL
kind	AppealEnums.Kind	kind	NOT NULL, enum (STRING)
mimeType	String	mime_type	
originalName	String	original_name	
fileSize	Long	file_size	
storagePath	String	storage_path	NOT NULL
fileId	String	file_id	

Building → jadval `building`

Maydon	Java turi	Ustun / bog'lanish	Cheklov
id	Long	id	PRIMARY KEY
school	School	ManyToOne → school_id	NOT NULL
name	String	name	NOT NULL

CalendarEvent → jadval `calendar_event`

Maydon	Java turi	Ustun / bog'lanish	Cheklov
id	Long	id	PRIMARY KEY
school	School	ManyToOne → school_id	NOT NULL
title	String	title	NOT NULL

Maydon	Java turi	Ustun / bog'lanish	Cheklov
description	String	description	
type	CalendarEventType	type	NOT NULL, enum (STRING)
startDate	LocalDate	start_date	NOT NULL
endDate	LocalDate	end_date	NOT NULL
seeded	boolean	seeded	
createdDate	LocalDateTime	created_date	

District → jadval `district`

Maydon	Java turi	Ustun / bog'lanish	Cheklov
id	Long	id	PRIMARY KEY
region	Region	ManyToOne (LAZY) → region_id	NOT NULL
code	String	code	
nameUz	String	name_uz	NOT NULL
state	String	state	NOT NULL
createdAt	LocalDateTime	created_at	NOT NULL
updatedAt	LocalDateTime	updated_at	NOT NULL

Employee → jadval `employee`

Maydon	Java turi	Ustun / bog'lanish	Cheklov
id	Long	id	PRIMARY KEY
position	Position	ManyToOne → position_id	NOT NULL
school	School	ManyToOne → school_id	
firstName	String	first_name	NOT NULL
lastName	String	last_name	NOT NULL
phone	String	phone	NOT NULL, UNIQUE
status	EmployeeStatus	status	enum (STRING)
leaveFrom	LocalDate	leave_from	
leaveTo	LocalDate	leave_to	
createdDate	LocalDateTime	created_date	
updatedDate	LocalDateTime	updated_date	

Grade → jadval `grade`

Maydon	Java turi	Ustun / bog'lanish	Cheklov
id	Long	id	PRIMARY KEY
student	Student	ManyToOne → student_id	NOT NULL
subject	Subject	ManyToOne → subject_id	NOT NULL
gradeDate	LocalDate	grade_date	NOT NULL

Maydon	Java turi	Ustun / bog'lanish	Cheklov
score	Integer	score	NOT NULL
type	GradeType	type	NOT NULL, enum (STRING)
comment	String	comment	
createdDate	LocalDateTime	created_date	
createdBy	String	created_by	

LessonSlot → jadval `lesson_slot`

Maydon	Java turi	Ustun / bog'lanish	Cheklov
id	Long	id	PRIMARY KEY
schoolClass	SchoolClass	ManyToOne → class_id	NOT NULL
subject	Subject	ManyToOne → subject_id	NOT NULL
employee	Employee	ManyToOne → employee_id	NOT NULL
room	Room	ManyToOne → room_id	NOT NULL
weekday	String	weekday	NOT NULL
startTime	LocalTime	start_time	NOT NULL
endTime	LocalTime	end_time	NOT NULL

LibraryPermission → jadval `library_permission`

Maydon	Java turi	Ustun / bog'lanish	Cheklov
id	Long	id	PRIMARY KEY
school	School	ManyToOne (LAZY) → school_id	NOT NULL
grantor	Grantor	grantor	NOT NULL, enum (STRING)
grantorFullName	String	grantor_full_name	NOT NULL
phone	String	phone	
scope	String	scope	NOT NULL
validFrom	LocalDate	valid_from	NOT NULL
validUntil	LocalDate	valid_until	
documentPath	String	document_path	NOT NULL
documentType	String	document_type	NOT NULL
note	String	note	
revoked	Boolean	revoked	NOT NULL
revokedAt	LocalDateTime	revoked_at	
expiryWarnedAt	LocalDateTime	expiry_warned_at	
createdBy	String	created_by	
createdAt	LocalDateTime	created_at	NOT NULL

NotificationLog → jadval `notification_log`

Maydon	Java turi	Ustun / bog'lanish	Cheklov
id	Long	id	PRIMARY KEY
school	School	ManyToOne → school_id	NOT NULL
student	Student	ManyToOne → student_id	NOT NULL
chatId	Long	chat_id	NOT NULL
type	NotificationType	type	NOT NULL, enum (STRING)
referenceId	Long	reference_id	NOT NULL
recordDate	LocalDate	record_date	NOT NULL
text	String	text	NOT NULL
status	NotificationStatus	status	NOT NULL, enum (STRING)
attempts	Integer	attempts	NOT NULL
lastError	String	last_error	
createdAt	LocalDateTime	created_at	NOT NULL
scheduledAt	LocalDateTime	scheduled_at	NOT NULL
sentAt	LocalDateTime	sent_at	
replyMarkup	String	reply_markup	
quietBundle	Boolean	quiet_bundle	
media	String	media	
mediaParts	Integer	media_parts	

NotificationSettings → jadval `notification_settings`

Maydon	Java turi	Ustun / bog'lanish	Cheklov
id	Long	id	PRIMARY KEY
school	School	OneToOne → school_id	NOT NULL, UNIQUE
attendanceEnabled	Boolean	attendance_enabled	NOT NULL
gradeEnabled	Boolean	grade_enabled	NOT NULL
announcementEnabled	Boolean	announcement_enabled	NOT NULL
quietHoursEnabled	Boolean	quiet_hours_enabled	NOT NULL
quietHoursStart	LocalTime	quiet_hours_start	NOT NULL
quietHoursEnd	LocalTime	quiet_hours_end	NOT NULL

ParentMessage → jadval `parent_message`

Maydon	Java turi	Ustun / bog'lanish	Cheklov
id	Long	id	PRIMARY KEY
school	School	ManyToOne → school_id	NOT NULL
student	Student	ManyToOne → student_id	NOT NULL
chatId	Long	chat_id	NOT NULL

Maydon	Java turi	Ustun / bog'lanish	Cheklov
parentName	String	parent_name	
parentUsername	String	parent_username	
recipient	ParentMessageRecipient	recipient	NOT NULL, enum (STRING)
text	String	text	NOT NULL
photoFileId	String	photo_file_id	
status	ParentMessageStatus	status	NOT NULL, enum (STRING)
replyText	String	reply_text	
repliedBy	String	replied_by	
createdAt	LocalDateTime	created_at	NOT NULL
repliedAt	LocalDateTime	replied_at	

ParentSession → jadval `parent_session`

Maydon	Java turi	Ustun / bog'lanish	Cheklov
id	Long	id	PRIMARY KEY
chatId	Long	chat_id	NOT NULL, UNIQUE
selectedStudentId	Long	selected_student_id	
language	String	language	
firstName	String	first_name	
username	String	username	
pendingAction	String	pending_action	
pendingData	String	pending_data	
readAnnouncements	String	read_announcements	
notifyAbsence	Boolean	notify_absence	
notifyGrades	Boolean	notify_grades	
notifyLowGrade	Boolean	notify_low_grade	
notifyAnnouncements	Boolean	notify_announcements	
notifyTomorrowSchedule	Boolean	notify_tomorrow_schedule	
notifyWeeklyReport	Boolean	notify_weekly_report	
notifyEventReminder	Boolean	notify_event_reminder	
notifyMorningDigest	Boolean	notify_morning_digest	
notifyLibrary	Boolean	notify_library	
scheduleTime	LocalTime	schedule_time	
quietHoursEnabled	Boolean	quiet_hours_enabled	
quietStart	LocalTime	quiet_start	
quietEnd	LocalTime	quiet_end	
createdAt	LocalDateTime	created_at	
lastActiveAt	LocalDateTime	last_active_at	

ParentTelegramLink → jadval `parent_telegram_link`

Maydon	Java turi	Ustun / bog'lanish	Cheklov
id	Long	id	PRIMARY KEY
student	Student	ManyToOne → student_id	NOT NULL
chatId	Long	chat_id	NOT NULL
telegramUsername	String	telegram_username	
firstName	String	first_name	
linkedAt	LocalDateTime	linked_at	NOT NULL
active	Boolean	active	NOT NULL

Position → jadval `position`

Maydon	Java turi	Ustun / bog'lanish	Cheklov
id	Long	id	PRIMARY KEY
title	String	title	NOT NULL, UNIQUE
createdDate	LocalDateTime	created_date	
updatedDate	LocalDateTime	updated_date	

Region → jadval `region`

Maydon	Java turi	Ustun / bog'lanish	Cheklov
id	Long	id	PRIMARY KEY
code	String	code	NOT NULL
nameUz	String	name_uz	NOT NULL
state	String	state	NOT NULL
createdAt	LocalDateTime	created_at	NOT NULL
updatedAt	LocalDateTime	updated_at	NOT NULL

Room → jadval `room`

Maydon	Java turi	Ustun / bog'lanish	Cheklov
id	Long	id	PRIMARY KEY
building	Building	ManyToOne → building_id	NOT NULL
roomNumber	String	room_number	NOT NULL
capacity	Integer	capacity	NOT NULL
floor	Integer	floor	
type	RoomType	type	enum (STRING)

School → jadval `school`

Maydon	Java turi	Ustun / bog'lanish	Cheklov
id	Long	id	PRIMARY KEY
name	String	name	NOT NULL

Maydon	Java turi	Ustun / bog'lanish	Cheklov
address	String	address	NOT NULL
createdDate	LocalDateTime	created_date	
updatedDate	LocalDateTime	updated_date	

SchoolClass → jadval `school_class`

Maydon	Java turi	Ustun / bog'lanish	Cheklov
id	Long	id	PRIMARY KEY
academicYear	AcademicYear	ManyToOne → academic_year_id	NOT NULL
gradeNumber	Integer	grade_number	NOT NULL
sectionLetter	String	section_letter	NOT NULL
maxStudents	Integer	max_students	NOT NULL
classTeacher	Employee	ManyToOne → class_teacher_id	

Student → jadval `students`

Maydon	Java turi	Ustun / bog'lanish	Cheklov
id	Long	id	PRIMARY KEY
schoolClass	SchoolClass	ManyToOne → class_id	NOT NULL
firstName	String	first_name	NOT NULL
lastName	String	last_name	NOT NULL
birthDate	LocalDate	birth_date	NOT NULL
guardianName	String	guardian_name	
guardianPhone	String	guardian_phone	
guardianRelation	GuardianRelation	guardian_relation	enum (STRING)
birthDistrict	District	ManyToOne (LAZY) → birth_district_id	
residenceDistrict	District	ManyToOne (LAZY) → residence_district_id	
mahalla	String	mahalla	
street	String	street	
housingType	HousingType	housing_type	enum (STRING)
houseNumber	String	house_number	
apartmentNumber	String	apartment_number	
telegramLinkCode	String	telegram_link_code	UNIQUE

Subject → jadval `subject`

Maydon	Java turi	Ustun / bog'lanish	Cheklov
id	Long	id	PRIMARY KEY
school	School	ManyToOne → school_id	
name	String	name	NOT NULL
active	Boolean	active	

User → jadval `app_user`

Maydon	Java turi	Ustun / bog'lanish	Cheklov
`id`	`Long`	`id`	PRIMARY KEY
`username`	`String`	`username`	NOT NULL, UNIQUE
`password`	`String`	`password`	NOT NULL
`role`	`Role`	`role`	NOT NULL, enum (STRING)
`employee`	`Employee`	`ManyToOne → employee_id`	

D-ilova. Testlar (55 ta fayl)

Loyiha testlari: `src/test/java`. Integratsiya testlari (`*IntegrationTest`) haqiqiy HTTP va test bazasi bilan ishlaydi; `service/*Test` — Mockito bilan bazasiz unit testlar. `load` va `wikisource` tegli testlar oddiy `gradlew test` ga kirmaydi.

Test klassi	@Test soni	Nimani tekshiradi
`AccountBlockingIntegrationTest`	5	Ta'tildagi xodim: login 403, kirgan bo'lsa keyingi so'rov 401.
`AppealFlowIntegrationTest`	5	Murojaat boshidan oxirigacha: matn, rasm, video, ovoz, hujjat; javob va yopish.
`AttendanceCalendarIntegrationTest`	2	Oylik kalendar: kunni belgilash shu kunning barcha darslariga yoziladi.
`BotI18nTest`	9	Bot matnlari: uchala tilda ham har bir kalit bor.
`CallbackDataTest`	6	callback_data formati va 64 bayt chegarasi.
`KeyboardsTest`	5	Pastki klaviatura va tugmadan ekranni topish.
`BotScreensSeedIntegrationTest`	7	Botning har bir sahifasini haqiqiy router orqali chizib tekshiradi.
`GradeDateIntegrationTest`	3	Yakshanba va kelajak sanaga baho qo'yib bo'lmaydi (409).
`JournalIntegrationTest`	6	Sinf jurnali: baho qoidalari, oy kunlari, sinf paneli.
`JournalQueryCountTest`	1	Har endpoint 1–3 ta SQL so'rov, N+1 muammosi yo'q.
`LibraryBotFlowIntegrationTest`	1	Kutubxona bot oqimi: menyu → bo'lim → kitob → PDF.
`LibraryIntegrationTest`	10	Kutubxona qoidalari: huquqiy asos majburiy, faqat haqiqiy PDF, 50 MB, maktab chegarasi.
`LibraryLegalIntegrationTest`	11	Jamoat mulki (70 yil), ruxsatnomalar, Wikisource import va PDF harflari.
`BotLoadTest` `load`	1	Yuklama testi: 3000 ta soxta ota-ona (alohida yoqiladi).
`BotQueryCountProbe` `load`	1	Diagnostika: har update turida nechta SQL so'rov.
`MaktabBoshqaruvApplicationTests`	1	Spring konteksti xatosiz ko'tariladimi.
`ParentApiSecurityIntegrationTest`	2	Mini App API: ota-ona faqat o'z bolasini ko'radi; soxta initData 401.
`RegionIntegrationTest`	2	Hududlar: CRUD, kod takrori 409, soft delete, rollar.
`SchoolScopeIntegrationTest`	2	Boshqa maktab ma'lumoti 403 — path, query va body orqali.
`AbsenceRequestServiceTest`	4	Arizalar servisi (unit, Mockito).
`AcademicYearServiceTest`	5	O'quv yillari servisi (unit).
`AccountStatusServiceTest`	5	Bloklash qoidalari: o'zingizni va oxirgi adminni bloklab bo'lmaydi.
`AttendanceServiceTest`	4	Davomat servisi (unit).
`BotScheduledJobsTest`	5	Ertalabki digest faqat o'qish kunlari 07:30 da.
`BuildingServiceTest`	5	Binolar servisi (unit).
`EmployeeServiceTest`	7	Xodimlar servisi (unit).
`LessonSlotServiceTest`	8	Dars jadvali to'qnashuvlari (unit).
`LinkingServiceTest`	7	Ota-onani kod va telefon orqali bog'lash.

Test klassi	@Test soni	Nimani tekshiradi
`NotificationSenderTest`	8	Outbox yuboruvchi: limitlar, 429, 403, qayta urinish.
`NotificationServiceTest`	17	Event'lardan outbox qatorlari, dublikatlar, tinch soatlar.
`OutboxMediaTest`	2	429 bo'lsa albom qayta yuborilmaydi, faqat matn.
`ParentAccessServiceTest`	3	Begona o'quvchi id'si rad etiladi.
`ParentStatsTest`	9	Ota-ona ko'rinishlari hisob-kitoblari.
`PositionServiceTest`	6	Lavozimlar servisi (unit).
`RoomServiceTest`	3	Xonalar servisi (unit).
`SchoolClassServiceTest`	4	Sinflar servisi (unit).
`SchoolServiceTest`	6	Maktablar servisi (unit).
`StudentServiceTest`	4	O'quvchilar servisi (unit).
`SubjectServiceTest`	6	Fanlar servisi (unit).
`TelegramTestFixtures`	0	Telegram testlari uchun umumiy xotiradagi ob'ektlar (test emas).
`TimetablePdfServiceTest`	3	Dars jadvali PDF: har sinfga bitta sahifa, kirill matni bilan.
`UserServiceTest`	8	Foydalanuvchilar servisi (unit).
`StudentAddressIntegrationTest`	3	O'quvchi manzili: hudud → tuman, kvartira/hovli qoidalari, profil.
`SubjectLifecycleIntegrationTest`	2	Nofaol fanga baho qo'yilmaydi, eski baho fan nomini saqlaydi.
`ApiClient`	0	Integratsiya testlari uchun kichik HTTP mijoz (test emas).
`MutableClock`	0	Testlar vaqtni boshqara oladigan soat (test emas).
`MessageFormatterTest`	9	Xabarnoma matnlari formati.
`PhoneNormalizerTest`	4	Telefon raqamlarini normallashtirish.
`QuietHoursTest`	6	Tinch soatlar hisobi.
`TelegramInitDataValidatorTest`	6	initData imzosi: to'g'ri, soxta, eskirgan.
`TokenMaskerTest`	5	Tokenni matndan yashirish.
`TelegramAdminIntegrationTest`	5	Telegram admin: ota-onalar ro'yxati, albomli xabar, rollar.
`TelegramMockFlowIntegrationTest`	2	Mock rejimda to'liq oqim: bog'lanish → davomat → xabarnoma.
`JwtUtilTest`	3	JWT yaratish va tekshirish.
`WikisourceLiveCheck` `wikisource`	1	Haqiqiy Wikisource bilan tekshiruv (build'ga kirmaydi).